

Contents

1	Appendix: The Grandmaster Strategy Guide	8
1.1	1. The "REACTO" Framework (How to Interview)	8
1.2	2. Universal Decision Tree (Data Structures)	8
1.3	3. The "Unstuck" Protocol (What to do when blanking)	8
1.4	4. Pattern Deep Dives	8
1.4.1	A. The Sliding Window Template	8
1.4.2	B. The Binary Search Template (Minimize Errors)	9
1.4.3	C. The "Topological Sort" Template (Kahn's Algo)	9
1.5	5. Math Bit Manipulation Tricks	9
1.6	6. Complexity Cheat Sheet	9
1.7	7. Python-Specific Speed Tricks	10
1.8	8. Final Pre-Interview Checklist	10
2	Appendix: Python Syntax Superpowers	10
2.1	1. List Comprehensions & Generators	10
2.2	2. Advanced Sorting	10
2.3	3. The 'collections' Module	10
2.4	4. The 'itertools' Module	11
2.5	5. The 'heapq' Module (Min-Heap)	11
2.6	6. The 'bisect' Module (Binary Search)	11
2.7	7. String & Math Tricks	12
2.8	8. The 'else' Clause in Loops	12
3	Sliding Window: Fixed & Dynamic	12
3.1	Best Time to Buy and Sell Stock	12
3.2	Longest Substring Without Repeating Characters	14
3.3	Longest Repeating Character Replacement	16
3.4	Permutation in String	18
3.5	Minimum Size Subarray Sum	20
3.6	Minimum Window Substring	21
3.7	Sliding Window Maximum	23
4	Two Pointers: String Array Manipulation	24
4.1	Reverse String	24
4.2	Valid Palindrome	24
4.3	Valid Palindrome II	25
4.4	Merge Strings Alternately	26
4.5	Merge Sorted Array	27
4.6	Remove Duplicates from Sorted Array	28
4.7	Two Sum II - Input Array Is Sorted	29
5	Two Pointers: Target Sums & Containers	29
5.1	3Sum	29
5.2	4Sum	30
5.3	Rotate Array	32
5.4	Container With Most Water	32
5.5	Boats to Save People	33
5.6	Trapping Rain Water	34
6	Stack: Essentials & Simulation	35
6.1	Baseball Game	35
6.2	Valid Parentheses	37
6.3	Implement Stack Using Queues	38
6.4	Implement Queue Using Stacks	40
6.5	Min Stack	41
6.6	Evaluate Reverse Polish Notation	42
6.7	Asteroid Collision	43

7	Stack: Monotonic Patterns & Parsing	44
7.1	Daily Temperatures	44
7.2	Car Fleet	46
7.3	Simplify Path	47
7.4	Decode String	48
7.5	Largest Rectangle in Histogram (Hard)	50
8	Binary Search: The "Transition Point" Framework	51
8.1	Search Insert Position	51
8.2	Binary Search	52
8.3	Sqrt(x)	53
8.4	Guess Number Higher Or Lower	54
8.5	Koko Eating Bananas	55
8.6	Capacity to Ship Packages	56
8.7	Search a 2D Matrix	57
9	Binary Search: Rotated Arrays & Hard Variations	57
9.1	Find Minimum in Rotated Sorted Array	57
9.2	Search in Rotated Sorted Array	59
9.3	Time Based Key-Value Store	60
9.4	Median of Two Sorted Arrays	61
9.5	Search in Rotated Sorted Array II (Duplicates)	62
10	Linked List: Pointers & The Sentinel Pattern	62
10.1	Reverse Linked List	62
10.2	Merge Two Sorted Lists	64
10.3	Linked List Cycle	65
10.4	Reorder List	66
10.5	Remove Nth Node From End of List	67
10.6	Copy List With Random Pointer	68
10.7	Add Two Numbers	69
11	Linked List: Advanced Patterns & Design	69
11.1	Find the Duplicate Number	69
11.2	LRU Cache	72
11.3	Merge K Sorted Lists	74
11.4	Reverse Nodes in k-Group	76
12	Heap: Essentials & The "K" Pattern	77
12.1	Kth Largest Element in a Stream	77
12.2	Last Stone Weight	79
12.3	K Closest Points to Origin	80
12.4	Kth Largest Element in an Array	81
12.5	Task Scheduler	82
12.6	Design Twitter	84
13	Heap: Advanced Scheduling & Two Heaps	85
13.1	Single-Threaded CPU	85
13.2	Reorganize String	87
13.3	Longest Happy String	89
13.4	Car Pooling	91
13.5	Find Median From Data Stream (Two Heaps)	93
13.6	IPO (Hard)	95
14	Arrays & Hashing: Maps, Sets & Prefixes	96
14.1	Product of Array Except Self	96
14.2	Valid Sudoku	98
14.3	Longest Consecutive Sequence	100
14.4	Encode and Decode Strings	102

15 Arrays & Hashing: The Classics	103
15.1 Two Sum	103
15.2 Group Anagrams	105
15.3 Top K Frequent Elements	106
15.4 Valid Anagram	108
15.5 Sort Colors	109
16 Arrays & Hashing: Hard Patterns & Design	110
16.1 First Missing Positive	110
16.2 Subarray Sum Equals K	112
16.3 Majority Element	114
16.4 Range Sum Query 2D - Immutable	115
16.5 Design HashSet	116
16.6 Longest Common Prefix	117
16.7 Sort an Array	118
17 Arrays & Hashing: Final Missing Pieces	118
17.1 Concatenation of Array	118
17.2 Contains Duplicate	120
17.3 Remove Element	121
17.4 Design HashMap	122
17.5 Best Time to Buy and Sell Stock II	124
17.6 Majority Element II	125
18 Backtracking: The Decision Tree Blueprint	126
18.1 Subsets (The "Include/Exclude" Pattern)	126
18.2 Combination Sum (Unlimited Use)	128
18.3 Permutations	129
18.4 Subsets II (Handling Duplicates)	130
18.5 Combination Sum II (No Reuse, Duplicates)	131
18.6 Palindrome Partitioning	132
19 Backtracking: Grids & Logic Puzzles	132
19.1 Word Search	132
19.2 Letter Combinations of a Phone Number	135
19.3 N-Queens (Hard)	137
19.4 Sudoku Solver (Hard)	138
20 Backtracking: Variations & Partitioning	139
20.1 Combinations	139
20.2 Permutations II	141
20.3 Generate Parentheses	143
20.4 Matchsticks to Square	144
20.5 Partition to K Equal Sum Subsets	145
20.6 Word Break II	146
20.7 N-Queens II	147
21 Catch-Up: Missing High-Signal Problems	147
21.1 Online Stock Span	147
21.2 Maximum Frequency Stack	149
21.3 Split Array Largest Sum	150
21.4 Find in Mountain Array	151
21.5 Reverse Linked List II	152
21.6 LFU Cache	153
21.7 Design Circular Queue	154

22 Trees: Recursion Fundamentals	154
22.1 Invert Binary Tree	154
22.2 Maximum Depth of Binary Tree	156
22.3 Diameter of Binary Tree	157
22.4 Balanced Binary Tree	158
22.5 Same Tree	159
22.6 Subtree of Another Tree	160
22.7 Binary Tree Traversals (Iterative)	161
23 Trees: BST Properties & Level Traversal	161
23.1 Lowest Common Ancestor of a BST	161
23.2 Binary Tree Level Order Traversal	163
23.3 Binary Tree Right Side View	164
23.4 Count Good Nodes in Binary Tree	165
23.5 Validate Binary Search Tree	166
23.6 Kth Smallest Element in a BST	167
23.7 Insert into a Binary Search Tree	168
23.8 Delete Node in a BST	169
24 Trees: Construction & Hard Problems	169
24.1 Construct Binary Tree from Preorder and Inorder Traversal	169
24.2 Delete Leaves With a Given Value	171
24.3 House Robber III	172
24.4 Binary Tree Maximum Path Sum	173
24.5 Serialize and Deserialize Binary Tree	174
25 Trees: The Missing Piece & Tries	175
25.1 Construct Quad Tree	175
25.2 Implement Trie (Prefix Tree)	176
25.3 Design Add and Search Words Data Structure	177
25.4 Extra Characters in a String	178
25.5 Word Search II	179
26 Graphs: Grid DFS & BFS	179
26.1 Island Perimeter	179
26.2 Number of Islands	181
26.3 Max Area of Island	183
26.4 Walls and Gates	184
26.5 Rotting Oranges	186
26.6 Pacific Atlantic Water Flow	187
26.7 Surrounded Regions	188
27 Graphs: Dependencies & Disjoint Sets	188
27.1 Clone Graph	188
27.2 Course Schedule	190
27.3 Course Schedule II	192
27.4 Graph Valid Tree	193
27.5 Number of Connected Components in an Undirected Graph	194
27.6 Redundant Connection	195
27.7 Accounts Merge	196
28 Graphs: Shortest Path & Logic	196
28.1 Open The Lock	196
28.2 Evaluate Division	198
28.3 Minimum Height Trees	199
28.4 Course Schedule IV	200
28.5 Word Ladder	201

29 Advanced Graphs: Pathfinding & MST	201
29.1 Path with Minimum Effort	201
29.2 Network Delay Time	203
29.3 Min Cost to Connect All Points	204
29.4 Swim in Rising Water	206
30 Advanced Graphs: Hard Logic & Topo Sort	206
30.1 Cheapest Flights Within K Stops	206
30.2 Reconstruct Itinerary	208
30.3 Alien Dictionary	209
30.4 Build a Matrix With Conditions	210
31 Advanced Graphs: Critical Edges & GCD	210
31.1 Find Critical and Pseudo-Critical Edges in MST	210
31.2 Greatest Common Divisor Traversal	213
32 1-D DP: Basics & Palindromes	214
32.1 Climbing Stairs	214
32.2 Min Cost Climbing Stairs	215
32.3 House Robber	216
32.4 House Robber II	217
32.5 Longest Palindromic Substring	218
32.6 Palindromic Substrings	219
33 1-D DP: Strings, Products & Subsequences	219
33.1 N-th Tribonacci Number	219
33.2 Decode Ways	220
33.3 Coin Change	221
33.4 Maximum Product Subarray	222
33.5 Word Break	223
33.6 Longest Increasing Subsequence	224
34 1-D DP: Knapsacks & Math	224
34.1 Partition Equal Subset Sum	224
34.2 Combination Sum IV	226
34.3 Perfect Squares	227
34.4 Integer Break	228
34.5 Delete and Earn	229
35 2-D DP: Grids & Sequences	229
35.1 Unique Paths	229
35.2 Unique Paths II	231
35.3 Minimum Path Sum	232
35.4 Longest Common Subsequence	233
35.5 Last Stone Weight II	234
35.6 Best Time to Buy and Sell Stock with Cooldown	235
36 2-D DP: Knapsacks, Strings & Memoization	235
36.1 Coin Change II	235
36.2 Target Sum	237
36.3 Interleaving String	238
36.4 Longest Increasing Path in a Matrix	239
36.5 Distinct Subsequences	240
36.6 Edit Distance	241

37 2-D DP: Game Theory & Hard Intervals	241
37.1 Stone Game	241
37.2 Stone Game II	243
37.3 Burst Balloons	244
37.4 Regular Expression Matching	245
38 Greedy: Change, Subarrays & Jumps	245
38.1 Lemonade Change	245
38.2 Maximum Subarray	247
38.3 Maximum Sum Circular Subarray	248
38.4 Longest Turbulent Subarray	249
38.5 Jump Game	250
39 Greedy: Advanced Jumps & Simulation	250
39.1 Jump Game II	250
39.2 Jump Game VII	252
39.3 Gas Station	253
39.4 Hand of Straights	254
39.5 Dota2 Senate	255
40 Greedy: Constructive Logic	255
40.1 Merge Triplets to Form Target Triplet	255
40.2 Partition Labels	257
40.3 Valid Parenthesis String	258
40.4 Candy	259
41 Intervals: Merging & Schedules	259
41.1 Insert Interval	259
41.2 Merge Intervals	261
41.3 Non-overlapping Intervals	262
41.4 Meeting Rooms	263
41.5 Meeting Rooms II	264
42 Intervals: Advanced Heaps	264
42.1 Meeting Rooms III	264
42.2 Minimum Interval to Include Each Query	266
43 Math & Geometry: Matrix Manipulation	266
43.1 Transpose Matrix	266
43.2 Rotate Image	268
43.3 Spiral Matrix	269
43.4 Set Matrix Zeroes	270
43.5 Detect Squares	271
44 Math & Geometry: Number Theory & GCD	271
44.1 Greatest Common Divisor of Strings	271
44.2 Insert Greatest Common Divisors in Linked List	273
44.3 Happy Number	274
44.4 Plus One	275
45 Math & Geometry: String Logic & Arithmetic	275
45.1 Roman to Integer	275
45.2 Excel Sheet Column Title	277
45.3 Pow(x, n)	278
45.4 Multiply Strings	279

46 Bit Manipulation: XOR & Essentials	279
46.1 Single Number	279
46.2 Number of 1 Bits	281
46.3 Counting Bits	282
46.4 Missing Number	283
46.5 Reverse Bits	284
47 Bit Manipulation: Arithmetic & Ranges	284
47.1 Add Binary	284
47.2 Sum of Two Integers	286
47.3 Reverse Integer	287
47.4 Bitwise AND of Numbers Range	288
47.5 Minimum Array End	289

1 Appendix: The Grandmaster Strategy Guide

1.1 1. The "REACTO" Framework (How to Interview)

Don't just code. Follow this flow for **every single problem**.

1. **R - Repeat & Clarify:** Paraphrase the question. Ask about constraints (Is $N < 0$? Is the graph connected? Max value of N ?).
2. **E - Examples:** Write down input/output. Include edge cases (Empty? Single element? Sorted? All negative?).
3. **A - Approaches:**
 - **Brute Force:** "I could check every pair ($O(N^2)$)..."
 - **Optimize:** "But since it's sorted, we can use Two Pointers ($O(N)$)."
 - **Trade-offs:** Discuss Time vs. Space.
4. **C - Code:** Write clean, Pythonic code. Use helper functions.
5. **T - Test (Dry Run):** Trace your code with variables step-by-step.
6. **O - Optimize:** Can we use less space? Can we exit early?

1.2 2. Universal Decision Tree (Data Structures)

- **Input is Sorted?** → Binary Search ($O(\log N)$) or Two Pointers.
- **Find Top K elements?** → Min-Heap (size K).
- **Subarrays / Substrings?** → Sliding Window or Prefix Sums.
- **Tree/Graph Shortest Path?** → BFS (Unweighted) or Dijkstra (Weighted).
- **Tree/Graph "All Paths" or "Connectivity"?** → DFS.
- **Permutations / Subsets?** → Backtracking.
- **Optimization (Min/Max) with Overlapping Subproblems?** → Dynamic Programming.
- **Common Strings (Prefixes)?** → Trie.
- **Cycle Detection?** → Floyd's Tortoise & Hare (LL) or Union-Find (Graph).

1.3 3. The "Unstuck" Protocol (What to do when blanking)

If you are staring at the screen with no idea, try these:

1. **Smallest Example:** Solve it manually for $N = 1, 2, 3$. Do you see a pattern?
2. **Sorting:** "If I sort the input, does the problem become trivial?"
3. **Reverse It:** Try solving from the back (End to Start).
4. **HashMap It:** "Can I store something I've seen before to save time?"
5. **Pre-computation:** Can I build a Prefix Sum or Frequency Map first?

1.4 4. Pattern Deep Dives

1.4.1 A. The Sliding Window Template

Use for "Longest/Shortest substring satisfying condition".

```
1 l = 0
2 for r in range(len(nums)):
3     # 1. Add nums[r] to window (update sum/map)
4     # 2. While window is INVALID (sum > target, etc.):
5         # Remove nums[l]
6         # l += 1
7     # 3. Update global result (max len = r - l + 1)
```

1.4.2 B. The Binary Search Template (Minimize Errors)

Always use 'l := r' and 'mid = l + (r - l) // 2'.

```
1 l, r = 0, len(nums) - 1
2 while l <= r:
3     mid = l + (r - l) // 2
4     if nums[mid] == target:
5         return mid
6     elif nums[mid] < target:
7         l = mid + 1
8     else:
9         r = mid - 1
10 return -1
```

1.4.3 C. The "Topological Sort" Template (Kahn's Algo)

Use for dependencies (courses, build order).

```
1 # 1. Build Graph & Indegrees
2 adj = {i:[] for i in nodes}
3 indegree = {i:0 for i in nodes}
4 for u, v in edges:
5     adj[u].append(v)
6     indegree[v] += 1
7 # 2. Queue sources (indegree 0)
8 q = deque([i for i in indegree if indegree[i] == 0])
9 # 3. Process
10 while q:
11     node = q.popleft()
12     for nei in adj[node]:
13         indegree[nei] -= 1
14         if indegree[nei] == 0: q.append(nei)
```

1.5 5. Math Bit Manipulation Tricks

- **XOR:** $x \oplus x = 0$, $x \oplus 0 = x$. (Single Number).
- **Rightmost Set Bit:** $x \& -x$.
- **Clear Rightmost Set Bit:** $x \& (x - 1)$. (Hamming Weight).
- **Even/Odd:** $(x \& 1) == 0$ is even.
- **Divide/Multiply by 2:** $x \gg 1$ and $x \ll 1$.
- **Sum of 1..N:** $N \times (N + 1) // 2$.
- **GCD:** 'math.gcd(a, b)' (Euclidean: $a, b = b, a \% b$).
- **Modulo Arithmetic:** $(a + b) \% k = ((a \% k) + (b \% k)) \% k$.

1.6 6. Complexity Cheat Sheet

Data Structure	Operation	Complexity
Array/List	Access	$O(1)$
	Search	$O(N)$
	Insert/Delete	$O(N)$
Linked List	Access	$O(N)$
	Insert/Delete	$O(1)$ (if pointer known)
Hash Map	Put/Get	$O(1)$ Avg
BST (Balanced)	Search/Insert	$O(\log N)$
Heap	Peek	$O(1)$
	Push/Pop	$O(\log N)$
Sorting	Merge/Quick	$O(N \log N)$

1.7 7. Python-Specific Speed Tricks

- **Reversing:** `arr[::-1]` is faster than `arr.reverse()`.
- **Max/Min:** `val = float('inf')` and `float('-inf')`.
- **Default Dict:** `from collections import defaultdict`. No key errors.
- **Counter:** `from collections import Counter`. Instant frequency map.
- **Swapping:** `a, b = b, a`.
- **Sorting:** `arr.sort(key=lambda x: x[1])` to sort by specific element.

1.8 8. Final Pre-Interview Checklist

1. **Speak** while thinking. Silence is a red flag.
2. **Validate** assumptions. "Is the array sorted?" "Are inputs unique?"
3. **Name variables** meaningfully. `'index'` vs `'i'`, `'count'` vs `'c'`.
4. **Modularize**. If logic is complex, say "I'll assume a helper function `'isValid()'` exists..." and write it later.
5. **Be honest**. If you don't know, say "I'm not sure, but here is how I would derive it..."

2 Appendix: Python Syntax Superpowers

2.1 1. List Comprehensions & Generators

Write concise loops. Be careful with readability.

```
1 # Filter and Map: Squares of even numbers
2 evens = [x**2 for x in range(10) if x % 2 == 0]
3
4 # 2D Grid Initialization (CRITICAL)
5 # Do NOT use [[0]*5]*5 (creates references to same row)
6 rows, cols = 5, 5
7 grid = [[0] * cols for _ in range(rows)]
8
9 # Flatten a 2D list
10 flat = [val for row in grid for val in row]
```

2.2 2. Advanced Sorting

Python's `'sort'` is stable (Timsort). You can sort by multiple keys easily.

```
1 data = [(1, 3), (1, 2), (2, 5)]
2
3 # Sort by 1st element ASC, then 2nd element DESC
4 # Trick: Negate numbers for descending order
5 data.sort(key=lambda x: (x[0], -x[1]))
6 # Result: [(1, 3), (1, 2), (2, 5)]
7
8 # Sort by length of string
9 strs = ["apple", "banana", "kiwi"]
10 strs.sort(key=len)
```

2.3 3. The `'collections'` Module

This module is your best friend.

- **Counter:** Frequency map.

```

1 from collections import Counter
2 counts = Counter("banana") # {'a': 3, 'n': 2, 'b': 1}
3 # Most common
4 top_k = counts.most_common(2) # [('a', 3), ('n', 2)]
5 # Subtract counts
6 counts.subtract("bana")

```

- **defaultdict**: No 'KeyError'. Code cleaner graphs.

```

1 from collections import defaultdict
2 adj = defaultdict(list)
3 adj[0].append(1) # No need to check if 0 exists

```

- **deque**: $O(1)$ append/pop from both ends (BFS).

```

1 from collections import deque
2 q = deque([1, 2, 3])
3 q.appendleft(0)
4 q.pop()

```

2.4 4. The 'itertools' Module

Don't write nested loops if you don't have to.

```

1 import itertools
2
3 # Permutations (Order matters)
4 perms = list(itertools.permutations([1, 2, 3], 2))
5
6 # Combinations (Order doesn't matter)
7 combs = list(itertools.combinations([1, 2, 3], 2))
8
9 # Cartesian Product (Great for grid neighbors or multiple arrays)
10 # Replaces: for x in A: for y in B:
11 prod = list(itertools.product([0, 1], repeat=3)) # 000 to 111
12
13 # Accumulate (Prefix Sums)
14 nums = [1, 2, 3, 4]
15 prefix = list(itertools.accumulate(nums)) # [1, 3, 6, 10]

```

2.5 5. The 'heapq' Module (Min-Heap)

Python only has a Min-Heap. For Max-Heap, invert values.

```

1 import heapq
2 arr = [3, 1, 4, 1, 5]
3 heapq.heapify(arr) #  $O(N)$ 
4
5 min_val = heapq.heappop(arr) # 1
6 heapq.heappush(arr, 2)
7
8 # Max-Heap Workaround: Store negative values
9 max_h = [-x for x in nums]
10 heapq.heapify(max_h)
11 peak = -heapq.heappop(max_h)
12
13 # Get K largest/smallest efficiently
14 large = heapq.nlargest(3, arr)
15 small = heapq.nsmallest(3, arr)

```

2.6 6. The 'bisect' Module (Binary Search)

Don't write binary search from scratch unless asked.

```

1 import bisect
2 arr = [1, 3, 4, 4, 8]
3
4 # Find insertion point (Left side) -> First '4'
5 idx_l = bisect.bisect_left(arr, 4) # 2

```

```

6
7 # Find insertion point (Right side) -> After last '4'
8 idx_r = bisect.bisect_right(arr, 4) # 4
9
10 # Count occurrences of x
11 count = bisect.bisect_right(arr, x) - bisect.bisect_left(arr, x)

```

2.7 7. String & Math Tricks

```

1 # Reverse string/list
2 s = "hello"
3 rev = s[::-1]
4
5 # ASCII values
6 code = ord('a') # 97
7 char = chr(97) # 'a'
8
9 # Check Alphanumeric
10 s.isalnum() # True if "a1", False if "a!"
11
12 # Infinity
13 inf = float('inf')
14 neg_inf = float('-inf')
15
16 # Divmod (Quotient and Remainder)
17 q, r = divmod(10, 3) # (3, 1)
18
19 # Zip (Iterate multiple lists)
20 names = ["A", "B"]
21 scores = [10, 20]
22 for name, score in zip(names, scores):
23     print(name, score)
24
25 # Unpacking
26 head, *tail = [1, 2, 3, 4] # head=1, tail=[2, 3, 4]

```

2.8 8. The 'else' Clause in Loops

A rare but useful trick for search loops.

```

1 for n in nums:
2     if n == target:
3         print("Found")
4         break
5 else:
6     # Runs ONLY if loop completed without 'break'
7     print("Not Found")

```

3 Sliding Window: Fixed & Dynamic

3.1 Best Time to Buy and Sell Stock

```

1 """
2 1. CLARIFYING QUESTIONS & EXPECTED ANSWERS
3 - Q: Can I sell before I buy?
4   A: No, time moves forward. Buy index < Sell index.
5 - Q: Can I buy and sell multiple times?
6   A: No, exactly one transaction (Buy once, Sell once).
7 - Q: What if prices keep dropping?
8   A: Return 0 (no transaction).
9
10 2. APPROACHES TO CONSIDER & REJECT
11 - Approach 1: Brute Force (Double Loop)
12   - Logic: For each day i, check every future day j. Max(price[j] - price[i]).
13   - Complexity: O(N^2).
14   - Reject: Too slow for N=10^5.
15 - Approach 2: One Pass (Greedy / Two Pointers)
16   - Logic: Keep track of the 'min_price' seen so far. Profit = current - min_price.

```

```

17     - Complexity:  $O(N)$  Time,  $O(1)$  Space.
18     - Accept: Optimal.
19
20 3. INDENTED ENGLISH (LOGIC)
21 function maxProfit(prices):
22     min_price = infinity
23     max_p = 0
24     for price in prices:
25         if price < min_price:
26             update min_price
27         else:
28             profit = price - min_price
29             max_p = max(max_p, profit)
30     return max_p
31 """

```

The Magic Question (Get Buy-In)

"I will iterate through the prices once, keeping track of the lowest price seen so far. At each step, I calculate the potential profit. This gives us $O(N)$ time and $O(1)$ space. Should I code this?"

```

1 def maxProfit(prices: List[int]) -> int:
2     # Guard Clause: Empty input
3     if not prices:
4         return 0
5
6     l, r = 0, 1 # Left=Buy, Right=Sell
7     maxP = 0
8
9     while r < len(prices):
10        # Is this a profitable transaction?
11        if prices[l] < prices[r]:
12            profit = prices[r] - prices[l]
13            maxP = max(maxP, profit)
14        else:
15            # Found a new lower price? Shift 'buy' to current 'sell' index
16            # This is the "Greedy" jump
17            l = r
18            r += 1
19
20    return maxP
21
22 """
23 5. DRY RUN (HUMAN DEBUGGER)
24 prices = [7, 1, 5, 3, 6, 4]
25 - l=0(7), r=1(1). 7 < 1 False. New low found. l=1. r=2.
26 - l=1(1), r=2(5). Profit=4. maxP=4.
27 - l=1(1), r=3(3). Profit=2. maxP=4.
28 - l=1(1), r=4(6). Profit=5. maxP=5.
29 - Return 5.
30 """

```

3.2 Longest Substring Without Repeating Characters

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Case sensitive?
4   A: Yes, 'a' != 'A'.
5 - Q: What constitutes a "substring"?
6   A: Contiguous sequence of characters.
7 - Q: Input constraints?
8   A: s length up to 5 * 10^4.
9
10 2. APPROACHES TO CONSIDER & REJECT
11 - Approach 1: Brute Force
12   - Logic: Check every substring s[i:j]. Use a set to validate.
13   - Complexity: O(N^3).
14   - Reject: Terrible.
15 - Approach 2: Sliding Window + Hash Set
16   - Logic: Expand Right pointer. If duplicate found, shrink Left pointer
17     until duplicate is removed.
18   - Complexity: O(N) Time, O(min(N, charset)) Space.
19   - Accept: Optimal.
20
21 3. INDENTED ENGLISH (LOGIC)
22 function lengthOfLongestSubstring(s):
23     charSet = empty set
24     l = 0
25     res = 0
26     for r from 0 to len(s):
27         while s[r] in charSet:
28             remove s[l] from charSet
29             l++
30         add s[r] to charSet
31         res = max(res, r - l + 1)
32     return res
33 """
```

The Magic Question (Get Buy-In)

"I'll use a dynamic sliding window with a Hash Set. If I encounter a duplicate, I'll shrink the window from the left until the duplicate is removed. This guarantees $O(N)$ time. Should I code this?"

```
1 def lengthOfLongestSubstring(s: str) -> int:
2     charSet = set()
3     l = 0
4     res = 0
5
6     for r in range(len(s)):
7         # Shrink Phase: Ensure window is valid (no duplicates)
8         # We must remove elements from the left until s[r] is no longer in set
9         while s[r] in charSet:
10             charSet.remove(s[l])
11             l += 1
12
13         # Expand Phase
14         charSet.add(s[r])
15         res = max(res, r - l + 1)
16
17     return res
18
19 """
20 5. DRY RUN (HUMAN DEBUGGER)
21 s = "pwwkew"
22 - r=0('p'): set={p}, res=1
23 - r=1('w'): set={p,w}, res=2
24 - r=2('w'): Duplicate 'w'.
25   - Remove 'p' (l=1). set={w}. 'w' still in set.
26   - Remove 'w' (l=2). set={}.
27   - Add 'w'. set={w}. res=max(2, 1) -> 2.
28 - r=3('k'): set={w,k}, res=2
```

```
29     - r=4('e'): set={w,k,e}, res=3. Return 3.  
30     """
```

3.3 Longest Repeating Character Replacement

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Uppercase English letters only?
4   A: Yes.
5 - Q: Can k be 0?
6   A: Yes, works as finding longest consecutive segment.
7
8 2. APPROACHES TO CONSIDER & REJECT
9 - Approach 1: Brute Force
10  - Logic: For every substring, check if (len - max_freq) <= k.
11  - Complexity:  $O(N^2)$ .
12  - Reject: TLE.
13 - Approach 2: Sliding Window (Validity Check)
14  - Key Math: valid = (window_len - max_freq) <= k.
15  - Optimization: We don't need to decrement 'max_freq' when shrinking.
16    The window only expands when we find a NEW max_freq.
17  - Complexity:  $O(N)$ .
18  - Accept: Optimal "Strong Hire" solution.
19
20 3. INDENTED ENGLISH (LOGIC)
21  function characterReplacement(s, k):
22      count = map
23      res = 0, l = 0, maxf = 0
24      for r from 0 to len(s):
25          count[s[r]]++
26          maxf = max(maxf, count[s[r]])
27
28          # Validity Check: Replacements needed > k?
29          if (r - l + 1) - maxf > k:
30              count[s[l]]--
31              l++
32          res = max(res, r - l + 1)
33 """
```

The Magic Question (Get Buy-In)

"I'll use a sliding window and track the count of the most frequent character in the current window. If $(\text{WindowLength} - \text{MaxFreq}) \leq k$, the window is valid. This is $O(N)$ time. Agreed?"

```
1 def characterReplacement(s: str, k: int) -> int:
2     count = {}
3     res = 0
4     l = 0
5     maxf = 0 # Tracks max frequency in current window
6
7     for r in range(len(s)):
8         count[s[r]] = 1 + count.get(s[r], 0)
9
10        # Optimization: We only care if maxf increases to potentially maximize result
11        maxf = max(maxf, count[s[r]])
12
13        # Check Validity: Do we need more than k replacements?
14        # length = r - l + 1
15        if (r - l + 1) - maxf > k:
16            # Shrink window
17            count[s[l]] -= 1
18            l += 1
19
20        # At this point, window is valid (or size is preserved)
21        res = max(res, r - l + 1)
22
23    return res
24
25 """
26 5. DRY RUN (HUMAN DEBUGGER)
27 s = "AABABBA", k = 1
28 - r=0..3 ("AABA"): maxf=3('A'). Len=4. 4-3 = 1 <= k. Valid. res=4.
29 - r=4 ("AABAB"): maxf=3('A'). Len=5. 5-3 = 2 > k. INVALID.
```

```
30     - Shrink: l=1. Window effectively "ABAB" (size 4 preserved).
31     - count['A'] decreases.
32     - Note: We don't shrink size, we just shift the window right until valid again.
33 """
```

3.4 Permutation in String

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Does s2 contain a permutation of s1?
4 - Q: What defines a permutation?
5 A: Same characters, same counts. Order doesn't matter.
6 - Q: Case sensitive?
7 A: Yes.
8
9 2. APPROACHES TO CONSIDER & REJECT
10 - Approach 1: Sort Substrings
11 - Logic: For every substring of length s1 in s2, sort and compare.
12 - Complexity:  $O(N * M \log M)$ .
13 - Reject: Too slow.
14 - Approach 2: Sliding Window with Hash Maps (Fixed Size)
15 - Logic: Window size is fixed at len(s1). Maintain matches count.
16 - Complexity:  $O(26 * N) \rightarrow O(N)$ .
17 - Accept: Optimal.
18
19 3. INDENTED ENGLISH (LOGIC)
20 function checkInclusion(s1, s2):
21     if len1 > len2: return False
22     count1, count2 = array[26]
23     fill initial window
24     matches = compare(count1, count2)
25
26     for r from len1 to len2:
27         if matches == 26: return True
28         add s2[r], remove s2[l]
29         update matches
30     return matches == 26
31 """
```

The Magic Question (Get Buy-In)

"Since permutations have the same length, I'll use a fixed-size sliding window of 'len(s1)'. I will maintain two character count arrays and a 'matches' variable to compare in $O(1)$. Total time $O(N)$. Should I code this?"

```
1 def checkInclusion(s1: str, s2: str) -> bool:
2     if len(s1) > len(s2):
3         return False
4
5     # 1. Initialize First Window
6     # Using arrays for  $O(1)$  comparison (size 26)
7     s1Count, s2Count = [0] * 26, [0] * 26
8     for i in range(len(s1)):
9         s1Count[ord(s1[i]) - ord('a')] += 1
10        s2Count[ord(s2[i]) - ord('a')] += 1
11
12    matches = 0
13    for i in range(26):
14        matches += (1 if s1Count[i] == s2Count[i] else 0)
15
16    # 2. Slide Window
17    l = 0
18    for r in range(len(s1), len(s2)):
19        if matches == 26:
20            return True
21
22        # Add new char (r)
23        index = ord(s2[r]) - ord('a')
24        s2Count[index] += 1
25        if s2Count[index] == s1Count[index]:
26            matches += 1
27        elif s2Count[index] == s1Count[index] + 1:
28            matches -= 1 # We just broke a match
29
30        # Remove old char (l)
31        index = ord(s2[l]) - ord('a')
32        s2Count[index] -= 1
33        if s2Count[index] == s1Count[index]:
34            matches += 1
35        elif s2Count[index] == s1Count[index] - 1:
36            matches -= 1 # We just broke a match
37        l += 1
```

```

31     index = ord(s2[l]) - ord('a')
32     s2Count[index] -= 1
33     if s2Count[index] == s1Count[index]:
34         matches += 1
35     elif s2Count[index] == s1Count[index] - 1:
36         matches -= 1
37     l += 1
38
39     return matches == 26
40
41 """
42 5. DRY RUN (HUMAN DEBUGGER)
43 s1="ab", s2="eidbaooo"
44 - Init Window "ei": s1=[a:1, b:1], s2=[e:1, i:1]. Matches=24 (zeros match).
45 - Slide 1 "id": Remove 'e', Add 'd'. Matches still 24.
46 - Slide 2 "db": Remove 'i', Add 'b'. 'b' count becomes 1 (match!). Matches=25.
47 - Slide 3 "ba": Remove 'd', Add 'a'. 'a' count becomes 1 (match!). Matches=26.
48 - Return True.
49 """

```

3.5 Minimum Size Subarray Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Are numbers positive?
4     A: Yes. (Crucial: Allows simple window shrinking).
5   - Q: What if no subarray exists?
6     A: Return 0.
7   - Q: Constraints?
8     A: N=10^5. O(N) required.
9
10 2. APPROACHES TO CONSIDER & REJECT
11   - Approach 1: Nested Loops
12     - Complexity: O(N^2).
13     - Reject: TLE.
14   - Approach 2: Sliding Window (Dynamic)
15     - Logic: Expand 'r' until sum >= target. Then shrink 'l' as much as possible
16       while maintaining sum >= target.
17     - Complexity: O(N).
18     - Accept: Optimal.
19
20 3. INDENTED ENGLISH (LOGIC)
21 function minSubArrayLen(target, nums):
22     l, total = 0, 0
23     res = infinity
24     for r in nums:
25         total += nums[r]
26         while total >= target:
27             res = min(res, r - l + 1)
28             total -= nums[l]
29             l++
30     return 0 if infinity else res
31 """
```

The Magic Question (Get Buy-In)

"I'll expand the window to find a valid sum, then shrink from the left to minimize length. This runs in $O(N)$ time and $O(1)$ space. Should I code this?"

```
1 def minSubArrayLen(target: int, nums: List[int]) -> int:
2     l, total = 0, 0
3     res = float("inf")
4
5     for r in range(len(nums)):
6         # 1. Expand
7         total += nums[r]
8
9         # 2. Shrink (While Valid)
10        while total >= target:
11            res = min(res, r - l + 1)
12            total -= nums[l]
13            l += 1
14
15    return 0 if res == float("inf") else res
16
17 """
18 5. DRY RUN (HUMAN DEBUGGER)
19 target=7, nums=[2,3,1,2,4,3]
20 - r=0..3 sum=8 ([2,3,1,2]). Valid. res=4.
21 - Shrink l=0(2) -> sum=6. Invalid. Window [3,1,2].
22 - r=4(4). sum=10 ([3,1,2,4]). Valid. res=min(4,4)=4.
23 - Shrink l=1(3) -> sum=7. Valid. res=3. Window [1,2,4].
24 - Shrink l=2(1) -> sum=6. Invalid.
25 - r=5(3). sum=9. Valid... eventually res=2 ([4,3]).
26 """
```

3.6 Minimum Window Substring

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Does T have duplicates?
4     A: Yes ("AA" needs 2 A's).
5   - Q: Return empty string if not found?
6     A: Yes.
7   - Q: Is  $O(N)$  required?
8     A: Yes, brute force  $O(N^3)$  is unacceptable.
9
10 2. APPROACHES TO CONSIDER & REJECT
11   - Approach 1: Brute Force
12     - Reject:  $O(N^3)$ .
13   - Approach 2: Sliding Window with 'Have/Need' counters
14     - Logic: Avoid comparing full hash maps ( $O(26)$  or  $O(52)$ ) every iteration.
15       Use integer 'have' variable to track how many unique chars strictly meet the
16       requirement.
17     - Complexity:  $O(N)$ .
18     - Accept: Optimal.
19
20 3. INDENTED ENGLISH (LOGIC)
21   function minWindow(s, t):
22       countT = map(t)
23       window = {}
24       have = 0, need = len(countT)
25       res = [-1, -1], resLen = infinity
26       l = 0
27
28       for r in range(len(s)):
29           add s[r] to window
30           if s[r] meets requirement: have++
31
32           while have == need:
33               update res if smaller
34               remove s[l] from window
35               if s[l] no longer meets req: have--
36               l++
37       return s[res]
```

The Magic Question (Get Buy-In)

"I'll use 'have' and 'need' variables to track validity in $O(1)$ inside the sliding window loop, avoiding expensive map comparisons. Total time $O(N)$. Should I code this?"

```
1 def minWindow(s: str, t: str) -> str:
2     if t == "": return ""
3
4     countT, window = {}, {}
5     for c in t:
6         countT[c] = 1 + countT.get(c, 0)
7
8     have, need = 0, len(countT)
9     res, resLen = [-1, -1], float("inf")
10    l = 0
11
12    for r in range(len(s)):
13        c = s[r]
14        window[c] = 1 + window.get(c, 0)
15
16        # Only increment 'have' if count matches EXACTLY the requirement
17        if c in countT and window[c] == countT[c]:
18            have += 1
19
20        # Shrink while valid
21        while have == need:
22            # Update result
23            if (r - l + 1) < resLen:
24                res = [l, r]
```

```

25         resLen = (r - l + 1)
26
27         # Pop from left
28         window[s[l]] -= 1
29         if s[l] in countT and window[s[l]] < countT[s[l]]:
30             have -= 1
31         l += 1
32
33     l, r = res
34     return s[l : r + 1] if resLen != float("inf") else ""
35
36 """
37 5. DRY RUN (HUMAN DEBUGGER)
38 s = "ADOBECODEBANC", t = "ABC"
39 - need=3 (A, B, C)
40 - Expand until r=5 ("ADOBEC"):
41     - window has A:1, B:1, C:1. have=3. Valid! res="ADOBEC" (6).
42     - Shrink l=0 ('A'). window A=0. have=2. Stop shrinking.
43 - Continue...
44 - r=10 ("...CODEBA"): Valid. Shrink 'C', 'O', 'D', 'E'.
45 - Window becomes "BANC" (4). res="BANC".
46 """

```

3.7 Sliding Window Maximum

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Is k <= len(nums)?
4   A: Yes.
5 - Q: Return list of maxes?
6   A: Yes.
7 - Q: Can I use Heap?
8   A: O(N log k) is okay, but O(N) with Deque is "Strong Hire".
9
10 2. APPROACHES TO CONSIDER & REJECT
11 - Approach 1: Brute Force. O(N*k). Reject.
12 - Approach 2: Max Heap. O(N log k). Good.
13 - Approach 3: Monotonic Decreasing Deque.
14   - Logic: Store INDICES. Keep values in deque decreasing.
15     The front of the deque is always the Max of current window.
16   - Complexity: O(N) Time (Each element added/popped once).
17   - Accept: Optimal.
18
19 3. INDENTED ENGLISH (LOGIC)
20 function maxSlidingWindow(nums, k):
21     q = deque() # indices
22     res = []
23     for r in range(len(nums)):
24         while q and nums[q.back] < nums[r]:
25             q.pop()
26         q.append(r)
27
28         if l > q.front:
29             q.popleft()
30
31         if window size >= k:
32             add nums[q.front] to res
33     return res
34 """
```

The Magic Question (Get Buy-In)

"I will use a Monotonic Decreasing Deque to store indices. This ensures the maximum is always at the front for $O(1)$ access. Total Time is $O(N)$. Should I code this?"

```
1 from collections import deque
2
3 def maxSlidingWindow(nums: List[int], k: int) -> List[int]:
4     output = []
5     q = deque() # Stores indices
6     l = 0
7
8     for r in range(len(nums)):
9         # 1. Maintain Monotonic Decreasing Order
10        # Pop smaller values from back (They are useless now)
11        while q and nums[q[-1]] < nums[r]:
12            q.pop()
13        q.append(r)
14
15        # 2. Remove left val from window if it's the max
16        # If the index at the front is outside the window [l, r], remove it
17        if l > q[0]:
18            q.popleft()
19
20        # 3. Append max to output
21        # Only start adding to result when window is at least size k
22        if (r + 1) >= k:
23            output.append(nums[q[0]])
24            l += 1
25
26    return output
27
28 """
```

```

29 5. DRY RUN (HUMAN DEBUGGER)
30     nums=[1,3,-1,-3,5,3,6,7], k=3
31     - r=0(1): q=[0]
32     - r=1(3): 3>1. Pop 0. q=[1].
33     - r=2(-1): -1<3. q=[1, 2]. Window [1,3,-1]. Max=nums[q[0]]=3.
34     - r=3(-3): q=[1, 2, 3]. Window [3,-1,-3]. Max=3.
35     - r=4(5): 5>-3 pop, 5>-1 pop, 5>3 pop. q=[4]. Window [-1,-3,5]. Max=5.
36     - Return [3, 3, 5, 5, 6, 7]
37 """

```

4 Two Pointers: String Array Manipulation

4.1 Reverse String

```

1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Can I allocate new space? A: No, modify in-place O(1) space.
4     - Q: Input is list of chars or string? A: List of characters (mutable).
5
6 2. APPROACHES
7     - Approach 1: Stack. Push all, pop all. O(N) Space. Reject.
8     - Approach 2: Two Pointers. Swap l and r. Move inward.
9         - Complexity: O(N) Time, O(1) Space.
10        - Accept: Optimal.
11
12 3. INDENTED ENGLISH
13     function reverseString(s):
14         l, r = 0, len(s)-1
15         while l < r:
16             swap s[l] and s[r]
17             l++, r--
18 """
19
20 # --- THE MAGIC QUESTION ---
21 # "I will use two pointers starting at both ends and swapping elements until they meet.
22 # This modifies the array in-place with O(1) space. Should I code this?"
23
24 def reverseString(s: List[str]) -> None:
25     l, r = 0, len(s) - 1
26     while l < r:
27         s[l], s[r] = s[r], s[l]
28         l += 1
29         r -= 1
30
31 """
32 5. DRY RUN
33     s = ['h','e','l','l','o']
34     - l=0('h'), r=4('o') -> Swap -> ['o','e','l','l','h']
35     - l=1('e'), r=3('l') -> Swap -> ['o','l','l','e','h']
36     - l=2('l'), r=2('l') -> l is not < r. Stop.
37     - Result: ['o','l','l','e','h']
38 """

```

4.2 Valid Palindrome

```

1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Case sensitive? A: No.
4     - Q: Non-alphanumeric characters? A: Skip them.
5     - Q: Empty string? A: Valid.
6
7 2. APPROACHES
8     - Approach 1: Filter + Reverse. new_s = filter(s); return new_s == new_s[::-1].
9         - Complexity: O(N) Time, O(N) Space.
10        - Reject: Interviewer usually wants O(1) space.
11     - Approach 2: Two Pointers.
12         - Logic: Move inward. If char not alphanumeric, skip. Else compare.
13         - Complexity: O(N) Time, O(1) Space.
14         - Accept: Optimal.

```

```

15
16 3. INDENTED ENGLISH
17 function isPalindrome(s):
18     l, r = 0, len(s)-1
19     while l < r:
20         if s[l] not alnum: l++; continue
21         if s[r] not alnum: r--; continue
22         if lower(s[l]) != lower(s[r]): return False
23         l++, r--
24     return True
25 """
26
27 # --- THE MAGIC QUESTION ---
28 # "I'll use two pointers moving inward, skipping non-alphanumeric characters
29 # and comparing case-insensitively. This avoids allocating new strings, keeping it O(1)
30 # space."
31
32 def isPalindrome(s: str) -> bool:
33     l, r = 0, len(s) - 1
34
35     while l < r:
36         # 1. Skip non-alphanumeric from left
37         while l < r and not s[l].isalnum():
38             l += 1
39
40         # 2. Skip non-alphanumeric from right
41         while r > l and not s[r].isalnum():
42             r -= 1
43
44         # 3. Compare characters
45         if s[l].lower() != s[r].lower():
46             return False
47
48         l, r = l + 1, r - 1
49
50     return True
51 """
52 5. DRY RUN
53 s = "A man, a plan..."
54 - l=0('A'), r=end('.') -> r skips '.' -> r='n'
55 - Compare 'a' == 'n'? No wait, "A man, a plan, a canal: Panama"
56 - l='A', r='a' (last 'a' of Panama). Match.
57 - ... eventual middle ...
58 - s = "OP"
59 - l='O', r='P'. Match? No. Return False.
60 """

```

4.3 Valid Palindrome II

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: How many deletions allowed? A: At most one.
4 - Q: What if it's already a palindrome? A: Return True.
5
6 2. APPROACHES
7 - Approach 1: Brute Force. Try deleting every char. O(N^2). Reject.
8 - Approach 2: Greedy Two Pointers.
9   - Logic: Match until mismatch. If mismatch at (l, r), try skipping 'l' OR skipping 'r'.
10  - If either resulting substring is a palindrome, return True.
11  - Complexity: O(N) Time, O(1) Space.
12  - Accept: Optimal.
13
14 3. INDENTED ENGLISH
15 function validPalindrome(s):
16     l, r = 0, len(s)-1
17     while l < r:
18         if s[l] != s[r]:
19             # Try deleting left OR deleting right
20             return isPal(s[l+1...r]) or isPal(s[l...r-1])
21         l++, r--

```

```

22         return True
23     """
24
25     # --- THE MAGIC QUESTION ---
26     # "I'll iterate with two pointers. Upon the first mismatch, I will check if the
27     # substring
28     # formed by skipping the left character OR the right character is a valid palindrome. O(
29     # N) Time."
30
31     def validPalindrome(s: str) -> bool:
32         # Helper to check strict palindrome
33         def isPal(l, r):
34             while l < r:
35                 if s[l] != s[r]: return False
36                 l, r = l + 1, r - 1
37             return True
38
39         l, r = 0, len(s) - 1
40         while l < r:
41             if s[l] != s[r]:
42                 # Mismatch found: try skipping L or skipping R
43                 return isPal(l + 1, r) or isPal(l, r - 1)
44             l += 1
45             r -= 1
46
47         return True
48
49     """
50     5. DRY RUN
51     s = "abca"
52     - l=0('a'), r=3('a'). Match.
53     - l=1('b'), r=2('c'). Mismatch!
54     - Try Skip L (isPal(2, 2)): s[2] 'c'. Single char is True.
55     - Return True.
56
57     s = "abc"
58     - l=0('a'), r=2('c'). Mismatch.
59     - Try Skip L (isPal(1, 2)): "bc". 'b'!='c'. False.
60     - Try Skip R (isPal(0, 1)): "ab". 'a'!='b'. False.
61     - Return False.
62     """

```

4.4 Merge Strings Alternately

```

1     """
2     1. CLARIFYING QUESTIONS
3     - Q: Strings different lengths? A: Append the remainder of the longer string.
4     - Q: Empty strings? A: Handle gracefully.
5
6     2. APPROACHES
7     - Approach 1: Iterative Pointers.
8     - Logic: Loop while i < len(word1) or i < len(word2). Append if index valid.
9     - Complexity: O(N + M).
10    - Accept: Optimal.
11
12    3. INDENTED ENGLISH
13    function mergeAlternately(word1, word2):
14        i = 0
15        res = []
16        while i < len1 or i < len2:
17            if i < len1: append word1[i]
18            if i < len2: append word2[i]
19            i++
20        return join(res)
21    """
22
23    # --- THE MAGIC QUESTION ---
24    # "I'll iterate through both strings with a single index, appending characters from each
25    # if the index is within bounds. This handles different lengths automatically. O(N+M)
26    # Time."
27
28    def mergeAlternately(word1: str, word2: str) -> str:

```

```

28     i = 0
29     res = []
30
31     # Iterate as long as one string has characters
32     while i < len(word1) or i < len(word2):
33         if i < len(word1):
34             res.append(word1[i])
35         if i < len(word2):
36             res.append(word2[i])
37         i += 1
38
39     return "".join(res)
40
41 """
42 5. DRY RUN
43 word1 = "ab", word2 = "pqrs"
44 - i=0: App 'a', App 'p'. res=['a','p']
45 - i=1: App 'b', App 'q'. res=['a','p','b','q']
46 - i=2: 'ab' done. App 'r'. res=..., 'r']
47 - i=3: 'ab' done. App 's'. res=..., 'r','s']
48 - Return "apbqrs".
49 """

```

4.5 Merge Sorted Array

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Is nums1 large enough? A: Yes, size is m + n.
4 - Q: In-place? A: Yes, do not return anything.
5 - Q: Order? A: Non-decreasing.
6
7 2. APPROACHES
8 - Approach 1: Merge and Sort. nums1[:] = sorted(nums1[:m] + nums2).
9   - Complexity: O((N+M) log(N+M)). Reject (naive).
10 - Approach 2: Three Pointers (Start from End).
11   - Logic: Since nums1 has empty space at the END, fill from right to left to avoid
12     overwriting.
13   - Complexity: O(N + M) Time, O(1) Space.
14   - Accept: Optimal.
15
16 3. INDENTED ENGLISH
17 function merge(nums1, m, nums2, n):
18     last = m + n - 1
19     while m > 0 and n > 0:
20         if nums1[m-1] > nums2[n-1]:
21             nums1[last] = nums1[m-1]
22             m--
23         else:
24             nums1[last] = nums2[n-1]
25             n--
26         last--
27     # Fill remaining nums2 if any
28     while n > 0:
29         nums1[last] = nums2[n-1]
30         n--, last--
31
32 # --- THE MAGIC QUESTION ---
33 # "I will fill nums1 starting from the back (index m+n-1) to avoid overwriting elements.
34 # I'll compare the tails of both arrays and place the larger element. O(N+M) Time."
35
36 def merge(nums1: List[int], m: int, nums2: List[int], n: int) -> None:
37     # last index in nums1
38     last = m + n - 1
39
40     # Merge in reverse order
41     while m > 0 and n > 0:
42         if nums1[m - 1] > nums2[n - 1]:
43             nums1[last] = nums1[m - 1]
44             m -= 1
45         else:
46             nums1[last] = nums2[n - 1]

```

```

47         n -= 1
48         last -= 1
49
50     # Edge Case: If nums2 is not empty yet (nums1 elements exhausted)
51     while n > 0:
52         nums1[last] = nums2[n - 1]
53         n, last = n - 1, last - 1
54
55     """
56 5. DRY RUN
57     nums1=[1,2,3,0,0,0], m=3, nums2=[2,5,6], n=3
58     - last=5. Compare 3(n1) vs 6(n2). Set n1[5]=6. n=2.
59     - last=4. Compare 3(n1) vs 5(n2). Set n1[4]=5. n=1.
60     - last=3. Compare 3(n1) vs 2(n2). Set n1[3]=3. m=2.
61     - last=2. Compare 2(n1) vs 2(n2). Set n1[2]=2. n=0.
62     - Loop ends. nums1=[1,2,2,3,5,6].
63     """

```

4.6 Remove Duplicates from Sorted Array

```

1 """
2 1. CLARIFYING QUESTIONS
3     - Q: In-place? A: Yes.
4     - Q: Return value? A: The new length k.
5     - Q: Remaining elements? A: Don't matter.
6
7 2. APPROACHES
8     - Approach 1: Use Set. O(N) Space. Reject.
9     - Approach 2: Two Pointers (Left = Insert Pos, Right = Scanner).
10        - Logic: If nums[r] != nums[r-1], we found a new unique. Place it at nums[l].
11        - Complexity: O(N) Time, O(1) Space.
12        - Accept: Optimal.
13
14 3. INDENTED ENGLISH
15     function removeDuplicates(nums):
16         l = 1
17         for r from 1 to len(nums):
18             if nums[r] != nums[r-1]:
19                 nums[l] = nums[r]
20                 l++
21         return l
22 """
23
24 # --- THE MAGIC QUESTION ---
25 # "I'll use a left pointer to track the position for the next unique element
26 # and a right pointer to scan. When a new unique value is found, I'll move it to the
27 # left pointer. O(N)."
28
29 def removeDuplicates(nums: List[int]) -> int:
30     if not nums: return 0
31
32     l = 1 # Points to where the next unique value goes
33
34     for r in range(1, len(nums)):
35         # Found a new unique value?
36         if nums[r] != nums[r - 1]:
37             nums[l] = nums[r]
38             l += 1
39
40     return l
41
42 """
43 5. DRY RUN
44     nums = [1, 1, 2]
45     - l=1, r=1. nums[1]==nums[0]. Duplicate. Ignore.
46     - l=1, r=2. nums[2](2) != nums[1](1). Unique.
47       - nums[l] = 2. -> nums=[1, 2, 2]
48       - l becomes 2.
49     - Return 2. (First 2 elements are [1, 2]).
50 """

```

4.7 Two Sum II - Input Array Is Sorted

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: 1-indexed? A: Yes.
4     - Q: Solution exists? A: Exactly one.
5     - Q: Can I reuse elements? A: No.
6
7 2. APPROACHES
8     - Approach 1: Dictionary. O(N) Space. Reject (Problem usually asks for O(1) space).
9     - Approach 2: Binary Search. O(N log N). Reject.
10    - Approach 3: Two Pointers.
11        - Logic: Sum > Target? Decrease R. Sum < Target? Increase L.
12        - Complexity: O(N) Time, O(1) Space.
13        - Accept: Optimal.
14
15 3. INDENTED ENGLISH
16    function twoSum(numbers, target):
17        l, r = 0, len-1
18        while l < r:
19            s = numbers[l] + numbers[r]
20            if s > target: r--
21            else if s < target: l++
22            else: return [l+1, r+1]
23 """
24
25 # --- THE MAGIC QUESTION ---
26 # "Since it's sorted, I can shrink the search space from both ends.
27 # Too small? Move left. Too big? Move right. O(N) Time, O(1) Space."
28
29 def twoSum(numbers: List[int], target: int) -> List[int]:
30     l, r = 0, len(numbers) - 1
31
32     while l < r:
33         curSum = numbers[l] + numbers[r]
34
35         if curSum > target:
36             r -= 1
37         elif curSum < target:
38             l += 1
39         else:
40             # 1-indexed return
41             return [l + 1, r + 1]
42     return []
43
44 """
45 5. DRY RUN
46     nums = [2, 7, 11, 15], target = 9
47     - l=0(2), r=3(15). Sum=17 > 9. r--
48     - l=0(2), r=2(11). Sum=13 > 9. r--
49     - l=0(2), r=1(7). Sum=9 == 9. Match.
50     - Return [1, 2].
51 """
```

5 Two Pointers: Target Sums & Containers

5.1 3Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Unique triplets? A: Yes, no duplicate sets like [-1, 0, 1] and [0, -1, 1].
4     - Q: Can I modify input? A: Yes, sorting is recommended.
5     - Q: Constraints? A: N=3000. O(N^2) is acceptable.
6
7 2. APPROACHES
8     - Approach 1: Brute Force O(N^3). Reject.
9     - Approach 2: Sort + Two Pointers.
10        - Logic: Sort first. Iterate 'i'. For each 'i', solve Two Sum II on the rest.
11        - Handling Duplicates:
12            1. Skip 'i' if same as previous.
```

```

13     2. Skip 'l' after finding a valid triplet if same as previous.
14     - Complexity:  $O(N^2)$  Time,  $O(1)$  Space (depending on sort implementation).
15     - Accept: Optimal.
16
17 3. INDENTED ENGLISH
18 function threeSum(nums):
19     sort(nums)
20     res = []
21     for i from 0 to len:
22         if i > 0 and nums[i] == nums[i-1]: continue
23         l, r = i+1, len-1
24         while l < r:
25             sum = nums[i] + nums[l] + nums[r]
26             if sum > 0: r--
27             else if sum < 0: l++
28             else:
29                 add triplet
30                 l++
31                 while l < r and nums[l] == nums[l-1]: l++
32     return res
33 """
34
35 # --- THE MAGIC QUESTION ---
36 # "I'll sort the array to handle duplicates efficiently. Then I'll iterate through the
37 # list
38 # and use the 'Two Sum II' pointer logic for the remaining elements.  $O(N^2)$  Time."
39 def threeSum(nums: List[int]) -> List[List[int]]:
40     res = []
41     nums.sort()
42
43     for i in range(len(nums)):
44         # 1. Skip Duplicate 'i'
45         if i > 0 and nums[i] == nums[i-1]:
46             continue
47
48         l, r = i + 1, len(nums) - 1
49         while l < r:
50             threeSum = nums[i] + nums[l] + nums[r]
51
52             if threeSum > 0:
53                 r -= 1
54             elif threeSum < 0:
55                 l += 1
56             else:
57                 res.append([nums[i], nums[l], nums[r]])
58                 l += 1
59                 # 2. Skip Duplicate 'l' to avoid duplicate triplets
60                 while l < r and nums[l] == nums[l - 1]:
61                     l += 1
62
63     return res
64
65 """
66 5. DRY RUN
67 nums = [-1, 0, 1, 2, -1, -4] -> Sorted: [-4, -1, -1, 0, 1, 2]
68 - i=0 (-4): l=1(-1), r=5(2). sum=-3. l++.
69 - ... (no solution for -4)
70 - i=1 (-1):
71     - l=2(-1), r=5(2). Sum=0. Found [-1, -1, 2].
72     - l moves to 3(0).
73     - l=3(0), r=5(2). Sum=1. r--.
74 - i=2 (-1): Skip (duplicate).
75 """

```

5.2 4Sum

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Can k be general? A: No, specifically 4.
4 - Q: Constraints? A: N=200.  $O(N^3)$  is acceptable.
5 - Q: Overflow? A: Sum can exceed 32-bit integer (Python handles this automatically).

```

```

6
7 2. APPROACHES
8 - Approach 1: Generalized K-Sum (Recursion). Good for general K.
9 - Approach 2: Nested Loops + Two Pointers.
10 - Logic: Fix i, Fix j, then Two Pointers (l, r).
11 - Pruning: Break early if smallest sum > target.
12 - Complexity:  $O(N^3)$ .
13 - Accept: Standard for 4Sum.
14
15 3. INDENTED ENGLISH
16 function fourSum(nums, target):
17     sort(nums)
18     res = []
19     for i in range(N):
20         if duplicate i: continue
21         for j in range(i+1, N):
22             if duplicate j: continue
23             l, r = j+1, N-1
24             while l < r:
25                 sum = nums[i]+nums[j]+nums[l]+nums[r]
26                 if sum == target: add, move l, skip dupe l
27                 else if sum < target: l++
28                 else: r--
29     return res
30 """
31
32 # --- THE MAGIC QUESTION ---
33 # "I'll extend the 3Sum logic by adding one more nested loop. I will fix two numbers
34 # and run the two-pointer search on the rest. I'll carefully skip duplicates for both
35 # loops.  $O(N^3)$ ."
36
37 def fourSum(nums: List[int], target: int) -> List[List[int]]:
38     nums.sort()
39     res, quad = [], []
40
41     # Generic kSum helper is cleaner, but iterative is often expected for fixed k=4
42     def kSum(k, start, target):
43         if k != 2:
44             for i in range(start, len(nums) - k + 1):
45                 # Skip duplicate
46                 if i > start and nums[i] == nums[i - 1]:
47                     continue
48                 quad.append(nums[i])
49                 kSum(k - 1, i + 1, target - nums[i])
50                 quad.pop()
51         return
52
53     # Base Case: Two Sum
54     l, r = start, len(nums) - 1
55     while l < r:
56         if nums[l] + nums[r] < target:
57             l += 1
58         elif nums[l] + nums[r] > target:
59             r -= 1
60         else:
61             res.append(quad + [nums[l], nums[r]])
62             l += 1
63             while l < r and nums[l] == nums[l - 1]:
64                 l += 1
65
66     kSum(4, 0, target)
67     return res
68 """
69
70 5. DRY RUN
71 nums=[1,0,-1,0,-2,2], target=0. Sorted: [-2,-1,0,0,1,2]
72 - kSum(4): Fix -2. Need 2Sum(3) for target 2.
73   - Fix -1. Need 2Sum(2) for target 3.
74     - l=0, r=2. Sum=2 (Too small).
75 - ... Eventually finds [-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1].
76 """

```

5.3 Rotate Array

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: In-place? A: Yes. O(1) space.
4     - Q: k > len(nums)? A: Yes, handle with k % len.
5
6 2. APPROACHES
7     - Approach 1: Pop and Prepend. O(N*k). TLE.
8     - Approach 2: Extra Array. Copy elements. O(N) Space. Reject.
9     - Approach 3: Triple Reverse.
10        - Logic: 1. Reverse entire array. 2. Reverse first k. 3. Reverse rest.
11        - Complexity: O(N) Time, O(1) Space.
12        - Accept: Optimal "Trick".
13
14 3. INDENTED ENGLISH
15     function rotate(nums, k):
16         k = k % len(nums)
17         reverse(nums, 0, len-1)
18         reverse(nums, 0, k-1)
19         reverse(nums, k, len-1)
20 """
21
22 # --- THE MAGIC QUESTION ---
23 # "I'll use the reverse trick: reverse the whole array, then reverse the first k
24 # elements,
25 # and finally the remaining elements. This achieves O(N) time and O(1) space."
26
27 def rotate(nums: List[int], k: int) -> None:
28     k = k % len(nums)
29
30     # Helper to reverse a portion of array
31     def reverse(l, r):
32         while l < r:
33             nums[l], nums[r] = nums[r], nums[l]
34             l, r = l + 1, r - 1
35
36     # 1. Reverse All
37     reverse(0, len(nums) - 1)
38     # 2. Reverse First k
39     reverse(0, k - 1)
40     # 3. Reverse Rest
41     reverse(k, len(nums) - 1)
42 """
43 5. DRY RUN
44     nums=[1,2,3,4,5,6,7], k=3
45     - Rev All: [7,6,5,4,3,2,1]
46     - Rev 0..2: [5,6,7, 4,3,2,1]
47     - Rev 3..6: [5,6,7, 1,2,3,4] -> Correct.
48 """
```

5.4 Container With Most Water

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Slanted lines? A: No.
4     - Q: Can lines be 0 height? A: Yes.
5
6 2. APPROACHES
7     - Approach 1: Brute Force Pairs. O(N^2). Reject.
8     - Approach 2: Greedy Two Pointers.
9        - Logic: Start widest. Move the SHORTER line.
10        - Why? Area is limited by the short line. Moving the tall line inward can
11        only decrease width without potentially increasing height.
12        - Complexity: O(N).
13        - Accept: Optimal.
14
15 3. INDENTED ENGLISH
16     function maxArea(height):
17         l, r = 0, len-1
```

```

18     res = 0
19     while l < r:
20         area = (r-l) * min(h[l], h[r])
21         res = max(res, area)
22         if h[l] < h[r]: l++
23         else: r--
24     return res
25 """
26
27 # --- THE MAGIC QUESTION ---
28 # "I'll start with the widest possible container. At each step, I'll move the shorter
29 # pointer
30 # inward to try and find a taller line. This is O(N) time."
31 def maxArea(height: List[int]) -> int:
32     l, r = 0, len(height) - 1
33     res = 0
34
35     while l < r:
36         area = (r - l) * min(height[l], height[r])
37         res = max(res, area)
38
39         # Greedy move
40         if height[l] < height[r]:
41             l += 1
42         else:
43             r -= 1
44
45     return res
46
47 """
48 5. DRY RUN
49 h = [1,8,6,2,5,4,8,3,7]
50 - l=0(1), r=8(7). w=8, minH=1. Area=8. Move l (1<7).
51 - l=1(8), r=8(7). w=7, minH=7. Area=49. Max=49. Move r (7<8).
52 - ...
53 """

```

5.5 Boats to Save People

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Max people per boat? A: Exactly 2.
4 - Q: Weight limit? A: Strictly <= limit.
5
6 2. APPROACHES
7 - Approach 1: Greedy Two Pointers.
8 - Logic: Sort people. Pair the Heaviest with the Lightest.
9 - If heavier + lighter <= limit, they share a boat.
10 - If not, heavier goes alone (lighter waits for next heavy).
11 - Complexity: O(N log N) for sort.
12 - Accept: Optimal.
13
14 3. INDENTED ENGLISH
15 function numRescueBoats(people, limit):
16     sort(people)
17     l, r = 0, len-1, boats = 0
18     while l <= r:
19         if people[l] + people[r] <= limit:
20             l++ # Light person gets on board
21             r-- # Heavy person always gets on board
22             boats++
23     return boats
24 """
25
26 # --- THE MAGIC QUESTION ---
27 # "I'll sort the people by weight. Then, I'll try to pair the heaviest person with the
28 # lightest person. If they fit, they go together; otherwise, the heaviest goes alone. O(
29 # N log N)."
30
31 def numRescueBoats(people: List[int], limit: int) -> int:
32     people.sort()

```

```

32     l, r = 0, len(people) - 1
33     boats = 0
34
35     while l <= r:
36         remain = limit - people[r]
37         r -= 1 # Heavy person is always on the boat
38         boats += 1
39
40         # Can the light person fit too?
41         if l <= r and people[l] <= remain:
42             l += 1
43
44     return boats
45
46 """
47 5. DRY RUN
48 people=[3,2,2,1], limit=3. Sorted: [1,2,2,3]
49 - l=0(1), r=3(3). Sum=4 > 3. Heavy(3) goes alone. boats=1. r=2.
50 - l=0(1), r=2(2). Sum=3 <= 3. Pair (1,2) go. boats=2. l=1, r=1.
51 - l=1(2), r=1(2). Same person. Condition l<=r holds.
52 - Heavy(2) goes alone. boats=3. l=1, r=0. Loop end.
53 """

```

5.6 Trapping Rain Water

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Can I use O(N) space? A: O(1) is preferred for "Strong Hire".
4 - Q: Negative heights? A: No.
5
6 2. APPROACHES
7 - Approach 1: Dynamic Programming (Precompute MaxLeft and MaxRight). O(N) Space.
8 - Approach 2: Two Pointers (Space Optimized).
9   - Logic: Maintain maxLeft and maxRight. Process the side with the SMALLER max.
10  - Why? The water level at 'i' is limited by min(maxL, maxR). If maxL < maxR,
11    we know the limit is maxL, regardless of what's between i and R.
12  - Complexity: O(N) Time, O(1) Space.
13  - Accept: Optimal.
14
15 3. INDENTED ENGLISH
16 function trap(height):
17     l, r = 0, len-1
18     maxL, maxR = height[l], height[r]
19     res = 0
20     while l < r:
21         if maxL < maxR:
22             l++
23             maxL = max(maxL, height[l])
24             res += maxL - height[l]
25         else:
26             r--
27             maxR = max(maxR, height[r])
28             res += maxR - height[r]
29     return res
30 """
31
32 # --- THE MAGIC QUESTION ---
33 # "I'll use two pointers. I'll process the side with the smaller maximum height because
34 # the water level is always limited by the shorter wall. This allows O(N) time and O(1)
35 # space."
36
37 def trap(height: List[int]) -> int:
38     if not height: return 0
39
40     l, r = 0, len(height) - 1
41     maxL, maxR = height[l], height[r]
42     res = 0
43
44     while l < r:
45         if maxL < maxR:
46             l += 1
47             # Water level is limited by maxL.

```

```

47         # If current height < maxL, we trap water.
48         # If current height > maxL, we update maxL (and trap 0).
49         maxL = max(maxL, height[l])
50         res += maxL - height[l]
51     else:
52         r -= 1
53         maxR = max(maxR, height[r])
54         res += maxR - height[r]
55     return res
56
57
58 """
59 5. DRY RUN
60 height = [0,1,0,2,1,0,1,3,2,1,2,1]
61 - l=0(0), r=11(1). maxL=0, maxR=1. maxL < maxR.
62   - Move l to 1. h=1. maxL=1. res+=0.
63 - l=1(1), r=11(1). maxL=1, maxR=1. Equal (else branch).
64   - Move r to 10. h=2. maxR=2. res+=0.
65 - l=1(1), r=10(2). maxL=1 < maxR=2.
66   - Move l to 2. h=0. maxL=1. res += (1-0) = 1.
67 - ...
68 """

```

6 Stack: Essentials & Simulation

6.1 Baseball Game

```

1  """
2  1. CLARIFYING QUESTIONS & EXPECTED ANSWERS
3      - Q: Are operations always valid?
4        A: Yes, e.g., 'C' only appears if there is a score to invalidate.
5      - Q: Negative scores allowed?
6        A: Yes.
7      - Q: Result constraint?
8        A: Fits in 32-bit integer.
9
10 2. APPROACHES TO CONSIDER & REJECT
11     - Approach 1: Simulation with List/Stack
12       - Logic: Iterate through ops. Append numbers. 'C' pops. 'D' doubles last. '+' adds
13         last two.
14       - Complexity: O(N) Time, O(N) Space.
15       - Accept: Optimal.
16
17 3. INDENTED ENGLISH (LOGIC)
18 function calPoints(operations):
19     stack = []
20     for op in operations:
21         if op == '+':
22             stack.append(stack[-1] + stack[-2])
23         elif op == 'D':
24             stack.append(2 * stack[-1])
25         elif op == 'C':
26             stack.pop()
27         else:
28             stack.append(int(op))
29     return sum(stack)
30 """

```

The Magic Question (Get Buy-In)

"I will simulate the record keeping using a stack to handle the 'invalidate' (pop) and 'previous score' (peek) operations efficiently in $O(N)$. Should I code this?"

```

1 def calPoints(operations: List[str]) -> int:
2     stack = []
3
4     for op in operations:
5         if op == "+":

```

```

6         # Add sum of last two scores
7         stack.append(stack[-1] + stack[-2])
8     elif op == "D":
9         # Double the last score
10        stack.append(2 * stack[-1])
11    elif op == "C":
12        # Invalidate previous score
13        stack.pop()
14    else:
15        # Record a new score
16        stack.append(int(op))
17
18    return sum(stack)
19
20 """
21 5. DRY RUN (HUMAN DEBUGGER)
22 ops = ["5","2","C","D","+"]
23 - "5": stack=[5]
24 - "2": stack=[5, 2]
25 - "C": Pop 2. stack=[5]
26 - "D": 5*2=10. stack=[5, 10]
27 - "+": 5+10=15. stack=[5, 10, 15]
28 - Sum: 30.
29 """

```

6.2 Valid Parentheses

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Empty string valid?
4     A: Yes.
5   - Q: Only ()[]{} chars?
6     A: Yes.
7   - Q: Order matters?
8     A: Yes, "([)]" is invalid. Must close in reverse order of opening (LIFO).
9
10 2. APPROACHES TO CONSIDER & REJECT
11   - Approach 1: Counter?
12     A: No, counters can't handle order (e.g., "([)]").
13   - Approach 2: Stack
14     Logic: Push open brackets. When closing comes, check if matches stack top.
15     Complexity: O(N) Time, O(N) Space.
16     Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19   function isValid(s):
20       map = {'(': ')', '[': ']', '{': '}'
21       stack = []
22       for c in s:
23           if c in map: # is closing
24               if stack empty or stack.pop() != map[c]:
25                   return False
26           else: # is opening
27               stack.append(c)
28       return stack is empty
29 """
```

The Magic Question (Get Buy-In)

"I'll use a stack to track open brackets. A hash map will verify that each closing bracket matches the most recently opened one. This is $O(N)$. Agreed?"

```
1 def isValid(s: str) -> bool:
2     # Map closing to opening for easy lookup
3     closeToOpen = {")": "(", "]": "[", "}": "{"}
4     stack = []
5
6     for c in s:
7         # 1. Is it a Closing Bracket?
8         if c in closeToOpen:
9             # Check if stack is not empty AND top matches
10            if stack and stack[-1] == closeToOpen[c]:
11                stack.pop()
12            else:
13                return False
14            # 2. It's an Opening Bracket
15            else:
16                stack.append(c)
17
18            # Valid only if all opened brackets were closed
19            return True if not stack else False
20
21 """
22 5. DRY RUN (HUMAN DEBUGGER)
23   s = "([)]"
24   - '(': push. stack=['(']
25   - '[': push. stack=['(', '[']
26   - ')': is closing. map[')']='('. Top is '['. Mismatch!
27   - Return False.
28
29   s = "()[]{}"
30   - '(': push. ')': matches '('. pop. stack=[]
31   - '[': push. ']': matches '['. pop. stack=[]
32   - Return True (stack empty).
33 """
```

6.3 Implement Stack Using Queues

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Use standard library Deque?
4   A: Yes, but only use push_back (append) and pop_front (popleft).
5   - Q: One queue or two?
6   A: One queue is optimal for space.
7
8 2. APPROACHES TO CONSIDER & REJECT
9   - Approach 1: Two Queues.
10    Logic: Move all elements to q2, add x, move back. Expensive.
11   - Approach 2: One Queue (Rotate).
12    Logic: To push 'x' and keep LIFO order:
13        1. Append 'x'.
14        2. Rotate all previous elements (popleft and append) behind 'x'.
15    Complexity: Push  $O(N)$ , Pop  $O(1)$ .
16    Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19 class MyStack:
20     q = deque()
21
22     push(x):
23         q.append(x)
24         for i in range(len(q) - 1):
25             q.append(q.popleft())
26
27     pop():
28         return q.popleft()
29 """
```

The Magic Question (Get Buy-In)

"I can implement this using a single queue. When pushing a new element, I'll rotate the existing elements behind it to maintain LIFO order at the front. Push is $O(N)$, Pop is $O(1)$. Should I code this?"

```
1 from collections import deque
2
3 class MyStack:
4
5     def __init__(self):
6         self.q = deque()
7
8     def push(self, x: int) -> None:
9         self.q.append(x)
10        # Rotate all previous elements to the back
11        # This makes the new 'x' the front (Top of Stack)
12        for _ in range(len(self.q) - 1):
13            self.q.append(self.q.popleft())
14
15    def pop(self) -> int:
16        return self.q.popleft() #  $O(1)$ 
17
18    def top(self) -> int:
19        return self.q[0] #  $O(1)$ 
20
21    def empty(self) -> bool:
22        return len(self.q) == 0
23
24 """
25 5. DRY RUN (HUMAN DEBUGGER)
26 push(1): q=[1]
27 push(2):
28     - append(2) -> q=[1, 2]
29     - loop range(1):
30     - popleft(1), append(1) -> q=[2, 1]
31 push(3):
32     - append(3) -> q=[2, 1, 3]
```

```
33     - loop range(2):
34         - popleft(2), append(2) -> q=[1, 3, 2]
35         - popleft(1), append(1) -> q=[3, 2, 1] (Correct Stack Order!)
36     pop(): returns 3.
37 """
```

6.4 Implement Queue Using Stacks

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Amortized O(1)?
4   A: Yes, pop/peek should be amortized O(1).
5 - Q: Two stacks allowed?
6   A: Yes.
7
8 2. APPROACHES TO CONSIDER & REJECT
9 - Approach 1: Transfer on Push. O(N) push.
10 - Approach 2: Transfer on Pop (Lazy).
11   Logic: Use 'input' stack for pushes. Use 'output' stack for pops.
12   Only move elements from input->output when output is empty.
13   Complexity: Amortized O(1).
14   Accept: Optimal.
15
16 3. INDENTED ENGLISH (LOGIC)
17 class MyQueue:
18     s1 (in), s2 (out)
19
20     push(x): s1.append(x)
21
22     pop():
23         if not s2:
24             while s1: s2.append(s1.pop())
25         return s2.pop()
26 """
```

The Magic Question (Get Buy-In)

"I'll use two stacks: 's1' for input and 's2' for output. I'll only transfer elements from 's1' to 's2' when 's2' is empty. This ensures 'Amortized O(1)' time for operations. Should I code this?"

```
1 class MyQueue:
2
3     def __init__(self):
4         self.s1 = [] # Input stack
5         self.s2 = [] # Output stack
6
7     def push(self, x: int) -> None:
8         self.s1.append(x)
9
10    def pop(self) -> int:
11        self.peek() # Ensure s2 has elements
12        return self.s2.pop()
13
14    def peek(self) -> int:
15        # Only transfer if output stack is empty
16        if not self.s2:
17            while self.s1:
18                self.s2.append(self.s1.pop())
19        return self.s2[-1]
20
21    def empty(self) -> bool:
22        return not self.s1 and not self.s2
23
24 """
25 5. DRY RUN (HUMAN DEBUGGER)
26 push(1), push(2): s1=[1, 2], s2=[]
27 peek():
28     - s2 empty. Transfer 2, then 1. s1=[], s2=[2, 1].
29     - Returns 1 (Front).
30 pop():
31     - s2 not empty. Pops 1. s2=[2].
32 push(3): s1=[3], s2=[2]. (Order preserved).
33 """
```

6.5 Min Stack

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: O(1) time for all ops?
4   A: Yes, including getMin().
5 - Q: Space constraints?
6   A: O(N) is standard.
7
8 2. APPROACHES TO CONSIDER & REJECT
9 - Approach 1: Calculate Min on the fly.
10  Reject: O(N) time.
11 - Approach 2: Value/Min Pair.
12  Logic: Each element in stack is (val, min_so_far).
13  Complexity: O(1) Time, O(N) Space.
14  Accept: Optimal.
15
16 3. INDENTED ENGLISH (LOGIC)
17 class MinStack:
18     stack = []
19     minStack = [] # Stores min values corresponding to main stack
20
21     push(x):
22         stack.append(x)
23         val = min(x, minStack[-1])
24         minStack.append(val)
25
26     pop(): stack.pop(), minStack.pop()
27     getMin(): return minStack[-1]
28 """
```

The Magic Question (Get Buy-In)

"I'll maintain a secondary 'minStack' in parallel. Every time I push a value, I'll also push the minimum seen so far onto the 'minStack'. This gives $O(1)$ access to the minimum. Agreed?"

```
1 class MinStack:
2
3     def __init__(self):
4         self.stack = []
5         self.minStack = []
6
7     def push(self, val: int) -> None:
8         self.stack.append(val)
9         # What is the new minimum?
10        # Either the new val OR the previous minimum
11        val = min(val, self.minStack[-1] if self.minStack else val)
12        self.minStack.append(val)
13
14    def pop(self) -> None:
15        self.stack.pop()
16        self.minStack.pop()
17
18    def top(self) -> int:
19        return self.stack[-1]
20
21    def getMin(self) -> int:
22        return self.minStack[-1]
23
24 """
25 5. DRY RUN (HUMAN DEBUGGER)
26 push(-2): stack=[-2], minStack=[-2]
27 push(0):  stack=[-2, 0], minStack=[-2, -2] (min of 0 and -2 is -2)
28 push(-3): stack=[-2, 0, -3], minStack=[-2, -2, -3]
29 getMin(): returns -3.
30 pop():    stack=[-2, 0], minStack=[-2, -2].
31 getMin(): returns -2.
32 """
```

6.6 Evaluate Reverse Polish Notation

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Division behavior?
4     A: Truncate toward zero (int conversion).
5   - Q: Valid input?
6     A: Yes, always valid RPN.
7
8 2. APPROACHES TO CONSIDER & REJECT
9   - Approach 1: Stack.
10    Logic: If number, push. If operator, pop two, operate, push result.
11    Complexity: O(N) Time.
12    Accept: Optimal.
13
14 3. INDENTED ENGLISH (LOGIC)
15 function evalRPN(tokens):
16     stack = []
17     for t in tokens:
18         if t is operator:
19             b, a = stack.pop(), stack.pop()
20             res = apply(a, b, t)
21             stack.append(res)
22         else:
23             stack.append(int(t))
24     return stack[0]
25 """
```

The Magic Question (Get Buy-In)

"I'll process tokens linearly. Operands go onto a stack. When I see an operator, I pop the last two operands, perform the calculation, and push the result back. This handles the order of operations naturally in $O(N)$."

```
1 def evalRPN(tokens: List[str]) -> int:
2     stack = []
3
4     for c in tokens:
5         if c == "+":
6             stack.append(stack.pop() + stack.pop())
7         elif c == "-":
8             a, b = stack.pop(), stack.pop()
9             stack.append(b - a) # Order matters: b - a
10        elif c == "*":
11            stack.append(stack.pop() * stack.pop())
12        elif c == "/":
13            a, b = stack.pop(), stack.pop()
14            # Python / is float division, // is floor (rounds down).
15            # We need truncate towards zero. int(b / a) handles this correctly.
16            stack.append(int(b / a))
17        else:
18            stack.append(int(c))
19
20    return stack[0]
21
22 """
23 5. DRY RUN (HUMAN DEBUGGER)
24 tokens = ["4","13","5","/","+"]
25 - push 4, push 13, push 5. stack=[4, 13, 5]
26 - "/": pop 5, pop 13. int(13/5) = 2. push 2. stack=[4, 2]
27 - "+": pop 2, pop 4. 4+2=6. push 6.
28 - Return 6.
29 """
```

6.7 Asteroid Collision

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Collision rules?
4   A: Right (+) hits Left (-). Smaller explodes. Equal both explode.
5 - Q: [-5, 5] collide?
6   A: No, moving away from each other.
7 - Q: [5, -5] collide?
8   A: Yes, moving towards each other.
9
10 2. APPROACHES TO CONSIDER & REJECT
11 - Approach 1: Simulation with Stack.
12   Logic: Iterate asteroids. If moving Right (+), push.
13   If moving Left (-), check stack for collisions.
14   Loop while stack top is (+) and < abs(current).
15   Complexity: O(N). Each asteroid pushed/popped once.
16   Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19 function asteroidCollision(asteroids):
20     stack = []
21     for a in asteroids:
22         while stack and a < 0 and stack[-1] > 0:
23             diff = a + stack[-1]
24             if diff < 0: # Stack top smaller, it explodes
25                 stack.pop()
26             elif diff > 0: # Current smaller, it explodes
27                 a = 0 # Mark current as destroyed
28             else: # Equal size
29                 a = 0
30                 stack.pop()
31         if a: stack.append(a)
32     return stack
33 """
```

The Magic Question (Get Buy-In)

"I'll use a stack to track stable asteroids. When a 'left-moving' asteroid appears, I'll check for collisions against the 'right-moving' asteroids on top of the stack. This simulates the collisions in $O(N)$ time. Should I code this?"

```
1 def asteroidCollision(asteroids: List[int]) -> List[int]:
2     stack = []
3
4     for a in asteroids:
5         # Collision Loop:
6         # Only happens if 'a' is Left (-) AND Stack Top is Right (+)
7         while stack and a < 0 and stack[-1] > 0:
8             diff = a + stack[-1]
9
10            if diff < 0:
11                # Stack top is smaller (e.g. 5 vs -10). Pop stack.
12                stack.pop()
13                # Continue loop to compare 'a' with next stack top
14            elif diff > 0:
15                # 'a' is smaller (e.g. 10 vs -5). 'a' explodes.
16                a = 0
17                break
18            else:
19                # Equal (e.g. 5 vs -5). Both explode.
20                a = 0
21                stack.pop()
22                break
23
24        # If 'a' survived (not 0), append it
25        if a:
26            stack.append(a)
27
28    return stack
```

```

29
30 """
31 5. DRY RUN (HUMAN DEBUGGER)
32   asteroids = [5, 10, -5]
33   - 5: push. stack=[5]
34   - 10: push. stack=[5, 10]
35   - -5: < 0. stack[-1]=10 (>0). Collision!
36     - diff = -5 + 10 = 5 (>0).
37     - 'a' becomes 0. Break.
38   - Result: [5, 10]
39
40   asteroids = [8, -8]
41   - 8: push.
42   - -8: diff = -8 + 8 = 0. 'a'=0, pop stack.
43   - Result: []
44 """

```

7 Stack: Monotonic Patterns & Parsing

7.1 Daily Temperatures

```

1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Output format?
4     A: List of integers (days to wait).
5   - Q: If no warmer day?
6     A: 0.
7
8 2. APPROACHES TO CONSIDER & REJECT
9   - Approach 1: Brute Force.
10    Logic: For each day, look ahead.
11    Complexity:  $O(N^2)$ .
12    Reject: TLE for  $N=10^5$ .
13   - Approach 2: Monotonic Decreasing Stack.
14    Logic: Stack stores indices of days *waiting* for a warmer day.
15    If curr > stack_top, we found the warmer day! Pop and record difference.
16    Complexity:  $O(N)$  Time (each index pushed/popped once).
17    Accept: Optimal.
18
19 3. INDENTED ENGLISH (LOGIC)
20   function dailyTemperatures(temps):
21       res = [0] * len(temps)
22       stack = [] # pair: [temp, index]
23       for i, t in enumerate(temps):
24           while stack and t > stack.top.temp:
25               stackT, stackInd = stack.pop()
26               res[stackInd] = i - stackInd
27               stack.append([t, i])
28       return res
29 """

```

The Magic Question (Get Buy-In)

"I'll use a Monotonic Decreasing Stack to store indices of days that haven't found a warmer day yet. When I encounter a higher temperature, I resolve all waiting days in the stack. This is $O(N)$ time. Should I code this?"

```

1 def dailyTemperatures(temperatures: List[int]) -> List[int]:
2     res = [0] * len(temperatures)
3     stack = [] # Stores indices
4
5     for i, t in enumerate(temperatures):
6         # Process Stack: Is current temp 't' warmer than the stack top?
7         while stack and t > temperatures[stack[-1]]:
8             stackInd = stack.pop()
9             res[stackInd] = i - stackInd
10
11         stack.append(i)

```

```

12
13     return res
14
15 """
16 5. DRY RUN (HUMAN DEBUGGER)
17     temps = [73, 74, 75, 71, 69, 72]
18     - i=0(73): push 0. stack=[0]
19     - i=1(74): 74 > 73. pop 0. res[0] = 1-0 = 1. push 1. stack=[1]
20     - i=2(75): 75 > 74. pop 1. res[1] = 2-1 = 1. push 2. stack=[2]
21     - i=3(71): 71 < 75. push 3. stack=[2, 3]
22     - i=4(69): 69 < 71. push 4. stack=[2, 3, 4]
23     - i=5(72):
24         - 72 > 69. pop 4. res[4] = 5-4 = 1.
25         - 72 > 71. pop 3. res[3] = 5-3 = 2.
26         - 72 < 75. Stop. push 5.
27     - Result: [1, 1, 0, 2, 1, 0] (Correct logic, 75 has no warmer day so 0).
28 """

```

7.2 Car Fleet

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Can cars pass each other?
4     A: No, if A is faster but behind B, A slows down to match B (forms a fleet).
5   - Q: Input sorted?
6     A: No.
7
8 2. APPROACHES TO CONSIDER & REJECT
9   - Approach 1: Physics Simulation (Time Step).
10    Reject: Floating point mess and slow.
11   - Approach 2: Sort by Position + Stack.
12    Logic: Calculate time to reach target: (target - pos) / speed.
13    Sort cars by position (descending/right-to-left).
14    If a car behind arrives SOONER (smaller time) than the car ahead, it catches up.
15    Stack stores "fleet arrival times".
16    Complexity:  $O(N \log N)$  for sort.
17    Accept: Optimal.
18
19 3. INDENTED ENGLISH (LOGIC)
20 function carFleet(target, position, speed):
21     pair = zip(position, speed)
22     sort pair descending by position
23     stack = []
24     for p, s in pair:
25         time = (target - p) / s
26         stack.append(time)
27         if len(stack) >= 2 and stack[-1] <= stack[-2]:
28             stack.pop() # Merged into the fleet ahead
29     return len(stack)
30 """
```

The Magic Question (Get Buy-In)

"I'll sort the cars by position in reverse order (closest to target first). I'll calculate the arrival time for each. If a car arrives faster than the one ahead of it, it effectively joins that fleet. A stack can track the distinct fleets. $O(N \log N)$ due to sorting."

```
1 def carFleet(target: int, position: List[int], speed: List[int]) -> int:
2     pair = [[p, s] for p, s in zip(position, speed)]
3
4     # Sort by position descending (Start from car closest to target)
5     stack = []
6     for p, s in sorted(pair)[::-1]:
7         time = (target - p) / s
8         stack.append(time)
9
10        # If current car arrives faster (smaller time) than previous fleet,
11        # it catches up and merges. We keep the slower time (stack[-2]).
12        if len(stack) >= 2 and stack[-1] <= stack[-2]:
13            stack.pop()
14
15    return len(stack)
16
17 """
18 5. DRY RUN (HUMAN DEBUGGER)
19 target=12, pos=[10, 8, 0], spd=[2, 4, 1]
20 - Pairs: (10,2), (8,4), (0,1). Sorted Rev: (10,2), (8,4), (0,1).
21 - Car (10, 2): Time = (12-10)/2 = 1. stack=[1].
22 - Car (8, 4): Time = (12-8)/4 = 1. stack=[1, 1].
23   - 1 <= 1? Yes. Pop. stack=[1]. (Catches up).
24 - Car (0, 1): Time = (12-0)/1 = 12. stack=[1, 12].
25   - 12 <= 1? No. It's slower, can't catch up. New fleet.
26 - Return 2.
27 """
```

7.3 Simplify Path

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Trailing slashes? A: Remove them.
4   - Q: Multiple slashes '//'? A: Treat as single '/'.
5   - Q: '..' at root? A: Stay at root.
6
7 2. APPROACHES
8   - Approach 1: Stack.
9     Logic: Split string by '/'.
10    Iterate tokens:
11      - "..": pop stack (if not empty).
12      - "." or "": continue.
13      - Name: push to stack.
14    Join stack with '/'.
15    Complexity: O(N).
16    Accept: Optimal.
17
18 3. INDENTED ENGLISH
19 function simplifyPath(path):
20     stack = []
21     parts = path.split("/")
22     for p in parts:
23         if p == "..": if stack: stack.pop()
24         elif p == "." or p == "": continue
25         else: stack.append(p)
26     return "/" + "/".join(stack)
27 """
```

The Magic Question (Get Buy-In)

"I'll split the path by slashes and process the tokens. A stack is perfect for managing directory depth: valid names push, '..' pops, and '.' is ignored. Finally, I'll reconstruct the canonical path. $O(N)$."

```
1 def simplifyPath(path: str) -> str:
2     stack = []
3     cur = ""
4
5     # Adding trailing slash to force processing of last token
6     for c in path + "/":
7         if c == "/":
8             if cur == "..":
9                 if stack: stack.pop()
10            elif cur != "" and cur != ".":
11                stack.append(cur)
12            cur = ""
13        else:
14            cur += c
15
16    return "/" + "/".join(stack)
17
18 """
19 5. DRY RUN (HUMAN DEBUGGER)
20 path = "/a./b/../../c/"
21 - "a": push. stack=["a"]
22 - ".": ignore.
23 - "b": push. stack=["a", "b"]
24 - "..": pop "b". stack=["a"]
25 - "..": pop "a". stack=[]
26 - "c": push. stack=["c"]
27 - Return "/c"
28 """
```

7.4 Decode String

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Nested brackets? A: Yes '3[a2[c]]'.
4   - Q: Valid input? A: Yes.
5
6 2. APPROACHES
7   - Approach 1: Recursion.
8     Reject: Stack is iterative and cleaner for interviews (avoids recursion depth).
9   - Approach 2: Stack.
10    Logic: Push chars. When ']' is hit:
11      1. Pop chars until '[' to get substring.
12      2. Pop '[' discard.
13      3. Pop digits to get k.
14      4. Push substring * k back to stack.
15    Complexity: O(N).
16    Accept: Optimal.
17
18 3. INDENTED ENGLISH
19 function decodeString(s):
20     stack = []
21     for c in s:
22         if c != ']': stack.append(c)
23         else:
24             substr = pop until '['
25             k = pop digits
26             stack.append(substr * k)
27     return join(stack)
28 """
```

The Magic Question (Get Buy-In)

"I'll use a stack. When I hit a closing bracket ']', I will pop characters to build the substring, pop the opening bracket, and then pop the multiplier 'k'. I'll push the expanded string back onto the stack to handle nesting. $O(N)$."

```
1 def decodeString(s: str) -> str:
2     stack = []
3
4     for c in s:
5         if c != "]":
6             stack.append(c)
7         else:
8             # 1. Build Substring
9             substr = ""
10            while stack[-1] != "[":
11                substr = stack.pop() + substr
12                stack.pop() # Remove '['
13
14            # 2. Get Multiplier (k)
15            k = ""
16            while stack and stack[-1].isdigit():
17                k = stack.pop() + k
18
19            # 3. Push Expanded String
20            stack.append(int(k) * substr)
21
22    return "".join(stack)
23
24 """
25 5. DRY RUN (HUMAN DEBUGGER)
26 s = "3[a2[c]]"
27 - push '3', '[' , 'a', '2', '[', 'c'.
28 - Hit ']':
29   - substr='c'. pop '['. k='2'.
30   - push 'cc'. stack=['3', '[', 'a', 'cc']
31 - Hit ']':
32   - substr='acc'. pop '['. k='3'.
33   - push 'accaccacc'.
```

```
34     - Return "accaccacc".
35     """
```

7.5 Largest Rectangle in Histogram (Hard)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: All bars positive? A: Yes.
4 - Q: Empty array? A: 0.
5
6 2. APPROACHES
7 - Approach 1: Brute Force. Expand from every bar.  $O(N^2)$ . Reject.
8 - Approach 2: Monotonic Increasing Stack.
9   Logic: We can extend a bar 'h' to the right as long as upcoming bars are  $\geq h$ .
10  When we see a SMALLER bar 'i', we know the bars in stack CANNOT extend past 'i'.
11  We pop them and calculate area:  $H = \text{popped\_height}$ ,  $W = i - \text{stack\_top\_index} - 1$ .
12  Complexity:  $O(N)$ .
13  Accept: Optimal.
14
15 3. INDENTED ENGLISH
16 function largestRectangleArea(heights):
17     stack = [-1] # boundary
18     maxArea = 0
19     heights.append(0) # trigger pop for remaining
20     for i, h in enumerate(heights):
21         while stack.top != -1 and heights[stack.top] >= h:
22             H = heights[stack.pop()]
23             W = i - stack.top - 1
24             maxArea = max(maxArea, H * W)
25         stack.append(i)
26     return maxArea
27 """
```

The Magic Question (Get Buy-In)

"I'll use a Monotonic Increasing Stack. When I encounter a bar shorter than the stack top, it defines the right boundary for the taller bars. I'll pop them and calculate the max area using the current index as the width limit. $O(N)$."

```
1 def largestRectangleArea(heights: List[int]) -> int:
2     maxArea = 0
3     stack = [] # pair: (index, height)
4
5     for i, h in enumerate(heights):
6         start = i
7         # While current bar is shorter than stack top,
8         # stack top can't extend right anymore. Compute its area.
9         while stack and stack[-1][1] > h:
10             index, height = stack.pop()
11             maxArea = max(maxArea, height * (i - index))
12             start = index # This short bar can extend LEFT to where the tall bar was
13         stack.append((start, h))
14
15     # Process remaining stack (they extend to the very end)
16     for i, h in stack:
17         maxArea = max(maxArea, h * (len(heights) - i))
18
19     return maxArea
20
21 """
22 5. DRY RUN (HUMAN DEBUGGER)
23 h = [2, 1, 5, 6, 2, 3]
24 - i=0(2): push (0, 2)
25 - i=1(1): 1 < 2. Pop (0, 2). Area=2*(1-0)=2. start=0. push (0, 1).
26 - i=2(5): push (2, 5).
27 - i=3(6): push (3, 6).
28 - i=4(2):
29   - 2 < 6. Pop (3, 6). Area=6*(4-3)=6. start=3.
30   - 2 < 5. Pop (2, 5). Area=5*(4-2)=10. start=2.
31   - push (2, 2).
32 - ... End logic.
33 """
```

8 Binary Search: The "Transition Point" Framework

The "Transition Point" Mental Model

Instead of searching for a value, we search for the boundary between **FALSE** and **TRUE**.

- **Left (l):** Always points to **FALSE** (Too small / Invalid).
- **Right (r):** Always points to **TRUE** (Valid / Possible Answer).
- **Loop:** while $l + 1 < r$: Run until l and r are neighbors.
- **Update:** If `check(mid)` is True, $r = mid$ (Maintain Valid). Else $l = mid$.
- **Result:** r is the minimum valid value.

8.1 Search Insert Position

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Return index if found? A: Yes.
4   - Q: Return insertion point? A: Yes.
5
6 2. LOGIC (TRANSITION POINT)
7   - Mental Model: [F, F, F, T, T, T]
8   - Condition: val >= target.
9   - We want the FIRST index where this condition is True.
10  - Init: l = -1 (Always F), r = len (Always T, fictitious index).
11  - Update: If nums[mid] >= target: r = mid. Else: l = mid.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll define a condition 'x >= target'. I want the first index where this is True.
16 # I'll use the l+1 < r template to find the boundary. O(log N)."
17
18 def searchInsert(nums: List[int], target: int) -> int:
19     # l points to 'Too Small', r points to 'Greater or Equal'
20     l, r = -1, len(nums)
21
22     while l + 1 < r:
23         mid = l + (r - l) // 2
24
25         if nums[mid] >= target:
26             r = mid # Maintained 'True' Zone
27         else:
28             l = mid # Maintained 'False' Zone
29
30     return r
31
32 """
33 5. DRY RUN
34     nums=[1,3,5,6], target=5
35     - [F, F, T, T]. We want first T.
36     - l=-1, r=4. mid=1(3). 3>=5 False. l=1.
37     - l=1, r=4. mid=2(5). 5>=5 True. r=2.
38     - l=1, r=2. Neighbors! Stop.
39     - Return r=2. Correct.
40 """
```

8.2 Binary Search

```
1 """
2 2. LOGIC (TRANSITION POINT)
3 - Even exact search can use this template.
4 - Find first index >= target.
5 - Then check if nums[r] == target.
6 """
7
8 # --- THE MAGIC QUESTION ---
9 # "I'll use the boundary template to find the first index where 'val >= target'.
10 # Then I'll check if that value equals the target. O(log N)."
11
12 def search(nums: List[int], target: int) -> int:
13     l, r = -1, len(nums)
14
15     while l + 1 < r:
16         mid = l + (r - l) // 2
17         if nums[mid] >= target:
18             r = mid
19         else:
20             l = mid
21
22     # Post-processing: Did we find it?
23     if r < len(nums) and nums[r] == target:
24         return r
25     return -1
```

8.3 Sqrt(x)

```
1 """
2 2. LOGIC (TRANSITION POINT)
3 - Condition: val * val <= x.
4 - We want the LAST value where this is True.
5 - Invert: We want the boundary between T (<= x) and F (> x).
6 - l = Valid (0), r = Invalid (x+1).
7 - Return l (the largest valid).
8 """
9
10 # --- THE MAGIC QUESTION ---
11 # "I'll search for the boundary where 'val * val <= x' stops being True.
12 # l will track valid answers, r will track invalid (too big). O(log x)."
13
14 def mySqrt(x: int) -> int:
15     if x == 0: return 0
16     l, r = 1, x + 1
17
18     while l + 1 < r:
19         mid = l + (r - l) // 2
20         if mid * mid <= x:
21             l = mid # Valid, try bigger (move l up)
22         else:
23             r = mid # Invalid, too big
24
25     return l
26
27 """
28 5. DRY RUN
29 x = 8.
30 - l=1 (T), r=9 (F). mid=5. 25<=8 False. r=5.
31 - l=1, r=5. mid=3. 9<=8 False. r=3.
32 - l=1, r=3. mid=2. 4<=8 True. l=2.
33 - l=2, r=3. Neighbors. Return l=2. Correct.
34 """
```

8.4 Guess Number Higher Or Lower

```
1 """
2 2. LOGIC (TRANSITION POINT)
3 - Condition: guess(val) <= 0. (i.e., val is >= pick).
4 - [Higher, Higher, PICK, Lower, Lower] -> [-1, -1, 0, 1, 1]
5 - Wait, logic: We want the point where "Is valid?" becomes True.
6 """
7
8 def guessNumber(n: int) -> int:
9     l, r = 0, n + 1
10
11     while l + 1 < r:
12         mid = l + (r - l) // 2
13         res = guess(mid)
14
15         if res == 0:
16             return mid # Lucky hit
17         elif res == -1:
18             # My guess is High. Pick is lower.
19             # mid is "Too Big" (True side of boundary)
20             r = mid
21         else:
22             # My guess is Low. Pick is higher.
23             # mid is "Too Small" (False side)
24             l = mid
25
26     return r # Should be caught by res==0 check usually
```

8.5 Koko Eating Bananas

```
1 """
2 2. LOGIC (TRANSITION POINT)
3     - Condition: can_finish(speed) == True.
4     - Array: [F, F, F, T, T, T] (Slow speeds fail, Fast speeds work).
5     - Goal: First T.
6     - l = 0 (Impossible speed), r = max(piles) (Definitely works).
7 """
8
9 # --- THE MAGIC QUESTION ---
10 # "I'll define a function 'can_finish(k)'. I want the smallest 'k' where this is True.
11 # I'll use the boundary search 'l+1 < r'. O(N log MaxP)."
```

```
12
13 def minEatingSpeed(piles: List[int], h: int) -> int:
14     # l is False (0 speed impossible), r is True (Max pile speed always works)
15     l, r = 0, max(piles)
16
17     def can_finish(k):
18         hours = 0
19         for p in piles:
20             # Equivalent to math.ceil(p / k)
21             hours += (p + k - 1) // k
22         return hours <= h
23
24     while l + 1 < r:
25         mid = l + (r - l) // 2
26         if can_finish(mid):
27             r = mid # Valid, but can we go slower?
28         else:
29             l = mid # Invalid, need to speed up
30
31     return r
32
33 """
34 5. DRY RUN
35 piles=[3,6,7,11], h=8
36 - l=0, r=11. mid=5. Hours=6 <= 8. True. r=5.
37 - l=0, r=5. mid=2. Hours=15 > 8. False. l=2.
38 - l=2, r=5. mid=3. Hours=10 > 8. False. l=3.
39 - l=3, r=5. mid=4. Hours=8 <= 8. True. r=4.
40 - l=3, r=4. Neighbors. Return r=4.
41 """
```

8.6 Capacity to Ship Packages

```
1 """
2 2. LOGIC (TRANSITION POINT)
3 - Condition: can_ship(capacity) == True.
4 - Array: [F, F, T, T, T].
5 - Goal: First T.
6 - l = max(weights) - 1 (Impossible), r = sum(weights) (Certain).
7 """
8
9 # --- THE MAGIC QUESTION ---
10 # "I'll search for the minimum capacity. l tracks 'Too Small', r tracks 'Works'.
11 # The transition point is the answer. O(N log SumW)."
12
13 def shipWithinDays(weights: List[int], days: int) -> int:
14     l, r = max(weights) - 1, sum(weights)
15
16     def can_ship(cap):
17         ships = 1
18         curr = 0
19         for w in weights:
20             if curr + w > cap:
21                 ships += 1
22                 curr = 0
23             curr += w
24         return ships <= days
25
26     while l + 1 < r:
27         mid = l + (r - l) // 2
28         if can_ship(mid):
29             r = mid # Valid, try to squeeze tighter
30         else:
31             l = mid # Cannot ship, increase capacity
32
33     return r
```

8.7 Search a 2D Matrix

```
1 """
2 2. LOGIC (TRANSITION POINT)
3 - Flatten 2D to 1D index: 0 to (R*C - 1).
4 - Condition: matrix[val] >= target.
5 - Goal: First True.
6 """
7
8 def searchMatrix(matrix: List[List[int]], target: int) -> bool:
9     R, C = len(matrix), len(matrix[0])
10     l, r = -1, R * C
11
12     while l + 1 < r:
13         mid = l + (r - l) // 2
14         val = matrix[mid // C][mid % C]
15
16         if val >= target:
17             r = mid
18         else:
19             l = mid
20
21     # r is the index of the first element >= target
22     if r < R * C:
23         val = matrix[r // C][r % C]
24         return val == target
25     return False
```

9 Binary Search: Rotated Arrays & Hard Variations

9.1 Find Minimum in Rotated Sorted Array

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Duplicates? A: No (simplifies pivot finding).
4 - Q: Rotated? A: Yes, [3,4,5,1,2].
5 - Q: Goal? A: Find the minimum element (the pivot).
6
7 2. LOGIC (TRANSITION POINT)
8 - Observation: In a rotated sorted array, the "Right Sorted Portion"
9   contains elements strictly <= last element.
10   [4, 5, 6, 7, 0, 1, 2]
11   Check vs nums[-1] (2):
12   [F, F, F, F, T, T, T] (4<=2 False, 0<=2 True)
13 - Goal: First True.
14 - l = -1, r = len-1. (Since last element is always T, r can start there).
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll use the transition point framework. My condition is 'val <= nums[-1]'.
19 # The array is split into [False, False, ..., True, True]. The first True is the minimum
20 # . O(log N)."
21
22 def findMin(nums: List[int]) -> int:
23     l, r = -1, len(nums) - 1
24     target = nums[-1] # Compare against the last element
25
26     while l + 1 < r:
27         mid = l + (r - l) // 2
28         if nums[mid] <= target:
29             r = mid # In the right sorted portion (True)
30         else:
31             l = mid # In the left sorted portion (False)
32
33     return nums[r]
34
35 """
36 5. DRY RUN
37 nums = [3, 4, 5, 1, 2]
38 - target = 2.
```

```
38 - l=-1, r=4. mid=1(4). 4<=2 False. l=1.  
39 - l=1, r=4. mid=2(5). 5<=2 False. l=2.  
40 - l=2, r=4. mid=3(1). 1<=2 True. r=3.  
41 - l=2, r=3. Neighbors. Return nums[3]=1. Correct.  
42 """
```

9.2 Search in Rotated Sorted Array

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: One pass required? A: Usually "O(log N)" is required.
4     - Q: Two steps ok? A: Yes, O(log N) + O(log N) is still O(log N).
5
6 2. LOGIC (TWO STEP "ROBOT" MODE)
7     - Step 1: Find the Pivot Index (Minimum) using logic from Problem 8.
8     - Step 2: Determine which side the target is on.
9         - If target >= nums[pivot] and target <= nums[-1]: Search Right side.
10        - Else: Search Left side.
11     - Step 3: Standard Binary Search on that specific subarray.
12     - Why? Eliminates the "nested if-else" hell of the one-pass solution.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "Instead of complex branching, I'll use a robust two-step approach:
17 # 1. Find the pivot (minimum) index.
18 # 2. Binary search the correct subarray.
19 # This is cleaner and remains O(log N). Should I code this?"
20
21 def search(nums: List[int], target: int) -> int:
22     # 1. Find Pivot (First element <= nums[-1])
23     l, r = -1, len(nums) - 1
24     last = nums[-1]
25
26     while l + 1 < r:
27         mid = l + (r - l) // 2
28         if nums[mid] <= last:
29             r = mid
30         else:
31             l = mid
32     pivot = r
33
34     # 2. Choose Subarray
35     # If target is in the right sorted portion:
36     if pivot <= len(nums)-1 and target >= nums[pivot] and target <= nums[-1]:
37         l, r = pivot - 1, len(nums)
38     else:
39         # Target must be in the left sorted portion
40         l, r = -1, pivot
41
42     # 3. Standard BS on chosen range
43     while l + 1 < r:
44         mid = l + (r - l) // 2
45         if nums[mid] >= target:
46             r = mid
47         else:
48             l = mid
49
50     if r < len(nums) and nums[r] == target:
51         return r
52     return -1
53
54 """
55 5. DRY RUN
56     nums=[4,5,6,7,0,1,2], target=0
57     - Step 1: Find Pivot.
58       - [F,F,F,F,T,T,T]. Pivot index=4 (val 0).
59     - Step 2: Choose Side.
60       - target(0) >= nums[4](0) AND <= nums[6](2)? Yes.
61       - Search range: indices [4, 6]. (l=3, r=7 for template).
62     - Step 3: BS on [0, 1, 2].
63       - Finds 0 at index 4.
64 """
```

9.3 Time Based Key-Value Store

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Timestamps strictly increasing? A: Yes (append only).
4     - Q: If exact timestamp missing? A: Return largest timestamp_prev <= timestamp.
5
6 2. LOGIC (TRANSITION POINT)
7     - Data Structure: Map { key : List[(time, value)] }
8     - Since times are increasing, the list is sorted.
9     - Search: Find LAST pair where pair.time <= timestamp.
10    - Invert: Boundary between T (<= target) and F (> target).
11    - l = Valid, r = Invalid. Return l.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll store the values in a Hash Map where the value is a list of (timestamp, value)
16 # tuples.
17 # Since timestamps are increasing, I can Binary Search the list to find the latest valid
18 # timestamp. O(log N)."
19
20 from collections import defaultdict
21
22 class TimeMap:
23
24     def __init__(self):
25         self.store = defaultdict(list) # key : [ [time, val] ]
26
27     def set(self, key: str, value: str, timestamp: int) -> None:
28         self.store[key].append([timestamp, value])
29
30     def get(self, key: str, timestamp: int) -> str:
31         res = ""
32         values = self.store[key]
33
34         # Binary Search: Find largest t <= timestamp
35         # l points to Valid, r points to Invalid (> timestamp)
36         l, r = -1, len(values)
37
38         while l + 1 < r:
39             mid = l + (r - 1) // 2
40             if values[mid][0] <= timestamp:
41                 l = mid # Valid, try to find a later one
42             else:
43                 r = mid # Invalid (future timestamp)
44
45         return values[l][1] if l >= 0 else ""
46
47 """
48 5. DRY RUN
49     set("foo", "bar", 1). store["foo"] = [[1, "bar"]]
50     get("foo", 3).
51     - values = [[1, "bar"]]
52     - l=-1, r=1. mid=0. values[0][0](1) <= 3. True. l=0.
53     - Loop ends. Return values[0][1] -> "bar".
54 """
```

9.4 Median of Two Sorted Arrays

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Time complexity? A:  $O(\log(\min(N, M)))$ .
4 - Q: Can I merge? A: No, merge is  $O(N+M)$ .
5 - Q: Double or Int? A: Float (average of two middle if even).
6
7 2. LOGIC (BINARY SEARCH ON PARTITION)
8 - We want to partition both arrays such that:
9   LeftPart size == RightPart size (roughly).
10  max(LeftPart) <= min(RightPart).
11 - We BS on the cut position of the SMALLER array (A).
12 - Cut B is derived:  $j = \text{HalfTotal} - i$ .
13 - Predicate:  $A[i] \leq B[j+1]$  AND  $B[j] \leq A[i+1]$ .
14   (Are the cross-elements ordered correctly?)
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I will perform a Binary Search on the partition index of the smaller array.
19 # By deriving the partition for the second array, I can check if the left halves are
20 # strictly smaller than the right halves in  $O(1)$ . Total time  $O(\log(\min(N, M)))$ ."
21
22 def findMedianSortedArrays(nums1: List[int], nums2: List[int]) -> float:
23     A, B = nums1, nums2
24     total = len(nums1) + len(nums2)
25     half = total // 2
26
27     # Ensure A is the smaller array to minimize BS range
28     if len(B) < len(A):
29         A, B = B, A
30
31     l, r = 0, len(A) - 1
32
33     while True: # Will always find a valid cut
34         i = (l + r) // 2 # A's index (floored)
35         j = half - (i + 1) - 1 # B's index
36
37         # Edge cases: If index out of bounds, use Infinity
38         Aleft = A[i] if i >= 0 else float("-inf")
39         Aright = A[i+1] if (i+1) < len(A) else float("inf")
40         Bleft = B[j] if j >= 0 else float("-inf")
41         Bright = B[j+1] if (j+1) < len(B) else float("inf")
42
43         # Valid Partition?
44         if Aleft <= Bright and Bleft <= Aright:
45             # Odd length: min of Right side
46             if total % 2:
47                 return min(Aright, Bright)
48             # Even length: avg of max(Left) and min(Right)
49             return (max(Aleft, Bleft) + min(Aright, Bright)) / 2
50         elif Aleft > Bright:
51             r = i - 1 # A's left is too big, shrink A
52         else:
53             l = i + 1 # A's left is too small, expand A
54
55 """
56 5. DRY RUN
57 A=[1, 2], B=[3, 4]. Total=4, Half=2.
58 - l=0, r=1. i=0 (val 1). j = 2 - 1 - 1 = 0 (val 3).
59 - Aleft=1, Aright=2. Bleft=3, Bright=4.
60 - 1 <= 4 (Ok). 3 <= 2 (Fail). Bleft > Aright.
61 - Need more A. l=1.
62 - i=1 (val 2). j = 2 - 2 - 1 = -1.
63 - Aleft=2, Aright=inf. Bleft=-inf, Bright=3.
64 - 2 <= 3 (Ok). -inf <= inf (Ok). Valid.
65 - Even:  $\max(2, -\text{inf}) + \min(\text{inf}, 3) / 2 = (2+3)/2 = 2.5$ .
66 """
```

9.5 Search in Rotated Sorted Array II (Duplicates)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Duplicates allowed? A: Yes.
4 - Q: Complexity? A: Worst case becomes O(N) because e.g. [1,1,1,3,1].
5   We can't tell if right/left is sorted if nums[l] == nums[mid] == nums[r].
6
7 2. LOGIC (TRIMMING)
8 - Same as Search Rotated (Problem 9), but handling the duplicate edge case.
9 - If nums[l] == nums[mid] == nums[r], we cannot decide.
10 - Strategy: Shrink the window linearly (l++, r--) until items differ.
11 """
12
13 def search(nums: List[int], target: int) -> bool:
14     l, r = 0, len(nums) - 1
15
16     while l <= r:
17         mid = l + (r - l) // 2
18         if nums[mid] == target:
19             return True
20
21         # Edge Case: Duplicates making it impossible to determine sorted side
22         if nums[l] == nums[mid] and nums[mid] == nums[r]:
23             l += 1
24             r -= 1
25             continue
26
27         # Left Sorted Portion
28         if nums[l] <= nums[mid]:
29             if nums[l] <= target < nums[mid]:
30                 r = mid - 1
31             else:
32                 l = mid + 1
33         # Right Sorted Portion
34         else:
35             if nums[mid] < target <= nums[r]:
36                 l = mid + 1
37             else:
38                 r = mid - 1
39
40     return False
```

10 Linked List: Pointers & The Sentinel Pattern

The "Sentinel" Mental Model

Never handle the "Head" as a special case.

- **The Problem:** The head might change (deletion, merge, reorder), forcing ugly if head is None logic.
- **The Fix:** Create a dummy node. Point dummy.next to head.
- **The Action:** Always iterate using a pointer starting at dummy.
- **The Return:** return dummy.next.

10.1 Reverse Linked List

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Iterative or Recursive? A: Both are good. Iterative is O(1) space.
4 - Q: Empty list? A: Return None.
5
6 2. LOGIC (POINTER MANIPULATION)
7 - Mental Model: "Link Breaker".
8 - We need three pointers: prev, curr, nxt.
```

```

9     - Save next, Break link (point backwards), Move forward.
10    """
11
12    # --- THE MAGIC QUESTION ---
13    # "I will use three pointers (prev, curr, next) to reverse the links in-place.
14    # This runs in O(N) time and O(1) space. Should I code this?"
15
16    def reverseList(head: Optional[ListNode]) -> Optional[ListNode]:
17        prev, curr = None, head
18
19        while curr:
20            # 1. Save next connection
21            nxt = curr.next
22            # 2. Reverse the link
23            curr.next = prev
24            # 3. Move pointers forward
25            prev = curr
26            curr = nxt
27
28        return prev
29
30    """
31    5. DRY RUN
32    1 -> 2 -> 3 -> None
33    - prev=None, curr=1. nxt=2. 1->None. prev=1, curr=2.
34    - prev=1, curr=2. nxt=3. 2->1. prev=2, curr=3.
35    - prev=2, curr=3. nxt=None. 3->2. prev=3, curr=None.
36    - Return prev (3). List is 3->2->1.
37    """

```

10.2 Merge Two Sorted Lists

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: New list or in-place? A: Splice the existing nodes together.
4   - Q: Lists can be different lengths? A: Yes.
5
6 2. LOGIC (SENTINEL PATTERN)
7   - We don't know which list has the smaller head.
8   - Use a 'dummy' node to avoid writing 'if l1 < l2: head = l1'.
9   - Build the tail starting from dummy.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use a Sentinel (dummy) node to start the merged list. I'll iterate through both
14 # lists,
15 # attaching the smaller node to my tail pointer. This avoids edge cases for the head. O(
16 # N+M)."
17
18 def mergeTwoLists(list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[
19     ListNode]:
20     dummy = ListNode()
21     tail = dummy
22
23     while list1 and list2:
24         if list1.val < list2.val:
25             tail.next = list1
26             list1 = list1.next
27         else:
28             tail.next = list2
29             list2 = list2.next
30         tail = tail.next
31
32     # Append the remaining nodes (one list might still have elements)
33     if list1:
34         tail.next = list1
35     elif list2:
36         tail.next = list2
37
38     return dummy.next
39
40 """
41 5. DRY RUN
42 l1=[1,2,4], l2=[1,3,4]
43 - dummy(0). tail=dummy.
44 - 1 vs 1. tail->1(l1). l1=2. tail=1.
45 - 2 vs 1. tail->1(l2). l2=3. tail=1.
46 - 2 vs 3. tail->2(l1). l1=4. tail=2.
47 - ...
48 """
```

10.3 Linked List Cycle

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: O(1) Space? A: Yes, "Strong Hire" requirement[cite: 166].
4   - Q: Max length? A: 104.
5
6 2. LOGIC (TORTOISE AND HARE)
7   - Use two pointers: Slow (1 step) and Fast (2 steps).
8   - If there is a cycle, Fast will eventually lap Slow and they will meet.
9   - If Fast reaches None, there is no cycle.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use Floyd's Cycle Finding Algorithm (Tortoise and Hare).
14 # Slow moves 1 step, Fast moves 2. If they collide, there's a cycle.
15 # This is O(N) time and O(1) space."
16
17 def hasCycle(head: Optional[ListNode]) -> bool:
18     slow, fast = head, head
19
20     while fast and fast.next:
21         slow = slow.next
22         fast = fast.next.next
23
24         if slow == fast:
25             return True
26
27     return False
28
29 """
30 5. DRY RUN
31   3 -> 2 -> 0 -> -4 -> (points back to 2)
32   - S=3, F=3
33   - S=2, F=0
34   - S=0, F=2 (lap 1)
35   - S=-4, F=-4 (COLLISION) -> Return True.
36 """
```

10.4 Reorder List

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: L0 -> Ln -> L1 -> Ln-1? A: Yes, merge first half with reversed second half.
4   - Q: In-place? A: Yes.
5
6 2. LOGIC (CLUSTER B - IMPLEMENTATION)
7   - This tests "can you write 50 lines of bug-free code"[cite: 129].
8   - Step 1: Find Middle (Slow/Fast).
9   - Step 2: Reverse Second Half (Reverse List Logic).
10  - Step 3: Merge Two Halves (Merge Logic).
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll split this into three steps: find the middle, reverse the second half,
15 # and merge the two halves. This reuses standard patterns and runs in O(N) time."
16
17 def reorderList(head: Optional[ListNode]) -> None:
18     # 1. Find Middle
19     slow, fast = head, head.next
20     while fast and fast.next:
21         slow = slow.next
22         fast = fast.next.next
23
24     # 2. Reverse Second Half
25     second = slow.next
26     slow.next = None # Cut the connection
27     prev = None
28     while second:
29         tmp = second.next
30         second.next = prev
31         prev = second
32         second = tmp
33
34     # 3. Merge Two Halves
35     first, second = head, prev
36     while second:
37         tmp1, tmp2 = first.next, second.next
38         first.next = second
39         second.next = tmp1
40         first, second = tmp1, tmp2
41
42 """
43 5. DRY RUN
44 1->2->3->4
45 - Middle: slow=2. second=3->4.
46 - Reverse: 4->3. prev=4.
47 - Merge:
48   - 1->4. 4->2. 2->3.
49   - Result: 1->4->2->3.
50 """
```

10.5 Remove Nth Node From End of List

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: n is valid? A: Yes.
4     - Q: Can n = length? A: Yes, remove head.
5
6 2. LOGIC (SENTINEL + TWO POINTERS)
7     - Use a Sentinel because we might remove the head.
8     - Right pointer advances 'n' times.
9     - Then Left and Right move together.
10    - When Right hits end, Left is just before the target.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use a Sentinel node to handle the edge case of removing the head.
15 # I'll use two pointers separated by 'n' nodes. When the right pointer hits the end,
16 # the left pointer is at the removal point. O(N)."
17
18 def removeNthFromEnd(head: Optional[ListNode], n: int) -> Optional[ListNode]:
19     dummy = ListNode(0, head)
20     left = dummy
21     right = head
22
23     # 1. Advance Right n times
24     while n > 0 and right:
25         right = right.next
26         n -= 1
27
28     # 2. Move both until Right hits end
29     while right:
30         left = left.next
31         right = right.next
32
33     # 3. Delete node
34     left.next = left.next.next
35
36     return dummy.next
37
38 """
39 5. DRY RUN
40     1->2->3->4->5, n=2 (remove 4)
41     - dummy->1. L=dummy, R=1.
42     - Advance R by 2: R=3.
43     - Move: L=1, R=4.
44     - Move: L=2, R=5.
45     - Move: L=3, R=None.
46     - Delete: L(3).next = 3.next.next (5).
47     - List: 1->2->3->5.
48 """
```

10.6 Copy List With Random Pointer

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Null random pointers? A: Yes.
4   - Q: O(1) Space? A: Standard is O(N) with Map. O(1) is Interweaving.
5
6 2. LOGIC (HASH MAP)
7   - Two passes.
8   - Pass 1: Create copies of all nodes. Store in Map {old_node : new_node}.
9   - Pass 2: Connect next and random pointers using the map.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use a Hash Map to map original nodes to their copies.
14 # First pass creates the nodes, second pass links the pointers.
15 # This is O(N) time and O(N) space. Should I code this?"
16
17 def copyRandomList(head: 'Optional[Node]') -> 'Optional[Node]':
18     oldToCopy = {None : None}
19
20     # 1. Create Copies
21     curr = head
22     while curr:
23         oldToCopy[curr] = Node(curr.val)
24         curr = curr.next
25
26     # 2. Link Pointers
27     curr = head
28     while curr:
29         copy = oldToCopy[curr]
30         copy.next = oldToCopy[curr.next]
31         copy.random = oldToCopy[curr.random]
32         curr = curr.next
33
34     return oldToCopy[head]
35
36 """
37 5. DRY RUN
38 7 -> 13 (random 7).
39 - Map: {7:Node(7), 13:Node(13)}.
40 - Link: Node(7).next = Map[13]. Node(13).random = Map[7].
41 """
```

10.7 Add Two Numbers

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Different lengths? A: Yes.
4   - Q: New list? A: Yes.
5   - Q: Order? A: Reverse order (digits are 1s, 10s, 100s). Easy math.
6
7 2. LOGIC (SENTINEL + CARRY)
8   - Use Sentinel for result head.
9   - Loop while l1 OR l2 OR carry.
10  - Sum = v1 + v2 + carry.
11  - Node val = sum % 10.
12  - New carry = sum // 10.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll use a Sentinel node and a 'carry' variable. I'll iterate while either list has
17 #   nodes
18 # or a carry exists, performing column-wise addition. O(N)."
```

```
19 def addTwoNumbers(l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
20     dummy = ListNode()
21     curr = dummy
22     carry = 0
23
24     while l1 or l2 or carry:
25         v1 = l1.val if l1 else 0
26         v2 = l2.val if l2 else 0
27
28         # Math
29         val = v1 + v2 + carry
30         carry = val // 10
31         val = val % 10
32
33         curr.next = ListNode(val)
34         curr = curr.next
35
36         if l1: l1 = l1.next
37         if l2: l2 = l2.next
38
39     return dummy.next
40
41 """
42 5. DRY RUN
43     l1=[2,4,3], l2=[5,6,4] (342 + 465 = 807)
44     - 2+5=7. Node(7). carry=0.
45     - 4+6=10. Node(0). carry=1.
46     - 3+4+1=8. Node(8). carry=0.
47     - Result: 7->0->8.
48 """
```

11 Linked List: Advanced Patterns & Design

11.1 Find the Duplicate Number

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Can I modify the input array (e.g., sort it)?
4     A: No, read-only.
5   - Q: Can I use extra space (Set/Map)?
6     A: No, must be O(1) space.
7   - Q: Time complexity?
8     A: O(N).
9   - Q: Why is this a Linked List problem?
10    A: Because values point to indices.
11    [1,3,4,2,2] -> Index 0 points to 1, Index 1 points to 3... A duplicate value means
12    two indices point to the same location, creating a Cycle.
13
14 2. APPROACHES TO CONSIDER & REJECT
```

```

14 - Approach 1: Sorting.
15   - Logic: Sort and check adjacent elements.
16   - Complexity:  $O(N \log N)$ .
17   - Reject: Modifies input (forbidden).
18 - Approach 2: Hash Set.
19   - Logic: Store seen numbers.
20   - Complexity:  $O(N)$  Space.
21   - Reject: violates  $O(1)$  space constraint.
22 - Approach 3: Floyd's Cycle Detection (Tortoise and Hare).
23   - Logic: Treat array as a linked list ('next = nums[index]'). Use Slow/Fast
24   pointers to detect cycle (collision) and then find the entrance (duplicate).
25   - Complexity:  $O(N)$  Time,  $O(1)$  Space.
26   - Accept: Optimal.
27
28 3. INDENTED ENGLISH (LOGIC)
29 function findDuplicate(nums):
30     slow, fast = 0, 0
31     # Phase 1: Find collision
32     do:
33         slow = nums[slow]
34         fast = nums[nums[fast]]
35     while slow != fast
36
37     # Phase 2: Find entrance
38     slow2 = 0
39     while slow != slow2:
40         slow = nums[slow]
41         slow2 = nums[slow2]
42     return slow

```

The Magic Question (Get Buy-In)

"This problem can be modeled as a Linked List Cycle where values are pointers. I'll use Floyd's Cycle Detection to find the cycle entrance, which corresponds to the duplicate number. This guarantees $O(N)$ time and $O(1)$ space. Should I code this?"

```

1 def findDuplicate(nums: List[int]) -> int:
2     slow, fast = 0, 0
3
4     # 1. Phase 1: Find the intersection point of the two runners.
5     while True:
6         slow = nums[slow]          # Move 1 step
7         fast = nums[nums[fast]]    # Move 2 steps
8         if slow == fast:
9             break
10
11     # 2. Phase 2: Find the "entrance" to the cycle.
12     # Reset a second pointer to the start. Move both 1 step at a time.
13     slow2 = 0
14     while True:
15         if slow == slow2:
16             return slow
17         slow = nums[slow]
18         slow2 = nums[slow2]
19
20 """
21 5. DRY RUN (HUMAN DEBUGGER)
22 nums = [1, 3, 4, 2, 2]
23 - Indices: 0->1, 1->3, 3->2, 2->4, 4->2 (Cycle points to 2)
24 - Phase 1:
25   - S=1, F=3
26   - S=3, F=4 (via 2->4)
27   - S=2, F=4 (via 2->4)
28   - S=4, F=4 (Meet!)
29 - Phase 2:
30   - S=4, S2=0.
31   - S=nums[4]=2, S2=nums[0]=1.
32   - S=nums[2]=4, S2=nums[1]=3.
33   - S=nums[4]=2, S2=nums[3]=2. (Meet!)

```

```
34     - Return 2.  
35     """
```

11.2 LRU Cache

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: What happens if I put a key that exists?
4     A: Update value and mark as Most Recently Used (MRU).
5   - Q: Time complexity requirements?
6     A: O(1) for both 'get' and 'put'.
7   - Q: Capacity constraint?
8     A: Remove Least Recently Used (LRU) if full.
9
10 2. APPROACHES TO CONSIDER & REJECT
11   - Approach 1: Array / List.
12     - Logic: Store items in a list. Move accessed item to end.
13     - Complexity: O(N) to find/remove item.
14     - Reject: Too slow.
15   - Approach 2: Hash Map Only.
16     - Logic: O(1) access, but Map doesn't track "Order" (in standard theory).
17     - Reject: Cannot identify LRU efficiently.
18   - Approach 3: Hash Map + Doubly Linked List.
19     - Logic: Map stores {key: Node}. List maintains order (Head=LRU, Tail=MRU).
20     - Moving a node requires changing pointers (O(1)). Map gives direct access to Node (O(1)).
21     - Accept: Optimal.
22
23 3. INDENTED ENGLISH (LOGIC)
24 Class Node: val, prev, next
25 Class LRUCache:
26     map = {}
27     left (LRU), right (MRU) dummy nodes
28
29     get(key):
30         if key in map:
31             remove node from list
32             insert node at right
33             return val
34         return -1
35
36     put(key, val):
37         if key in map: remove node
38         create new node
39         map[key] = node
40         insert at right
41         if len > cap:
42             remove left.next (LRU)
43             delete from map
44 """
```

The Magic Question (Get Buy-In)

"I'll use a Hash Map for $O(1)$ lookups and a Doubly Linked List to maintain order. I'll use two sentinel nodes (Left and Right) to avoid edge-case checks when adding/removing. Total time $O(1)$. Should I code this?"

```
1 class Node:
2     def __init__(self, key, val):
3         self.key, self.val = key, val
4         self.prev, self.next = None, None
5
6 class LRUCache:
7
8     def __init__(self, capacity: int):
9         self.cap = capacity
10        self.cache = {} # Map Key -> Node
11
12        # Sentinel Nodes: Left=LRU, Right=MRU
13        self.left, self.right = Node(0, 0), Node(0, 0)
14        self.left.next, self.right.prev = self.right, self.left
15
16        # Helper: Remove node from LL
```

```

17 def remove(self, node):
18     prev, nxt = node.prev, node.next
19     prev.next, nxt.prev = nxt, prev
20
21 # Helper: Insert at Right (MRU)
22 def insert(self, node):
23     prev, nxt = self.right.prev, self.right
24     prev.next = nxt.prev = node
25     node.next, node.prev = nxt, prev
26
27 def get(self, key: int) -> int:
28     if key in self.cache:
29         # Update to MRU by removing and re-inserting
30         self.remove(self.cache[key])
31         self.insert(self.cache[key])
32         return self.cache[key].val
33     return -1
34
35 def put(self, key: int, value: int) -> None:
36     if key in self.cache:
37         self.remove(self.cache[key])
38
39     self.cache[key] = Node(key, value)
40     self.insert(self.cache[key])
41
42     if len(self.cache) > self.cap:
43         # Evict LRU (node immediately after Left Sentinel)
44         lru = self.left.next
45         self.remove(lru)
46         del self.cache[lru.key]
47
48 """
49 5. DRY RUN (HUMAN DEBUGGER)
50 Cap=2.
51 - Put(1,1): Map={1:N1}. List: L->N1->R.
52 - Put(2,2): Map={1:N1, 2:N2}. List: L->N1->N2->R. (N1 is LRU).
53 - Get(1): Remove N1, Insert Right. List: L->N2->N1->R. (N2 is LRU).
54 - Put(3,3): Over capacity.
55   - Evict L.next (N2). Map={1:N1, 3:N3}.
56   - List: L->N1->N3->R.
57 """

```

11.3 Merge K Sorted Lists

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input size? A: k lists, total N nodes.
4   - Q: Can lists be empty? A: Yes.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Iterative Merge (Merge 1 & 2, then result & 3...).
8     - Complexity:  $O(k * N)$ .
9     - Reject: Inefficient for large k.
10  - Approach 2: Min-Heap.
11    - Logic: Add head of all k lists to Heap. Pop min, add to result, push min.next.
12    - Complexity:  $O(N \log k)$ .
13    - Accept: Optimal.
14  - Approach 3: Divide & Conquer.
15    - Logic: Pair up lists and merge. reduces k by half each step.
16    - Complexity:  $O(N \log k)$ .
17    - Accept: Also optimal (good alternative if heaps forbidden).
18
19 3. INDENTED ENGLISH (LOGIC)
20  function mergeKLists(lists):
21      heap = []
22      for l in lists: if l: heap.push(l.val, i, l)
23
24      dummy = Node(), curr = dummy
25      while heap:
26          val, i, node = heap.pop()
27          curr.next = node
28          curr = curr.next
29          if node.next:
30              heap.push(node.next.val, i, node.next)
31      return dummy.next
32 """
```

The Magic Question (Get Buy-In)

"I'll use a Min-Heap to track the smallest node among the k lists. I extract the minimum, attach it to the result, and push its next node back into the heap. This runs in $O(N \log k)$ time. Should I code this?"

```
1 import heapq
2
3 def mergeKLists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
4     # Stores tuples (val, index, node). Index is for tie-breaking.
5     minHeap = []
6
7     # 1. Initialize Heap
8     for i, l in enumerate(lists):
9         if l:
10             heapq.heappush(minHeap, (l.val, i, l))
11
12     dummy = ListNode()
13     curr = dummy
14
15     # 2. Process Heap
16     while minHeap:
17         val, i, node = heapq.heappop(minHeap)
18         curr.next = node
19         curr = curr.next
20
21         if node.next:
22             heapq.heappush(minHeap, (node.next.val, i, node.next))
23
24     return dummy.next
25
26 """
27 5. DRY RUN (HUMAN DEBUGGER)
28  L1=[1,4,5], L2=[1,3,4], L3=[2,6]
29  - Heap Init: [(1,0,N1a), (1,1,N1b), (2,2,N2)]
```

```
30 - Pop (1,0,N1a). curr->1. Push N1a.next (4).
31   Heap: [(1,1,N1b), (2,2,N2), (4,0,N4)]
32 - Pop (1,1,N1b). curr->1->1. Push N1b.next (3).
33   Heap: [(2,2,N2), (3,1,N3), (4,0,N4)]
34 - Pop (2,2,N2). curr->1->1->2.
35 - ...
36 """
```

11.4 Reverse Nodes in k-Group

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Can I use  $O(k)$  space?
4   A: No,  $O(1)$  space is required.
5 - Q: Leftover nodes  $< k$ ?
6   A: Leave them as is.
7
8 2. APPROACHES TO CONSIDER & REJECT
9 - Approach 1: Extract to Array.
10  - Logic: Dump  $k$  nodes to array, reverse, rebuild list.
11  - Complexity:  $O(N)$  Space.
12  - Reject: Violates space constraint.
13 - Approach 2: Pointer Surgery (Iterative).
14  - Logic: Check if  $k$  nodes exist. If yes, reverse them in-place. Connect 'prevGroup'
15  to 'newHead', and 'newTail' to 'nextGroup'.
16  - Complexity:  $O(N)$  Time,  $O(1)$  Space.
17  - Accept: Optimal.
18
19 3. INDENTED ENGLISH (LOGIC)
20 function reverseKGroup(head, k):
21     dummy -> head
22     groupPrev = dummy
23     while True:
24         kth = get_kth(groupPrev, k)
25         if not kth: break
26         groupNext = kth.next
27
28         # Reverse inner
29         prev, curr = kth.next, groupPrev.next
30         while curr != groupNext:
31             tmp = curr.next
32             curr.next = prev
33             prev = curr
34             curr = tmp
35
36         # Stitch
37         tmp = groupPrev.next
38         groupPrev.next = kth
39         groupPrev = tmp
40     return dummy.next
41 """
```

The Magic Question (Get Buy-In)

"I'll iterate through the list. For every k nodes, I'll identify the segment and reverse it in-place. I'll maintain a 'groupPrev' pointer to stitch the reversed segments back together. This is $O(N)$ time and $O(1)$ space. Agreed?"

```
1 def reverseKGroup(head: Optional[ListNode], k: int) -> Optional[ListNode]:
2     dummy = ListNode(0, head)
3     groupPrev = dummy
4
5     while True:
6         # 1. Check if we have k nodes left
7         kth = groupPrev
8         for i in range(k):
9             kth = kth.next
10            if not kth:
11                return dummy.next # Less than k nodes, done
12
13            groupNext = kth.next
14
15            # 2. Reverse Group
16            # prev starts at groupNext so the first node points to the next group after
17            reversal
18            prev, curr = kth.next, groupPrev.next
19
20            while curr != groupNext:
```

```

20         tmp = curr.next
21         curr.next = prev
22         prev = curr
23         curr = tmp
24
25     # 3. Stitch it back
26     # groupPrev.next points to the OLD head (now tail).
27     # We need to save it to move groupPrev there later.
28     tmp = groupPrev.next
29     groupPrev.next = kth # kth is the NEW head
30     groupPrev = tmp # Move prev to the end of this group
31
32     return dummy.next
33
34 """
35 5. DRY RUN (HUMAN DEBUGGER)
36 1->2->3->4->5, k=2
37 - dummy->1. groupPrev=dummy.
38 - kth=2. groupNext=3.
39 - Reverse:
40   - Init: prev=3, curr=1.
41   - Loop: 1->3. prev=1, curr=2.
42   - Loop: 2->1. prev=2, curr=3. (curr==groupNext, Stop).
43 - Stitch:
44   - groupPrev.next (dummy.next) = 2. (List: D->2->1->3).
45   - groupPrev = 1 (Old head, now tail).
46 - Next Group: kth=4...
47 """

```

12 Heap: Essentials & The "K" Pattern

12.1 Kth Largest Element in a Stream

```

1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Will k change? A: No.
4   - Q: Can inputs be negative? A: Yes.
5   - Q: Complexity? A: add() should be faster than O(N log N).
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: Sort list on every add().
9     - Complexity: O(N log N).
10    - Reject: Too slow for a stream.
11   - Approach 2: Min-Heap of Size K.
12     - Logic: Maintain a heap with exactly K elements. The root (min) is the Kth largest
13       .
14     - If new val > root, pop root and push new val.
15     - Complexity: O(log K) per add.
16     - Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19 class KthLargest:
20     minHeap = []
21     size = k
22
23     def init(k, nums):
24         for n in nums:
25             heapq.push(minHeap, n)
26             if len(minHeap) > k: heapq.pop(minHeap)
27
28     def add(val):
29         heapq.push(minHeap, val)
30         if len(minHeap) > k: heapq.pop(minHeap)
31         return minHeap[0]
32 """

```

The Magic Question (Get Buy-In)

"I'll maintain a Min-Heap of size K . The root of the heap will always represent the K -th largest element. Adding a new number takes $O(\log K)$ time. Should I code this?"

```
1 import heapq
2
3 class KthLargest:
4
5     def __init__(self, k: int, nums: List[int]):
6         self.minHeap, self.k = nums, k
7         heapq.heapify(self.minHeap)
8
9         # Shrink to size k
10        while len(self.minHeap) > k:
11            heapq.heappop(self.minHeap)
12
13    def add(self, val: int) -> int:
14        heapq.heappush(self.minHeap, val)
15
16        # Maintain invariant: Size <= k
17        if len(self.minHeap) > self.k:
18            heapq.heappop(self.minHeap)
19
20        return self.minHeap[0]
21
22 """
23 5. DRY RUN (HUMAN DEBUGGER)
24 k=3, nums=[4, 5, 8, 2]
25 - Init: Heapify -> [2, 4, 8, 5]. Pop (2). Heap=[4, 5, 8]. Root=4 (3rd largest).
26 - add(3): Push 3. Heap=[3, 4, 8, 5]. Len=4 > 3. Pop 3. Heap=[4, 5, 8]. Ret 4.
27 - add(5): Push 5. Heap=[4, 5, 8, 5]. Len=4 > 3. Pop 4. Heap=[5, 5, 8]. Ret 5.
28 - add(10): Push 10. Pop 5. Heap=[5, 8, 10]. Ret 5.
29 """
```

12.2 Last Stone Weight

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: What if 1 stone left? A: Return its weight.
4 - Q: What if 0 stones left? A: Return 0.
5 - Q: Python Heap behavior? A: Python has Min-Heap. Need to negate values for Max-Heap
6 .
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Sort in Loop.
9   - Logic: Sort, pop two, insert, resort.
10  - Complexity:  $O(N * N \log N)$ .
11  - Reject: Too slow.
12 - Approach 2: Max-Heap Simulation.
13   - Logic: Put all stones in heap (negated). Pop two smallest (heaviest). Smash. Push
14   result.
15   - Complexity:  $O(N \log N)$ .
16   - Accept: Optimal.
17 3. INDENTED ENGLISH (LOGIC)
18 function lastStoneWeight(stones):
19     maxHeap = [-s for s in stones]
20     heapify(maxHeap)
21
22     while len(maxHeap) > 1:
23         y = -pop(maxHeap) # Heaviest
24         x = -pop(maxHeap) # Second heaviest
25         if x != y:
26             push(-(y - x))
27
28     return -maxHeap[0] if maxHeap else 0
29 """
```

The Magic Question (Get Buy-In)

"I'll use a Max-Heap to efficiently access the two heaviest stones. Since Python standard library only has a Min-Heap, I'll store the values as negatives. This runs in $O(N \log N)$ time."

```
1 def lastStoneWeight(stones: List[int]) -> int:
2     # Python heaps are min-heaps. Use negative values to simulate max-heap.
3     stones = [-s for s in stones]
4     heapq.heapify(stones)
5
6     while len(stones) > 1:
7         first = heapq.heappop(stones) # Heaviest (e.g., -8)
8         second = heapq.heappop(stones) # Second heaviest (e.g., -7)
9
10        if second > first: # Logic inverted: -7 > -8 means 7 < 8
11            # Push the difference (negative)
12            heapq.heappush(stones, first - second)
13
14        stones.append(0) # Edge case: if empty, return 0
15        return abs(stones[0])
16
17 """
18 5. DRY RUN (HUMAN DEBUGGER)
19 stones = [2, 7, 4, 1, 8, 1]
20 - Heap: [-8, -7, -4, -2, -1, -1]
21 - Pop -8, -7. Diff = -1. Push -1. Heap: [-4, -2, -1, -1, -1]
22 - Pop -4, -2. Diff = -2. Push -2. Heap: [-2, -2, -1, -1, -1]
23 - Pop -2, -2. Diff = 0. Nothing pushed. Heap: [-1, -1, -1]
24 - Pop -1, -1. Diff = 0. Heap: [-1].
25 - Return 1.
26 """
```

12.3 K Closest Points to Origin

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Distance formula? A: Euclidean (sqrt(x^2 + y^2)).
4 - Q: Return order? A: Any.
5 - Q: Tie breaking? A: Any.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Sort All.
9   - Complexity: O(N log N).
10  - Reject: We only need K closest, not full sort.
11 - Approach 2: Max-Heap of Size K.
12   - Logic: Maintain heap of K *closest* points.
13   - To keep the SMALLEST distances, we use a MAX-Heap to evict the LARGEST distance.
14   - Complexity: O(N log K).
15   - Accept: Optimal for N >> K.
16
17 3. INDENTED ENGLISH (LOGIC)
18 function kClosest(points, k):
19     maxHeap = []
20     for x, y in points:
21         dist = -(x^2 + y^2) # Negative for Max Heap behavior
22         push(maxHeap, (dist, x, y))
23         if len > k: pop(maxHeap) # Remove the farthest point
24     return [[x, y] for d, x, y in maxHeap]
25 """
```

The Magic Question (Get Buy-In)

"I'll use a Max-Heap of size K to store the closest points. By popping the 'max' (farthest point) whenever the heap size exceeds K , I ensure only the K closest remain. Time complexity is $O(N \log K)$."

```
1 def kClosest(points: List[List[int]], k: int) -> List[List[int]]:
2     maxHeap = []
3
4     for x, y in points:
5         # We want to keep SMALL distances.
6         # Max Heap pops the LARGE distances.
7         # In Python Min-Heap, store negative distance to simulate Max Heap.
8         dist = -(x ** 2 + y ** 2)
9         heapq.heappush(maxHeap, [dist, x, y])
10
11     if len(maxHeap) > k:
12         heapq.heappop(maxHeap) # Removes the point with largest distance
13
14     return [[x, y] for dist, x, y in maxHeap]
15
16 """
17 5. DRY RUN (HUMAN DEBUGGER)
18 pts=[[1,3], [-2,2]], k=1
19 - [1,3]: dist = -(1+9) = -10. Push [-10, 1, 3].
20 - [-2,2]: dist = -(4+4) = -8. Push [-8, -2, 2].
21 - Heap: [-10, -8] (Since -10 < -8, -10 is 'smaller' in python logic, but represents
    dist 10).
22 - Wait, Python heap pops SMALLEST. -10 is smaller than -8.
23 - So pop(-10) removes distance 10. Correct.
24 - Remaining: [-8, -2, 2]. Result [[-2, 2]].
25 """
```

12.4 Kth Largest Element in an Array

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Can I modify input? A: Yes.
4   - Q: Complexity requirement? A:  $O(N)$  is best (QuickSelect),  $O(N \log K)$  is safe.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Sort.  $O(N \log N)$ . Reject (too easy).
8   - Approach 2: Min-Heap of Size  $K$ .
9     - Logic: Keep top  $K$  elements. Root is the  $K$ th.
10    - Complexity:  $O(N \log K)$ . Safe bet.
11   - Approach 3: QuickSelect (Partitioning).
12     - Logic: Pick pivot, partition. If pivot index ==  $k$ , done. Else recurse.
13     - Complexity:  $O(N)$  average,  $O(N^2)$  worst case.
14     - Accept: "Strong Hire" if you can implement correctly.
15
16 3. INDENTED ENGLISH (HEAP APPROACH)
17 function findKthLargest(nums, k):
18     heap = nums[:k]
19     heapify(heap)
20     for n in nums[k:]:
21         if n > heap[0]:
22             heapreplace(heap, n)
23     return heap[0]
24 """
```

The Magic Question (Get Buy-In)

"I can use a Min-Heap of size K which guarantees $O(N \log K)$. Alternatively, I can use QuickSelect for $O(N)$ average time, though it has a worst-case $O(N^2)$. Which approach do you prefer?"

```
1 # Implementation: Min-Heap Strategy (Safer/Cleaner)
2 def findKthLargest(nums: List[int], k: int) -> int:
3     # Min-Heap of size K
4     # The smallest element in a heap of "K Largest" items IS the Kth Largest.
5
6     heap = nums[:k]
7     heapq.heapify(heap)
8
9     for n in nums[k:]:
10         if n > heap[0]:
11             heapq.heapreplace(heap, n) # Optimized pop+push
12
13     return heap[0]
14
15 """
16 5. DRY RUN (HUMAN DEBUGGER)
17 nums=[3,2,1,5,6,4], k=2
18 - Init Heap(2): [2, 3]. (Size 2, holding 3 and 2).
19 - Process 1: 1 < 2. Ignore.
20 - Process 5: 5 > 2. Replace 2 with 5. Heap=[3, 5].
21 - Process 6: 6 > 3. Replace 3 with 6. Heap=[5, 6].
22 - Process 4: 4 < 5. Ignore.
23 - Return 5. Correct.
24 """
```

12.5 Task Scheduler

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input format? A: List of chars ["A","A","B"].
4 - Q: Idle behavior? A: CPU stays idle if no task is ready.
5 - Q: Goal? A: Minimum units of time.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Simulation (Heap + Queue).
9   - Logic: Max-Heap stores counts of available tasks. Queue stores (count, idle_time)
   for tasks on cooldown.
10  - Complexity:  $O(N * M)$  where  $M$  is idle time? No,  $O(\text{Total Time})$ .
11  - Accept: Intuitive.
12 - Approach 2: Math / Greedy.
13   - Logic: Max frequency determines the frame.  $(\text{max\_f} - 1) * (n + 1) + \text{count\_of\_max\_f}$ 
14   .
15   - Complexity:  $O(N)$ .
16   - Trade-off: Harder to explain "why" it works. Simulation is safer.
17
18 3. INDENTED ENGLISH (SIMULATION)
19 function leastInterval(tasks, n):
20     count = Counter(tasks)
21     maxHeap = [-c for c in count]
22     q = Deque() # (count, time_to_active)
23     time = 0
24
25     while maxHeap or q:
26         time += 1
27         if maxHeap:
28             cnt = pop(maxHeap) + 1 # Decrement count (it was negative)
29             if cnt != 0: q.append([cnt, time + n])
30         if q and q[0].time == time:
31             push(maxHeap, q.popleft().count)
32     return time
33 """
```

The Magic Question (Get Buy-In)

"I'll simulate the CPU scheduler. I'll use a Max-Heap to always pick the most frequent available task, and a Queue to manage tasks currently on cooldown. This simulates the exact process in linear time relative to the total intervals."

```
1 def leastInterval(tasks: List[str], n: int) -> int:
2     # Frequency map
3     count = Counter(tasks)
4     # Max Heap (stores negative counts)
5     maxHeap = [-c for c in count.values()]
6     heapq.heapify(maxHeap)
7
8     time = 0
9     # Queue stores [count, idle_until_time]
10    q = deque()
11
12    while maxHeap or q:
13        time += 1
14
15        if maxHeap:
16            # Process task: 1 + (-count) -> effectively count - 1
17            cnt = 1 + heapq.heappop(maxHeap)
18            if cnt:
19                # Task still needs processing, add to cooldown queue
20                q.append([cnt, time + n])
21
22        # Check if any task in queue is ready to run
23        if q and q[0][1] == time:
24            heapq.heappush(maxHeap, q.popleft()[0])
25
26    return time
27
```

```

28 """
29 5. DRY RUN (HUMAN DEBUGGER)
30 tasks=["A","A","B"], n=2
31 - Heap: [-2(A), -1(B)]. q=[]. time=0.
32 - T=1: Pop A. Rem A=-1. q=[[-1, 3]]. Heap=[-1(B)].
33 - T=2: Pop B. Rem B=0. Done. q=[[-1, 3]]. Heap=[].
34 - T=3: Heap empty. q front time is 3. Push A(-1) back to heap. q=[]. Heap=[-1(A)].
35     Pop A. Rem A=0. Done.
36 - Return 3.
37 """

```

12.6 Design Twitter

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Feed size? A: 10 most recent.
4   - Q: Does following self matter? A: No, user follows self implicitly or explicitly.
5   - Q: Complexity? A: getNewsFeed should be efficient.
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: Sort all tweets.  $O(N \log N)$ . Reject.
9   - Approach 2: Merge K Sorted Lists (Heap).
10    - Logic: User follows K people. Each person has a list of tweets (sorted by time).
11    - We need the top 10 from K sorted lists.
12    - Data Structures: Map<User, Set<Followee>>, Map<User, List<Tweet>>.
13    - Complexity:  $O(1)$  post,  $O(K)$  getFeed (where K is follow count).
14    - Accept: Optimal "System Design Lite".
15
16 3. INDENTED ENGLISH (LOGIC)
17 class Twitter:
18     tweetMap = { user : list of [time, tweetId] }
19     followMap = { user : set of followees }
20     timer = 0
21
22     postTweet(user, tweet):
23         tweetMap[user].append([timer--, tweet]) # Decrement for Max Heap
24
25     getNewsFeed(user):
26         res = []
27         minHeap = []
28         users = followMap[user] + [user]
29         for u in users:
30             if u has tweets: push latest tweet to heap
31         while heap and len(res) < 10:
32             pop max, add to res
33             push next tweet from that user
34         return res
35 """
```

The Magic Question (Get Buy-In)

"I'll use Hash Maps to track followers and tweets. Generating the feed is essentially 'Merging K Sorted Lists.' I'll use a Max-Heap to pull the 10 most recent tweets from the user's followees. Should I code this class?"

```
1 class Twitter:
2
3     def __init__(self):
4         self.count = 0
5         self.tweetMap = defaultdict(list) # userId -> list of [count, tweetIds]
6         self.followMap = defaultdict(set) # userId -> set of followeeId
7
8     def postTweet(self, userId: int, tweetId: int) -> None:
9         # Use count to track global order.
10        # Decrement count for Max Heap simulation later
11        self.tweetMap[userId].append([self.count, tweetId])
12        self.count -= 1
13
14    def getNewsFeed(self, userId: int) -> List[int]:
15        res = []
16        minHeap = []
17
18        # User follows themselves
19        self.followMap[userId].add(userId)
20
21        # 1. Initial Heap Population (Heads of lists)
22        for followeeId in self.followMap[userId]:
23            if followeeId in self.tweetMap:
24                # Get most recent tweet
25                index = len(self.tweetMap[followeeId]) - 1
26                count, tweetId = self.tweetMap[followeeId][index]
```

```

27         # Push: (count, tweetId, followeeId, index-1)
28         heapq.heappush(minHeap, [count, tweetId, followeeId, index - 1])
29
30     # 2. Extract Top 10
31     while minHeap and len(res) < 10:
32         count, tweetId, followeeId, index = heapq.heappop(minHeap)
33         res.append(tweetId)
34
35         # If user has more tweets, push the next one
36         if index >= 0:
37             count, tweetId = self.tweetMap[followeeId][index]
38             heapq.heappush(minHeap, [count, tweetId, followeeId, index - 1])
39
40     return res
41
42     def follow(self, followerId: int, followeeId: int) -> None:
43         self.followMap[followerId].add(followeeId)
44
45     def unfollow(self, followerId: int, followeeId: int) -> None:
46         if followeeId in self.followMap[followerId]:
47             self.followMap[followerId].remove(followeeId)
48
49 """
50 5. DRY RUN (HUMAN DEBUGGER)
51 - post(1, 101) -> count=-1. Map={1: [[-1, 101]]}
52 - post(2, 102) -> count=-2. Map={2: [[-2, 102]]}
53 - follow(1, 2). FollowMap={1: {2}}
54 - getNewsFeed(1):
55   - Heap Init: 1 follows {1, 2}.
56   - Push 1's latest: [-1, 101, 1, index=-1]
57   - Push 2's latest: [-2, 102, 2, index=-1]
58   - Pop [-2, 102...]. res=[102]. Index<0, no push.
59   - Pop [-1, 101...]. res=[102, 101].
60 """

```

13 Heap: Advanced Scheduling & Two Heaps

13.1 Single-Threaded CPU

```

1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input order? A: Tasks not sorted.
4   - Q: Tie-breaking? A: Shortest processing time, then smallest index.
5   - Q: Idle CPU? A: If no task available, jump to next available task time.
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: Simple Sorting.
9     - Logic: Sort by enqueue time.
10    - Reject: Doesn't handle processing time priority once tasks are queued.
11   - Approach 2: Simulation with Min-Heap.
12     - Logic: Sort tasks by enqueue time. Use a time pointer.
13     - Push all available tasks (enqueue <= time) to Min-Heap.
14     - Heap sorts by (processing_time, index).
15     - Complexity: O(N log N).
16     - Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19 function getOrder(tasks):
20     add index to tasks
21     sort tasks by enqueue time
22     minHeap = []
23     time = tasks[0].enqueue
24     i = 0
25     while minHeap or i < len(tasks):
26         while i < len and tasks[i].enqueue <= time:
27             push(minHeap, (tasks[i].proc, tasks[i].index))
28             i++
29         if not minHeap:
30             time = tasks[i].enqueue
31         else:
32             proc, index = pop(minHeap)

```

```

33         time += proc
34         res.append(index)
35     return res
36 """

```

The Magic Question (Get Buy-In)

"I'll sort the tasks by start time to manage the timeline. I'll use a Min-Heap to store available tasks, prioritizing shorter processing times. I'll advance a 'current time' variable to simulate the CPU execution. $O(N \log N)$."

```

1  import heapq
2
3  def getOrder(tasks: List[List[int]]) -> List[int]:
4      # Add index to track original order: [enqueue, process, index]
5      for i, t in enumerate(tasks):
6          t.append(i)
7
8      # Sort by Enqueue Time
9      tasks.sort(key=lambda x: x[0])
10
11     res, minHeap = [], []
12     i, time = 0, tasks[0][0] # Jump to first task start
13
14     while minHeap or i < len(tasks):
15         # 1. Add all tasks that have become available by 'time'
16         while i < len(tasks) and time >= tasks[i][0]:
17             # Heap sorts by: Processing Time, then Index
18             heapq.heappush(minHeap, [tasks[i][1], tasks[i][2]])
19             i += 1
20
21         # 2. Process task or Advance time
22         if not minHeap:
23             # CPU idle, jump to next task
24             time = tasks[i][0]
25         else:
26             procTime, index = heapq.heappop(minHeap)
27             time += procTime
28             res.append(index)
29
30     return res
31
32 """
33 5. DRY RUN (HUMAN DEBUGGER)
34 tasks=[[1,2],[2,4],[3,2],[4,1]]
35 - Sort: Same. time=1.
36 - T=1: Push [2,0]. i=1. Heap=[[2,0]].
37 - Pop [2,0]. time=1+2=3. res=[0].
38 - T=3: Push [4,1], [2,2]. i=3. Heap=[[2,2], [4,1]].
39 - Pop [2,2]. time=3+2=5. res=[0, 2].
40 - T=5: Push [1,3]. i=4. Heap=[[1,3], [4,1]].
41 - Pop [1,3]. time=6. res=[0, 2, 3].
42 - Pop [4,1]. res=[0, 2, 3, 1].
43 """

```

13.2 Reorganize String

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: If impossible? A: Return "".
4 - Q: Valid result? A: No adjacent chars are same.
5 - Q: Case sensitive? A: Yes.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Greedy filling (A, B, A, B...).
9 - Logic: Count chars. Always pick the most frequent char that isn't the previous
10 one.
11 - Data Structure: Max-Heap.
12 - Complexity:  $O(N \log K)$  where  $K=26$ . Effectively  $O(N)$ .
13 - Accept: Optimal.
14
15 3. INDENTED ENGLISH (LOGIC)
16 function reorganizeString(s):
17     count = Counter(s)
18     maxHeap = [(-cnt, char)]
19     prev = None
20     res = ""
21     while heap or prev:
22         if not heap and prev: return "" # Impossible
23         cnt, char = pop(heap)
24         res += char
25         cnt += 1 # Decrement count (it's negative)
26
27         if prev: push(prev) # Push back the ONE we set aside
28
29         if cnt != 0: prev = (cnt, char)
30         else: prev = None
31     return res
32 """
```

The Magic Question (Get Buy-In)

"I'll use a Greedy approach with a Max-Heap. I always pick the most frequent character available. I'll temporarily hold the 'just used' character aside to ensure I don't pick it immediately again. $O(N)$ time."

```
1 def reorganizeString(s: str) -> str:
2     count = Counter(s)
3     # Max Heap (negative counts)
4     maxHeap = [(-cnt, char) for char, cnt in count.items()]
5     heapq.heapify(maxHeap)
6
7     prev = None
8     res = ""
9
10    while maxHeap or prev:
11        # If heap empty but we still have a 'prev' holding a char, impossible
12        if not maxHeap and prev:
13            return ""
14
15        # Get most frequent
16        cnt, char = heapq.heappop(maxHeap)
17        res += char
18        cnt += 1 # Decrement frequency (closer to 0)
19
20        # Push BACK the previous char (now valid to use again)
21        if prev:
22            heapq.heappush(maxHeap, prev)
23            prev = None
24
25        # Set current char as 'prev' if it still has count left
26        if cnt != 0:
27            prev = [cnt, char]
28
29    return res
```

```

30
31 """
32 5. DRY RUN (HUMAN DEBUGGER)
33     s = "aab"
34     - Heap: [[-2, 'a'], [-1, 'b']]. prev=None.
35     - Pop 'a'. res="a". cnt=-1. prev=[-1, 'a']. Heap=[[-1, 'b']].
36     - Pop 'b'. res="ab". cnt=0. Push prev('a'). Heap=[[-1, 'a']]. prev=None.
37     - Pop 'a'. res="aba". cnt=0. prev=None.
38     - Done.
39 """

```

13.3 Longest Happy String

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Constraint? A: No "aaa", "bbb", "ccc" (max 2 consecutive).
4 - Q: Goal? A: Maximize length.
5
6 2. APPROACHES TO CONSIDER & REJECT
7 - Approach 1: Greedy Max-Heap.
8 - Logic: Always try to add most frequent.
9 - Check: If adding most freq creates "aaa", try 2nd most freq.
10 - Complexity: O(a+b+c).
11 - Accept: Optimal.
12
13 3. INDENTED ENGLISH (LOGIC)
14 function longestDiverseString(a, b, c):
15     heap = [(-a,'a'), (-b,'b'), (-c,'c')]
16     res = ""
17     while heap:
18         cnt, char = pop(heap)
19         if len(res)>1 and res[-1]==res[-2]==char:
20             if not heap: break # No other char to break sequence
21             cnt2, char2 = pop(heap)
22             res += char2
23             cnt2++
24             if cnt2: push(cnt2)
25             push(cnt, char) # Push original back
26         else:
27             res += char
28             cnt++
29             if cnt: push(cnt)
30     return res
31 """
```

The Magic Question (Get Buy-In)

"I'll greedily pick the character with the highest remaining count using a Max-Heap. If picking it would violate the 'no triple' rule, I'll peek at the second most frequent character to break the streak. O(N) time."

```
1 def longestDiverseString(a: int, b: int, c: int) -> str:
2     res, maxHeap = "", []
3     for count, char in [(-a, "a"), (-b, "b"), (-c, "c")]:
4         if count != 0:
5             heapq.heappush(maxHeap, (count, char))
6
7     while maxHeap:
8         cnt, char = heapq.heappop(maxHeap)
9
10        # Check for violation (last 2 chars are same as current)
11        if len(res) > 1 and res[-1] == res[-2] == char:
12            if not maxHeap:
13                break # Cannot proceed
14
15            # Use 2nd most frequent
16            cnt2, char2 = heapq.heappop(maxHeap)
17            res += char2
18            cnt2 += 1 # used one
19
20            if cnt2:
21                heapq.heappush(maxHeap, (cnt2, char2))
22
23            # Don't forget to put the 1st one back!
24            heapq.heappush(maxHeap, (cnt, char))
25        else:
26            res += char
27            cnt += 1
28            if cnt:
29                heapq.heappush(maxHeap, (cnt, char))
30
```

```

31     return res
32
33 """
34 5. DRY RUN (HUMAN DEBUGGER)
35     a=1, b=1, c=7. Heap=[(-7,c), (-1,a), (-1,b)]
36     - Pop c. res="c". H=[(-6,c)...]
37     - Pop c. res="cc". H=[(-5,c)...]
38     - Pop c. Violation! Pop a. res="cca". H=[(-5,c), (-1,b)].
39     - Pop c. res="ccac".
40     - Pop c. res="ccacc".
41     - Pop c. Violation! Pop b. res="ccaccb".
42     - ... "ccaccbcc"
43 """

```

13.4 Car Pooling

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: trips=[[passengers, start, end]], capacity.
4   - Q: Same location drop/pickup? A: Drop happen BEFORE pickup at same location.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Min-Heap (Simulation).
8     - Logic: Sort trips by start. Loop through trips.
9     - Heap stores [end_time, num_passengers].
10    - Pop from heap if end_time <= curr_start (passengers leave).
11    - Check capacity.
12    - Complexity: O(N log N).
13    - Accept: Standard.
14   - Approach 2: Bucket Sort / Difference Array.
15     - Logic: timestamp array[1001]. arr[start] += num, arr[end] -= num.
16     - Prefix sum to check capacity.
17     - Complexity: O(N + Range).
18     - Accept: Faster if range is small (1000).
19
20 3. INDENTED ENGLISH (HEAP APPROACH)
21 function carPooling(trips, capacity):
22     sort trips by start
23     minHeap = [] # (end, num)
24     curPass = 0
25     for t in trips:
26         num, start, end = t
27         while heap and heap[0].end <= start:
28             curPass -= heap.pop().num
29         curPass += num
30         if curPass > capacity: return False
31         push(heap, (end, num))
32     return True
33 """
```

The Magic Question (Get Buy-In)

"I'll sort trips by start location. I'll use a Min-Heap to track active trips, ordered by their end location. As I iterate, I'll drop off passengers whose end location is $\leq$ current start location. O(N log N)."

```
1 def carPooling(trips: List[List[int]], capacity: int) -> bool:
2     # Sort by start location
3     trips.sort(key=lambda t: t[1])
4
5     minHeap = [] # [end, numPassengers]
6     curPass = 0
7
8     for t in trips:
9         num, start, end = t
10
11         # Drop off passengers who reached their destination
12         while minHeap and minHeap[0][0] <= start:
13             curPass -= minHeap[0][1]
14             heapq.heappop(minHeap)
15
16         curPass += num
17         if curPass > capacity:
18             return False
19
20         heapq.heappush(minHeap, [end, num])
21
22     return True
23
24 """
25 5. DRY RUN (HUMAN DEBUGGER)
26 trips=[[2,1,5],[3,3,7]], cap=4
27 - Sort: Same.
28 - T1 [2,1,5]: Heap empty. cur=2. Heap=[[5,2]].
```

```
29 - T2 [3,3,7]:
30   - Heap top end=5. 5 <= 3? False.
31   - cur = 2 + 3 = 5.
32   - 5 > 4. Return False.
33 """
```

13.5 Find Median From Data Stream (Two Heaps)

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Integer stream? A: Yes.
4   - Q: Median def? A: Middle element or avg of two middle.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Sort list.  $O(N \log N)$  add. Reject.
8   - Approach 2: Insertion Sort.  $O(N)$  add. Reject.
9   - Approach 3: Two Heaps.
10    - Logic: Max-Heap (Left Half, smaller nums), Min-Heap (Right Half, larger nums).
11    - Maintain balance: size diff  $\leq 1$ .
12    - Median is root of larger heap or avg of roots.
13    - Complexity:  $O(\log N)$  add,  $O(1)$  find.
14    - Accept: Optimal.
15
16 3. INDENTED ENGLISH (LOGIC)
17 class MedianFinder:
18     small = [] # Max Heap (negated)
19     large = [] # Min Heap
20
21     addNum(num):
22         push(small, -num)
23         # Ensure order: small.max <= large.min
24         if small and large and -small[0] > large[0]:
25             val = -pop(small)
26             push(large, val)
27         # Balance sizes
28         if len(small) > len(large) + 1:
29             move small -> large
30         if len(large) > len(small) + 1:
31             move large -> small
32 """
```

The Magic Question (Get Buy-In)

"I'll use the Two Heaps pattern. A Max-Heap for the smaller half of numbers and a Min-Heap for the larger half. By keeping their sizes balanced, the median is always accessible at the roots in $O(1)$ time."

```
1 class MedianFinder:
2
3     def __init__(self):
4         # Two heaps: small (Max Heap), large (Min Heap)
5         self.small, self.large = [], []
6
7     def addNum(self, num: int) -> None:
8         # Always push to small first (Max Heap simulation)
9         heapq.heappush(self.small, -1 * num)
10
11         # 1. Order Check: Max of small must be <= Min of large
12         if (self.small and self.large and
13             (-1 * self.small[0]) > self.large[0]):
14             val = -1 * heapq.heappop(self.small)
15             heapq.heappush(self.large, val)
16
17         # 2. Size Check: Uneven size?
18         if len(self.small) > len(self.large) + 1:
19             val = -1 * heapq.heappop(self.small)
20             heapq.heappush(self.large, val)
21         if len(self.large) > len(self.small) + 1:
22             val = heapq.heappop(self.large)
23             heapq.heappush(self.small, -1 * val)
24
25     def findMedian(self) -> float:
26         if len(self.small) > len(self.large):
27             return -1 * self.small[0]
28         if len(self.large) > len(self.small):
29             return self.large[0]
```

```

30
31         return (-1 * self.small[0] + self.large[0]) / 2.0
32
33     """
34 5. DRY RUN (HUMAN DEBUGGER)
35     add(1): Small=[-1], Large=[].
36     add(2): Small=[-2, -1]. -(-2) > large? No large.
37         Size check: 2 > 0 + 1. Move -2 to large.
38         Small=[-1], Large=[2].
39     add(3): Small=[-3, -1]. -(-3)=3 > 2 (Large min).
40         Move 3 to Large. Small=[-1], Large=[2, 3].
41         Size: 2 > 1 + 1 (False). Large > Small + 1? No.
42         Correction? Wait. Large has 2, Small has 1. Balanced.
43     findMedian: Large size > Small. Return 2.
44     (1, 2, 3 -> Median 2). Correct.
45     """

```

13.6 IPO (Hard)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Max k projects? A: Yes.
4 - Q: Constraint? A: Can only start project if Capital >= min_capital.
5 - Q: Goal? A: Maximize final capital.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Check all every time. O(k*N). TLE.
9 - Approach 2: Two Heaps (Min and Max).
10 - Logic: We have 'Available' projects (Capital <= W) and 'Unavailable'.
11 - Store Unavailable in Min-Heap (sorted by Capital needed).
12 - Move feasible projects to Max-Heap (sorted by Profit).
13 - Greedy: Always pick Max Profit from feasible.
14 - Complexity: O(N log N).
15 - Accept: Optimal.
16
17 3. INDENTED ENGLISH (LOGIC)
18 function findMaximizedCapital(k, w, profits, capital):
19     minHeap = [(c, p) for c, p in zip(capital, profits)]
20     heapify(minHeap)
21     maxHeap = []
22
23     for i in range(k):
24         while minHeap and minHeap[0].cap <= w:
25             c, p = pop(minHeap)
26             push(maxHeap, -p)
27             if not maxHeap: break
28             w += -pop(maxHeap)
29     return w
30 """
```

The Magic Question (Get Buy-In)

"I'll use a Min-Heap to store projects by capital requirement and a Max-Heap to store available projects by profit. In each step, I move all affordable projects to the Max-Heap and greedily choose the most profitable one. O(N log N)."

```
1 def findMaximizedCapital(k: int, w: int, profits: List[int], capital: List[int]) -> int:
2     # Min-Heap stores (capital, profit) sorted by capital
3     minHeap = [(c, p) for c, p in zip(capital, profits)]
4     heapq.heapify(minHeap)
5
6     # Max-Heap stores (-profit) for available projects
7     maxHeap = []
8
9     for i in range(k):
10         # 1. Move all affordable projects to Max-Heap
11         while minHeap and minHeap[0][0] <= w:
12             c, p = heapq.heappop(minHeap)
13             heapq.heappush(maxHeap, -p)
14
15         # 2. Choose best project
16         if not maxHeap:
17             break # No projects can be funded
18
19         w += -1 * heapq.heappop(maxHeap)
20
21     return w
22
23 """
24 5. DRY RUN (HUMAN DEBUGGER)
25 k=2, w=0, profits=[1,2,3], capital=[0,1,1]
26 - minHeap=[(0,1), (1,2), (1,3)]
27 - i=0:
28   - 0 <= 0? Yes. Pop (0,1). Push -1 to max.
29   - 1 <= 0? No.
30   - Pick -1. w = 0 + 1 = 1.
31 - i=1:
```

```

32     - 1 <= 1? Yes. Pop (1,2). Push -2.
33     - 1 <= 1? Yes. Pop (1,3). Push -3.
34     - MaxHeap=[-3, -2]. Pick -3. w = 1 + 3 = 4.
35     - Return 4.
36 """

```

14 Arrays & Hashing: Maps, Sets & Prefixes

14.1 Product of Array Except Self

```

1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Can I use division?
4     A: No. (Crucial constraint).
5     - Q: Time complexity?
6     A: O(N).
7     - Q: Space complexity?
8     A: O(1) extra space (excluding the output array).
9     - Q: Zeros in input?
10    A: Yes (makes division impossible anyway without handling 0 cases).
11
12 2. APPROACHES TO CONSIDER & REJECT
13     - Approach 1: Brute Force.
14       - Logic: For each element i, loop through all others and multiply.
15       - Complexity: O(N^2).
16       - Reject: TLE.
17     - Approach 2: Division.
18       - Logic: Calculate total product, then res[i] = total / nums[i].
19       - Reject: Explicitly forbidden. Fails if nums contains 0.
20     - Approach 3: Prefix and Suffix Products.
21       - Logic: res[i] = (product of all left) * (product of all right).
22       - Optimization: Use the output array to store prefix products first, then multiply
23       by suffix on the fly.
24       - Complexity: O(N) Time, O(1) Extra Space.
25       - Accept: Optimal.
26
27 3. INDENTED ENGLISH (LOGIC)
28 function productExceptSelf(nums):
29     res = [1] * len(nums)
30
31     # Pass 1: Prefix (Left to Right)
32     prefix = 1
33     for i from 0 to N:
34         res[i] = prefix
35         prefix *= nums[i]
36
37     # Pass 2: Suffix (Right to Left)
38     postfix = 1
39     for i from N-1 down to 0:
40         res[i] *= postfix
41         postfix *= nums[i]
42
43     return res
44 """

```

The Magic Question (Get Buy-In)

"I will calculate the prefix products in a first pass and store them in the result array. Then, I'll do a second pass from right to left to multiply by the suffix products. This achieves $O(N)$ time and $O(1)$ extra space (excluding output). Should I code this?"

```

1 def productExceptSelf(nums: List[int]) -> List[int]:
2     # Initialize result array with 1s
3     res = [1] * len(nums)
4
5     # 1. Prefix Pass (Left -> Right)
6     # res[i] will contain product of nums[0]...nums[i-1]

```

```

7     prefix = 1
8     for i in range(len(nums)):
9         res[i] = prefix
10        prefix *= nums[i]
11
12    # 2. Suffix Pass (Right -> Left)
13    # Multiply existing prefix in res[i] by suffix product
14    postfix = 1
15    for i in range(len(nums) - 1, -1, -1):
16        res[i] *= postfix
17        postfix *= nums[i]
18
19    return res
20
21 """
22 5. DRY RUN (HUMAN DEBUGGER)
23     nums = [1, 2, 3, 4]
24
25     Pass 1 (Prefix):
26     - i=0: res[0]=1. pre=1*1=1.
27     - i=1: res[1]=1. pre=1*2=2.
28     - i=2: res[2]=2. pre=2*3=6.
29     - i=3: res[3]=6. pre=6*4=24.
30     - Res after P1: [1, 1, 2, 6] (Note: res[3] is product of 1,2,3).
31
32     Pass 2 (Suffix):
33     - i=3: res[3] = 6*1 = 6. post=1*4=4.
34     - i=2: res[2] = 2*4 = 8. post=4*3=12.
35     - i=1: res[1] = 1*12 = 12. post=12*2=24.
36     - i=0: res[0] = 1*24 = 24. post=24*1=24.
37     - Final: [24, 12, 8, 6]. Correct.
38 """

```

14.2 Valid Sudoku

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Is the board completely filled?
4   A: No, partial board. Ignore empty cells ".".
5 - Q: What makes it valid?
6   A: No duplicates in any Row, Col, or 3x3 Box.
7 - Q: Do I need to solve it?
8   A: No, just validate current state.
9
10 2. APPROACHES TO CONSIDER & REJECT
11 - Approach 1: Iteration with Lists.
12   - Logic: Check every row, then every col, then every box.
13   - Complexity: 3 passes.  $O(3 * N^2)$ .
14   - Trade-off: Code is repetitive/messy.
15 - Approach 2: Single Pass with Hash Sets.
16   - Logic: Iterate board once. Maintain sets for rows, cols, boxes.
17   - Key: Box index =  $(r // 3) * 3 + (c // 3)$ .
18   - Complexity:  $O(N^2)$  Time,  $O(N^2)$  Space.
19   - Accept: Cleanest implementation.
20
21 3. INDENTED ENGLISH (LOGIC)
22 function isValidSudoku(board):
23     rows = [set() for 9]
24     cols = [set() for 9]
25     boxes = [set() for 9]
26
27     for r in 0..8:
28         for c in 0..8:
29             val = board[r][c]
30             if val == ".": continue
31
32             box_idx = (r // 3) * 3 + (c // 3)
33             if val in rows[r] or val in cols[c] or val in boxes[box_idx]:
34                 return False
35
36             add val to rows[r], cols[c], boxes[box_idx]
37     return True
38 """
```

The Magic Question (Get Buy-In)

"I'll iterate through the board once. I'll use Hash Sets to track seen numbers for each Row, Column, and 3x3 Box. If I see a duplicate in any set, I return False immediately. $O(1)$ time since the board size is fixed."

```
1 import collections
2
3 def isValidSudoku(board: List[List[str]]) -> bool:
4     rows = collections.defaultdict(set)
5     cols = collections.defaultdict(set)
6     boxes = collections.defaultdict(set) # Key: (r // 3, c // 3)
7
8     for r in range(9):
9         for c in range(9):
10             val = board[r][c]
11
12             if val == ".":
13                 continue
14
15             # Check validity
16             # Box Key: (r // 3, c // 3) creates unique keys (0,0) to (2,2)
17             if (val in rows[r] or
18                 val in cols[c] or
19                 val in boxes[(r // 3, c // 3)]):
20                 return False
21
22             # Add to sets
23             rows[r].add(val)
```

```

24         cols[c].add(val)
25         boxes[(r // 3, c // 3)].add(val)
26
27     return True
28
29 """
30 5. DRY RUN (HUMAN DEBUGGER)
31 Board partial: (0,0)='5', (0,4)='5'.
32 - r=0, c=0. val='5'.
33   - rows[0] has '5'? No. Add.
34   - cols[0] has '5'? No. Add.
35   - box(0,0) has '5'? No. Add.
36 - ... iterate ...
37 - r=0, c=4. val='5'.
38   - rows[0] has '5'? YES. Return False.
39 """

```

14.3 Longest Consecutive Sequence

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Unsorted? A: Yes.
4 - Q: Duplicates? A: Yes.
5 - Q: Time Constraint? A: O(N). This is the hard constraint.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Sort.
9 - Logic: Sort and count streaks.
10 - Complexity: O(N log N).
11 - Reject: Violates O(N) constraint.
12 - Approach 2: Hash Set (Intelligent Sequence Start).
13 - Logic: Put all nums in Set.
14 - Iterate nums. If (n-1) is NOT in set, 'n' is the START of a sequence.
15 - While (n+length) in set, increment length.
16 - Complexity: O(N). We visit each number at most twice (once to put in set, once to
17   count).
18 - Accept: Optimal.
19
20 3. INDENTED ENGLISH (LOGIC)
21 function longestConsecutive(nums):
22     numSet = set(nums)
23     longest = 0
24
25     for n in nums:
26         # Only start counting if 'n' is the start of a sequence
27         if (n - 1) not in numSet:
28             length = 0
29             while (n + length) in numSet:
30                 length += 1
31             longest = max(longest, length)
32     return longest
33 """
```

The Magic Question (Get Buy-In)

"To achieve $O(N)$, I'll convert the array to a Hash Set for $O(1)$ lookups. I'll iterate through the set and only attempt to build a sequence if the current number is the *start* of a sequence (i.e., 'n-1' is missing). This prevents redundant counting."

```
1 def longestConsecutive(nums: List[int]) -> int:
2     numSet = set(nums)
3     longest = 0
4
5     for n in nums:
6         # Check if 'n' is the start of a sequence
7         if (n - 1) not in numSet:
8             length = 0
9             # Expand sequence
10            while (n + length) in numSet:
11                length += 1
12            longest = max(longest, length)
13
14    return longest
15
16 """
17 5. DRY RUN (HUMAN DEBUGGER)
18 nums = [100, 4, 200, 1, 3, 2]
19 - Set = {100, 4, 200, 1, 3, 2}
20 - n=100: 99 not in set. Start. 101 not in set. Len=1.
21 - n=4: 3 IS in set. Skip (not start).
22 - n=200: 199 not in set. Start. 201 not in set. Len=1.
23 - n=1: 0 not in set. Start.
24   - 1 in set? Yes.
25   - 2 in set? Yes.
26   - 3 in set? Yes.
27   - 4 in set? Yes.
28   - 5 in set? No. Len=4. Max=4.
```

```
29     - n=3: 2 in set. Skip.  
30     - n=2: 1 in set. Skip.  
31     - Return 4.  
32 """
```

14.4 Encode and Decode Strings

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Can strings contain ANY character?
4     A: Yes, including delimiters like '#' or '/'.
5   - Q: Stateless?
6     A: Yes, no preserving state between calls.
7
8 2. APPROACHES TO CONSIDER & REJECT
9   - Approach 1: Simple Delimiter (e.g., ",").
10    - Reject: Fails if the input string contains the delimiter itself.
11   - Approach 2: Escaping.
12    - Logic: Replace "#" with "##".
13    - Reject: Complex to implement and slow.
14   - Approach 3: Length-Prefix Encoding.
15    - Logic: Store "Length#String". e.g., "Hello" -> "5#Hello".
16    - Parsing: Read integer until '#', then read that many chars.
17    - Complexity: O(N) encode/decode.
18    - Accept: Standard System Design pattern.
19
20 3. INDENTED ENGLISH (LOGIC)
21 encode(strs):
22     res = ""
23     for s in strs:
24         res += str(len(s)) + "#" + s
25     return res
26
27 decode(s):
28     res = []
29     i = 0
30     while i < len(s):
31         j = i
32         while s[j] != "#": j++
33         length = int(s[i:j])
34         res.append(s[j+1 : j+1+length])
35         i = j + 1 + length
36     return res
37 """
```

The Magic Question (Get Buy-In)

"I'll use a length-prefix scheme. For every string, I'll encode it as 'lengthstring'. During decoding, I'll read the integer length until the delimiter '#', then read exactly that many characters. This handles all special characters correctly. O(N)."

```
1 class Codec:
2     def encode(self, strs: List[str]) -> str:
3         """Encodes a list of strings to a single string."""
4         res = ""
5         for s in strs:
6             # Prefix with length and delimiter
7             res += str(len(s)) + "#" + s
8         return res
9
10    def decode(self, s: str) -> List[str]:
11        """Decodes a single string to a list of strings."""
12        res, i = [], 0
13
14        while i < len(s):
15            # 1. Find the delimiter '#'
16            j = i
17            while s[j] != "#":
18                j += 1
19
20            # 2. Parse length
21            length = int(s[i:j])
22
23            # 3. Extract string
24            # Start after '#', end at start + length
```

```

25         res.append(s[j + 1 : j + 1 + length])
26
27         # 4. Move pointer to start of next segment
28         i = j + 1 + length
29
30     return res
31
32 """
33 5. DRY RUN (HUMAN DEBUGGER)
34 strs = ["Hello", "World"]
35 - Encode:
36     - "5#Hello"
37     - "5#Hello5#World"
38 - Decode:
39     - i=0. j finds '#' at 1. length=5.
40     - Extract s[2 : 2+5] -> "Hello".
41     - i moves to 2+5 = 7.
42     - i=7. j finds '#' at 8. length=5.
43     - Extract s[9 : 9+5] -> "World".
44     - i moves to 14. Loop end.
45 """

```

15 Arrays & Hashing: The Classics

15.1 Two Sum

```

1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Sorted input? A: No. (If yes, use Two Pointers).
4     - Q: One solution? A: Yes, exactly one.
5     - Q: Can I use the same element twice? A: No.
6
7 2. APPROACHES TO CONSIDER & REJECT
8     - Approach 1: Brute Force.
9         - Logic: Double loop. Check nums[i] + nums[j] == target.
10        - Complexity:  $O(N^2)$ .
11        - Reject: Too slow.
12     - Approach 2: Hash Map (One Pass).
13        - Logic: We need 'target - nums[i]'. Store 'val -> index' in map.
14        - If complement exists in map, we found the pair.
15        - Complexity:  $O(N)$  Time,  $O(N)$  Space.
16        - Accept: Optimal standard.
17
18 3. INDENTED ENGLISH (LOGIC)
19 function twoSum(nums, target):
20     prevMap = {} # val -> index
21     for i, n in nums:
22         diff = target - n
23         if diff in prevMap:
24             return [prevMap[diff], i]
25     prevMap[n] = i
26 """

```

The Magic Question (Get Buy-In)

"I'll use a Hash Map to store the values and indices I've seen so far. For each number, I check if 'target - number' exists in the map. This allows me to find the pair in one pass, $O(N)$ time. Should I code this?"

```

1 def twoSum(nums: List[int], target: int) -> List[int]:
2     prevMap = {} # val : index
3
4     for i, n in enumerate(nums):
5         diff = target - n
6         if diff in prevMap:
7             return [prevMap[diff], i]
8         prevMap[n] = i
9     return []

```

```
10
11 """
12 5. DRY RUN (HUMAN DEBUGGER)
13    nums=[2, 7, 11, 15], target=9
14    - i=0, n=2. diff=7. Map empty. Map={2:0}.
15    - i=1, n=7. diff=2. 2 in Map? Yes (index 0).
16    - Return [0, 1].
17 """
```

15.2 Group Anagrams

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Empty strings? A: Group them together.
4   - Q: Output order? A: Doesn't matter.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Sorting Strings.
8     - Logic: Map key = sorted(string). Value = list of strings.
9     - Complexity:  $O(N * K \log K)$ .
10    - Trade-off: Acceptable, but we can optimize the key generation.
11   - Approach 2: Character Count Array (Tuple).
12     - Logic: Map key = tuple of 26 character counts (0-25).
13     - Complexity:  $O(N * K)$ .
14     - Accept: Optimal "Strong Hire" solution (removes the log K factor).
15
16 3. INDENTED ENGLISH (LOGIC)
17 function groupAnagrams(strs):
18     res = defaultdict(list)
19     for s in strs:
20         count = [0] * 26
21         for c in s:
22             count[ord(c) - ord('a')] += 1
23         res[tuple(count)].append(s)
24     return res.values()
25 """
```

The Magic Question (Get Buy-In)

"Instead of sorting each string which takes $O(K \log K)$, I'll use a tuple of character counts as the Hash Map key. This reduces the key generation to $O(K)$, giving a total time complexity of $O(N \cdot K)$. Agreed?"

```
1 def groupAnagrams(strs: List[str]) -> List[List[str]]:
2     # Key: Tuple of 26 counts, Value: List of anagrams
3     ans = collections.defaultdict(list)
4
5     for s in strs:
6         count = [0] * 26
7         for c in s:
8             count[ord(c) - ord("a")] += 1
9
10        # Lists are mutable (unhashable), so convert to tuple for Key
11        ans[tuple(count)].append(s)
12
13    return list(ans.values())
14
15 """
16 5. DRY RUN (HUMAN DEBUGGER)
17 strs = ["eat", "tea", "tan"]
18 - "eat": count[a=1, e=1, t=1...]. Key=(1,0,0,0,1...). Map={Key1: ["eat"]}
19 - "tea": count[same]. Key=Key1. Map={Key1: ["eat", "tea"]}
20 - "tan": count[a=1, n=1, t=1...]. Key=Key2. Map={..., Key2: ["tan"]}
21 """
```

15.3 Top K Frequent Elements

```

1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Is answer unique? A: Yes.
4   - Q: Can  $O(N \log N)$  work? A: Yes, but  $O(N)$  is the goal.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Sort by Frequency.
8     - Complexity:  $O(N \log N)$ .
9     - Reject: Naive.
10  - Approach 2: Max Heap.
11    - Logic: Count freq, push to Heap. Pop k times.
12    - Complexity:  $O(N + k \log N)$ .
13    - Accept: Good standard answer.
14  - Approach 3: Bucket Sort (Frequency Buckets).
15    - Logic: Array where index = frequency. value = list of numbers.
16    - Max frequency is N. Iterate buckets backwards.
17    - Complexity:  $O(N)$ .
18    - Accept: Optimal "Strong Hire" answer.
19
20 3. INDENTED ENGLISH (LOGIC)
21  function topKFrequent(nums, k):
22      count = Counter(nums)
23      freq = [[] for _ in range(len(nums) + 1)]
24
25      for n, c in count:
26          freq[c].append(n)
27
28      res = []
29      for i from N down to 1:
30          for n in freq[i]:
31              res.append(n)
32              if len(res) == k: return res
33 """

```

The Magic Question (Get Buy-In)

"I can use a Heap for $O(N \log K)$, but since the maximum frequency is bounded by N , I can use Bucket Sort to achieve linear $O(N)$ time. I'll map frequency indices to lists of numbers. Should I code the linear approach?"

```

1 def topKFrequent(nums: List[int], k: int) -> List[int]:
2     count = {}
3     # Index is frequency, Value is list of numbers with that frequency
4     freq = [[] for _ in range(len(nums) + 1)]
5
6     for n in nums:
7         count[n] = 1 + count.get(n, 0)
8
9     for n, c in count.items():
10        freq[c].append(n)
11
12    res = []
13    # Iterate backwards (highest frequency first)
14    for i in range(len(freq) - 1, 0, -1):
15        for n in freq[i]:
16            res.append(n)
17            if len(res) == k:
18                return res
19    return []
20
21 """
22 5. DRY RUN (HUMAN DEBUGGER)
23  nums=[1,1,1,2,2,3], k=2
24  - Count: {1:3, 2:2, 3:1}
25  - Freq: size 7. freq[3]=[1], freq[2]=[2], freq[1]=[3]
26  - Loop backwards:
27    - i=6..4: empty
28    - i=3: res=[1]. len!=k.

```

```
29         - i=2: res=[1,2]. len==k. Return.  
30     """
```

15.4 Valid Anagram

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Case sensitive? A: Yes.
4 - Q: Spaces? A: Ignore or count? Usually count in interview.
5 - Q: Unicode? A: If yes, use HashMap. If ASCII, array[26] is slightly faster.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Sort both strings.
9   - Check sorted(s) == sorted(t).
10  - Complexity:  $O(N \log N)$ .
11  - Reject: Too slow.
12 - Approach 2: Hash Map / Array Count.
13   - Logic: Count chars in S (+1), count in T (-1). Check all zero.
14   - Complexity:  $O(N)$ .
15   - Accept: Optimal.
16
17 3. INDENTED ENGLISH (LOGIC)
18 function isAnagram(s, t):
19     if len(s) != len(t): return False
20     countS, countT = {}, {}
21     for i in range(len(s)):
22         countS[s[i]] += 1
23         countT[t[i]] += 1
24     return countS == countT
25 """
```

The Magic Question (Get Buy-In)

"I'll verify the lengths match, then use a frequency map to count characters in both strings. If the maps are identical, it's an anagram. This is $O(N)$ time and $O(1)$ space (since character set is fixed)."

```
1 def isAnagram(s: str, t: str) -> bool:
2     if len(s) != len(t):
3         return False
4
5     countS, countT = {}, {}
6
7     for i in range(len(s)):
8         countS[s[i]] = 1 + countS.get(s[i], 0)
9         countT[t[i]] = 1 + countT.get(t[i], 0)
10
11     return countS == countT
12
13 """
14 5. DRY RUN (HUMAN DEBUGGER)
15 s="rat", t="car"
16 - len equal? Yes.
17 - S: {r:1, a:1, t:1}
18 - T: {c:1, a:1, r:1}
19 - Compare: 't' in S not in T. Return False.
20 """
```

15.5 Sort Colors

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Integers 0, 1, 2 only? A: Yes.
4   - Q: Built-in sort allowed? A: No.
5   - Q: One pass? A: Yes, "Strong Hire" constraint.
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: Count Sort (Two Pass).
9     - Logic: Count 0s, 1s, 2s. Overwrite array.
10    - Complexity:  $O(N)$ .
11    - Trade-off: Valid, but 2 passes.
12   - Approach 2: Dutch National Flag (One Pass).
13     - Logic: 3 Pointers. L (end of 0s), R (start of 2s), i (scanner).
14     - If 0: swap(i, L), L++, i++.
15     - If 2: swap(i, R), R-- (Don't increment i, check swapped val).
16     - If 1: i++.
17     - Complexity:  $O(N)$  One pass.
18     - Accept: Optimal.
19
20 3. INDENTED ENGLISH (LOGIC)
21 function sortColors(nums):
22     l, r = 0, len-1
23     i = 0
24     while i <= r:
25         if nums[i] == 0:
26             swap(i, l)
27             l++, i++
28         elif nums[i] == 2:
29             swap(i, r)
30             r-- # don't i++
31         else:
32             i++
33 """
```

The Magic Question (Get Buy-In)

"I'll use the Dutch National Flag algorithm with three pointers: 'l' for 0s, 'r' for 2s, and 'i' to scan. This sorts the array in-place in a single pass $O(N)$."

```
1 def sortColors(nums: List[int]) -> None:
2     l, r = 0, len(nums) - 1
3     i = 0
4
5     def swap(a, b):
6         nums[a], nums[b] = nums[b], nums[a]
7
8     while i <= r:
9         if nums[i] == 0:
10             swap(l, i)
11             l += 1
12             # Since we swapped from the left (processed), we know it's a 1 or 0
13             i += 1
14         elif nums[i] == 2:
15             swap(i, r)
16             r -= 1
17             # Don't increment i. The value swapped from r is unknown.
18         else:
19             i += 1
20
21 """
22 5. DRY RUN (HUMAN DEBUGGER)
23 nums = [2, 0, 2, 1, 1, 0]
24 - i=0, val=2: swap(0, 5). r=4. nums=[0, 0, 2, 1, 1, 2]
25 - i=0, val=0: swap(0, 0). l=1, i=1. nums=[0, 0, 2, 1, 1, 2]
26 - i=1, val=0: swap(1, 1). l=2, i=2. nums=[0, 0, 2, 1, 1, 2]
27 - i=2, val=2: swap(2, 4). r=3. nums=[0, 0, 1, 1, 2, 2]
28 - i=2, val=1: i=3.
29 - i=3, val=1: i=4.
```

```

30 - i=4 > r(3). Stop.
31 """

```

16 Arrays & Hashing: Hard Patterns & Design

16.1 First Missing Positive

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Time complexity? A: O(N).
4 - Q: Space complexity? A: O(1) auxiliary. This is the hard part.
5 - Q: Input range? A: Unsorted integers, negatives allowed.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Sort.
9   - Logic: Sort and find first gap.
10  - Complexity: O(N log N).
11  - Reject: Too slow.
12 - Approach 2: Hash Set.
13   - Logic: Add all to set. Check 1, 2, 3...
14   - Complexity: O(N) Space.
15   - Reject: Violates O(1) space constraint.
16 - Approach 3: Index as Hash Key (Cyclic Sort logic).
17   - Logic: We want num 'x' to be at index 'x-1'.
18   - Iterate and swap numbers to their correct positions.
19   - Pass 2: Find first index i where nums[i] != i+1.
20   - Complexity: O(N) Time, O(1) Space.
21   - Accept: Optimal.
22
23 3. INDENTED ENGLISH (LOGIC)
24 function firstMissingPositive(nums):
25     N = len(nums)
26     # Pass 1: Place numbers in correct spots
27     for i in range(N):
28         while 0 < nums[i] <= N and nums[nums[i]-1] != nums[i]:
29             swap(nums[i], nums[nums[i]-1])
30
31     # Pass 2: Find mismatch
32     for i in range(N):
33         if nums[i] != i + 1:
34             return i + 1
35     return N + 1
36 """

```

The Magic Question (Get Buy-In)

"To achieve $O(N)$ time and $O(1)$ space, I'll use the array indices themselves as a Hash Map. I'll swap each number 'x' to the index 'x-1' (if valid). Then I'll iterate to find the first index that doesn't match. Should I code this?"

```

1 def firstMissingPositive(nums: List[int]) -> int:
2     n = len(nums)
3
4     # 1. Place numbers in correct spots (Cyclic Sort logic)
5     for i in range(n):
6         # Loop while the current number:
7         # a) Is in valid range [1, n]
8         # b) Is NOT already in the correct spot
9         while 0 < nums[i] <= n and nums[nums[i] - 1] != nums[i]:
10             # Swap nums[i] with the number at target index
11             target_idx = nums[i] - 1
12             nums[i], nums[target_idx] = nums[target_idx], nums[i]
13
14     # 2. Find the first missing number
15     for i in range(n):
16         if nums[i] != i + 1:
17             return i + 1
18

```

```

19     # Edge Case: All numbers 1..n are present
20     return n + 1
21
22 """
23 5. DRY RUN (HUMAN DEBUGGER)
24     nums = [3, 4, -1, 1]
25     - i=0 (3): Target idx 2. Swap 3 & -1. nums=[-1, 4, 3, 1].
26       - -1 invalid. Stop while.
27     - i=1 (4): Target idx 3. Swap 4 & 1. nums=[-1, 1, 3, 4].
28       - Now nums[i]=1. Target idx 0. Swap 1 & -1. nums=[1, -1, 3, 4].
29       - nums[i]=-1. Invalid. Stop.
30     - i=2 (3): Correct spot.
31     - i=3 (4): Correct spot.
32     - Final nums: [1, -1, 3, 4].
33     - Scan: i=0(1==1), i=1(-1!=2). Return 2.
34 """

```

16.2 Subarray Sum Equals K

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Negative numbers? A: Yes (so Sliding Window won't work).
4   - Q: Output? A: Total count of subarrays.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Brute Force.
8     - Logic: Check all subarrays.
9     - Complexity:  $O(N^2)$ .
10    - Reject: TLE.
11   - Approach 2: Sliding Window.
12     - Reject: Only works for positive numbers (monotonic sums).
13   - Approach 3: Prefix Sum + Hash Map.
14     - Logic:  $\text{curSum} - \text{oldSum} = k \Rightarrow \text{oldSum} = \text{curSum} - k$ .
15     - We need to know how many times 'oldSum' has appeared before.
16     - Map: { prefix_sum : count }.
17     - Complexity:  $O(N)$ .
18     - Accept: Optimal.
19
20 3. INDENTED ENGLISH (LOGIC)
21   function subarraySum(nums, k):
22       count = 0
23       curSum = 0
24       prefixMap = {0 : 1} # Base case: sum 0 occurs once (empty array)
25
26       for n in nums:
27           curSum += n
28           diff = curSum - k
29           if diff in prefixMap:
30               count += prefixMap[diff]
31           prefixMap[curSum] += 1
32       return count
33 """
```

The Magic Question (Get Buy-In)

"Since input contains negatives, Sliding Window fails. I'll use the Prefix Sum Hash Map pattern. The equation 'CurrentSum - PrefixSum = k' implies I need to find how many times 'CurrentSum - k' has occurred previously. $O(N)$."

```
1 def subarraySum(nums: List[int], k: int) -> int:
2     res = 0
3     curSum = 0
4     # Map prefix_sum -> count of occurrences
5     # Init with {0:1} to handle case where a subarray starting at index 0 equals k
6     prefixCounts = {0: 1}
7
8     for n in nums:
9         curSum += n
10        diff = curSum - k
11
12        # If (curSum - k) exists in map, it means there are subarrays
13        # ending here that sum to k
14        res += prefixCounts.get(diff, 0)
15
16        # Add current sum to map
17        prefixCounts[curSum] = 1 + prefixCounts.get(curSum, 0)
18
19    return res
20
21 """
22 5. DRY RUN (HUMAN DEBUGGER)
23   nums=[1, -1, 1], k=1
24   - Init: map={0:1}, sum=0, res=0
25   - n=1: sum=1. diff=0. map[0]=1. res+=1. map={0:1, 1:1}
26   - n=-1: sum=0. diff=-1. map[-1]=0. res+=0. map={0:2, 1:1}
27   - n=1: sum=1. diff=0. map[0]=2. res+=2 (Total 3). map={...}
28   - Return 3. ([1], [1,-1,1], [-1,1]? No wait [1] at end).
```


16.3 Majority Element

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Definition? A: Element appearing > n/2 times.
4   - Q: Guaranteed to exist? A: Yes.
5   - Q: O(1) Space? A: Yes, "Strong Hire".
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: Hash Map.
9     - Complexity: O(N) Time, O(N) Space.
10    - Reject: Trivial.
11   - Approach 2: Sorting.
12     - Logic: Return nums[n//2].
13     - Complexity: O(N log N).
14     - Reject: Slow.
15   - Approach 3: Boyer-Moore Voting Algorithm.
16     - Logic: Count variable. If count==0, new candidate. Same? count++. Different?
17       count--.
18     - Complexity: O(N) Time, O(1) Space.
19     - Accept: Optimal.
20
21 3. INDENTED ENGLISH (LOGIC)
22   function majorityElement(nums):
23     res, count = 0, 0
24     for n in nums:
25       if count == 0: res = n
26       count += (1 if n == res else -1)
27   return res
28 """
```

The Magic Question (Get Buy-In)

"I'll use the Boyer-Moore Voting algorithm. I maintain a candidate and a counter. If the counter drops to zero, I pick a new candidate. This finds the majority element in $O(N)$ time and $O(1)$ space."

```
1 def majorityElement(nums: List[int]) -> int:
2     res = 0
3     count = 0
4
5     for n in nums:
6         if count == 0:
7             res = n
8
9         # Increment if matches candidate, decrement otherwise
10        count += (1 if n == res else -1)
11
12    return res
13
14 """
15 5. DRY RUN (HUMAN DEBUGGER)
16    nums=[2, 2, 1, 1, 1, 2, 2]
17    - n=2: count=0 -> res=2. count=1.
18    - n=2: matches. count=2.
19    - n=1: differs. count=1.
20    - n=1: differs. count=0.
21    - n=1: count=0 -> res=1. count=1.
22    - n=2: differs. count=0.
23    - n=2: count=0 -> res=2. count=1.
24    - Return 2.
25 """
```

16.4 Range Sum Query 2D - Immutable

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Many queries? A: Yes, optimize for fast retrieval.
4 - Q: Matrix changes? A: No (Immutable).
5
6 2. APPROACHES TO CONSIDER & REJECT
7 - Approach 1: Brute Force loop.  $O(N*M)$  per query. Reject.
8 - Approach 2: 2D Prefix Sum.
9   - Logic: Precompute sum from (0,0) to (r,c).
10  - Formula:  $\text{Sum}(r1, c1, r2, c2) =$ 
11    $P(r2, c2) - P(r1-1, c2) - P(r2, c1-1) + P(r1-1, c1-1).$ 
12  - Complexity:  $O(1)$  per query.
13  - Accept: Optimal.
14
15 3. INDENTED ENGLISH (LOGIC)
16 init(matrix):
17     P = matrix + extra row/col
18     for r, c:
19         P[r][c] = val + left + top - diag_overlap
20 sumRegion(r1, c1, r2, c2):
21     return P[r2][c2] - left - top + diag_overlap
22 """
```

The Magic Question (Get Buy-In)

"I'll precompute a 2D Prefix Sum matrix. 'prefix[r][c]' will store the sum of the rectangle from '(0,0)' to '(r,c)'. This allows calculating any sub-rectangle sum in $O(1)$ time using the inclusion-exclusion principle."

```
1 class NumMatrix:
2
3     def __init__(self, matrix: List[List[int]]):
4         ROWS, COLS = len(matrix), len(matrix[0])
5         # Pad with extra row/col to avoid boundary checks
6         self.sumMat = [[0] * (COLS + 1) for _ in range(ROWS + 1)]
7
8         for r in range(ROWS):
9             for c in range(COLS):
10                 # Current + Top + Left - TopLeft (avoid double counting)
11                 self.sumMat[r + 1][c + 1] = (matrix[r][c] +
12                                                self.sumMat[r][c + 1] +
13                                                self.sumMat[r + 1][c] -
14                                                self.sumMat[r][c])
15
16     def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
17         r1, c1, r2, c2 = row1 + 1, col1 + 1, row2 + 1, col2 + 1
18
19         return (self.sumMat[r2][c2] -
20                 self.sumMat[r1 - 1][c2] -
21                 self.sumMat[r2][c1 - 1] +
22                 self.sumMat[r1 - 1][c1 - 1])
```

16.5 Design HashSet

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Can I use built-in dict? A: No.
4   - Q: Collision handling? A: Chaining (Linked List) or Open Addressing.
5   - Q: Size? A: e.g., 104 ops.
6
7 2. APPROACHES
8   - Logic: Array of Buckets.
9   - Hash Function: key % len(buckets).
10  - Each bucket is a list (or dummy node for LL).
11  - Add: Append if not present. Remove: Delete from list. Contains: Scan list.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use the Chaining method. I'll create a fixed-size array of buckets (e.g., 1000).
16 # Each bucket will hold a list of keys to handle collisions. The hash function will be '
17 #   key % size'.
18 # Should I implement this?"
19
20 class MyHashSet:
21     def __init__(self):
22         self.size = 1000
23         self.buckets = [[] for _ in range(self.size)]
24
25     def _hash(self, key):
26         return key % self.size
27
28     def add(self, key: int) -> None:
29         idx = self._hash(key)
30         if key not in self.buckets[idx]:
31             self.buckets[idx].append(key)
32
33     def remove(self, key: int) -> None:
34         idx = self._hash(key)
35         if key in self.buckets[idx]:
36             self.buckets[idx].remove(key)
37
38     def contains(self, key: int) -> bool:
39         idx = self._hash(key)
40         return key in self.buckets[idx]
```

16.6 Longest Common Prefix

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Empty list? A: Return "".
4   - Q: Only lowercase? A: Yes.
5
6 2. APPROACHES
7   - Approach 1: Horizontal. P = strs[0]. P = LCP(P, strs[1])...
8   - Approach 2: Vertical Scanning.
9     - Logic: Check column 0 for all strings. If match, check col 1.
10    - Stop if mismatch or end of shortest string.
11    - Complexity: O(S) where S is sum of all chars.
12    - Accept: Cleanest.
13
14 3. INDENTED ENGLISH
15 function longestCommonPrefix(strs):
16     for i in range(len(strs[0])):
17         char = strs[0][i]
18         for s in strs:
19             if i == len(s) or s[i] != char:
20                 return strs[0][:i]
21     return strs[0]
22 """
23
24 # --- THE MAGIC QUESTION ---
25 # "I'll use Vertical Scanning. I'll take the first string as a reference and iterate
26 # through its characters.
27 # For each character, I'll verify if it matches in all other strings. The first mismatch
28 # defines the prefix length."
29
30 def longestCommonPrefix(strs: List[str]) -> str:
31     res = ""
32
33     # Iterate through chars of the first string
34     for i in range(len(strs[0])):
35         char = strs[0][i]
36
37         for s in strs:
38             # Check bounds OR mismatch
39             if i == len(s) or s[i] != char:
40                 return res
41
42         res += char
43
44     return res
```

16.7 Sort an Array

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Constraints? A: N=5*10^4. O(N log N) required.
4   - Q: Built-in sort? A: No, implement one.
5   - Q: Stable? A: Merge Sort is stable. Quick Sort is not.
6
7 2. LOGIC (MERGE SORT)
8   - Divide: Split array in half.
9   - Conquer: Recursively sort halves.
10  - Combine: Merge two sorted halves into one.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll implement Merge Sort. It guarantees  $O(N \log N)$  time complexity even in the
15 # and is a stable sort. I'll split the array recursively and merge the sorted halves."
16
17 def sortArray(nums: List[int]) -> List[int]:
18
19     def merge(left, right):
20         res = []
21         i, j = 0, 0
22         while i < len(left) and j < len(right):
23             if left[i] < right[j]:
24                 res.append(left[i])
25                 i += 1
26             else:
27                 res.append(right[j])
28                 j += 1
29         res.extend(left[i:])
30         res.extend(right[j:])
31         return res
32
33     def mergeSort(arr):
34         if len(arr) <= 1:
35             return arr
36
37         mid = len(arr) // 2
38         left = mergeSort(arr[:mid])
39         right = mergeSort(arr[mid:])
40
41         return merge(left, right)
42
43     return mergeSort(nums)
```

17 Arrays & Hashing: Final Missing Pieces

17.1 Concatenation of Array

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: New array? A: Yes, length 2n.
4   - Q: Reference or Copy? A: Copy (distinct memory).
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Loop twice.
8     - Logic: Iterate nums, append to ans. Iterate nums again, append.
9     - Complexity: O(N).
10    - Accept: Standard.
11   - Approach 2: Array extension (Language specific).
12     - Logic: ans = nums + nums.
13     - Complexity: O(N).
14     - Accept: Pythonic/Optimal.
15
16 3. INDENTED ENGLISH (LOGIC)
17     function getConcatenation(nums):
18         ans = []
19         for i in range(2):
```

```

20         for n in nums:
21             ans.append(n)
22     return ans
23 """

```

The Magic Question (Get Buy-In)

"I'll create a new array of size $2n$. I can simply traverse the input array twice (or extend it by itself) to populate the result. This runs in $O(N)$ time."

```

1 def getConcatenation(nums: List[int]) -> List[int]:
2     # Pythonic way: Create a new list combining nums with itself
3     return nums + nums
4
5     # Alternative (Instructional):
6     # ans = []
7     # for i in range(2):
8     #     for n in nums:
9     #         ans.append(n)
10    # return ans
11
12 """
13 5. DRY RUN (HUMAN DEBUGGER)
14     nums = [1, 2]
15     - nums + nums -> [1, 2] + [1, 2]
16     - Result: [1, 2, 1, 2]
17 """

```

17.2 Contains Duplicate

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q:  $O(N^2)$  allowed? A: No.
4   - Q: Space limit? A:  $O(N)$  is fine.
5
6 2. APPROACHES TO CONSIDER & REJECT
7   - Approach 1: Brute Force.  $O(N^2)$ . Reject.
8   - Approach 2: Sorting.
9     - Logic: Sort and check adjacent.
10    - Complexity:  $O(N \log N)$ .
11    - Reject: Slower than  $O(N)$ .
12   - Approach 3: Hash Set.
13     - Logic: Add to set. If already in set, return True.
14     - Complexity:  $O(N)$  Time,  $O(N)$  Space.
15     - Accept: Optimal.
16
17 3. INDENTED ENGLISH (LOGIC)
18     function containsDuplicate(nums):
19         seen = set()
20         for n in nums:
21             if n in seen: return True
22             seen.add(n)
23         return False
24 """
```

The Magic Question (Get Buy-In)

"I'll use a Hash Set to track numbers I've seen. As I iterate, I check if the current number is already in the set. This gives $O(N)$ time complexity."

```
1 def containsDuplicate(nums: List[int]) -> bool:
2     seen = set()
3
4     for n in nums:
5         if n in seen:
6             return True
7         seen.add(n)
8
9     return False
10
11 """
12 5. DRY RUN (HUMAN DEBUGGER)
13     nums = [1, 2, 3, 1]
14     - n=1: seen={1}
15     - n=2: seen={1, 2}
16     - n=3: seen={1, 2, 3}
17     - n=1: 1 in seen? Yes. Return True.
18 """
```

17.3 Remove Element

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: In-place? A: Yes, O(1) space.
4   - Q: Return? A: New length k.
5   - Q: Order? A: Can be changed.
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: New Array. Reject (Must be in-place).
9   - Approach 2: Two Pointers (Reader/Writer).
10    - Logic: 'i' reads, 'k' writes.
11    - If nums[i] != val, we keep it: nums[k] = nums[i], k++.
12    - If nums[i] == val, we skip it.
13    - Complexity: O(N).
14    - Accept: Optimal.
15
16 3. INDENTED ENGLISH (LOGIC)
17   function removeElement(nums, val):
18       k = 0
19       for i in range(len(nums)):
20           if nums[i] != val:
21               nums[k] = nums[i]
22               k++
23       return k
24 """
```

The Magic Question (Get Buy-In)

"I'll use a 'Writer' pointer 'k'. I'll iterate through the array, and whenever I encounter a value that is NOT the target, I'll overwrite 'nums[k]' with it and increment 'k'. This effectively filters out the target in-place."

```
1 def removeElement(nums: List[int], val: int) -> int:
2     k = 0 # The write pointer
3
4     for i in range(len(nums)):
5         if nums[i] != val:
6             nums[k] = nums[i]
7             k += 1
8
9     return k
10
11 """
12 5. DRY RUN (HUMAN DEBUGGER)
13   nums=[3, 2, 2, 3], val=3
14   - i=0(3): == val. Skip. k=0.
15   - i=1(2): != val. nums[0]=2. k=1. Array=[2, 2, 2, 3].
16   - i=2(2): != val. nums[1]=2. k=2. Array=[2, 2, 2, 3].
17   - i=3(3): == val. Skip.
18   - Return k=2. (First 2 elements are [2, 2]).
19 """
```

17.4 Design HashMap

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: No built-in dict? A: Correct.
4 - Q: Key type? A: Integers.
5 - Q: Collision Strategy? A: Chaining (Linked List).
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: Big Array.
9   - Logic: Array of size 10^6.
10  - Reject: Memory inefficient if keys are sparse.
11 - Approach 2: Array of Buckets + Chaining.
12   - Logic: Hash = key % 1000.
13   - Bucket is a list of (key, value) pairs.
14   - Put: Update if exists, else append.
15   - Get: Scan bucket.
16   - Complexity: O(N/K) -> O(1) avg.
17   - Accept: Standard.
18
19 3. INDENTED ENGLISH (LOGIC)
20 class MyHashMap:
21     buckets = [[] for _ in range(1000)]
22
23     def put(self, key, val):
24         bucket = buckets[key % 1000]
25         for i, (k, v) in enumerate(bucket):
26             if k == key:
27                 bucket[i] = (key, val)
28                 return
29         bucket.append((key, val))
30
31     def get(self, key):
32         bucket = buckets[key % 1000]
33         for k, v in bucket:
34             if k == key: return v
35         return -1
36 """
```

The Magic Question (Get Buy-In)

"I'll use Chaining with an array of buckets. Each bucket will store a list of '(key, value)' pairs to handle collisions. My hash function will be 'key

```
1 class MyHashMap:
2
3     def __init__(self):
4         self.size = 1000
5         self.buckets = [[] for _ in range(self.size)]
6
7     def _hash(self, key):
8         return key % self.size
9
10    def put(self, key: int, value: int) -> None:
11        idx = self._hash(key)
12        bucket = self.buckets[idx]
13
14        # Update existing
15        for i, (k, v) in enumerate(bucket):
16            if k == key:
17                bucket[i] = (key, value)
18                return
19
20        # Or append new
21        bucket.append((key, value))
22
23    def get(self, key: int) -> int:
24        idx = self._hash(key)
25        bucket = self.buckets[idx]
26
```

```
27         for k, v in bucket:
28             if k == key:
29                 return v
30         return -1
31
32     def remove(self, key: int) -> None:
33         idx = self._hash(key)
34         bucket = self.buckets[idx]
35
36         for i, (k, v) in enumerate(bucket):
37             if k == key:
38                 del bucket[i]
39         return
```

17.5 Best Time to Buy and Sell Stock II

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Multiple transactions? A: Yes.
4   - Q: Simultaneous? A: No, sell before buying again.
5   - Q: Strategy? A: Capture every upward slope.
6
7 2. APPROACHES TO CONSIDER & REJECT
8   - Approach 1: DP.
9     - Logic: Buy/Sell states.
10    - Complexity:  $O(N)$ .
11    - Accept: Good, but overkill.
12   - Approach 2: Greedy.
13     - Logic: If  $price[i] > price[i-1]$ , we buy at  $i-1$  and sell at  $i$ .
14     - Why? Sum of parts equals whole.  $(5-1) = (5-3) + (3-1)$ .
15     - Complexity:  $O(N)$ .
16     - Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19   function maxProfit(prices):
20       profit = 0
21       for i from 1 to len:
22           if prices[i] > prices[i-1]:
23               profit += prices[i] - prices[i-1]
24       return profit
25 """
```

The Magic Question (Get Buy-In)

"Since I can make unlimited transactions, the optimal strategy is Greedy: simply capture every daily price increase. If today's price is higher than yesterday's, I add the difference to my profit. $O(N)$."

```
1 def maxProfit(prices: List[int]) -> int:
2     profit = 0
3
4     for i in range(1, len(prices)):
5         # If price went up, capture the gain immediately
6         if prices[i] > prices[i - 1]:
7             profit += prices[i] - prices[i - 1]
8
9     return profit
10
11 """
12 5. DRY RUN (HUMAN DEBUGGER)
13   prices = [7, 1, 5, 3, 6, 4]
14   - 1 > 7? No.
15   - 5 > 1? Yes. Profit += 4. (Buy 1, Sell 5)
16   - 3 > 5? No.
17   - 6 > 3? Yes. Profit += 3. (Buy 3, Sell 6) -> Total 7.
18   - 4 > 6? No.
19   - Return 7.
20 """
```

17.6 Majority Element II

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Definition? A: Elements appearing > n/3 times.
4 - Q: Max such elements? A: At most 2. (Since 3 * (n/3 + 1) > n).
5 - Q: O(1) Space? A: Yes, required.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: HashMap.
9 - Complexity: O(N) Space.
10 - Reject: Space constraint.
11 - Approach 2: Boyer-Moore Voting (Extended).
12 - Logic: Maintain 2 candidates and 2 counters.
13 - Pass 1: Find potential candidates.
14 - Pass 2: Verify counts (mandatory, as candidates might be false positives).
15 - Complexity: O(N) Time, O(1) Space.
16 - Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19 function majorityElement(nums):
20     cand1, cnt1 = None, 0
21     cand2, cnt2 = None, 0
22     # Pass 1: Find Candidates
23     for n in nums:
24         if n == cand1: cnt1++
25         elif n == cand2: cnt2++
26         elif cnt1 == 0: cand1, cnt1 = n, 1
27         elif cnt2 == 0: cand2, cnt2 = n, 1
28         else: cnt1--, cnt2--
29
30     # Pass 2: Verify
31     res = []
32     if nums.count(cand1) > n//3: res.append(cand1)
33     if nums.count(cand2) > n//3: res.append(cand2)
34     return res
35 """
```

The Magic Question (Get Buy-In)

"Since we need elements appearing more than $N/3$ times, there can be at most 2 such elements. I'll use the Extended Boyer-Moore algorithm with two candidates and two counters. I'll filter candidates in a first pass and verify their counts in a second pass. $O(N)$ Time, $O(1)$ Space."

```
1 def majorityElement(nums: List[int]) -> List[int]:
2     cand1, count1 = None, 0
3     cand2, count2 = None, 0
4
5     # Pass 1: Find potential candidates
6     for n in nums:
7         if n == cand1:
8             count1 += 1
9         elif n == cand2:
10            count2 += 1
11        elif count1 == 0:
12            cand1, count1 = n, 1
13        elif count2 == 0:
14            cand2, count2 = n, 1
15        else:
16            # Both counts decrease
17            count1 -= 1
18            count2 -= 1
19
20    # Pass 2: Verify candidates
21    res = []
22    threshold = len(nums) // 3
23
24    # We must reset counts to verify actual frequency
25    if nums.count(cand1) > threshold:
26        res.append(cand1)
```

```

27     if nums.count(cand2) > threshold:
28         # Check cand2 != cand1 (handles small inputs implicitly usually, but good
29         hygiene)
30         if cand2 != cand1:
31             res.append(cand2)
32
33     return res
34
35 """
36 5. DRY RUN (HUMAN DEBUGGER)
37 nums = [3, 2, 3]
38 - n=3: c1=3, cnt1=1.
39 - n=2: c2=2, cnt2=1.
40 - n=3: matches c1. cnt1=2.
41 - Candidates: 3, 2.
42 - Verify: count(3)=2 > 1. Add 3. count(2)=1 != 1.
43 - Return [3].
44 """

```

18 Backtracking: The Decision Tree Blueprint

The Backtracking Template

Every backtracking function needs three things:

- **State:** Where are we? (index, current_path).
- **Base Case:** Are we done? (index == n). Add copy of path to result.
- **Recursive Step:**
 1. **Choose:** Add element to path.
 2. **Explore:** Call recursive function (next index).
 3. **Unchoose:** Pop element from path (Backtrack).

18.1 Subsets (The "Include/Exclude" Pattern)

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Duplicates in input? A: No (distinct integers).
4 - Q: Order matters? A: No, [1,2] is same as [2,1].
5 - Q: Empty set included? A: Yes.
6
7 2. LOGIC (DECISION TREE)
8 - For every number, we have exactly TWO choices:
9     1. Include it in the current subset.
10    2. Exclude it (don't add it).
11 - Tree Depth = N. Branches = 2. Complexity = O(2^N).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use a standard Depth First Search (DFS) approach. At each index, I'll branch
16 # twice:
17 # once including the current number and once skipping it. Time complexity is O(2^N).
18 # Agreed?"
19
20 def subsets(nums: List[int]) -> List[List[int]]:
21     res = []
22     subset = []
23
24     def dfs(i):
25         # Base Case: Processed all numbers
26         if i >= len(nums):
27             res.append(subset.copy()) # Important: Copy the mutable list
28             return

```

```

28     # Branch 1: Include nums[i]
29     subset.append(nums[i])
30     dfs(i + 1)
31
32     # Branch 2: Exclude nums[i] (Backtrack)
33     subset.pop()
34     dfs(i + 1)
35
36     dfs(0)
37     return res
38
39 """
40 5. DRY RUN
41 nums = [1, 2]
42 - dfs(0): Include 1. sub=[1].
43   - dfs(1): Include 2. sub=[1, 2].
44     - dfs(2): Base case. Add [1, 2]. Return.
45   - Backtrack: Pop 2. sub=[1].
46 - dfs(1): Exclude 2. sub=[1].
47   - dfs(2): Base case. Add [1]. Return.
48 - Backtrack: Pop 1. sub=[].
49 - dfs(0): Exclude 1. sub=[].
50   - dfs(1): Include 2. sub=[2]. -> Add [2].
51   - dfs(1): Exclude 2. sub=[]. -> Add [].
52 """

```

18.2 Combination Sum (Unlimited Use)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Can I reuse numbers? A: Yes, unlimited times.
4 - Q: All positive? A: Yes (prevents infinite loops with 0).
5 - Q: Stop condition? A: Sum > target or Sum == target.
6
7 2. LOGIC (DECISION TREE VARIATION)
8 - Choice 1: Include nums[i] AND STAY at i (allows reuse).
9 - Choice 2: Exclude nums[i] AND MOVE to i + 1.
10 - Pruning: If total > target, stop immediately.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "Since I can reuse numbers, my recursive step will have two branches:
15 # one where I add the current number and stay at the same index,
16 # and one where I skip the number and move to the next index. O(2^Target)."
17
18 def combinationSum(candidates: List[int], target: int) -> List[List[int]]:
19     res = []
20
21     def dfs(i, cur, total):
22         # Base Case: Success
23         if total == target:
24             res.append(cur.copy())
25             return
26         # Base Case: Failure / Out of bounds
27         if i >= len(candidates) or total > target:
28             return
29
30         # Branch 1: Include nums[i]
31         cur.append(candidates[i])
32         # RECURSE i (NOT i + 1) allows reusing the same number
33         dfs(i, cur, total + candidates[i])
34
35         # Branch 2: Exclude nums[i] (Backtrack)
36         cur.pop()
37         dfs(i + 1, cur, total)
38
39     dfs(0, [], 0)
40     return res
41
42 """
43 5. DRY RUN
44 nums=[2], target=4
45 - dfs(0, [], 0)
46   - Include 2: dfs(0, [2], 2)
47     - Include 2: dfs(0, [2,2], 4) -> Target met! Add [2,2]. Return.
48     - Backtrack. Exclude 2: dfs(1, [2], 2) -> Out of bounds.
49   - Backtrack. Exclude 2: dfs(1, [], 0) -> Out of bounds.
50 """
```

18.3 Permutations

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Duplicates? A: No.
4     - Q: Length? A: Output is N! size.
5
6 2. LOGIC (ITERATIVE CHOICES)
7     - Unlike subsets (index based), permutations iterate through ALL available numbers at
8       every step.
9     - We need to track "used" numbers (or simply modify the list).
10    - Simpler: Base case len(path) == len(nums). Loop through nums, if not in path, add.
11 """
12 # --- THE MAGIC QUESTION ---
13 # "I'll use DFS. At each step, I will iterate through all numbers.
14 # If a number hasn't been used in the current path, I'll add it and recurse.
15 # This generates all N!$ permutations."
16
17 def permute(nums: List[int]) -> List[List[int]]:
18     res = []
19
20     # Or use a boolean array 'used' if explicit tracking needed
21     def dfs(path):
22         if len(path) == len(nums):
23             res.append(path.copy())
24             return
25
26         for n in nums:
27             if n not in path: # O(N) check - valid for small N
28                 path.append(n)
29                 dfs(path)
30                 path.pop()
31
32     dfs([])
33     return res
34
35 # OPTIMIZED VERSION (Swap-based, O(1) space)
36 # def permute(self, nums):
37 #     def backtrack(start):
38 #         if start == len(nums): res.append(nums[:]); return
39 #         for i in range(start, len(nums)):
40 #             nums[start], nums[i] = nums[i], nums[start]
41 #             backtrack(start + 1)
42 #             nums[start], nums[i] = nums[i], nums[start]
```

18.4 Subsets II (Handling Duplicates)

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Duplicates in input? A: Yes [1, 2, 2].
4   - Q: Output? A: No duplicate subsets ([2] and [2] are same).
5
6 2. LOGIC (SORT + SKIP)
7   - Step 1: SORT the input. This groups duplicates [1, 2, 2].
8   - Step 2: Standard Include/Exclude DFS.
9   - Step 3: THE TRICK. When "Excluding", we must skip ALL instances of the current
10     number.
11     Why? "Include" branch handles the [2], [2,2] cases.
12     "Exclude" means "I am done with 2s". If we pick the next 2, we repeat work.
13 """
14 # --- THE MAGIC QUESTION ---
15 # "I'll sort the array first to group duplicates. In the DFS, if I choose to 'exclude' a
16   number,
17 # I will skip all subsequent identical numbers to avoid generating duplicate subsets. 0(
18   N * 2^N)."
19
20 def subsetsWithDup(nums: List[int]) -> List[List[int]]:
21     res = []
22     nums.sort() # Critical Step
23
24     def dfs(i, subset):
25         if i == len(nums):
26             res.append(subset.copy())
27             return
28
29         # Branch 1: Include nums[i]
30         subset.append(nums[i])
31         dfs(i + 1, subset)
32         subset.pop()
33
34         # Branch 2: Exclude nums[i]
35         # SKIP DUPLICATES: While next number is same as current, skip it.
36         while i + 1 < len(nums) and nums[i] == nums[i + 1]:
37             i += 1
38         dfs(i + 1, subset)
39
40     dfs(0, [])
41     return res
42
43 """
44 5. DRY RUN
45 nums=[1,2,2]. Sort -> [1,2,2].
46 - dfs(0, []): Include 1 -> [1] -> ... [1,2,2], [1,2].
47 - Backtrack to 1. Exclude 2(idx 1).
48   - Skip next 2(idx 2). i becomes 2. dfs(3).
49   - Adds [1]. (Correctly skipped [1,2] duplicate).
50 """
```

18.5 Combination Sum II (No Reuse, Duplicates)

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Reuse numbers? A: No (each number used once).
4   - Q: Duplicates in input? A: Yes.
5   - Q: Unique combinations? A: Yes.
6
7 2. LOGIC (SORT + SKIP RECURSION)
8   - Similar to Subsets II. Sort first.
9   - Loop method: Iterate i from start to end.
10  - If i > start and nums[i] == nums[i-1]: continue (Skip duplicates in same tree level
11   ).
12 """
13 # --- THE MAGIC QUESTION ---
14 # "I'll sort the candidates. I'll use a loop-based DFS. Inside the loop,
15 # if I encounter a duplicate number that isn't the first one in the current iteration,
16 # I skip it to prevent duplicate combinations. O(2^N)."
17
18 def combinationSum2(candidates: List[int], target: int) -> List[List[int]]:
19     candidates.sort()
20     res = []
21
22     def dfs(start, cur, total):
23         if total == target:
24             res.append(cur.copy())
25             return
26         if total > target:
27             return
28
29         for i in range(start, len(candidates)):
30             # Skip duplicates (only allow first instance in this recursion level)
31             if i > start and candidates[i] == candidates[i - 1]:
32                 continue
33
34             cur.append(candidates[i])
35             dfs(i + 1, cur, total + candidates[i]) # i+1 because no reuse
36             cur.pop()
37
38     dfs(0, [], 0)
39     return res
40
41 """
42 5. DRY RUN
43 nums=[1,1,2], target=3. Sorted.
44 - dfs(start=0):
45   - i=0 (1): cur=[1]. dfs(1, [1], 1)
46   - i=1 (1): cur=[1,1]. dfs(2, [1,1], 2)
47     - i=2 (2): cur=[1,1,2]. Sum=4 > 3. Return.
48   - i=2 (2): cur=[1,2]. Sum=3. Add [1,2]. Return.
49   - i=1 (1): 1 > start(0) and nums[1]==nums[0]. SKIP.
50   - i=2 (2): cur=[2]. Sum=2. ...
51 """
```

18.6 Palindrome Partitioning

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input string? A: "aab".
4 - Q: Goal? A: Partition such that EVERY substring is a palindrome.
5 - Q: Return? A: All possible lists of partitions.
6
7 2. LOGIC (CHOICE IS SUBSTRING LENGTH)
8 - State: Index 'i' (start of current substring).
9 - Choice: Try every possible end index 'j'.
10 - Constraint: Is s[i...j] a palindrome?
11 - If yes: Add s[i...j] to path, recurse from j+1.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use DFS starting at index 0. I'll iterate through all possible split points 'j'.
16 # If the substring s[i:j+1] is a palindrome, I'll add it to my current path and recurse
17 # on the rest of the string. O(N * 2^N)."
18
19 def partition(s: str) -> List[List[str]]:
20     res = []
21     part = []
22
23     def dfs(i):
24         if i >= len(s):
25             res.append(part.copy())
26             return
27
28         for j in range(i, len(s)):
29             # Check if substring s[i...j] is palindrome
30             if self.isPali(s, i, j):
31                 part.append(s[i : j + 1])
32                 dfs(j + 1)
33                 part.pop()
34
35     dfs(0)
36     return res
37
38 def isPali(self, s, l, r):
39     while l < r:
40         if s[l] != s[r]: return False
41         l, r = l + 1, r - 1
42     return True
43
44 """
45 5. DRY RUN
46 s = "aab"
47 - dfs(0):
48   - j=0 ("a"): Pali? Yes. path=["a"]. dfs(1).
49   - j=1 ("a"): Pali? Yes. path=["a","a"]. dfs(2).
50     - j=2 ("b"): Pali? Yes. path=["a","a","b"]. dfs(3) -> Add.
51   - j=2 ("ab"): No.
52   - j=1 ("aa"): Pali? Yes. path=["aa"]. dfs(2).
53     - j=2 ("b"): Pali? Yes. path=["aa","b"]. dfs(3) -> Add.
54   - j=2 ("aab"): No.
55 """
```

19 Backtracking: Grids & Logic Puzzles

19.1 Word Search

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Can I reuse cells? A: No, same cell cannot be used twice in one word.
4 - Q: Can I move diagonally? A: No, only horizontal/vertical.
5 - Q: Complexity? A: O(N * M * 4^L) where L is word length.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: BFS.
```

```

9     - Reject: BFS finds shortest path, but tracking "visited" per path in BFS is memory
10     heavy.
11     - Approach 2: DFS (Backtracking).
12     - Logic: Iterate every cell. If cell matches word[0], start DFS.
13     - DFS: Check bounds, check match, check visited.
14     - Mark visited -> Recurse -> Unmark (Backtrack).
15     - Accept: Optimal standard.
16
17 3. INDENTED ENGLISH (LOGIC)
18 function exist(board, word):
19     ROWS, COLS = dim(board)
20     path = set()
21
22     def dfs(r, c, i):
23         if i == len(word): return True
24         if r,c out of bounds or board[r][c] != word[i] or (r,c) in path:
25             return False
26
27         path.add((r,c))
28         res = dfs(r+1,c, i+1) or dfs(r-1,c, i+1) ...
29         path.remove((r,c)) # Backtrack
30         return res
31
32     for r in ROWS:
33         for c in COLS:
34             if dfs(r, c, 0): return True
35     return False

```

The Magic Question (Get Buy-In)

"I'll traverse the grid. Whenever I find the first character, I'll launch a DFS to verify the rest of the word. I'll mark cells as visited temporarily to avoid reuse, then unmark them (backtrack) when returning. $O(N \cdot M \cdot 4^L)$."

```

1 def exist(board: List[List[str]], word: str) -> bool:
2     ROWS, COLS = len(board), len(board[0])
3     path = set() # To track visited cells in current DFS path
4
5     def dfs(r, c, i):
6         # Base Case: Found the whole word
7         if i == len(word):
8             return True
9
10        # Boundary / Validity Checks
11        if (r < 0 or c < 0 or r >= ROWS or c >= COLS or
12            word[i] != board[r][c] or
13            (r, c) in path):
14            return False
15
16        # Choose
17        path.add((r, c))
18
19        # Explore (Up, Down, Left, Right)
20        res = (dfs(r + 1, c, i + 1) or
21              dfs(r - 1, c, i + 1) or
22              dfs(r, c + 1, i + 1) or
23              dfs(r, c - 1, i + 1))
24
25        # Unchoose (Backtrack)
26        path.remove((r, c))
27
28        return res
29
30    # Optimization: Check if word is even possible (frequency count)
31    # count = Counter(sum(board, []))
32    # if count[word[0]] > count[word[-1]]: word = word[::-1]
33
34    for r in range(ROWS):
35        for c in range(COLS):

```

```

36         if dfs(r, c, 0):
37             return True
38
39     return False
40
41 """
42 5. DRY RUN (HUMAN DEBUGGER)
43 board=[['A','B'],['C','D']], word="AB"
44 - r=0, c=0 ('A'). Matches 'A'. path={(0,0)}.
45   - dfs(0, 1, 1) ('B'). Matches 'B'. path={(0,0), (0,1)}.
46     - dfs(...) i=2 == len. Return True.
47   - Bubble up True. Return True.
48 """

```

19.2 Letter Combinations of a Phone Number

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Empty string? A: Return [].
4 - Q: Digits 0 or 1? A: Map to nothing usually (skip).
5 - Q: Input constraints? A: digits.length <= 4.
6
7 2. APPROACHES
8 - Approach 1: Iterative (BFS).
9   - Start with [""]. For next digit, take every existing string and add new chars.
10 - Approach 2: Recursive (DFS/Backtracking).
11   - Logic: index 'i' maps to digit. Loop through letters of digit.
12   - Recurse to i + 1.
13   - Complexity:  $O(4^N)$ .
14   - Accept: Standard.
15
16 3. INDENTED ENGLISH
17 function letterCombinations(digits):
18     if not digits: return []
19     map = {'2': 'abc', ...}
20     res = []
21
22     def backtrack(i, curStr):
23         if i == len(digits):
24             res.append(curStr)
25             return
26         for c in map[digits[i]]:
27             backtrack(i + 1, curStr + c)
28
29     backtrack(0, "")
30     return res
31 """
```

The Magic Question (Get Buy-In)

"I'll use a Hash Map to store the digit-to-letter mappings. Then I'll perform a DFS where at each level 'i' (representing the i -th digit), I iterate through its possible letters and recurse. Time is $O(4^N)$."

```
1 def letterCombinations(digits: str) -> List[str]:
2     res = []
3     digitToChar = {
4         "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
5         "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"
6     }
7
8     if not digits:
9         return []
10
11     def backtrack(i, curStr):
12         # Base case: we have built a string of length N
13         if len(curStr) == len(digits):
14             res.append(curStr)
15             return
16
17         # Loop through all chars for the current digit
18         for c in digitToChar[digits[i]]:
19             backtrack(i + 1, curStr + c)
20
21     backtrack(0, "")
22     return res
23
24 """
25 5. DRY RUN
26 digits = "23"
27 - i=0 ('2' -> abc).
28   - 'a': backtrack(1, "a")
29   - i=1 ('3' -> def).
30     - 'd': backtrack(2, "ad") -> Add "ad".
```

```
31         - 'e': backtrack(2, "ae") -> Add "ae".
32         - 'f': backtrack(2, "af") -> Add "af".
33     - 'b': ... "bd", "be", "bf".
34 """
```

19.3 N-Queens (Hard)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: What defines a valid placement?
4   A: No two queens share row, col, or diagonal.
5 - Q: Output format? A: List of boards (strings).
6
7 2. LOGIC (CONSTRAINT TRACKING)
8 - Backtracking Row by Row. (r=0 to N).
9 - Constraints:
10 - Column: track 'cols' set.
11 - Pos Diag (r+c): track 'posDiag' set.
12 - Neg Diag (r-c): track 'negDiag' set.
13 - If (c, r+c, r-c) not in sets -> Place Q -> Recurse -> Remove Q.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll place queens row by row. To check validity in O(1), I'll use three Hash Sets:
18 # one for columns, one for positive diagonals (r+c), and one for negative diagonals (r-c
19 # ).
20 # This avoids the O(N) scan. Agreed?"
21
22 def solveNQueens(n: int) -> List[List[str]]:
23     col = set()
24     posDiag = set() # (r + c)
25     negDiag = set() # (r - c)
26
27     res = []
28     board = [ "." * n for _ in range(n)]
29
30     def backtrack(r):
31         # Base Case: N queens placed successfully
32         if r == n:
33             copy = [" ".join(row) for row in board]
34             res.append(copy)
35             return
36
37         for c in range(n):
38             # Check constraints
39             if c in col or (r + c) in posDiag or (r - c) in negDiag:
40                 continue
41
42             # Choose
43             col.add(c)
44             posDiag.add(r + c)
45             negDiag.add(r - c)
46             board[r][c] = "Q"
47
48             # Explore
49             backtrack(r + 1)
50
51             # Unchoose (Backtrack)
52             col.remove(c)
53             posDiag.remove(r + c)
54             negDiag.remove(r - c)
55             board[r][c] = "."
56
57     backtrack(0)
58     return res
59
60 """
61 5. DRY RUN
62 n=4.
63 - r=0, c=0. Q at (0,0). Sets: col{0}, pos{0}, neg{0}.
64 - r=1. Try c=0 (in col). Try c=1 (1-1=0 in neg). Try c=2. Valid.
65   - Place Q at (1,2). Sets: ..., col{0,2}, pos{0,3}, neg{0,-1}.
66   - r=2. Try...
67 - Backtracks automatically when stuck.
68 """
```

19.4 Sudoku Solver (Hard)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input valid? A: Yes, partial board valid.
4 - Q: Single solution? A: Guaranteed unique solution.
5
6 2. APPROACHES
7 - Approach 1: Backtracking with Sets.
8 - Logic: Find empty cell. Try 1-9. Check valid. Recurse.
9 - Optimizing Check: Use Sets for Rows, Cols, and 3x3 Boxes.
10 - Box Index: (r // 3) * 3 + (c // 3).
11 - Complexity:  $O(9^M)$  where M is empty cells.
12
13 3. INDENTED ENGLISH
14 function solveSudoku(board):
15     rows, cols, boxes = sets of sets
16
17     def backtrack():
18         r, c = find_empty()
19         if no empty: return True # Solved
20
21         for num in "1".."9":
22             if valid(num):
23                 place num
24                 add to sets
25                 if backtrack(): return True
26                 remove from sets (backtrack)
27                 unplace num
28         return False
29 """
```

The Magic Question (Get Buy-In)

"I'll solve this using Backtracking. I'll maintain Hash Sets for rows, columns, and the 3x3 sub-boxes to validate moves in $O(1)$. I'll fill the first empty cell I find, try digits 1-9, and recurse. If I hit a dead end, I'll backtrack."

```
1 def solveSudoku(board: List[List[str]]) -> None:
2     # 1. Pre-process board to fill constraint sets
3     rows = [set() for _ in range(9)]
4     cols = [set() for _ in range(9)]
5     boxes = [set() for _ in range(9)]
6
7     for r in range(9):
8         for c in range(9):
9             if board[r][c] != ".":
10                 val = board[r][c]
11                 rows[r].add(val)
12                 cols[c].add(val)
13                 boxes[(r // 3) * 3 + (c // 3)].add(val)
14
15     def backtrack():
16         for r in range(9):
17             for c in range(9):
18                 if board[r][c] == ".":
19                     # Found empty cell, try digits 1-9
20                     box_idx = (r // 3) * 3 + (c // 3)
21
22                     for k in range(1, 10):
23                         val = str(k)
24                         # Constraint Check
25                         if (val in rows[r] or
26                             val in cols[c] or
27                             val in boxes[box_idx]):
28                             continue
29
30                     # Choose
31                     board[r][c] = val
32                     rows[r].add(val)
```

```

33         cols[c].add(val)
34         boxes[box_idx].add(val)
35
36         # Explore
37         if backtrack():
38             return True
39
40         # Unchoose
41         board[r][c] = "."
42         rows[r].remove(val)
43         cols[c].remove(val)
44         boxes[box_idx].remove(val)
45
46         return False # Tried 1-9, none worked
47     return True # No empty cells left
48
49     backtrack()

```

20 Backtracking: Variations & Partitioning

20.1 Combinations

```

1  """
2  1. CLARIFYING QUESTIONS
3      - Q: Range? A: 1 to n.
4      - Q: Size? A: k.
5      - Q: Order? A: Order doesn't matter ([1,2] == [2,1]).
6
7  2. LOGIC (STANDARD BACKTRACKING)
8      - Similar to Subsets, but base case is len(path) == k.
9      - Loop from 'start' to n.
10     - Pruning: If remaining numbers < needed, stop.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use a standard DFS starting from 1. I'll add numbers to the combination
15 # until the size reaches k. I can prune branches where there aren't enough
16 # numbers left to fill the combination. 0(k * nCk)."
17
18 def combine(n: int, k: int) -> List[List[int]]:
19     res = []
20
21     def backtrack(start, comb):
22         if len(comb) == k:
23             res.append(comb.copy())
24             return
25
26         # Optimization: If we need 'need' more items,
27         # but only (n - i + 1) items are left, stop.
28         need = k - len(comb)
29
30         for i in range(start, n + 1):
31             if n - i + 1 < need: break # Pruning
32
33             comb.append(i)
34             backtrack(i + 1, comb)
35             comb.pop()
36
37     backtrack(1, [])
38     return res
39
40 """
41 5. DRY RUN
42 n=4, k=2
43 - start=1. add 1.
44   - start=2. add 2. len=2. RES=[1,2]. pop 2.
45   - start=3. add 3. len=2. RES=[1,3]. pop 3.
46   - start=4. add 4. len=2. RES=[1,4]. pop 4.
47 - pop 1.
48 - start=2. add 2.
49   - start=3...

```


20.2 Permutations II

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Duplicates in input? A: Yes.
4 - Q: Output? A: Unique permutations only.
5
6 2. APPROACHES
7 - Approach 1: Set of Tuples. Reject (Slow).
8 - Approach 2: Sort + Hash Map (Counter).
9   - Logic: Use a Counter for available counts.
10  - Loop distinct keys. If count > 0, use and recurse.
11  - Accept: Clean.
12 - Approach 3: Sort + Skip Logic (Index based).
13   - Logic: Sort nums. Use boolean 'used' array.
14   - If nums[i] == nums[i-1] and !used[i-1], SKIP.
15   - Why? If previous identical number wasn't used in this position,
16     using current one creates a duplicate permutation.
17   - Accept: Optimal standard.
18
19 3. INDENTED ENGLISH (LOGIC)
20 function permuteUnique(nums):
21     sort(nums)
22     used = [False] * len
23     def dfs(path):
24         if len == N: add path; return
25         for i in range(N):
26             if used[i]: continue
27             if i > 0 and nums[i]==nums[i-1] and not used[i-1]: continue
28
29             used[i] = True
30             dfs(path + [nums[i]])
31             used[i] = False
32 """
```

The Magic Question (Get Buy-In)

"I'll sort the numbers first. During DFS, if I encounter a duplicate 'nums[i] == nums[i-1]' and the previous number wasn't used in the current tree branch, I skip it to avoid generating the same permutation. O(N!)."

```
1 def permuteUnique(nums: List[int]) -> List[List[int]]:
2     nums.sort()
3     res = []
4     used = [False] * len(nums)
5
6     def backtrack(path):
7         if len(path) == len(nums):
8             res.append(path.copy())
9             return
10
11         for i in range(len(nums)):
12             if used[i]:
13                 continue
14
15             # Duplicate handling magic:
16             # If current is same as prev, AND prev was NOT used in this "level"
17             # (meaning we backtracked from it), then skipping it prevents dups.
18             if i > 0 and nums[i] == nums[i-1] and not used[i-1]:
19                 continue
20
21             used[i] = True
22             path.append(nums[i])
23             backtrack(path)
24             path.pop()
25             used[i] = False
26
27         backtrack([])
28     return res
29
```

```

30 """
31 5. DRY RUN
32     nums=[1,1,2]
33     - Use 1st '1'. path=[1_a]. recurse.
34       - Use 2nd '1'. path=[1_a, 1_b]. recurse.
35         - Use '2'. path=[1_a, 1_b, 2]. ADD.
36     - Backtrack. path=[].
37     - i=1 (2nd '1'). nums[1]==nums[0]. used[0] is False. SKIP.
38     - (Prevents generating [1_b, ...])
39 """

```

20.3 Generate Parentheses

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Valid? A: Every open must have a close. Correct order.
4   - Q: n pairs? A: Output length 2n.
5
6 2. LOGIC (CONSTRAINT BACKTRACKING)
7   - We build string char by char.
8   - Choice 1: Add '('? Allowed if open_count < n.
9   - Choice 2: Add ')'? Allowed if close_count < open_count.
10  - Base Case: len == 2n.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll build the string incrementally. I can add an open parenthesis if I haven't
15 # reached 'n' yet,
16 # and a closing parenthesis if strictly fewer closing ones have been added than opening
17 # ones. O(4^n / sqrt(n))."
18
19 def generateParenthesis(n: int) -> List[str]:
20     res = []
21
22     def backtrack(open_c, close_c, s):
23         if open_c == close_c == n:
24             res.append(s)
25             return
26
27         if open_c < n:
28             backtrack(open_c + 1, close_c, s + "(")
29
30         if close_c < open_c:
31             backtrack(open_c, close_c + 1, s + ")")
32
33     backtrack(0, 0, "")
34     return res
35
36 """
37 5. DRY RUN
38 n=2
39 - ( : 1,0
40 - (( : 2,0
41   - (() : 2,1
42   - (()) : 2,2 -> ADD
43 - () : 1,1
44 - ()( : 2,1
45   - ()() : 2,2 -> ADD
46 """
```

20.4 Matchsticks to Square

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Must use all matchsticks? A: Yes.
4     - Q: Square? A: 4 equal sides. Target = sum(nums) / 4.
5     - Q: Constraints? A: N <= 15 (Suggests Exponential/Backtracking).
6
7 2. LOGIC (PARTITION INTO K SUBSETS)
8     - Sort descending (crucial for pruning large invalid numbers early).
9     - Use array 'sides = [0,0,0,0]'.
10    - For each matchstick, try putting it in side 0, 1, 2, or 3.
11    - If side + match <= target, recurse.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "This is a partitioning problem. I need to fill 4 buckets each with sum 'total // 4'.
16 # I'll sort the matchsticks descending to fail fast. I'll backtrack by trying to place
17 # each stick into one of the 4 buckets. O(4^N)."
18
19 def makesquare(matchsticks: List[int]) -> bool:
20     total = sum(matchsticks)
21     if total % 4 != 0: return False
22
23     length = total // 4
24     matchsticks.sort(reverse=True) # Fail fast
25     sides = [0] * 4
26
27     def backtrack(i):
28         if i == len(matchsticks):
29             return True
30
31         for k in range(4):
32             if sides[k] + matchsticks[i] <= length:
33                 sides[k] += matchsticks[i]
34                 if backtrack(i + 1): return True
35                 sides[k] -= matchsticks[i]
36
37         # Optimization: If this side is empty and we failed,
38         # putting it in another empty side is equivalent. Skip.
39         if sides[k] == 0: break
40
41     return False
42
43 return backtrack(0)
```

20.5 Partition to K Equal Sum Subsets

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Same as matchsticks but general K? A: Yes.
4   - Q: Can elements be 0? A: Constraint says positive.
5
6 2. LOGIC
7   - Identical to Matchsticks logic but 'sides' array is size K.
8   - The "empty side" optimization is CRITICAL here or TLE.
9 """
10
11 def canPartitionKSubsets(nums: List[int], k: int) -> bool:
12     total = sum(nums)
13     if total % k != 0: return False
14     target = total // k
15     nums.sort(reverse=True)
16     sides = [0] * k
17
18     def backtrack(i):
19         if i == len(nums): return True
20
21         for j in range(k):
22             if sides[j] + nums[i] <= target:
23                 sides[j] += nums[i]
24                 if backtrack(i + 1): return True
25                 sides[j] -= nums[i]
26
27             # CRITICAL PRUNING
28             # 1. If this bucket is empty and it failed, trying other
29             #    empty buckets is redundant.
30             if sides[j] == 0: break
31
32             # 2. If the current number fit exactly and failed,
33             #    trying to fit smaller numbers later won't fix it.
34             #    (Usually 0 check is enough for LeetCode)
35         return False
36
37     return backtrack(0)
```

20.6 Word Break II

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Return all sentences? A: Yes.
4     - Q: Space separated? A: Yes.
5
6 2. LOGIC (BACKTRACKING + MEMOIZATION)
7     - Choice: Where to put the space?
8     - Iterate i from start to end.
9     - If s[start:i] is a valid word:
10        - Recurse on s[i:].
11        - Combine current word with all results from recursion.
12     - Memoize: Store results for index 'start' to avoid re-work.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll use DFS with memoization. For the current substring, I'll iterate through valid
17 # prefixes.
18 # If a prefix is in the dictionary, I'll recurse on the suffix and combine the results.
19 # O(2^N)."
20
21 def wordBreak(s: str, wordDict: List[str]) -> List[str]:
22     wordSet = set(wordDict)
23     memo = {}
24
25     def backtrack(start):
26         if start in memo: return memo[start]
27         if start == len(s): return [""]
28
29         res = []
30         for end in range(start + 1, len(s) + 1):
31             word = s[start:end]
32             if word in wordSet:
33                 suffixes = backtrack(end)
34                 for suffix in suffixes:
35                     space = " " if suffix else ""
36                     res.append(word + space + suffix)
37
38         memo[start] = res
39         return res
40
41     return backtrack(0)
42
43 """
44 5. DRY RUN
45 s="catsanddog", dict=["cat","cats","and","sand","dog"]
46 - "cat" (valid) -> recurse "sanddog"
47   - "sand" (valid) -> recurse "dog"
48     - "dog" (valid) -> recurse "" -> returns [""]
49     - combine: "dog" + "" -> ["dog"]
50   - combine: "sand" + " " + "dog" -> ["sand dog"]
51 - combine: "cat" + " " + "sand dog" -> ["cat sand dog"]
52 - ... Backtrack try "cats" ...
53 """
```

20.7 N-Queens II

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Return count only? A: Yes.
4     - Q: Complexity? A: Same as N-Queens I.
5
6 2. LOGIC
7     - Identical logic to N-Queens I.
8     - Instead of storing board state, just increment a counter (or return 1 for leaf, 0
9       for invalid).
10 """
11 def totalNQueens(n: int) -> int:
12     col, posDiag, negDiag = set(), set(), set()
13     count = 0
14
15     def backtrack(r):
16         nonlocal count
17         if r == n:
18             count += 1
19             return
20
21         for c in range(n):
22             if c in col or (r + c) in posDiag or (r - c) in negDiag:
23                 continue
24
25             col.add(c)
26             posDiag.add(r + c)
27             negDiag.add(r - c)
28
29             backtrack(r + 1)
30
31             col.remove(c)
32             posDiag.remove(r + c)
33             negDiag.remove(r - c)
34
35     backtrack(0)
36     return count
```

21 Catch-Up: Missing High-Signal Problems

21.1 Online Stock Span

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Input stream? A: Yes, next() calls.
4     - Q: Span definition? A: Consecutive days prior (including today) <= today's price.
5
6 2. LOGIC (MONOTONIC STACK)
7     - Storing all past prices is O(N).
8     - Instead, merge smaller past spans into the current day.
9     - Stack: [(price, span)].
10    - While stack.top.price <= current_price:
11        current_span += stack.pop().span
12    - Push (current_price, current_span).
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll use a Monotonic Stack storing pairs of '(price, span)'. When a new price comes
17 # in,
18 # I'll pop elements smaller than the current price and accumulate their spans. Amortized
19 # O(1)."
```

```
18
19 class StockSpanner:
20
21     def __init__(self):
22         self.stack = [] # (price, span)
23
24     def next(self, price: int) -> int:
```

```

25     span = 1
26     while self.stack and self.stack[-1][0] <= price:
27         span += self.stack.pop()[1]
28
29     self.stack.append((price, span))
30     return span
31
32 """
33 5. DRY RUN
34 Input: [100, 80, 60, 70, 60, 75, 85]
35 - 100: Stack=[(100,1)]. Res=1.
36 - 80:  Stack=[(100,1), (80,1)]. Res=1.
37 - 60:  Stack=[..., (80,1), (60,1)]. Res=1.
38 - 70:  70 >= 60. span=1+1=2. Pop 60. Stack=[..., (80,1), (70,2)]. Res=2.
39 - ...
40 """

```

21.2 Maximum Frequency Stack

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Tie breaking? A: Pop the one closest to top (most recently added).
4   - Q: Operations? A: push, pop.
5
6 2. LOGIC (MAP OF STACKS)
7   - Need to track:
8     1. Frequency of each val (Map {val: freq}).
9     2. Elements at each frequency (Map {freq: Stack}).
10    3. Current max frequency (int).
11   - Push: Freq increases. Add to stack at new freq. Update maxFreq.
12   - Pop: Pop from stack at maxFreq. Decrement freq. If stack empty, maxFreq--.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll group elements by their frequency using a Hash Map of Stacks: '{freq: [values
17 #   ]}'".
18 # This lets me access the most recently added element of the highest frequency in O(1)."
19
20 class FreqStack:
21
22     def __init__(self):
23         self.cnt = collections.defaultdict(int) # val -> count
24         self.stacks = collections.defaultdict(list) # count -> stack of vals
25         self.maxFreq = 0
26
27     def push(self, val: int) -> None:
28         self.cnt[val] += 1
29         freq = self.cnt[val]
30
31         if freq > self.maxFreq:
32             self.maxFreq = freq
33
34         self.stacks[freq].append(val)
35
36     def pop(self) -> int:
37         val = self.stacks[self.maxFreq].pop()
38         self.cnt[val] -= 1
39
40         if not self.stacks[self.maxFreq]:
41             self.maxFreq -= 1
42
43         return val
```

21.3 Split Array Largest Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Split into k subarrays? A: Yes, contiguous.
4     - Q: Goal? A: Minimize the largest sum among these k subarrays.
5
6 2. LOGIC (ANSWER SPACE SEARCH)
7     - Identical to "Capacity to Ship Packages" (Problem 7 in BS Part 1).
8     - "Capacity" = "Max Sum Allowed".
9     - "Days" = "k subarrays".
10    - Range: [max(nums), sum(nums)].
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "This is exactly the same pattern as 'Capacity to Ship Packages'.
15 # I'll Binary Search on the 'Max Allowed Sum' (from max(nums) to sum(nums)).
16 # For each mid, I'll check if I can split the array into <= k parts."
17
18 def splitArray(nums: List[int], k: int) -> int:
19     l, r = max(nums), sum(nums)
20     res = r
21
22     def canSplit(maxSum):
23         subarrays = 1
24         curSum = 0
25         for n in nums:
26             if curSum + n > maxSum:
27                 subarrays += 1
28                 curSum = 0
29             curSum += n
30         return subarrays <= k
31
32     while l <= r:
33         mid = l + (r - l) // 2
34         if canSplit(mid):
35             res = mid
36             r = mid - 1
37         else:
38             l = mid + 1
39     return res
```

21.4 Find in Mountain Array

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Mountain Array? A: Increases then Decreases.
4   - Q: Constraint? A: Can't access indices directly (use API get(k)). Max 100 calls.
5
6 2. LOGIC (TRIPLE BINARY SEARCH)
7   - Step 1: Find Peak Index (Binary Search).
8     - Condition: arr[mid] < arr[mid+1] means we are on left slope.
9   - Step 2: Binary Search Target in Left Slope (0 to Peak).
10  - Step 3: Binary Search Target in Right Slope (Peak to End).
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll solve this in three steps. First, find the Peak index using Binary Search.
15 # Then, search the ascending left slope. If not found, search the descending right slope
16 # . O(log N)."
```

```
17 def findInMountainArray(target: int, mountain_arr: 'MountainArray') -> int:
18     length = mountain_arr.length()
19
20     # 1. Find Peak
21     l, r = 0, length - 1
22     peak = 0
23     while l < r:
24         mid = l + (r - l) // 2
25         if mountain_arr.get(mid) < mountain_arr.get(mid + 1):
26             l = mid + 1
27         else:
28             r = mid
29     peak = l
30
31     # 2. Search Left (Ascending)
32     l, r = 0, peak
33     while l <= r:
34         mid = l + (r - l) // 2
35         val = mountain_arr.get(mid)
36         if val == target: return mid
37         elif val < target: l = mid + 1
38         else: r = mid - 1
39
40     # 3. Search Right (Descending)
41     l, r = peak, length - 1
42     while l <= r:
43         mid = l + (r - l) // 2
44         val = mountain_arr.get(mid)
45         if val == target: return mid
46         elif val > target: l = mid + 1 # Logic inverted for descending
47         else: r = mid - 1
48
49     return -1
```

21.5 Reverse Linked List II

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: 1-indexed? A: Yes.
4     - Q: One pass? A: Yes.
5
6 2. LOGIC (POINTER SURGERY)
7     - Move 'leftPrev' to node before start of reversal.
8     - 'curr' starts at 'left'.
9     - Perform reversal 'right - left' times.
10    - In each step: Move 'curr.next' to the front of the reversed portion.
11 """
12
13 def reverseBetween(head: Optional[ListNode], left: int, right: int) -> Optional[ListNode]:
14     dummy = ListNode(0, head)
15     leftPrev, curr = dummy, head
16
17     # 1. Move to starting position
18     for _ in range(left - 1):
19         leftPrev, curr = curr, curr.next
20
21     # 2. Reverse sublist
22     # curr is fixed at the "tail" of the sublist being built
23     # We continually move 'nxt' to the front (after leftPrev)
24     prev = None
25     for _ in range(right - left + 1):
26         tmp = curr.next
27         curr.next = prev
28         prev = curr
29         curr = tmp
30
31     # 3. Connect ends
32     # leftPrev.next was the start of the sublist (now the end)
33     # It needs to point to 'curr' (the node after the sublist)
34     leftPrev.next.next = curr
35     leftPrev.next = prev # prev is the new head of sublist
36
37     return dummy.next
```

21.6 LFU Cache

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Tie breaking? A: LRU (Least Recently Used) among the LFU (Least Frequent).
4 - Q: O(1)? A: Yes.
5
6 2. LOGIC (TWO MAPS)
7 - valMap: {key: [val, freq]}
8 - freqMap: {freq: OrderedDict} (Using OrderedDict/LinkedList for LRU order).
9 - minFreq: Track minimum frequency to know which bucket to evict from.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use two Hash Maps. 'valMap' stores values and frequencies.
14 # 'freqMap' maps a frequency to an OrderedDict of keys (acting as an LRU list).
15 # I'll verify 'minFreq' to evict correctly in O(1)."
```

```
16
17 from collections import defaultdict, OrderedDict
18
19 class LFUCache:
20     def __init__(self, capacity: int):
21         self.cap = capacity
22         self.valMap = {} # key -> [val, freq]
23         self.freqMap = defaultdict(OrderedDict) # freq -> OrderedDict(key: None)
24         self.minFreq = 0
25
26     def get(self, key: int) -> int:
27         if key not in self.valMap: return -1
28
29         val, freq = self.valMap[key]
30
31         # Update freq
32         del self.freqMap[freq][key]
33         if not self.freqMap[freq]:
34             del self.freqMap[freq]
35         if self.minFreq == freq:
36             self.minFreq += 1
37
38         self.freqMap[freq + 1][key] = None
39         self.valMap[key] = [val, freq + 1]
40         return val
41
42     def put(self, key: int, value: int) -> None:
43         if not self.cap: return
44
45         if key in self.valMap:
46             self.valMap[key][0] = value
47             self.get(key) # Reuse get to update freq
48             return
49
50         if len(self.valMap) == self.cap:
51             # Evict LFU (and LRU within that freq)
52             k, _ = self.freqMap[self.minFreq].popitem(last=False)
53             del self.valMap[k]
54
55         self.valMap[key] = [value, 1]
56         self.freqMap[1][key] = None
57         self.minFreq = 1
```

21.7 Design Circular Queue

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Array or List? A: Array (fixed size) is cleaner.
4   - Q: Space? A: O(k) pre-allocated.
5
6 2. LOGIC (MODULO ARITHMETIC)
7   - Use 'head' index, 'count' size, and 'capacity'.
8   - Tail index = (head + count - 1) % capacity.
9   - Next insert index = (head + count) % capacity.
10 """
11
12 class MyCircularQueue:
13     def __init__(self, k: int):
14         self.q = [0] * k
15         self.k = k
16         self.head = 0
17         self.count = 0
18
19     def enqueue(self, value: int) -> bool:
20         if self.isFull(): return False
21         # Insert at (head + count) % k
22         tail = (self.head + self.count) % self.k
23         self.q[tail] = value
24         self.count += 1
25         return True
26
27     def dequeue(self) -> bool:
28         if self.isEmpty(): return False
29         # Just move head forward
30         self.head = (self.head + 1) % self.k
31         self.count -= 1
32         return True
33
34     def Front(self) -> int:
35         return -1 if self.isEmpty() else self.q[self.head]
36
37     def Rear(self) -> int:
38         if self.isEmpty(): return -1
39         return self.q[(self.head + self.count - 1) % self.k]
40
41     def isEmpty(self) -> bool: return self.count == 0
42     def isFull(self) -> bool: return self.count == self.k
```

22 Trees: Recursion Fundamentals

The Recursive Leap of Faith

For most Tree problems, don't try to visualize the whole stack. Ask:

- **Base Case:** What if the node is None? (Usually return 0, False, or None).
- **Recursive Step:** Call the function on `node.left` and `node.right`.
- **Logic:** How do I combine the results from left/right to answer for **current**?

22.1 Invert Binary Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Modify in-place? A: Yes.
4   - Q: Empty tree? A: Return None.
5
6 2. LOGIC (RECURSIVE SWAP)
7   - Visit root.
8   - Swap left and right children.
9   - Recursively invert left subtree.
```

```

10     - Recursively invert right subtree.
11     """
12
13     # --- THE MAGIC QUESTION ---
14     # "I'll visit every node. At each node, I'll swap its left and right children,
15     # then recursively call the function on both children. O(N)."
```

```

16
17     def invertTree(root: Optional[TreeNode]) -> Optional[TreeNode]:
18         if not root:
19             return None
20
21         # Swap
22         root.left, root.right = root.right, root.left
23
24         # Recurse
25         invertTree(root.left)
26         invertTree(root.right)
27
28         return root
29
30     """
31     5. DRY RUN
32     Root=4, L=2, R=7
33     - Swap: L=7, R=2.
34     - Call invert(7). Call invert(2).
35     - Result: Tree mirrored.
36     """

```

22.2 Maximum Depth of Binary Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Depth definition? A: Number of nodes along longest path from root to leaf.
4   - Q: Empty tree? A: 0.
5
6 2. LOGIC (RECURSIVE)
7   - Depth = 1 (self) + max(depth(left), depth(right)).
8   - Base case: If None, depth is 0.
9 """
10
11 # --- THE MAGIC QUESTION ---
12 # "I'll use recursion. The depth of a node is 1 plus the maximum depth of its children.
13 # Base case is 0 for a null node. O(N)."
```

```
14
15 def maxDepth(root: Optional[TreeNode]) -> int:
16     if not root:
17         return 0
18
19     return 1 + max(maxDepth(root.left), maxDepth(root.right))
20
21 """
22 5. DRY RUN
23 3 -> (9, 20->(15, 7))
24 - maxDepth(9) -> 1
25 - maxDepth(15) -> 1
26 - maxDepth(7) -> 1
27 - maxDepth(20) -> 1 + max(1, 1) = 2
28 - maxDepth(3) -> 1 + max(1, 2) = 3
29 """
```

22.3 Diameter of Binary Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Path pass through root? A: Not necessarily.
4 - Q: Edge or Node count? A: Length is edges (nodes - 1).
5
6 2. APPROACHES
7 - Approach 1: Calculate depth at every node.  $O(N^2)$ . Reject.
8 - Approach 2: DFS with Global Variable.
9 - Logic: At any node, the "Longest Path through Me" = depth(left) + depth(right).
10 - Update global max with this sum.
11 - Return my height (1 + max(L, R)) to parent.
12 - Complexity:  $O(N)$ .
13 - Accept: Optimal.
14
15 3. INDENTED ENGLISH
16 class Solution:
17     res = 0
18     def diameterOfBinaryTree(root):
19         def dfs(node):
20             if not node: return 0
21             left = dfs(node.left)
22             right = dfs(node.right)
23
24             # Update global diameter (edges = L height + R height)
25             res = max(res, left + right)
26
27             # Return height to parent
28             return 1 + max(left, right)
29         dfs(root)
30         return res
31 """
```

The Magic Question (Get Buy-In)

"I'll use a DFS that returns the height of the tree. While calculating the height, I'll update a global variable 'res' with 'left_{height} + right_{height}', which represents the longest path passing through the current node. $O(N)$."

```
1 def diameterOfBinaryTree(root: Optional[TreeNode]) -> int:
2     res = 0
3
4     def dfs(root):
5         nonlocal res
6         if not root:
7             return 0
8
9         left = dfs(root.left)
10        right = dfs(root.right)
11
12        # Diameter THROUGH this node = left height + right height
13        res = max(res, left + right)
14
15        # Return height of this node
16        return 1 + max(left, right)
17
18    dfs(root)
19    return res
20
21 """
22 5. DRY RUN
23 1 -> 2 -> (4, 5), 3
24 - dfs(4): returns 1.
25 - dfs(5): returns 1.
26 - dfs(2): L=1, R=1. res=max(0, 2)=2. returns 2.
27 - dfs(3): returns 1.
28 - dfs(1): L=2, R=1. res=max(2, 3)=3. returns 3.
29 - Return 3.
30 """
```

22.4 Balanced Binary Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Definition? A: Height of left and right subtrees differ by at most 1.
4   - Q: Must ALL subtrees be balanced? A: Yes.
5
6 2. LOGIC (BOTTOM-UP DFS)
7   - Similar to Max Depth.
8   - Instead of just returning height, return [isBalanced, height].
9   - If left or right is unbalanced, bubble up False.
10  - If abs(left_h - right_h) > 1, return False.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use a bottom-up DFS. Each node returns its height. If a node detects that its
15 # children
16 # are unbalanced or their height difference is > 1, it passes up a failure signal (like
17 # -1 or False). O(N)."
```

```
17 def isBalanced(root: Optional[TreeNode]) -> bool:
18
19     def dfs(root):
20         if not root:
21             return [True, 0] # [Balanced?, Height]
22
23         left = dfs(root.left)
24         right = dfs(root.right)
25
26         # Check 1: Children balanced?
27         # Check 2: Current node balanced?
28         balanced = (left[0] and right[0] and
29                     abs(left[1] - right[1]) <= 1)
30
31         return [balanced, 1 + max(left[1], right[1])]
32
33     return dfs(root)[0]
34
35 """
36 5. DRY RUN
37 3 -> 9, 20->(15, 7)
38 - 15, 7, 9 return [True, 1].
39 - 20: L=1, R=1. Diff=0. Returns [True, 2].
40 - 3: L=[T, 1] (9), R=[T, 2] (20). Diff=1. Balanced.
41 - Return True.
42 """
```

22.5 Same Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Value and Structure? A: Yes, both must match exactly.
4
5 2. LOGIC (SIMULTANEOUS TRAVERSAL)
6   - Base Cases:
7     1. Both None -> True.
8     2. One None, one Not -> False.
9     3. Values differ -> False.
10  - Recurse: isSame(p.left, q.left) AND isSame(p.right, q.right).
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll traverse both trees simultaneously. If both nodes are null, it's a match.
15 # If one is null or values differ, it's a mismatch. Otherwise, I recurse on children. O(
16   N)."
17
18 def isSameTree(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
19     # 1. Both empty
20     if not p and not q:
21         return True
22     # 2. One empty or values differ
23     if not p or not q or p.val != q.val:
24         return False
25
26     # 3. Recurse
27     return (isSameTree(p.left, q.left) and
28             isSameTree(p.right, q.right))
```

22.6 Subtree of Another Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Exact match? A: Yes, structure and values.
4   - Q: Subtree definition? A: Node and all its descendants.
5
6 2. LOGIC (NESTED RECURSION)
7   - For every node in 'root', check if the tree starting there is Identical to 'subRoot'.
8   - Use 'isSameTree' helper.
9   - Complexity: O(N * M).
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll iterate through every node of the main tree. At each node, I'll use a helper
14 #   function
15 # 'isSameTree' to check if the subtree rooted there matches 'subRoot'. O(N*M)."
16
17 def isSubtree(root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
18     if not subRoot: return True # Empty is always a subtree
19     if not root: return False # Main tree empty but sub isn't -> False
20
21     if isSameTree(root, subRoot):
22         return True
23
24     return (isSubtree(root.left, subRoot) or
25             isSubtree(root.right, subRoot))
26
27 def isSameTree(p, q):
28     if not p and not q: return True
29     if not p or not q or p.val != q.val: return False
30     return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

22.7 Binary Tree Traversals (Iterative)

```
1 """
2 Note: Recursive is trivial. Interviews ask for Iterative.
3
4 1. INORDER (Left, Root, Right) - BST Sorted Order
5   - Stack strategy: Go left as much as possible, pushing to stack.
6   - When can't go left, pop, add value, go right.
7
8 2. PREORDER (Root, Left, Right) - Cloning
9   - Stack strategy: Push root. While stack: pop, add val, push Right, push Left.
10
11 3. POSTORDER (Left, Right, Root) - Deletion
12   - Trick: Do Preorder (Root, Right, Left) then reverse result.
13 """
14
15 def inorderTraversal(root: Optional[TreeNode]) -> List[int]:
16     res, stack = [], []
17     cur = root
18
19     while cur or stack:
20         while cur:
21             stack.append(cur)
22             cur = cur.left
23         cur = stack.pop()
24         res.append(cur.val)
25         cur = cur.right
26     return res
27
28 def preorderTraversal(root: Optional[TreeNode]) -> List[int]:
29     if not root: return []
30     res, stack = [], [root]
31     while stack:
32         cur = stack.pop()
33         res.append(cur.val)
34         if cur.right: stack.append(cur.right)
35         if cur.left: stack.append(cur.left)
36     return res
```

23 Trees: BST Properties & Level Traversal

23.1 Lowest Common Ancestor of a BST

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Is it a BST? A: Yes (Left < Root < Right).
4   - Q: Guaranteed to exist? A: Yes.
5   - Q: Can a node be its own ancestor? A: Yes.
6
7 2. LOGIC (BST SPLIT)
8   - If both p and q are greater than root, LCA is in Right subtree.
9   - If both p and q are smaller than root, LCA is in Left subtree.
10  - If split (p < root < q) or one is root, THEN root is the LCA.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "Since this is a BST, I can search efficiently. If both values are smaller than the
15 #   root,
16 # I go left. If both are larger, I go right. The split point is the LCA. O(log N)."
```

```
17 def lowestCommonAncestor(root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
18     curr = root
19
20     while curr:
21         if p.val > curr.val and q.val > curr.val:
22             curr = curr.right
23         elif p.val < curr.val and q.val < curr.val:
24             curr = curr.left
25         else:
26             # We found the split point (or match)
```

```

27         return curr
28
29     return None
30
31 """
32 5. DRY RUN
33 Tree: 6 -> (2, 8). p=2, q=8.
34 - 2 < 6 and 8 > 6. Split. Return 6.
35 Tree: 6 -> (2->(0, 4)), 8. p=2, q=4.
36 - 2<6, 4<6. Go Left (2).
37 - curr=2. p=2, q=4. 2 is not < 2. Split/Match. Return 2.
38 """

```

23.2 Binary Tree Level Order Traversal

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Return format? A: List of lists [[level1], [level2]].
4   - Q: Empty tree? A: Return [].
5
6 2. LOGIC (BFS / QUEUE)
7   - Use a Deque.
8   - Inner loop processes 'len(q)' nodes (snapshot of current level).
9   - Add their children to queue for next level.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use BFS with a Queue. Inside the loop, I'll process exactly 'len(q)' nodes
14 # to capture one full level at a time before adding their children. O(N)."
15
16 from collections import deque
17
18 def levelOrder(root: Optional[TreeNode]) -> List[List[int]]:
19     if not root: return []
20
21     res = []
22     q = deque([root])
23
24     while q:
25         level = []
26         # Snapshot length of current level
27         for _ in range(len(q)):
28             node = q.popleft()
29             level.append(node.val)
30             if node.left: q.append(node.left)
31             if node.right: q.append(node.right)
32         res.append(level)
33
34     return res
35
36 """
37 5. DRY RUN
38 3 -> 9, 20
39 - q=[3]. Loop range(1). Pop 3. Add 9, 20. level=[3].
40 - q=[9, 20]. Loop range(2).
41   - Pop 9. Add nothing.
42   - Pop 20. Add nothing. level=[9, 20].
43 - q=[]. Stop.
44 """
```

23.3 Binary Tree Right Side View

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Definition? A: Visible nodes from right side (last node of each level).
4   - Q: Empty tree? A: [].
5
6 2. APPROACHES
7   - Approach 1: BFS (Level Order).
8     - Logic: Take the last element of the 'level' list.
9     - Accept: Standard.
10  - Approach 2: DFS (Reverse Preorder: Root, Right, Left).
11    - Logic: If we visit Right first, the first node we see at depth 'd' is the right
12      view node.
13    - Accept: Cool trick.
14
15 3. INDENTED ENGLISH (BFS)
16  function rightSideView(root):
17      q = [root]
18      while q:
19          rightMost = None
20          for i in range(len(q)):
21              node = pop()
22              rightMost = node
23              add children
24          res.append(rightMost.val)
25      return res
26 """
```

The Magic Question (Get Buy-In)

"I'll use BFS to traverse level by level. By grabbing the last node processed in each level iteration, I naturally capture the 'Right Side View'. O(N)."

```
1 def rightSideView(root: Optional[TreeNode]) -> List[int]:
2     if not root: return []
3
4     res = []
5     q = deque([root])
6
7     while q:
8         rightNode = None
9         # Process entire level
10        for _ in range(len(q)):
11            node = q.popleft()
12            rightNode = node # Overwrite until the last one
13            if node.left: q.append(node.left)
14            if node.right: q.append(node.right)
15
16        if rightNode:
17            res.append(rightNode.val)
18
19    return res
```

23.4 Count Good Nodes in Binary Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: "Good" definition? A: Path from root to X contains no nodes > X.
4   - Q: Root is good? A: Always.
5
6 2. LOGIC (DFS + MAX_SO_FAR)
7   - Pass 'maxVal' down the recursion.
8   - If node.val >= maxVal, it's good (count += 1). Update maxVal.
9   - Else, it's bad (count += 0). maxVal stays same.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use DFS, passing the 'max_so_far' value down the tree.
14 # If a node's value is greater than or equal to 'max_so_far', it's a 'Good Node'. O(N)."
15
16 def goodNodes(root: TreeNode) -> int:
17
18     def dfs(node, maxVal):
19         if not node:
20             return 0
21
22         res = 1 if node.val >= maxVal else 0
23         maxVal = max(maxVal, node.val)
24
25         res += dfs(node.left, maxVal)
26         res += dfs(node.right, maxVal)
27
28         return res
29
30     return dfs(root, root.val)
31
32 """
33 5. DRY RUN
34 3 -> 1, 4 -> (1, 5)
35 - Root(3): Good. max=3.
36 - Left(1): 1 < 3. Bad. max=3.
37 - Right(4): 4 >= 3. Good. max=4.
38   - R->L(1): 1 < 4. Bad.
39   - R->R(5): 5 >= 4. Good.
40 - Total: 3.
41 """
```

23.5 Validate Binary Search Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Duplicates? A: Usually BST implies Strict inequality (Left < Root < Right).
4   - Q: Definition? A: ALL nodes in Left subtree < Root, ALL in Right > Root.
5
6 2. LOGIC (RANGE CONSTRAINT)
7   - Root can be anything (-inf, inf).
8   - Left child must be (-inf, root.val).
9   - Right child must be (root.val, inf).
10  - Pass valid range down.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use DFS. Each node must fall strictly within a valid range '(min, max)'.
15 # When going left, I update the max to the current node's value. When going right, I
16   update the min. O(N)."
```

```
17 def isValidBST(root: Optional[TreeNode]) -> bool:
18
19     def valid(node, left, right):
20         if not node:
21             return True
22
23         if not (left < node.val < right):
24             return False
25
26         return (valid(node.left, left, node.val) and
27                 valid(node.right, node.val, right))
28
29     return valid(root, float("-inf"), float("inf"))
30
31 """
32 5. DRY RUN
33   2 -> 1, 3
34   - Root(2): Range (-inf, inf). OK.
35   - Left(1): Range (-inf, 2). OK.
36   - Right(3): Range (2, inf). OK.
37
38   5 -> 1, 4 -> (3, 6)
39   - Right(4): Range (5, inf). 4 is NOT > 5. Return False.
40 """
```

23.6 Kth Smallest Element in a BST

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: k is valid? A: Yes.
4 - Q: 1-indexed? A: Yes.
5
6 2. APPROACHES
7 - Approach 1: Inorder -> List. Return list[k-1]. O(N) space.
8 - Approach 2: Iterative Inorder. Stop at k. O(H) space.
9 - Logic: Stack-based inorder traversal. Counter++.
10 - Accept: Optimal.
11
12 3. INDENTED ENGLISH
13 function kthSmallest(root, k):
14     stack = []
15     cur = root
16     while cur or stack:
17         while cur: push cur; cur = cur.left
18         cur = pop()
19         k -= 1
20         if k == 0: return cur.val
21         cur = cur.right
22 """
```

The Magic Question (Get Buy-In)

"Since it's a BST, an Inorder Traversal gives sorted values. I'll use an iterative Stack approach to simulate the traversal and stop exactly when I've visited the k -th node. $O(H + k)$ Time."

```
1 def kthSmallest(root: Optional[TreeNode], k: int) -> int:
2     stack = []
3     curr = root
4
5     while curr or stack:
6         # 1. Go as Left as possible
7         while curr:
8             stack.append(curr)
9             curr = curr.left
10
11         # 2. Process Node
12         curr = stack.pop()
13         k -= 1
14         if k == 0:
15             return curr.val
16
17         # 3. Go Right
18         curr = curr.right
19
20     return -1
```

23.7 Insert into a Binary Search Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Duplicates? A: Assuming no, or insert logic defined.
4   - Q: Rebalancing? A: No, just standard insert.
5
6 2. LOGIC (BST SEARCH)
7   - If val > root.val: go Right.
8     - If right is None, right = NewNode.
9   - If val < root.val: go Left.
10  - Return root.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll traverse the BST. If the value is greater, I go right; if smaller, I go left.
15 # Once I hit a 'None', I attach the new node there. O(log N)."
```

```
16
17 def insertIntoBST(root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
18     if not root:
19         return TreeNode(val)
20
21     if val > root.val:
22         root.right = insertIntoBST(root.right, val)
23     else:
24         root.left = insertIntoBST(root.left, val)
25
26     return root
```

23.8 Delete Node in a BST

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Node exists? A: Maybe not.
4   - Q: Restructuring? A: Maintain BST property.
5
6 2. LOGIC (THREE CASES)
7   - Case 1: Leaf node -> Return None.
8   - Case 2: One child -> Return that child.
9   - Case 3: Two children ->
10    - Find Min of Right Subtree.
11    - Copy that value to current node.
12    - Recursively delete that Min node from Right Subtree.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "The hard case is deleting a node with two children. I'll find the minimum value
17 # in the right subtree, replace the current node's value with it,
18 # and then recursively delete that minimum node. O(log N)."
19
20 def deleteNode(root: Optional[TreeNode], key: int) -> Optional[TreeNode]:
21     if not root:
22         return None
23
24     if key > root.val:
25         root.right = deleteNode(root.right, key)
26     elif key < root.val:
27         root.left = deleteNode(root.left, key)
28     else:
29         # Found the node!
30         if not root.left:
31             return root.right
32         if not root.right:
33             return root.left
34
35         # Two Children: Find Min in Right Subtree
36         curr = root.right
37         while curr.left:
38             curr = curr.left
39
40         root.val = curr.val
41         # Delete the duplicate
42         root.right = deleteNode(root.right, curr.val)
43
44     return root
```

24 Trees: Construction & Hard Problems

24.1 Construct Binary Tree from Preorder and Inorder Traversal

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Duplicates? A: No (Critical for map lookup).
4   - Q: Empty? A: None.
5
6 2. LOGIC (ROOT SPLIT)
7   - Preorder: [Root, (Left...)..., (Right...)...] -> First is ALWAYS root.
8   - Inorder: [(Left...), Root, (Right...)] -> Root splits the tree.
9   - Strategy:
10    1. Pick 'preorder[0]' as Root.
11    2. Find Root index in Inorder.
12    3. Left Subtree = Inorder[:index]. Right Subtree = Inorder[index+1:].
13    4. Recurse.
14   - Optimization: Use a Hash Map for Inorder index lookup (O(1)).
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll use the Preorder array to identify the root. Then, I'll find that root in the
19 # Inorder array to split the left and right subtrees. I'll recurse using a Hash Map for
20 # O(1) lookups."
```

```

20
21 def buildTree(preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
22     # Optimization: Map value -> index for O(1) lookup
23     inorder_map = {val: i for i, val in enumerate(inorder)}
24     pre_iter = iter(preorder)
25
26     def build(left, right):
27         if left > right:
28             return None
29
30         # Get next root from preorder
31         root_val = next(pre_iter)
32         root = TreeNode(root_val)
33
34         # Split point in inorder
35         idx = inorder_map[root_val]
36
37         # Recurse (Preorder visits Left then Right)
38         root.left = build(left, idx - 1)
39         root.right = build(idx + 1, right)
40
41         return root
42
43     return build(0, len(inorder) - 1)
44
45 """
46 5. DRY RUN
47 Pre=[3,9,20], In=[9,3,20]
48 - Root=3. Split In at idx 1.
49 - Left: build(0, 0). Root=9. Idx=0.
50   - L: build(0, -1) -> None. R: build(1, 0) -> None. Ret 9.
51 - Right: build(2, 2). Root=20. ...
52 """

```

24.2 Delete Leaves With a Given Value

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: What if a parent becomes a leaf after deletion?
4     A: It should also be deleted if value matches.
5   - Q: Logic direction?
6     A: Post-order (Bottom-Up). Delete children first, then check self.
7
8 2. LOGIC (POST-ORDER)
9   - Recurse Left. Recurse Right.
10  - If node.left/right becomes None (deleted), and node itself is a leaf with target
    value...
11  - Return None (delete self).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use Post-Order Traversal. I process the children first. If a child is deleted,
16 # I check if the current node has become a leaf with the target value. If so, I delete
    it too."
17
18 def removeLeafNodes(root: Optional[TreeNode], target: int) -> Optional[TreeNode]:
19     if not root:
20         return None
21
22     # 1. Process children first (Post-Order)
23     root.left = removeLeafNodes(root.left, target)
24     root.right = removeLeafNodes(root.right, target)
25
26     # 2. Check current node
27     if not root.left and not root.right and root.val == target:
28         return None
29
30     return root
```

24.3 House Robber III

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Constraint? A: Can't rob connected nodes (Parent + Child).
4 - Q: Values? A: Non-negative.
5
6 2. APPROACHES
7 - Approach 1: Naive Recursion.
8   - rob(root) = max(rob + rob(grandchildren), rob(children)).
9   - Reject: TLE (Exponential).
10 - Approach 2: DFS returning Pair.
11   - Return [withRoot, withoutRoot].
12   - withRoot = root.val + left.without + right.without.
13   - withoutRoot = max(left pair) + max(right pair).
14   - Complexity: O(N).
15   - Accept: Optimal.
16
17 3. INDENTED ENGLISH
18 def dfs(node):
19     if not node: return [0, 0]
20     left = dfs(node.left)
21     right = dfs(node.right)
22
23     withNode = node.val + left[1] + right[1]
24     withoutNode = max(left) + max(right)
25     return [withNode, withoutNode]
26 """
```

The Magic Question (Get Buy-In)

"I'll use a DFS that returns a pair of values for each node: '[MaxWithNode, MaxWithoutNode]'. This allows me to make the optimal decision at each level in $O(N)$ time without re-calculating subproblems."

```
1 def rob(root: Optional[TreeNode]) -> int:
2
3     def dfs(node):
4         if not node:
5             return [0, 0]
6
7         left = dfs(node.left)
8         right = dfs(node.right)
9
10        # Option 1: Rob this node (Can't rob children)
11        # Add values from children's "withoutNode" state
12        withNode = node.val + left[1] + right[1]
13
14        # Option 2: Skip this node (Can rob children or skip them)
15        # Take max of both options from children
16        withoutNode = max(left) + max(right)
17
18        return [withNode, withoutNode]
19
20    return max(dfs(root))
```

24.4 Binary Tree Maximum Path Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Can path stop anywhere? A: Yes, doesn't need to be root-to-leaf.
4   - Q: Negative values? A: Yes (Tricky! Path might just be one node).
5
6 2. LOGIC (GLOBAL MAX, LOCAL RETURN)
7   - At any node, the "Split Path" (going L->Node->R) is potential global max.
8   - But to the parent, we can only return "Single Path" (Node + max(L, R)).
9   - Important: If a child returns negative, treat it as 0 (don't include it).
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll use Post-Order traversal. At each node, I'll calculate the max path that *splits
14 #   * # through it (Left + Root + Right) to update the global maximum. However, I'll
15 #   only return
16 # the max *straight* path (Root + max(Left, Right)) to the parent. O(N)."
```

```
17 def maxPathSum(root: Optional[TreeNode]) -> int:
18     res = float("-inf")
19
20     def dfs(node):
21         nonlocal res
22         if not node:
23             return 0
24
25         # 1. Recursive step (Treat negative paths as 0)
26         leftMax = max(dfs(node.left), 0)
27         rightMax = max(dfs(node.right), 0)
28
29         # 2. Update Global Max (Split Path)
30         # Is the path L -> Node -> R the new best?
31         res = max(res, node.val + leftMax + rightMax)
32
33         # 3. Return value to Parent (Single Path)
34         # Must choose either Left or Right path to continue upward
35         return node.val + max(leftMax, rightMax)
36
37     dfs(root)
38     return res
39 """
40 5. DRY RUN
41   -10 -> (9, 20->(15, 7))
42   - dfs(9): returns 9. res=9.
43   - dfs(15): returns 15. res=15.
44   - dfs(7): returns 7. res=15.
45   - dfs(20):
46     - L=15, R=7.
47     - Split: 20+15+7 = 42. res=42.
48     - Return: 20 + 15 = 35.
49   - dfs(-10):
50     - L=9, R=35.
51     - Split: -10 + 9 + 35 = 34. res=max(42, 34) = 42.
52     - Return: -10 + 35 = 25.
53   - Final res=42.
54 """
```

24.5 Serialize and Deserialize Binary Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Format? A: Your choice.
4 - Q: Values? A: Ints.
5 - Q: Null handling? A: Must encode nulls (e.g., "N").
6
7 2. APPROACHES
8 - Approach 1: BFS (Level Order).
9 - Good, but trailing nulls are messy.
10 - Approach 2: DFS (Preorder).
11 - Logic: Root, Left, Right.
12 - If null, append "N".
13 - Deserializing: Use an iterator. Pop val. If "N", return None. Else create node,
14   recurse L, recurse R.
15 - Accept: Easiest to code.
16
17 3. INDENTED ENGLISH
18 serialize(root):
19     res = []
20     dfs(node):
21         if not node: res.append("N"); return
22         res.append(str(node.val))
23         dfs(left)
24         dfs(right)
25     return ",".join(res)
26 """
```

The Magic Question (Get Buy-In)

"I'll use Preorder Traversal (DFS) for both steps. I'll encode null nodes as a special character 'N'. Because Preorder is 'Root-Left-Right', I can easily reconstruct the tree by always popping the first element of the list. $O(N)$."

```
1 class Codec:
2
3     def serialize(self, root):
4         res = []
5
6         def dfs(node):
7             if not node:
8                 res.append("N")
9                 return
10            res.append(str(node.val))
11            dfs(node.left)
12            dfs(node.right)
13
14        dfs(root)
15        return ",".join(res)
16
17    def deserialize(self, data):
18        vals = data.split(",")
19        # Iterator is crucial for O(1) next() access
20        self.i = 0
21
22        def dfs():
23            if self.i >= len(vals): return None
24
25            token = vals[self.i]
26            self.i += 1
27
28            if token == "N":
29                return None
30
31            node = TreeNode(int(token))
32            node.left = dfs()
33            node.right = dfs()
34            return node
35
```

```
36         return dfs()
```

25 Trees: The Missing Piece & Tries

25.1 Construct Quad Tree

```
1  """
2  1. CLARIFYING QUESTIONS
3      - Q: What is a Quad Tree?
4          A: A tree where each node has 4 children (TL, TR, BL, BR).
5             Used to compress 2D grids (0s and 1s).
6      - Q: Leaf definition?
7          A: A region where all values are the same (all 0s or all 1s).
8      - Q: Grid size?
9          A: N x N, where N is power of 2.
10
11  2. LOGIC (RECURSION)
12      - Base Case: Check if current grid region is uniform (all same val).
13        - If yes: Return Leaf Node(val, isLeaf=True).
14      - Recursive Step:
15        - Split grid into 4 sub-grids.
16        - Recurse: TL, TR, BL, BR.
17        - Return Non-Leaf Node(val=*, isLeaf=False, children...).
18  """
19
20  # --- THE MAGIC QUESTION ---
21  # "I'll use recursion. For any grid section, I check if all values are identical.
22  # If so, I create a leaf. If not, I split the grid into four quadrants and recurse.
23  # O(N^2) time to visit all cells."
24
25  def construct(grid: List[List[int]]) -> 'Node':
26
27      def dfs(r, c, n):
28          # 1. Check if all values in this n x n region are the same
29          all_same = True
30          for i in range(n):
31              for j in range(n):
32                  if grid[r + i][c + j] != grid[r][c]:
33                      all_same = False
34                      break
35              if not all_same: break
36
37          # 2. Base Case: Is Leaf
38          if all_same:
39              return Node(grid[r][c] == 1, True, None, None, None, None)
40
41          # 3. Recursive Step: Divide and Conquer
42          n //= 2
43          top_left = dfs(r, c, n)
44          top_right = dfs(r, c + n, n)
45          bot_left = dfs(r + n, c, n)
46          bot_right = dfs(r + n, c + n, n)
47
48          return Node(1, False, top_left, top_right, bot_left, bot_right)
49
50  return dfs(0, 0, len(grid))
```

25.2 Implement Trie (Prefix Tree)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Inputs? A: Lowercase English letters.
4 - Q: Methods? A: insert, search, startsWith.
5
6 2. LOGIC (HASH MAP NODES)
7 - TrieNode class:
8   - children: { char : TrieNode }
9   - endOfWord: bool
10 - Insert: Traverse char by char. Create node if missing. Mark end.
11 - Search: Traverse. If char missing, False. If end, check endOfWord.
12 - StartsWith: Traverse. If char missing, False. If end, True.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll define a TrieNode with a hash map for children and a boolean flag for word
17 #   endings.
18 # Insertion and Search will simply traverse these nodes character by character.
19 # O(L) time where L is word length."
20
21 class TrieNode:
22     def __init__(self):
23         self.children = {}
24         self.endOfWord = False
25
26 class Trie:
27     def __init__(self):
28         self.root = TrieNode()
29
30     def insert(self, word: str) -> None:
31         cur = self.root
32         for c in word:
33             if c not in cur.children:
34                 cur.children[c] = TrieNode()
35             cur = cur.children[c]
36         cur.endOfWord = True
37
38     def search(self, word: str) -> bool:
39         cur = self.root
40         for c in word:
41             if c not in cur.children:
42                 return False
43             cur = cur.children[c]
44         return cur.endOfWord
45
46     def startsWith(self, prefix: str) -> bool:
47         cur = self.root
48         for c in prefix:
49             if c not in cur.children:
50                 return False
51             cur = cur.children[c]
52         return True
```

25.3 Design Add and Search Words Data Structure

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Dot '.' character? A: Matches ANY letter.
4     - Q: Solution? A: Recursion (DFS) needed for dots.
5
6 2. LOGIC (DFS FOR WILDCARDS)
7     - Standard Trie for insert.
8     - Search with DFS:
9         - If char is normal: Move to child.
10        - If char is '.': Loop through ALL children and recurse.
11        If any return True, then True.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use a standard Trie. For the search method, I'll use DFS.
16 # When I encounter a '.', I'll recursively check all children of the current node.
17 # If any path matches, return True."
18
19 class WordDictionary:
20     def __init__(self):
21         self.root = TrieNode()
22
23     def addWord(self, word: str) -> None:
24         cur = self.root
25         for c in word:
26             if c not in cur.children:
27                 cur.children[c] = TrieNode()
28             cur = cur.children[c]
29         cur.endOfWord = True
30
31     def search(self, word: str) -> bool:
32
33         def dfs(j, root):
34             cur = root
35             for i in range(j, len(word)):
36                 c = word[i]
37                 if c == ".":
38                     # Wildcard: Try all possible children
39                     for child in cur.children.values():
40                         if dfs(i + 1, child):
41                             return True
42                     return False
43                 else:
44                     # Standard match
45                     if c not in cur.children:
46                         return False
47                     cur = cur.children[c]
48             return cur.endOfWord
49
50         return dfs(0, self.root)
```

25.4 Extra Characters in a String

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: string s, dictionary of words.
4   - Q: Goal? A: Min number of chars left over after breaking s into dictionary words.
5   - Q: Overlap? A: Dictionary words can be reused.
6
7 2. APPROACHES
8   - Approach 1: DP (1D).
9     - dp[i] = min extra chars for prefix s[:i].
10    - Default: dp[i] = dp[i-1] + 1 (treat s[i-1] as extra).
11    - Check dictionary: For every word matching s[j:i], dp[i] = min(dp[i], dp[j]).
12    - Optimization: Use Trie to check matches efficiently.
13
14 3. LOGIC (DP + TRIE)
15   - Build Trie from dictionary.
16   - Iterate i from 1 to N.
17   - Start at i-1. Traverse Trie backwards? No, traverse forward from every j.
18   - Simpler DP: 'dp[i]' min extra from index 'i' to end.
19 """
20
21 # --- THE MAGIC QUESTION ---
22 # "I'll use Dynamic Programming. 'dp[i]' represents the min extra characters for the
23 #   suffix 's[i:]'.
24 # At each index, I can either skip the character (cost+1) or match a word from the
25 #   dictionary (cost 0).
26 # A Trie speeds up the matching process."
27
28 def minExtraChar(s: str, dictionary: List[str]) -> int:
29     # Build Trie
30     root = TrieNode()
31     for w in dictionary:
32         cur = root
33         for c in w:
34             if c not in cur.children: cur.children[c] = TrieNode()
35             cur = cur.children[c]
36         cur.endOfWord = True
37
38     # DP: dp[i] = min extra chars for s[i:]
39     # Base case: dp[len(s)] = 0
40     n = len(s)
41     dp = [0] * (n + 1)
42
43     for i in range(n - 1, -1, -1):
44         # Option 1: Skip current char (1 extra error)
45         dp[i] = 1 + dp[i + 1]
46
47         # Option 2: Try to match words starting at i
48         cur = root
49         for j in range(i, n):
50             if s[j] not in cur.children:
51                 break
52             cur = cur.children[s[j]]
53             if cur.endOfWord:
54                 dp[i] = min(dp[i], dp[j + 1])
55
56     return dp[0]
```

25.5 Word Search II

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: Board of chars, List of words.
4 - Q: Output? A: All words found on board.
5 - Q: Performance? A: Running DFS for each word is too slow (Trie required).
6
7 2. LOGIC (BACKTRACKING + TRIE OPTIMIZATION)
8 - Step 1: Put all words into a Trie.
9 - Step 2: Run DFS from every cell (r, c).
10 - Step 3: Optimization - Pass the Trie Node into the DFS.
11 - If 'node.children[board[r][c]]' exists, proceed.
12 - If 'node.endOfWord', add to result.
13 - *Crucial Pruning*: Remove word from Trie after finding it to avoid duplicates/re-
14   work.
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll insert all words into a Trie. Then I'll iterate every cell on the board.
18 # If a cell matches a Trie root child, I start a DFS, passing the Trie node along.
19 # This lets me search for all words simultaneously. O(M*N*4^L)."
20
21 def findWords(board: List[List[str]], words: List[str]) -> List[str]:
22     root = TrieNode()
23     for w in words:
24         cur = root
25         for c in w:
26             if c not in cur.children: cur.children[c] = TrieNode()
27             cur = cur.children[c]
28         cur.endOfWord = True
29         cur.word = w # Store full word at leaf for easy access
30
31     ROWS, COLS = len(board), len(board[0])
32     res = set()
33     visited = set()
34
35     def dfs(r, c, node):
36         if (r < 0 or c < 0 or r >= ROWS or c >= COLS or
37             (r, c) in visited or
38             board[r][c] not in node.children):
39             return
40
41         visited.add((r, c))
42         node = node.children[board[r][c]]
43
44         if node.endOfWord:
45             res.add(node.word)
46             # Pruning: Optional - remove word from trie or mark false
47
48         dfs(r + 1, c, node)
49         dfs(r - 1, c, node)
50         dfs(r, c + 1, node)
51         dfs(r, c - 1, node)
52
53         visited.remove((r, c))
54
55     for r in range(ROWS):
56         for c in range(COLS):
57             dfs(r, c, root)
58
59     return list(res)
```

26 Graphs: Grid DFS & BFS

26.1 Island Perimeter

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: Grid of 0s (water) and 1s (land).
```

```

4 - Q: Diagonals? A: No, only 4 directions.
5 - Q: Guaranteed island? A: Yes.
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: DFS.
9   - Logic: Visit every land cell. Add 4 for base perimeter. Subtract 1 for each land
10     neighbor.
11     - Complexity:  $O(R \cdot C)$ .
12     - Accept: Valid.
13 - Approach 2: Iterative Math (Optimal).
14   - Logic: Iterate grid. If cell is 1, add 4.
15   - Check Left and Top neighbors. If they are land, subtract 2 (because the shared
16     edge removes 1 perimeter from EACH).
17   - Why Left/Top? We avoid double counting edges we've already processed.
18   - Complexity:  $O(R \cdot C)$ .
19   - Accept: Cleanest.
20
21 3. INDENTED ENGLISH (LOGIC)
22 function islandPerimeter(grid):
23     res = 0
24     for r in rows:
25         for c in cols:
26             if grid[r][c] == 1:
27                 res += 4
28                 if r > 0 and grid[r-1][c] == 1: res -= 2
29                 if c > 0 and grid[r][c-1] == 1: res -= 2
30     return res
31 """

```

The Magic Question (Get Buy-In)

"I'll iterate through the grid. For every land cell, I add 4 to the perimeter. If I detect a land neighbor to the Top or Left, I subtract 2 because that shared edge is internal. $O(R \cdot C)$."

```

1 def islandPerimeter(grid: List[List[int]]) -> int:
2     ROWS, COLS = len(grid), len(grid[0])
3     peri = 0
4
5     for r in range(ROWS):
6         for c in range(COLS):
7             if grid[r][c] == 1:
8                 peri += 4
9
10                # Check Top Neighbor
11                if r > 0 and grid[r - 1][c] == 1:
12                    peri -= 2
13
14                # Check Left Neighbor
15                if c > 0 and grid[r][c - 1] == 1:
16                    peri -= 2
17
18    return peri
19
20 """
21 5. DRY RUN (HUMAN DEBUGGER)
22 [[0,1],
23  [1,1]]
24 - (0,1): peri = 4.
25 - (1,0): peri = 4 + 4 = 8.
26 - (1,1): peri = 8 + 4 = 12.
27   - Top (0,1) is 1. peri -= 2 -> 10.
28   - Left (1,0) is 1. peri -= 2 -> 8.
29 - Return 8. Correct.
30 """

```

26.2 Number of Islands

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Diagonals connect? A: No.
4 - Q: Modify grid? A: Yes, can mark visited by changing '1' to '0'.
5
6 2. APPROACHES TO CONSIDER & REJECT
7 - Approach 1: BFS.
8 - Use a queue for every island found.
9 - Approach 2: DFS (Recursion).
10 - Logic: Iterate grid. When '1' found, increment count, launch DFS to sink (mark
11 '0') all connected land.
12 - Complexity:  $O(R \cdot C)$ .
13 - Accept: Easiest to write.
14
15 3. INDENTED ENGLISH (LOGIC)
16 function numIslands(grid):
17     count = 0
18     def dfs(r, c):
19         if out_of_bounds or grid[r][c] == '0': return
20         grid[r][c] = '0' # Sink it
21         dfs(r+1, c), dfs(r-1, c), dfs(r, c+1), dfs(r, c-1)
22
23     for r in rows:
24         for c in cols:
25             if grid[r][c] == '1':
26                 count++
27                 dfs(r, c)
28     return count
29 """
```

The Magic Question (Get Buy-In)

"I'll iterate through every cell. Whenever I find a '1', I increment the island count and launch a DFS to 'sink' the entire island by turning connected '1's into '0's. This avoids double counting. $O(R \cdot C)$."

```
1 def numIslands(grid: List[List[str]]) -> int:
2     if not grid: return 0
3     ROWS, COLS = len(grid), len(grid[0])
4     islands = 0
5
6     def dfs(r, c):
7         # Boundary check and Water check
8         if (r < 0 or c < 0 or r >= ROWS or c >= COLS or
9             grid[r][c] == "0"):
10             return
11
12         # Mark as visited (Sink the island)
13         grid[r][c] = "0"
14
15         # Explore neighbors
16         dfs(r + 1, c)
17         dfs(r - 1, c)
18         dfs(r, c + 1)
19         dfs(r, c - 1)
20
21     for r in range(ROWS):
22         for c in range(COLS):
23             if grid[r][c] == "1":
24                 islands += 1
25                 dfs(r, c)
26
27     return islands
28
29 """
30 5. DRY RUN (HUMAN DEBUGGER)
31 grid = [["1","1"], ["0","1"]]
32 - (0,0)='1'. islands=1. dfs(0,0).
```

```
33     - Sink (0,0). Call (0,1).
34     - (0,1): Sink. Call (1,1).
35     - (1,1): Sink.
36     - Loop continues... all '1's are now '0'.
37     - Return 1.
38 """
```

26.3 Max Area of Island

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: If no islands? A: 0.
4     - Q: Modifying input allowed? A: Yes (for visited tracking).
5
6 2. LOGIC (DFS WITH RETURN VALUE)
7     - Same as Number of Islands, but DFS returns an integer (area).
8     - Area = 1 (self) + Up + Down + Left + Right.
9     - Update global max.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll iterate through the grid. When I find a '1', I'll start a DFS that returns the
14   size of the connected component. I'll maintain a variable for the maximum area found
15   so far. $O(R \cdot C)$."
16
17 def maxAreaOfIsland(grid: List[List[int]]) -> int:
18     ROWS, COLS = len(grid), len(grid[0])
19     max_area = 0
20
21     def dfs(r, c):
22         if (r < 0 or c < 0 or r >= ROWS or c >= COLS or
23             grid[r][c] == 0):
24             return 0
25
26         # Mark visited
27         grid[r][c] = 0
28
29         return (1 + dfs(r + 1, c) +
30                 dfs(r - 1, c) +
31                 dfs(r, c + 1) +
32                 dfs(r, c - 1))
33
34     for r in range(ROWS):
35         for c in range(COLS):
36             if grid[r][c] == 1:
37                 max_area = max(max_area, dfs(r, c))
38
39     return max_area
```

26.4 Walls and Gates

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: INF (Empty), -1 (Wall), 0 (Gate).
4 - Q: Goal? A: Fill INF with distance to nearest Gate.
5 - Q: Strategy? A: BFS starting from ALL gates simultaneously (Multi-source BFS).
6
7 2. APPROACHES TO CONSIDER & REJECT
8 - Approach 1: DFS from every empty room.
9   - Reject:  $O((RC)^2)$ . Too many redundant computations.
10 - Approach 2: BFS from every Gate (Sequential).
11   - Reject: Overwrites shorter paths with longer ones if not careful.
12 - Approach 3: Multi-Source BFS.
13   - Logic: Add ALL gates to queue first. Dist 0.
14   - Expand layer by layer. If neighbor is INF, set dist and add to queue.
15   - Complexity:  $O(R \cdot C)$ .
16   - Accept: Optimal.
17
18 3. INDENTED ENGLISH (LOGIC)
19 function wallsAndGates(rooms):
20     q = deque()
21     for r, c in grid:
22         if room == 0: q.append((r,c))
23
24     dist = 0
25     while q:
26         for _ in range(len(q)):
27             r, c = q.popleft()
28             for dr, dc in dirs:
29                 nr, nc = r+dr, c+dc
30                 if valid and rooms[nr][nc] == INF:
31                     rooms[nr][nc] = dist + 1
32                     q.append((nr, nc))
33     dist += 1
34 """
```

The Magic Question (Get Buy-In)

"This is a Multi-Source BFS problem. I'll initialize the queue with all the Gate coordinates (distance 0). I'll expand layer by layer, updating 'INF' rooms with 'distance + 1' only the first time I visit them. This guarantees the shortest path for everyone. $O(R \cdot C)$."

```
1 from collections import deque
2
3 def wallsAndGates(rooms: List[List[int]]) -> None:
4     ROWS, COLS = len(rooms), len(rooms[0])
5     q = deque()
6
7     # 1. Add all Gates to Queue
8     for r in range(ROWS):
9         for c in range(COLS):
10             if rooms[r][c] == 0:
11                 q.append((r, c))
12
13     dist = 0
14     directions = [[1,0], [-1,0], [0,1], [0,-1]]
15
16     # 2. Multi-Source BFS
17     while q:
18         dist += 1
19         for _ in range(len(q)):
20             r, c = q.popleft()
21
22             for dr, dc in directions:
23                 nr, nc = r + dr, c + dc
24
25                 # Check bounds and if room is empty (INF)
26                 # Note: 2147483647 is typical INF value
27                 if (0 <= nr < ROWS and 0 <= nc < COLS and
```

```

28         rooms[nr][nc] == 2147483647):
29
30         rooms[nr][nc] = dist
31         q.append((nr, nc))
32
33     """
34     5. DRY RUN
35     [[INF, -1], [0, INF]]
36     - Q = [(1,0)]. Dist=0.
37     - Pop (1,0). Neighbors: (0,0)=INF, (1,1)=INF.
38     - Set (0,0)=1, (1,1)=1. Q=[(0,0), (1,1)].
39     - Loop end. dist=1.
40     - Next layer...
41     """

```

26.5 Rotting Oranges

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: 0 (empty), 1 (fresh), 2 (rotten).
4 - Q: Logic? A: Rotten infect adjacent fresh every minute.
5 - Q: Result? A: Min minutes to rot all. -1 if impossible.
6
7 2. LOGIC (MULTI-SOURCE BFS)
8 - Identical pattern to Walls and Gates.
9 - Step 1: Count fresh oranges (to check -1 later) and add Rotten to Queue.
10 - Step 2: BFS. Decrement fresh count as we infect.
11 - Step 3: Return time. If fresh > 0, return -1.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use Multi-Source BFS starting from all rotten oranges. I'll track the number of
16 # fresh oranges.
17 # Each layer of BFS represents 1 minute. If fresh count is zero at the end, I return the
18 # time; otherwise -1."
19
20 def orangesRotting(grid: List[List[int]]) -> int:
21     ROWS, COLS = len(grid), len(grid[0])
22     q = deque()
23     fresh = 0
24
25     for r in range(ROWS):
26         for c in range(COLS):
27             if grid[r][c] == 1: fresh += 1
28             if grid[r][c] == 2: q.append((r, c))
29
30     minutes = 0
31     directions = [[1,0], [-1,0], [0,1], [0,-1]]
32
33     while q and fresh > 0:
34         minutes += 1
35         for _ in range(len(q)):
36             r, c = q.popleft()
37             for dr, dc in directions:
38                 nr, nc = r + dr, c + dc
39                 if (0 <= nr < ROWS and 0 <= nc < COLS and
40                     grid[nr][nc] == 1):
41                     grid[nr][nc] = 2 # Rot it
42                     fresh -= 1
43                     q.append((nr, nc))
44
45     return minutes if fresh == 0 else -1
```

26.6 Pacific Atlantic Water Flow

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Flow direction? A: High to Low (or Equal).
4 - Q: Goal? A: Cells that can reach BOTH Pacific (Top/Left) and Atlantic (Bot/Right).
5
6 2. APPROACHES
7 - Approach 1: Check every cell.  $O((RC)^2)$ . Slow.
8 - Approach 2: Inverse Flow (From Ocean to Land).
9 - Logic: Start from Pacific edges. DFS upwards (Low -> High). Mark reachable set 'pac'.
10 - Start from Atlantic edges. DFS upwards. Mark reachable set 'atl'.
11 - Result is intersection of 'pac' and 'atl'.
12 - Complexity:  $O(R*C)$ .
13 - Accept: Optimal.
14
15 3. INDENTED ENGLISH
16 function pacificAtlantic(heights):
17     pac, atl = set(), set()
18     dfs(r, c, visitSet, prevHeight):
19         if visited or heights[r][c] < prevHeight: return
20         add to visitSet
21         recurse neighbors
22
23     for c in cols: dfs(0, c, pac), dfs(rows-1, c, atl)
24     for r in rows: dfs(r, 0, pac), dfs(r, cols-1, atl)
25     return list(pac & atl)
26 """
```

The Magic Question (Get Buy-In)

"Instead of checking where water goes from each cell, I'll reverse the problem: where can water come *from*? I'll run DFS from the Pacific borders and the Atlantic borders upwards (Low to High). The intersection of cells reachable by both oceans is the answer."

```
1 def pacificAtlantic(heights: List[List[int]]) -> List[List[int]]:
2     ROWS, COLS = len(heights), len(heights[0])
3     pac, atl = set(), set()
4
5     def dfs(r, c, visit, prevHeight):
6         if ((r, c) in visit or
7             r < 0 or c < 0 or r >= ROWS or c >= COLS or
8             heights[r][c] < prevHeight): # Water can't flow up (inverse logic)
9             return
10
11         visit.add((r, c))
12         dfs(r + 1, c, visit, heights[r][c])
13         dfs(r - 1, c, visit, heights[r][c])
14         dfs(r, c + 1, visit, heights[r][c])
15         dfs(r, c - 1, visit, heights[r][c])
16
17     # Launch from Top/Bottom rows
18     for c in range(COLS):
19         dfs(0, c, pac, heights[0][c])
20         dfs(ROWS - 1, c, atl, heights[ROWS - 1][c])
21
22     # Launch from Left/Right cols
23     for r in range(ROWS):
24         dfs(r, 0, pac, heights[r][0])
25         dfs(r, COLS - 1, atl, heights[r][COLS - 1])
26
27     return list(pac.intersection(atl))
```

26.7 Surrounded Regions

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: What is "surrounded"?
4     A: Connected component of '0's not touching the border.
5   - Q: Capture logic? A: Flip '0' to 'X'.
6
7 2. LOGIC (REVERSE THINKING)
8   - Don't hunt for surrounded regions.
9   - Instead, find UNSURROUNDED regions (Safe '0's).
10  - Any '0' on the border is safe. Any '0' connected to a border '0' is safe.
11  - Phase 1: DFS from borders, mark '0' -> 'T' (Temporary Safe).
12  - Phase 2: Loop grid. '0' -> 'X' (Captured). 'T' -> '0' (Restored).
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "Instead of finding surrounded regions, I'll find the ones that *cannot* be captured.
17 # I'll run DFS from all border '0's and mark them as temporary 'T'.
18 # Finally, I'll flip remaining '0's to 'X' and restore 'T's to '0'. O(RC)."
19
20 def solve(board: List[List[str]]) -> None:
21     ROWS, COLS = len(board), len(board[0])
22
23     def capture(r, c):
24         if (r < 0 or c < 0 or r >= ROWS or c >= COLS or
25             board[r][c] != "0"):
26             return
27
28         board[r][c] = "T" # Mark as safe
29         capture(r + 1, c)
30         capture(r - 1, c)
31         capture(r, c + 1)
32         capture(r, c - 1)
33
34     # 1. Capture unsurrounded regions (Border 0's)
35     for r in range(ROWS):
36         for c in range(COLS):
37             if (board[r][c] == "0" and
38                 (r in [0, ROWS-1] or c in [0, COLS-1])):
39                 capture(r, c)
40
41     # 2. Final Sweep
42     for r in range(ROWS):
43         for c in range(COLS):
44             if board[r][c] == "0":
45                 board[r][c] = "X" # Capture surrounded
46             elif board[r][c] == "T":
47                 board[r][c] = "0" # Restore safe
```

27 Graphs: Dependencies & Disjoint Sets

27.1 Clone Graph

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Deep copy? A: Yes, distinct memory for nodes.
4   - Q: Graph type? A: Connected, undirected.
5   - Q: Empty graph? A: Return None.
6
7 2. LOGIC (MAP TRACKING)
8   - Challenge: Cycles. If A connects to B, and B to A, we loop infinitely.
9   - Fix: Use a Hash Map 'old_node -> new_node'.
10  - Logic:
11     - If node already in map, return map[node].
12     - Else, create copy, add to map.
13     - Recursively clone neighbors.
14 """
15
16 # --- THE MAGIC QUESTION ---
```

```

17 # "I'll use DFS with a Hash Map to map original nodes to their copies.
18 # This handles cycles gracefully: if I see a node I've already cloned, I simply return
    the reference from the map. O(V + E)."
```

```

19
20 def cloneGraph(node: Optional['Node']) -> Optional['Node']:
21     oldToNew = {}
22
23     def dfs(node):
24         if not node:
25             return None
26
27         if node in oldToNew:
28             return oldToNew[node]
29
30         # 1. Create Copy
31         copy = Node(node.val)
32         oldToNew[node] = copy
33
34         # 2. Clone Neighbors
35         for nei in node.neighbors:
36             copy.neighbors.append(dfs(nei))
37
38         return copy
39
40     return dfs(node)
```

27.2 Course Schedule

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: numCourses, prerequisites [[a, b]].
4 - Q: Goal? A: Return True if possible to finish all.
5 - Q: Conflict? A: [0, 1] and [1, 0] is a cycle. Impossible.
6
7 2. APPROACHES
8 - Approach 1: Topological Sort (Kahn's Algo).
9   - Count indegrees. Remove 0-indegree nodes. If nodes remaining > 0, cycle.
10 - Approach 2: DFS with States.
11   - States: 0 (Unvisited), 1 (Visiting), 2 (Visited).
12   - If we see a '1' while exploring, CYCLE detected.
13   - Accept: Standard for cycle detection.
14
15 3. INDENTED ENGLISH (DFS STATES)
16 function canFinish(n, prereqs):
17     adj = map
18     visit = [0] * n # 0:unvisited, 1:visiting, 2:visited
19
20     def dfs(crs):
21         if visit[crs] == 1: return False # Cycle
22         if visit[crs] == 2: return True  # Already validated
23
24         visit[crs] = 1
25         for pre in adj[crs]:
26             if not dfs(pre): return False
27         visit[crs] = 2
28         return True
29 """
```

The Magic Question (Get Buy-In)

"I'll model this as a directed graph. To check if the courses can be finished, I need to detect cycles. I'll use DFS with three states: Unvisited, Visiting, and Visited. If I encounter a node marked 'Visiting', a cycle exists. $O(V + E)$."

```
1 def canFinish(numCourses: int, prerequisites: List[List[int]]) -> bool:
2     # Build Adjacency List
3     preMap = {i: [] for i in range(numCourses)}
4     for crs, pre in prerequisites:
5         preMap[crs].append(pre)
6
7     # visitSet stores nodes currently in the recursion stack
8     visitSet = set()
9
10    def dfs(crs):
11        if crs in visitSet:
12            return False # Cycle detected
13        if preMap[crs] == []:
14            return True # No prereqs (or already cleared)
15
16        visitSet.add(crs)
17        for pre in preMap[crs]:
18            if not dfs(pre):
19                return False
20
21        visitSet.remove(crs)
22        preMap[crs] = [] # Optimization: Mark as cleared
23        return True
24
25    for crs in range(numCourses):
26        if not dfs(crs):
27            return False
28
29    return True
30
31 """
32 5. DRY RUN
```

```
33     0 -> 1 -> 0 (Cycle)
34     - dfs(0). Set={0}. Call dfs(1).
35     - dfs(1). Set={0,1}. Call dfs(0).
36     - dfs(0). 0 in Set. Return False.
37     """
```

27.3 Course Schedule II

```

1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Return? A: The ordering of courses.
4     - Q: If impossible? A: Return [].
5
6 2. LOGIC (KAHN'S ALGORITHM / BFS)
7     - This is slightly more intuitive for "Ordering" than DFS.
8     - Concept: "Peel the Onion".
9     - 1. Calculate Indegree (number of prereqs) for every node.
10    - 2. Add nodes with 0 Indegree to Queue.
11    - 3. Pop node, add to result, decrement neighbors' indegree.
12    - 4. If neighbor hits 0, add to Queue.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll use Kahn's Algorithm (BFS). I track the 'indegree' (number of prerequisites) for
17 # each course.
18 # I start by processing courses with 0 prerequisites, then 'remove' them to unlock
19 # dependent courses.
20 # If the result length equals 'numCourses', we succeeded. O(V + E)."
21
22 def findOrder(numCourses: int, prerequisites: List[List[int]]) -> List[int]:
23     adj = {i: [] for i in range(numCourses)}
24     indegree = [0] * numCourses
25
26     # 1. Build Graph & Indegrees
27     for crs, pre in prerequisites:
28         adj[pre].append(crs) # pre points to crs
29         indegree[crs] += 1
30
31     # 2. Init Queue with 0-indegree nodes
32     q = deque([i for i in range(numCourses) if indegree[i] == 0])
33     res = []
34
35     # 3. Process
36     while q:
37         node = q.popleft()
38         res.append(node)
39
40         for neighbor in adj[node]:
41             indegree[neighbor] -= 1
42             if indegree[neighbor] == 0:
43                 q.append(neighbor)
44
45     return res if len(res) == numCourses else []
46
47 """
48 5. DRY RUN
49     2->1, 1->0. (Must take 0, then 1, then 2).
50     - Adj: {0:[1], 1:[2], 2:[]}. Indegree: [0, 1, 1].
51     - Q: [0].
52     - Pop 0. res=[0]. Neighbor 1 indegree -> 0. Q=[1].
53     - Pop 1. res=[0,1]. Neighbor 2 indegree -> 0. Q=[2].
54     - Pop 2. res=[0,1,2].
55 """

```

27.4 Graph Valid Tree

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Definition of Tree?
4   A: 1. Fully Connected. 2. No Cycles.
5 - Q: Input? A: n nodes, edges list.
6
7 2. APPROACHES
8 - Approach 1: Union-Find.
9   - Logic: For each edge (u, v), if union(u, v) returns False (already connected),
10  Cycle!
11   - At end, check if num_components == 1.
12   - Accept: Optimal.
13 - Approach 2: DFS.
14   - Logic: Visit from node 0. Check for back-edges (Cycle). Check visited count == n
15   (Connected).
16
17 3. INDENTED ENGLISH (DFS)
18 function validTree(n, edges):
19     if len(edges) != n-1: return False # Tree property
20     adj = map
21     visit = set()
22
23     def dfs(i, prev):
24         if i in visit: return False # Cycle
25         visit.add(i)
26         for j in adj[i]:
27             if j == prev: continue
28             if not dfs(j, i): return False
29         return True
30
31     return dfs(0, -1) and len(visit) == n
32 """
```

The Magic Question (Get Buy-In)

"A graph is a valid tree if it has no cycles and is fully connected. I'll use DFS starting from node 0. I maintain a 'prev' pointer to avoid false cycle detection on the edge I just came from. Finally, I verify we visited all 'n' nodes. $O(V + E)$."

```
1 def validTree(n: int, edges: List[List[int]]) -> bool:
2     if not n:
3         return True
4
5     adj = {i: [] for i in range(n)}
6     for u, v in edges:
7         adj[u].append(v)
8         adj[v].append(u)
9
10    visit = set()
11
12    def dfs(curr, prev):
13        if curr in visit:
14            return False # Cycle detected
15
16        visit.add(curr)
17        for neighbor in adj[curr]:
18            if neighbor == prev:
19                continue
20            if not dfs(neighbor, curr):
21                return False
22
23        return True
24
25    # Check 1: No Cycles AND Check 2: Connected (visited all nodes)
26    return dfs(0, -1) and len(visit) == n
```

27.5 Number of Connected Components in an Undirected Graph

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Undirected? A: Yes.
4 - Q: Disconnected nodes count? A: Yes, a lone node is 1 component.
5
6 2. LOGIC (UNION-FIND)
7 - Start with 'n' components.
8 - For every edge (u, v):
9   - If union(u, v) is successful (they were separate), decrement 'n'.
10 - Return 'n'.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use the Union-Find (Disjoint Set) data structure. I start with 'n' components.
15 # For every edge, I try to union the two nodes. If they merge, I decrement the component
16 # count.
17 # O(E * alpha(n)) approx O(E)."
18
19 def countComponents(n: int, edges: List[List[int]]) -> int:
20     parent = [i for i in range(n)]
21     rank = [1] * n
22
23     def find(n):
24         p = parent[n]
25         while p != parent[p]:
26             parent[p] = parent[parent[p]] # Path compression
27             p = parent[p]
28         return p
29
30     def union(n1, n2):
31         p1, p2 = find(n1), find(n2)
32         if p1 == p2:
33             return 0 # Already merged
34
35         if rank[p1] > rank[p2]:
36             parent[p2] = p1
37             rank[p1] += rank[p2]
38         else:
39             parent[p1] = p2
40             rank[p2] += rank[p1]
41         return 1 # Successful merge
42
43     res = n
44     for n1, n2 in edges:
45         res -= union(n1, n2)
46
47     return res
```

27.6 Redundant Connection

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: What is redundant? A: An edge that creates a cycle in a tree.
4   - Q: Input? A: N nodes, N edges (Tree + 1 edge).
5   - Q: Return? A: The edge that creates the cycle (last one in input).
6
7 2. LOGIC (UNION-FIND CYCLE DETECTION)
8   - Iterate through edges.
9   - Try union(u, v).
10  - If union returns False (parents already same), then adding this edge creates a
    cycle.
11  - Return [u, v].
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll iterate through the edges and apply Union-Find. If I encounter an edge
    connecting two nodes
16 # that essentially share the same parent, that edge creates a cycle and is the answer. 0
    (N)."
17
18 def findRedundantConnection(edges: List[List[int]]) -> List[int]:
19     # N nodes (1 to N)
20     parent = [i for i in range(len(edges) + 1)]
21     rank = [1] * (len(edges) + 1)
22
23     def find(n):
24         p = parent[n]
25         while p != parent[p]:
26             parent[p] = parent[parent[p]]
27             p = parent[p]
28         return p
29
30     def union(n1, n2):
31         p1, p2 = find(n1), find(n2)
32         if p1 == p2:
33             return False # Cycle detected!
34
35         if rank[p1] > rank[p2]:
36             parent[p2] = p1
37             rank[p1] += rank[p2]
38         else:
39             parent[p1] = p2
40             rank[p2] += rank[p1]
41         return True
42
43     for n1, n2 in edges:
44         if not union(n1, n2):
45             return [n1, n2]
46
47     return []
```

27.7 Accounts Merge

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Merge rule? A: If two accounts share ANY email, they are the same person.
4   - Q: Output? A: [[Name, sorted_emails...]].
5
6 2. LOGIC (UNION-FIND ON EMAILS)
7   - Node = Email (not Account Index).
8   - 1. Map each email to the Name (for final output).
9   - 2. Union-Find: For each account [Name, e1, e2, e3], union(e1, e2), union(e2, e3).
10  - 3. Grouping: Map 'RootEmail -> List[Emails]'.
11  - 4. Format Output.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll treat every email as a node. Within each account, I'll union all emails together
16 # Then, I'll group emails by their Union-Find root and attach the associated name. O(N *
17   K log K)."
18
19 def accountsMerge(accounts: List[List[str]]) -> List[List[str]]:
20     uf = UnionFind(len(accounts) * 10) # Simplified logic below
21     # Actually, custom UF class is best, or map-based UF
22
23     parent = {}
24     emailToName = {}
25
26     def find(n):
27         # Path compression on the fly
28         if n not in parent: parent[n] = n
29         if n != parent[n]:
30             parent[n] = find(parent[n])
31         return parent[n]
32
33     def union(n1, n2):
34         p1, p2 = find(n1), find(n2)
35         if p1 != p2:
36             parent[p1] = p2 # Simple union
37
38     # 1. Union Emails
39     for acc in accounts:
40         name = acc[0]
41         first_email = acc[1]
42         for email in acc[1:]:
43             union(first_email, email)
44             emailToName[email] = name
45         emailToName[first_email] = name # Ensure first is recorded
46
47     # 2. Group by Root
48     emailGroup = collections.defaultdict(list)
49     for email in parent:
50         root = find(email)
51         emailGroup[root].append(email)
52
53     # 3. Format
54     res = []
55     for root, emails in emailGroup.items():
56         name = emailToName[root]
57         res.append([name] + sorted(emails))
58
59     return res
```

28 Graphs: Shortest Path & Logic

28.1 Open The Lock

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Start? A: "0000".
```

```

4 - Q: Deadends? A: If start or current node is in deadends, we stop/skip.
5 - Q: Moves? A: Rotate any of 4 wheels +/- 1. Wrap 0<->9.
6
7 2. LOGIC (BFS SHORTEST PATH)
8 - BFS is ideal for finding the minimum number of turns.
9 - Nodes: "0000" to "9999".
10 - Edges: +/- 1 on each digit (8 edges per node).
11 - Visited Set: Includes 'deadends' to block paths.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use BFS to find the shortest path. I'll treat 'deadends' as visited nodes so I
16 # never traverse them.
17 # From each state, I generate 8 possible moves (rotating each wheel up or down). O(10^4)
18 # ."
19
20 def openLock(deadends: List[str], target: str) -> int:
21     if "0000" in deadends: return -1
22
23     q = deque([("0000", 0)]) # (lock_state, turns)
24     visit = set(deadends)
25     visit.add("0000")
26
27     def children(lock):
28         res = []
29         for i in range(4):
30             digit = str((int(lock[i]) + 1) % 10) # +1 move
31             res.append(lock[:i] + digit + lock[i+1:])
32
33             digit = str((int(lock[i]) - 1 + 10) % 10) # -1 move
34             res.append(lock[:i] + digit + lock[i+1:])
35         return res
36
37     while q:
38         lock, turns = q.popleft()
39         if lock == target:
40             return turns
41
42         for child in children(lock):
43             if child not in visit:
44                 visit.add(child)
45                 q.append((child, turns + 1))
46
47     return -1

```

28.2 Evaluate Division

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: a/b = 2.0.
4 - Q: Query? A: a/c = ?.
5 - Q: Logic? A: Graph. a->b (2.0), b->a (0.5).
6 - Query a/c is path a -> ... -> c. Product of edge weights.
7
8 2. LOGIC (GRAPH CONSTRUCTION + DFS/BFS)
9 - 1. Build Adjacency List: {a: [(b, 2.0)], b: [(a, 0.5)]}.
10 - 2. For each query (start, end):
11 - Run DFS/BFS to find path.
12 - Keep track of running product.
13 - If unreachable, return -1.0.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll model variables as nodes and equations as directed edges with weights.
18 # 'a / b = 2.0' means an edge 'a -> b' with weight 2.0 and 'b -> a' with weight 0.5.
19 # For each query, I'll run BFS to find the path product."
20
21 def calcEquation(equations: List[List[str]], values: List[float], queries: List[List[str]]
22                 ) -> List[float]:
23     adj = collections.defaultdict(list)
24
25     # 1. Build Graph
26     for i, eq in enumerate(equations):
27         a, b = eq
28         adj[a].append((b, values[i]))
29         adj[b].append((a, 1 / values[i]))
30
31     def bfs(src, target):
32         if src not in adj or target not in adj:
33             return -1.0
34
35         q = deque([(src, 1.0)]) # (node, current_product)
36         visit = set()
37         visit.add(src)
38
39         while q:
40             n, w = q.popleft()
41             if n == target:
42                 return w
43
44             for neighbor, weight in adj[n]:
45                 if neighbor not in visit:
46                     visit.add(neighbor)
47                     q.append((neighbor, w * weight))
48
49         return -1.0
50
51     return [bfs(q[0], q[1]) for q in queries]
```

28.3 Minimum Height Trees

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Goal? A: Find root(s) that minimize tree height.
4   - Q: Output? A: List of roots (1 or 2 possible).
5
6 2. LOGIC (TOPOLOGICAL PEELING)
7   - Do NOT run BFS from every node (TLE).
8   - Insight: The "Center" of the graph yields min height.
9   - Algo:
10    1. Identify Leaves (Indegree 1).
11    2. Remove leaves layer by layer (Peel the onion).
12    3. The last remaining 1 or 2 nodes are the answer.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "The roots of Minimum Height Trees are the centroids of the graph. I'll use a
17 # topological sort approach:
18 # repeatedly trim leaf nodes (nodes with 1 connection) until only 1 or 2 nodes remain. O
19 # (V)."
```

```
18
19 def findMinHeightTrees(n: int, edges: List[List[int]]) -> List[int]:
20     if n == 1: return [0]
21
22     adj = collections.defaultdict(list)
23     for u, v in edges:
24         adj[u].append(v)
25         adj[v].append(u)
26
27     # Initialize Leaves
28     leaves = [i for i in range(n) if len(adj[i]) == 1]
29     remaining_nodes = n
30
31     while remaining_nodes > 2:
32         remaining_nodes -= len(leaves)
33         new_leaves = []
34
35         for leaf in leaves:
36             # The only neighbor of a leaf
37             neighbor = adj[leaf].pop()
38             adj[neighbor].remove(leaf) # Remove edge
39
40             if len(adj[neighbor]) == 1:
41                 new_leaves.append(neighbor)
42
43         leaves = new_leaves
44
45     return leaves
```

28.4 Course Schedule IV

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Query? A: Is course 'u' a prerequisite of 'v'? (Direct or Indirect).
4   - Q: Constraints? A: N <= 100. Small enough for O(N^3).
5
6 2. APPROACHES
7   - Approach 1: BFS/DFS per query. O(Q * (V+E)).
8   - Approach 2: Floyd-Warshall (Transitive Closure).
9     - Logic: If A -> B and B -> C, then A -> C.
10    - Matrix 'reachable[i][j]'.
11    - Complexity: O(N^3).
12    - Accept: Optimal for dense queries on small graph.
13
14 3. INDENTED ENGLISH
15 function checkIfPrerequisite(numCourses, prereqs, queries):
16     reachable = [[False]*N for N]
17     for pre, crs in prereqs: reachable[pre][crs] = True
18
19     for k in N:
20         for i in N:
21             for j in N:
22                 reachable[i][j] |= (reachable[i][k] and reachable[k][j])
23     return [reachable[u][v] for u,v in queries]
24 """
```

The Magic Question (Get Buy-In)

"Since N is small (100), I can compute the Transitive Closure using the Floyd-Warshall algorithm. 'reachable[i][j]' will be true if 'i' is a direct or indirect prerequisite of 'j'. This allows O(1) query time after O(N³) *precomputation*."

```
1 def checkIfPrerequisite(numCourses: int, prerequisites: List[List[int]], queries: List[
2   List[int]]) -> List[bool]:
3     # Floyd-Warshall Algorithm
4     # reachable[i][j] means i is a prereq of j
5     reachable = [[False] * numCourses for _ in range(numCourses)]
6
7     for pre, crs in prerequisites:
8         reachable[pre][crs] = True
9
10    # k is the intermediate node
11    for k in range(numCourses):
12        for i in range(numCourses):
13            for j in range(numCourses):
14                # If i -> k AND k -> j, then i -> j
15                if reachable[i][k] and reachable[k][j]:
16                    reachable[i][j] = True
17
18    return [reachable[u][v] for u, v in queries]
```

28.5 Word Ladder

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Transformation? A: Change 1 letter at a time to a valid dictionary word.
4   - Q: Goal? A: Shortest sequence length.
5
6 2. LOGIC (BFS + PREPROCESSING)
7   - Naive Graph: Connect words if diff is 1 char.  $O(N^2 * L)$ . Too slow.
8   - Optimization: Intermediate states.
9     - "hot" -> "*ot", "h*t", "ho*".
10    - Map: 'pattern -> list of words'.
11    - Graph: Word connects to all words sharing a pattern.
12    - BFS: Layer by layer to find shortest path.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll preprocess the dictionary into a Hash Map where keys are generic patterns (like
17 #   '*ot')
18 # and values are lists of matching words. Then I'll run BFS starting from 'beginWord'.
19 # This reduces edge finding from  $O(N^2)$  to  $O(L^2)$ .  $O(M^2 * N)$ ."
20
21 def ladderLength(beginWord: str, endWord: str, wordList: List[str]) -> int:
22     if endWord not in wordList:
23         return 0
24
25     # 1. Preprocess Patterns
26     nei = collections.defaultdict(list)
27     wordList.append(beginWord)
28
29     for word in wordList:
30         for i in range(len(word)):
31             pattern = word[:i] + "*" + word[i+1:]
32             nei[pattern].append(word)
33
34     # 2. BFS
35     visit = set([beginWord])
36     q = deque([beginWord])
37     res = 1
38
39     while q:
40         for _ in range(len(q)):
41             word = q.popleft()
42             if word == endWord:
43                 return res
44
45             for i in range(len(word)):
46                 pattern = word[:i] + "*" + word[i+1:]
47                 for neighbor in nei[pattern]:
48                     if neighbor not in visit:
49                         visit.add(neighbor)
50                         q.append(neighbor)
51
52     res += 1
53     return 0
```

29 Advanced Graphs: Pathfinding & MST

29.1 Path with Minimum Effort

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: What is "Effort"?
4     A: Max absolute difference in height between two consecutive cells in the path.
5   - Q: Goal?
6     A: Minimize this "Max Difference" from (0,0) to (R-1, C-1).
7
8 2. APPROACHES
9   - Approach 1: Binary Search on Answer + DFS.
```

```

10     - Guess "k". Can we reach end using only edges <= k?
11     - Complexity:  $O(R \cdot C \log(\text{MaxHeight}))$ .
12     - Accept: Valid.
13     - Approach 2: Dijkstra's Algorithm (Modified).
14     - Logic: "Cost" to reach a cell is the max effort along the path so far.
15     - Priority Queue: (effort, r, c). Always expand lowest effort path.
16     - Complexity:  $O(R \cdot C \log(R \cdot C))$ .
17     - Accept: Optimal & Intuitive.
18
19 3. INDENTED ENGLISH (DIJKSTRA)
20 function minimumEffortPath(heights):
21     minHeap = [(0, 0, 0)] # (diff, r, c)
22     dist = [[INF]]
23     dist[0][0] = 0
24
25     while minHeap:
26         diff, r, c = pop(minHeap)
27         if (r,c) == target: return diff
28         if diff > dist[r][c]: continue
29
30         for nr, nc in neighbors:
31             newDiff = max(diff, abs(heights[r][c] - heights[nr][nc]))
32             if newDiff < dist[nr][nc]:
33                 dist[nr][nc] = newDiff
34                 push(minHeap, (newDiff, nr, nc))
35 """

```

The Magic Question (Get Buy-In)

"I'll use a modified Dijkstra's Algorithm. Instead of summing edge weights, the 'cost' to reach a cell is the maximum absolute difference encountered on the path so far. The Priority Queue ensures we always explore the path with the lowest 'Maximum Effort' first. $O(MN \log(MN))$."

```

1 import heapq
2
3 def minimumEffortPath(heights: List[List[int]]) -> int:
4     ROWS, COLS = len(heights), len(heights[0])
5
6     # Min-Heap stores (current_max_effort, r, c)
7     minHeap = [(0, 0, 0)]
8
9     # Track min effort to reach each cell
10    dist = [[float('inf')] * COLS for _ in range(ROWS)]
11    dist[0][0] = 0
12
13    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
14
15    while minHeap:
16        effort, r, c = heapq.heappop(minHeap)
17
18        # If we reached the target, this is guaranteed to be min effort
19        # because Dijkstra extracts smallest first.
20        if r == ROWS - 1 and c == COLS - 1:
21            return effort
22
23        # Optimization: If we found a better way to this cell already, skip
24        if effort > dist[r][c]:
25            continue
26
27        for dr, dc in directions:
28            nr, nc = r + dr, c + dc
29
30            if 0 <= nr < ROWS and 0 <= nc < COLS:
31                new_effort = max(effort, abs(heights[r][c] - heights[nr][nc]))
32
33                if new_effort < dist[nr][nc]:
34                    dist[nr][nc] = new_effort
35                    heapq.heappush(minHeap, (new_effort, nr, nc))
36
37    return 0

```

29.2 Network Delay Time

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: times=[u, v, w], n nodes, k start node.
4   - Q: Goal? A: Time for signal to reach ALL nodes.
5   - Q: Return -1? A: If any node is unreachable.
6
7 2. LOGIC (DIJKSTRA)
8   - Standard Shortest Path problem.
9   - 1. Build Adjacency List: {u: [(v, w)]}.
10  - 2. Min-Heap: (time, node). Start with (0, k).
11  - 3. Track visited time.
12  - 4. Result is max(visited_times). If len(visited) < n, return -1.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "This is a classic Dijkstra's Algorithm problem. I need the shortest time to reach
17 # every node from 'k'.
18 # The answer is the maximum of these shortest times. If I can't reach all 'n' nodes, I
19 # return -1. O(E log V)."
20
21 def networkDelayTime(times: List[List[int]], n: int, k: int) -> int:
22     adj = collections.defaultdict(list)
23     for u, v, w in times:
24         adj[u].append((v, w))
25
26     minHeap = [(0, k)] # (time_elapsed, node)
27     shortest = {} # node -> min_time
28
29     while minHeap:
30         w1, n1 = heapq.heappop(minHeap)
31
32         if n1 in shortest:
33             continue
34         shortest[n1] = w1
35
36         for n2, w2 in adj[n1]:
37             if n2 not in shortest:
38                 heapq.heappush(minHeap, (w1 + w2, n2))
39
40     # Did we reach all nodes?
41     return max(shortest.values()) if len(shortest) == n else -1
42
43 """
44 5. DRY RUN
45 times=[[2,1,1],[2,3,1],[3,4,1]], n=4, k=2
46 - Heap: [(0,2)].
47 - Pop (0,2). Shortest[2]=0. Push (1,1), (1,3).
48 - Pop (1,1). Shortest[1]=1.
49 - Pop (1,3). Shortest[3]=1. Push (1+1, 4) -> (2,4).
50 - Pop (2,4). Shortest[4]=2.
51 - Max(0,1,1,2) = 2.
52 """
```

29.3 Min Cost to Connect All Points

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Connect all points? A: Yes (Spanning Tree).
4   - Q: Cost? A: Manhattan Distance  $|x_1-x_2| + |y_1-y_2|$ .
5   - Q: Algorithm? A: Prim's or Kruskal's.
6
7 2. APPROACHES
8   - Approach 1: Kruskal's (Union-Find).
9     - Calculate all edges ( $N^2$ ), sort edges, union.
10    -  $O(N^2 \log N)$ . Good.
11   - Approach 2: Prim's (Min-Heap).
12     - Start at point 0. Push all edges to Heap.
13     - Pop min edge to UNVISITED node. Add cost. Push new edges.
14     -  $O(N^2 \log N)$  but usually faster in dense graphs like this.
15     - Accept: Optimal.
16
17 3. INDENTED ENGLISH (PRIM'S)
18 function minCostConnectPoints(points):
19     N = len(points)
20     adj = { i : [(cost, j)] } # Implicitly dense
21     minHeap = [(0, 0)] # (cost, node_index)
22     visit = set()
23     res = 0
24
25     while len(visit) < N:
26         cost, i = pop(minHeap)
27         if i in visit: continue
28
29         res += cost
30         visit.add(i)
31
32         for j in range(N):
33             if j not in visit:
34                 dist = abs(x[i]-x[j]) + abs(y[i]-y[j])
35                 push(minHeap, (dist, j))
36     return res
37 """
```

The Magic Question (Get Buy-In)

"This requires a Minimum Spanning Tree (MST). I'll use Prim's Algorithm. I start at an arbitrary point and greedily pick the shortest edge connecting a visited node to an unvisited node using a Min-Heap. The sum of these edges is the answer. $O(N^2 \log N)$."

```
1 def minCostConnectPoints(points: List[List[int]]) -> int:
2     N = len(points)
3     # Min-Heap: (cost, point_index)
4     minHeap = [(0, 0)]
5     visit = set()
6     total_cost = 0
7
8     while len(visit) < N:
9         cost, i = heapq.heappop(minHeap)
10
11         if i in visit:
12             continue
13
14         total_cost += cost
15         visit.add(i)
16
17         x1, y1 = points[i]
18
19         # Calculate distance to all other unvisited points
20         # Note: In dense graphs, we optimize by not pre-building adj list
21         for j in range(N):
22             if j not in visit:
23                 x2, y2 = points[j]
24                 dist = abs(x1 - x2) + abs(y1 - y2)
```

```
25         heapq.heappush(minHeap, (dist, j))
26
27     return total_cost
```

29.4 Swim in Rising Water

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Can I swim? A: Yes, if time t >= grid[r][c].
4 - Q: Start/End? A: (0,0) to (N-1, N-1).
5 - Q: Goal? A: Minimum time 't' to complete path.
6
7 2. LOGIC (DIJKSTRA)
8 - This is basically "Path with Minimum Effort" but simpler.
9 - The "cost" to enter a cell is max(current_time, grid[r][c]).
10 - We want to minimize the MAX height encountered on the path.
11 - Min-Heap: (max_height_so_far, r, c).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "This is equivalent to finding a path where the maximum cell value is minimized.
16 # I'll use Dijkstra's Algorithm. The 'cost' of a path is the maximum elevation
17 # encountered.
18 # The Priority Queue ensures we find the path with the lowest 'peak' first. O(N^2 log N)
19 ."
20
21 def swimInWater(grid: List[List[int]]) -> int:
22     N = len(grid)
23     visit = set()
24     # (time/max_height, r, c)
25     minHeap = [(grid[0][0], 0, 0)]
26     visit.add((0, 0))
27
28     directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
29
30     while minHeap:
31         t, r, c = heapq.heappop(minHeap)
32
33         if r == N - 1 and c == N - 1:
34             return t
35
36         for dr, dc in directions:
37             nr, nc = r + dr, c + dc
38
39             if (0 <= nr < N and 0 <= nc < N and
40                 (nr, nc) not in visit):
41
42                 visit.add((nr, nc))
43                 # The time to enter next cell is max of current path time OR cell height
44                 new_time = max(t, grid[nr][nc])
45                 heapq.heappush(minHeap, (new_time, nr, nc))
46
47     return -1
```

30 Advanced Graphs: Hard Logic & Topo Sort

30.1 Cheapest Flights Within K Stops

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: n cities, flights=[from, to, price], src, dst, k.
4 - Q: Constraint? A: At most k stops (k+1 edges).
5 - Q: Goal? A: Min price.
6
7 2. APPROACHES
8 - Approach 1: Dijkstra.
9 - Problem: Dijkstra greedily picks cheapest. A cheap path might have too many stops
10 .
11 - Fix: Include 'stops' in the Heap state. O(E log V * k).
12 - Approach 2: Bellman-Ford (Layered BFS).
13 - Logic: We are allowed k+1 edges.
14 - Loop k+1 times. In each loop, relax ALL edges.
15 - Use a 'temp' array to ensure we only use values from the previous iteration (
16 stops constraint).
```

```

15     - Complexity:  $O(E \cdot K)$ .
16     - Accept: Optimal and easiest to implement correctly.
17
18 3. INDENTED ENGLISH (BELLMAN-FORD)
19 function findCheapestPrice(n, flights, src, dst, k):
20     prices = [INF] * n
21     prices[src] = 0
22
23     for i in range(k + 1):
24         tmpPrices = prices.copy()
25         for s, d, p in flights:
26             if prices[s] == INF: continue
27             if prices[s] + p < tmpPrices[d]:
28                 tmpPrices[d] = prices[s] + p
29         prices = tmpPrices
30
31     return prices[dst] if != INF else -1
32 """

```

The Magic Question (Get Buy-In)

”Standard Dijkstra fails here because the cheapest path might exceed the ‘K’ stops limit. I’ll use the Bellman-Ford algorithm. I’ll iterate ‘K+1’ times (representing the allowed edges). In each iteration, I’ll relax all edges using a temporary array to enforce the ‘stops’ constraint. $O(E \cdot K)$.”

```

1 def findCheapestPrice(n: int, flights: List[List[int]], src: int, dst: int, k: int) ->
  int:
2     # Initialize prices to infinity
3     prices = [float("inf")] * n
4     prices[src] = 0
5
6     # We are allowed k stops, which means k + 1 edges (flights)
7     for i in range(k + 1):
8         # Create a copy to ensure we only use prices from the PREVIOUS iteration
9         # This guarantees we don't use a path with i+1 edges in the i-th step
10        tmpPrices = prices.copy()
11
12        for s, d, p in flights: # s=source, d=dest, p=price
13            if prices[s] == float("inf"):
14                continue
15
16            if prices[s] + p < tmpPrices[d]:
17                tmpPrices[d] = prices[s] + p
18
19        prices = tmpPrices
20
21    return prices[dst] if prices[dst] != float("inf") else -1

```

30.2 Reconstruct Itinerary

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Start? A: "JFK".
4   - Q: Rule? A: Use every ticket exactly once.
5   - Q: Graph? A: Eulerian Path (visit every edge).
6   - Q: Tie-breaking? A: Lexicographical order (small letter first).
7
8 2. LOGIC (HIERHOLZER'S ALGORITHM / DFS)
9   - 1. Build Adjacency List: {src: [dst1, dst2]}. Sort dsts reversed (for O(1) pop).
10  - 2. DFS from JFK.
11  - 3. Greedy: Always go to the smallest lexical neighbor first.
12  - 4. Stuck? If we get stuck (no neighbors), add node to result.
13  - 5. Reverse result at the end.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "This is an Eulerian Path problem (using every edge exactly once). I'll build an
18 # adjacency list and sort neighbors for lexical order. I'll use DFS. When I get stuck
19 # at a node (no unvisited edges), I add it to the result. Finally, I reverse the
20 # result list. O(E log E)."
```

```
18 def findItinerary(tickets: List[List[str]]) -> List[str]:
19     adj = {src: [] for src, dst in tickets}
20
21     # Sort backwards to pop efficiently (O(1)) from end
22     tickets.sort(key=lambda x: x[1], reverse=True)
23     for src, dst in tickets:
24         if src in adj: adj[src].append(dst)
25         else: adj[src] = [dst]
26
27     res = []
28
29     def dfs(src):
30         # Visit all outgoing edges
31         while src in adj and adj[src]:
32             next_dst = adj[src].pop()
33             dfs(next_dst)
34
35         # Post-order: Add to result when stuck
36         res.append(src)
37
38     dfs("JFK")
39     return res[::-1] # Reverse to get correct order
40
41 """
42 5. DRY RUN
43 Tickets: [[JFK, SFO], [JFK, ATL], [SFO, ATL], [ATL, JFK], [ATL, SFO]]
44 - Path: JFK -> ATL -> JFK -> SFO -> ATL -> SFO.
45 - DFS traces edges.
46 - Stuck at last SFO. Append SFO.
47 - Backtrack to ATL. Stuck. Append ATL.
48 - ...
49 - Res (reversed) = [JFK, ATL, JFK, SFO, ATL, SFO].
50 """
51 """
```

30.3 Alien Dictionary

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: List of words sorted by alien rules.
4   - Q: Goal? A: Return unique string of ordered characters.
5   - Q: Invalid? A: Cycle (a < b and b < a) or Prefix error ("abc" before "ab").
6
7 2. LOGIC (TOPO SORT)
8   - Step 1: Build Graph. Compare adjacent words (w1, w2).
9     - Find first diff char: c1 != c2.
10    - Edge: c1 -> c2.
11    - Edge case: if w2 is prefix of w1 (len(w2) < len(w1)), INVALID.
12   - Step 2: Kahn's Algo (BFS).
13     - Count indegrees.
14     - Queue 0-indegree.
15     - Pop, Add to Res, Decrement neighbors.
16   - Step 3: Check validity (Cycle?).
17     - If len(res) != unique_chars, Cycle exists -> return "".
18 """
19
20 # --- THE MAGIC QUESTION ---
21 # "I'll derive dependency rules by comparing adjacent words to find the first differing
22 # character. This builds a directed graph. Then, I'll perform a Topological Sort (BFS)
23 # . If the resulting order length doesn't match the unique character count, there's a
24 # cycle, so I return empty."
25
26 def alienOrder(words: List[str]) -> str:
27     adj = {c: set() for w in words for c in w}
28     indegree = {c: 0 for c in adj}
29
30     # 1. Build Graph
31     for i in range(len(words) - 1):
32         w1, w2 = words[i], words[i + 1]
33         minLen = min(len(w1), len(w2))
34
35         # Edge Case: "abc" before "ab" is invalid
36         if len(w1) > len(w2) and w1[:minLen] == w2[:minLen]:
37             return ""
38
39         for j in range(minLen):
40             if w1[j] != w2[j]:
41                 if w2[j] not in adj[w1[j]]:
42                     adj[w1[j]].add(w2[j])
43                     indegree[w2[j]] += 1
44                 break # Only first diff matters
45
46     # 2. Topo Sort (BFS)
47     q = deque([c for c in indegree if indegree[c] == 0])
48     res = []
49
50     while q:
51         char = q.popleft()
52         res.append(char)
53
54         for neighbor in adj[char]:
55             indegree[neighbor] -= 1
56             if indegree[neighbor] == 0:
57                 q.append(neighbor)
58
59     return "".join(res) if len(res) == len(indegree) else ""
```

30.4 Build a Matrix With Conditions

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Conditions? A: rowCond [[above, below]], colCond [[left, right]].
4 - Q: k? A: Numbers 1 to k. Matrix k x k.
5 - Q: Impossible? A: Return [].
6
7 2. LOGIC (DOUBLE TOPO SORT)
8 - The row positions and column positions are INDEPENDENT.
9 - 1. Topo Sort the 'rowConditions' to get a valid sequence (e.g., [3, 1, 2]).
10   - This means 3 is in row 0, 1 in row 1, 2 in row 2.
11 - 2. Topo Sort 'colConditions'.
12 - 3. Combine: matrix[row_pos[num]][col_pos[num]] = num.
13 - 4. If either sort fails (cycle), return [].
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "The row and column placements are independent problems. I'll perform Topological Sort
18   on the row conditions to determine the relative row order, and do the same for
19   columns. If both valid orders exist, I'll place the numbers into the matrix at their
20   respective '(row_idx, col_idx)'."
21
22 def buildMatrix(k: int, rowConditions: List[List[int]], colConditions: List[List[int]])
23     -> List[List[int]]:
24
25     def topo_sort(edges):
26         adj = collections.defaultdict(list)
27         indegree = {i: 0 for i in range(1, k + 1)}
28         for u, v in edges:
29             adj[u].append(v)
30             indegree[v] += 1
31
32         q = deque([i for i in range(1, k + 1) if indegree[i] == 0])
33         order = []
34         while q:
35             node = q.popleft()
36             order.append(node)
37             for nei in adj[node]:
38                 indegree[nei] -= 1
39                 if indegree[nei] == 0:
40                     q.append(nei)
41         return order if len(order) == k else []
42
43     row_order = topo_sort(rowConditions)
44     col_order = topo_sort(colConditions)
45
46     if not row_order or not col_order:
47         return []
48
49     # Map Value -> Index
50     val_to_row = {num: i for i, num in enumerate(row_order)}
51     val_to_col = {num: i for i, num in enumerate(col_order)}
52
53     res = [[0] * k for _ in range(k)]
54     for num in range(1, k + 1):
55         r, c = val_to_row[num], val_to_col[num]
56         res[r][c] = num
57
58     return res
```

31 Advanced Graphs: Critical Edges & GCD

31.1 Find Critical and Pseudo-Critical Edges in MST

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: What is Critical?
4   A: Deleting this edge INCREASES the MST weight.
5 - Q: What is Pseudo-Critical?
```

```

6     A: This edge CAN be in an MST (weight same), but isn't strictly required.
7 - Q: Strategy?
8     A: Kruskal's Algorithm (Union-Find).
9     1. Find standard MST weight.
10    2. Force Exclude edge -> If weight increases or disconnected, it's Critical.
11    3. Force Include edge -> If weight matches standard MST, it's Pseudo-Critical.
12
13 2. LOGIC (KRUSKAL'S SIMULATION)
14 - Sort edges by weight, but keep original indices.
15 - Helper function 'getMST(block_edge, force_edge)':
16   - Returns weight of MST under these constraints.
17 - Loop every edge:
18   - If excluding it makes weight > min_weight: Critical.
19   - Else if including it results in min_weight: Pseudo.
20 """
21
22 # --- THE MAGIC QUESTION ---
23 # "I'll use Kruskal's Algorithm. First, I find the baseline MST weight. Then, for every
24 # edge,
25 # I test two scenarios: forcing its exclusion (to check if it's Critical) and forcing
26 # its inclusion
27 # (to check if it's Pseudo-Critical). O(E^2)."
```

```

28 class UnionFind:
29     def __init__(self, n):
30         self.parent = list(range(n))
31         self.count = n # Connected components
32
33     def find(self, i):
34         if self.parent[i] != i:
35             self.parent[i] = self.find(self.parent[i])
36         return self.parent[i]
37
38     def union(self, i, j):
39         root_i, root_j = self.find(i), self.find(j)
40         if root_i != root_j:
41             self.parent[root_i] = root_j
42             self.count -= 1
43         return True
44         return False
45
46 class Solution:
47     def findCriticalAndPseudoCriticalEdges(self, n: int, edges: List[List[int]]) -> List
48     [List[int]]:
49         # Add original index to edges: [u, v, w, i]
50         for i, e in enumerate(edges):
51             e.append(i)
52
53         # Sort by weight
54         edges.sort(key=lambda x: x[2])
55
56     def getMST(block_idx, force_edge_idx):
57         uf = UnionFind(n)
58         weight = 0
59
60         # 1. Force Include specific edge
61         if force_edge_idx != -1:
62             # Find the edge in sorted list by original index logic (simplified here)
63             # Actually simpler: Iterate edges, pick the forced one first.
64             pass
65
66         # Proper Logic:
67         if force_edge_idx != -1:
68             for e in edges:
69                 if e[3] == force_edge_idx:
70                     uf.union(e[0], e[1])
71                     weight += e[2]
72                     break
73
74         # 2. Standard Kruskal's
75         for u, v, w, i in edges:
76             if i == block_idx: continue
77             if uf.union(u, v):

```

```

76         weight += w
77
78         return weight if uf.count == 1 else float('inf')
79
80     base_mst = getMST(-1, -1)
81     critical, pseudo = [], []
82
83     for i in range(len(edges)):
84         orig_idx = edges[i][3]
85
86         # Check Critical: Exclude this edge
87         if getMST(orig_idx, -1) > base_mst:
88             critical.append(orig_idx)
89         # Check Pseudo: Force include this edge
90         elif getMST(-1, orig_idx) == base_mst:
91             pseudo.append(orig_idx)
92
93     return [critical, pseudo]

```

31.2 Greatest Common Divisor Traversal

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Connection rule? A: Index i and j connected if gcd(nums[i], nums[j]) > 1.
4 - Q: Goal? A: Can we traverse between ANY pair? (Is Graph fully connected?)
5 - Q: Naive N^2 check? A: TLE (N=10^5).
6 - Q: Optimization? A: Connect indices via Prime Factors.
7   - 6 (2,3) and 15 (3,5) connected via '3'.
8
9 2. LOGIC (PRIME FACTOR UNION-FIND)
10 - Map: 'Prime -> List[Indices]'.
11 - Iterate nums. Factorize each num.
12 - Union the current index with the FIRST index seen for that prime.
13 - Finally, check if all indices belong to one component.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "Building an adjacency graph is O(N^2), which is too slow. Instead, I'll use Union-
18   Find
19 # to group indices that share a prime factor. I map each prime factor to the first index
20 # that has it, and union subsequent indices to that leader. O(N * sqrt(V))."
21
22 class Solution:
23     def canTraverseAllPairs(self, nums: List[int]) -> bool:
24         if len(nums) == 1: return True
25         if 1 in nums: return False # 1 has no prime factors, cannot connect
26
27         n = len(nums)
28         # parent array for Union-Find (indices 0 to n-1)
29         parent = list(range(n))
30         count = n # components
31
32         def find(i):
33             if parent[i] != i:
34                 parent[i] = find(parent[i])
35             return parent[i]
36
37         def union(i, j):
38             nonlocal count
39             root_i, root_j = find(i), find(j)
40             if root_i != root_j:
41                 parent[root_i] = root_j
42                 count -= 1
43
44         # Map: Prime Factor -> Index of first number with this factor
45         prime_to_index = {}
46
47         for i, num in enumerate(nums):
48             d = 2
49             temp = num
50             while d * d <= temp:
51                 if temp % d == 0:
52                     if d in prime_to_index:
53                         union(i, prime_to_index[d])
54                     else:
55                         prime_to_index[d] = i
56                     while temp % d == 0:
57                         temp //= d
58                 d += 1
59             if temp > 1:
60                 if temp in prime_to_index:
61                     union(i, prime_to_index[temp])
62                 else:
63                     prime_to_index[temp] = i
64
65         return count == 1
```

32 1-D DP: Basics & Palindromes

32.1 Climbing Stairs

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Steps? A: 1 or 2 at a time.
4 - Q: Start? A: 0. End? A: n.
5
6 2. LOGIC (FIBONACCI)
7 - To reach step i, we could come from i-1 OR i-2.
8 - dp[i] = dp[i-1] + dp[i-2].
9 - Base: dp[1]=1, dp[2]=2.
10 - Optimization: Only need last two variables.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "This is the Fibonacci sequence. To reach step 'n', I must have come from 'n-1' or 'n-2'."
15 # I'll just maintain two variables to track the number of ways to reach the previous two steps. O(N)."
16
17 def climbStairs(n: int) -> int:
18     one, two = 1, 1
19
20     for i in range(n - 1):
21         temp = one
22         one = one + two
23         two = temp
24
25     return one
```

32.2 Min Cost Climbing Stairs

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Start? A: Index 0 or 1.
4   - Q: Goal? A: Reach top (index len(cost)).
5
6 2. LOGIC (REVERSE ITERATION)
7   - dp[i] = cost[i] + min(dp[i+1], dp[i+2]).
8   - If I am at step i, I pay cost[i], then take min path from next possible steps.
9 """
10
11 # --- THE MAGIC QUESTION ---
12 # "I'll iterate backwards from the end. At each step, the cost to reach the top is
13 # the current step's cost plus the minimum of the costs of the next two steps. O(N)."
14
15 def minCostClimbingStairs(cost: List[int]) -> int:
16     cost.append(0) # Top of stairs (cost 0)
17
18     for i in range(len(cost) - 3, -1, -1):
19         cost[i] += min(cost[i + 1], cost[i + 2])
20
21     return min(cost[0], cost[1])
```

32.3 House Robber

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Restriction? A: Cannot rob adjacent houses.
4   - Q: Goal? A: Maximize money.
5
6 2. LOGIC (PICK OR SKIP)
7   - At house i, choices:
8     1. Rob: nums[i] + max_loot(i-2).
9     2. Skip: max_loot(i-1).
10  - Recurrence: rob = max(arr[i] + rob[i-2], rob[i-1]).
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "At every house, I have two choices: rob it and add it to the loot from two houses
15 #   back,
16 # or skip it and keep the loot from the previous house. I'll maintain two variables to
17 #   track this. O(N)."
```

```
17 def rob(nums: List[int]) -> int:
18     rob1, rob2 = 0, 0
19
20     for n in nums:
21         # [rob1, rob2, n, ...]
22         newRob = max(n + rob1, rob2)
23         rob1 = rob2
24         rob2 = newRob
25
26     return rob2
```

32.4 House Robber II

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Circular? A: First and Last are neighbors.
4     - Q: Impact? A: Can't rob BOTH first and last.
5
6 2. LOGIC (BREAK THE CIRCLE)
7     - Case 1: Rob houses 0 to n-2 (Exclude Last).
8     - Case 2: Rob houses 1 to n-1 (Exclude First).
9     - Result: max(Case 1, Case 2).
10    - Use helper from House Robber I.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "Since the houses are circular, I can't rob both the first and the last.
15 # I'll simply run the standard House Robber algorithm twice: once excluding the last
16 # house,
17 # and once excluding the first. The answer is the maximum of the two. O(N)."
18
19 def rob(nums: List[int]) -> int:
20     if len(nums) == 0: return 0
21     if len(nums) == 1: return nums[0]
22
23     def helper(arr):
24         rob1, rob2 = 0, 0
25         for n in arr:
26             newRob = max(n + rob1, rob2)
27             rob1 = rob2
28             rob2 = newRob
29         return rob2
30
31     return max(helper(nums[:-1]), helper(nums[1:]))
```

32.5 Longest Palindromic Substring

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Substring vs Subsequence? A: Contiguous.
4   - Q: Input? A: "babad". Output? "bab" or "aba".
5
6 2. LOGIC (EXPAND AROUND CENTER)
7   - Palindromes mirror around a center.
8   - Center can be a character ('aba') or a gap ('abba').
9   - Iterate i from 0 to N.
10    - Expand (i, i) -> Odd length.
11    - Expand (i, i+1) -> Even length.
12    - Track maxLen.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll iterate through each character and consider it as the center of a palindrome.
17 # I'll expand outwards for both odd cases (single center) and even cases (double center)
18 # This is  $O(N^2)$  but uses  $O(1)$  space, better than DP  $O(N^2)$  space."
19
20 def longestPalindrome(s: str) -> str:
21     res = ""
22
23     def expand(l, r):
24         while l >= 0 and r < len(s) and s[l] == s[r]:
25             l -= 1
26             r += 1
27         return s[l + 1 : r] # Correct slice after loop breaks
28
29     for i in range(len(s)):
30         # Odd length
31         s1 = expand(i, i)
32         if len(s1) > len(res): res = s1
33
34         # Even length
35         s2 = expand(i, i + 1)
36         if len(s2) > len(res): res = s2
37
38     return res
```

32.6 Palindromic Substrings

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Goal? A: Count ALL palindromic substrings.
4   - Q: "aaa"? A: "a", "a", "a", "aa", "aa", "aaa" (6).
5
6 2. LOGIC (EXPAND AROUND CENTER)
7   - Same logic as Longest Palindrome.
8   - Instead of tracking max, just increment 'count' every time s[l] == s[r].
9 """
10
11 # --- THE MAGIC QUESTION ---
12 # "I'll use the 'Expand Around Center' technique. For each index, I expand for odd and
13 # even palindromes.
14 # Every time the characters match, I increment my count. O(N^2)."
```

```
15 def countSubstrings(s: str) -> int:
16     count = 0
17
18     def countPali(l, r):
19         res = 0
20         while l >= 0 and r < len(s) and s[l] == s[r]:
21             res += 1
22             l -= 1
23             r += 1
24         return res
25
26     for i in range(len(s)):
27         count += countPali(i, i)      # Odd
28         count += countPali(i, i + 1) # Even
29
30     return count
```

33 1-D DP: Strings, Products & Subsequences

33.1 N-th Tribonacci Number

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Formula? A: T(n) = T(n-1) + T(n-2) + T(n-3).
4   - Q: Base cases? A: 0, 1, 1.
5
6 2. LOGIC (SPACE OPTIMIZATION)
7   - Same as Fibonacci but maintain 3 variables instead of 2.
8   - Loop from 3 to n.
9 """
10
11 # --- THE MAGIC QUESTION ---
12 # "This is just Fibonacci extended to three terms. Instead of two variables, I'll
13 # maintain three
14 # (t0, t1, t2) and shift them in each iteration. O(N) time, O(1) space."
```

```
15 def tribonacci(n: int) -> int:
16     t = [0, 1, 1]
17
18     if n < 3:
19         return t[n]
20
21     for i in range(3, n + 1):
22         # Calculate next
23         next_val = t[0] + t[1] + t[2]
24         # Shift
25         t[0], t[1], t[2] = t[1], t[2], next_val
26
27     return t[2]
```

33.2 Decode Ways

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: String of digits "12".
4 - Q: Mapping? A: 'A'->1 ... 'Z'->26.
5 - Q: '0' handling? A: "06" invalid. "10" is 'J'. "0" invalid.
6
7 2. LOGIC (DP WITH PREVIOUS STATES)
8 - dp[i] = number of ways to decode s[:i].
9 - Choice 1: One digit. If s[i] != '0', we can add dp[i-1].
10 - Choice 2: Two digits. If "10" <= s[i-1:i+1] <= "26", we can add dp[i-2].
11 - Base case: dp[0] = 1 (empty string has 1 way).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use DP where 'dp[i]' represents ways to decode the prefix of length 'i'.
16 # At each step, I check if the single digit is valid (1-9) and if the two-digit pair is
17 # valid (10-26).
18 # If valid, I add the corresponding previous counts. O(N).
19
20 def numDecodings(s: str) -> int:
21     # dp[i] maps to s[i-1]
22     dp = {len(s): 1}
23
24     # Iterate backwards (or forwards, symmetric)
25     for i in range(len(s) - 1, -1, -1):
26         # Case 1: Single digit '0' is invalid start
27         if s[i] == "0":
28             dp[i] = 0
29         else:
30             dp[i] = dp[i + 1]
31
32         # Case 2: Two digits (10-26)
33         if (i + 1 < len(s) and
34             (s[i] == "1" or
35              (s[i] == "2" and s[i + 1] in "0123456"))):
36             dp[i] += dp[i + 2]
37
38     return dp[0]
39
40 """
41 5. DRY RUN
42 s = "12"
43 - Init: dp[2] = 1.
44 - i=1 ('2'): != '0', dp[1] = dp[2] = 1.
45 - i=0 ('1'): != '0', dp[0] = dp[1] = 1.
46   - Check "12": Valid. dp[0] += dp[2] -> 1 + 1 = 2.
47 - Return 2. ("AB", "L")
48 """
```

33.3 Coin Change

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Goal? A: Minimum coins to make amount.
4     - Q: Infinite supply? A: Yes.
5     - Q: Impossible? A: Return -1.
6
7 2. LOGIC (BOTTOM-UP DP)
8     - dp[a] = min coins to make amount 'a'.
9     - Initialize dp array with amount + 1 (representing infinity).
10    - dp[0] = 0.
11    - For each coin 'c', dp[a] = min(dp[a], 1 + dp[a - c]).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use an array 'dp' where 'dp[i]' is the minimum coins needed for amount 'i'.
16 # I initialize with a value larger than the max possible coins. For every amount, I try
17 # subtracting
18 # each coin denomination and taking the minimum result. O(Amount * Coins)."
19
20 def coinChange(coins: List[int], amount: int) -> int:
21     # Init with amount + 1 (effectively infinity)
22     dp = [amount + 1] * (amount + 1)
23     dp[0] = 0
24
25     for a in range(1, amount + 1):
26         for c in coins:
27             if a - c >= 0:
28                 dp[a] = min(dp[a], 1 + dp[a - c])
29
30     return dp[amount] if dp[amount] != amount + 1 else -1
31 """
32 5. DRY RUN
33     coins=[1,2], amount=3
34     - dp=[0, 4, 4, 4]
35     - a=1: use 1-> dp[1]=min(4, 1+dp[0])=1.
36     - a=2: use 1-> dp[2]=2. use 2-> dp[2]=min(2, 1+dp[0])=1.
37     - a=3: use 1-> dp[3]=1+dp[2]=2. use 2-> dp[3]=1+dp[1]=2.
38     - Return 2.
39 """
```

33.4 Maximum Product Subarray

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Negatives? A: Yes (negative * negative = positive).
4     - Q: Zeros? A: Yes (resets product).
5     - Q: Subarray? A: Contiguous.
6
7 2. LOGIC (TRACK MIN & MAX)
8     - Because of negatives, the "minimum" product (large negative) can become the Maximum
9       if multiplied by another negative.
10    - Maintain 'curMax' and 'curMin'.
11    - If n is negative, swap max and min.
12    - Update: curMax = max(n, n*curMax). curMin = min(n, n*curMin).
13 """
14 # --- THE MAGIC QUESTION ---
15 # "Since a negative number flips the sign, I need to track both the current Maximum and
16 #   current Minimum product.
17 # When I encounter a negative number, the Minimum becomes the candidate for the new
18 #   Maximum. O(N)."
19
20 def maxProduct(nums: List[int]) -> int:
21     res = max(nums)
22     curMin, curMax = 1, 1
23
24     for n in nums:
25         if n == 0:
26             curMin, curMax = 1, 1
27             continue
28
29         tmp = curMax * n
30         curMax = max(n * curMax, n * curMin, n) # n itself starts new subarray
31         curMin = min(tmp, n * curMin, n)
32
33         res = max(res, curMax)
34
35     return res
36 """
37 5. DRY RUN
38     nums = [2, 3, -2, 4]
39     - n=2: Max=2, Min=2. Res=2.
40     - n=3: Max=6, Min=3. Res=6.
41     - n=-2:
42         - tmp = 6*-2 = -12.
43         - Max = max(-12, 3*-2, -2) = -2.
44         - Min = min(-12, -6, -2) = -12.
45     - n=4:
46         - Max = max(-8, -48, 4) = 4.
47         - Res=6.
48 """
```

33.5 Word Break

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Reuse dictionary words? A: Yes.
4   - Q: Goal? A: Return True if s can be segmented.
5
6 2. LOGIC (BOOLEAN DP)
7   - dp[i] = True if s[:i] can be segmented.
8   - Base: dp[0] = True.
9   - Loop i from 1 to len(s).
10  - Loop j from 0 to i.
11  - If dp[j] is True AND s[j:i] in dict -> dp[i] = True.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use a boolean array 'dp' where 'dp[i]' means the prefix 's[:i]' is valid.
16 # I iterate through the string, and for each position 'i', I check all valid start
17 # points 'j'
18 # to see if 's[j:i]' is a word. O(N^2 * M)."
```

```
19 def wordBreak(s: str, wordDict: List[str]) -> bool:
20     dp = [False] * (len(s) + 1)
21     dp[len(s)] = True # Base case (matching end)
22
23     # Iterate backwards for cleanliness
24     for i in range(len(s) - 1, -1, -1):
25         for w in wordDict:
26             # Check if word fits and matches
27             if (i + len(w) <= len(s) and
28                 s[i : i + len(w)] == w):
29
30                 dp[i] = dp[i + len(w)]
31                 if dp[i]:
32                     break
33
34     return dp[0]
```

33.6 Longest Increasing Subsequence

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Contiguous? A: No, subsequence.
4 - Q: Strict increasing? A: Yes (nums[i] > nums[j]).
5
6 2. APPROACHES
7 - Approach 1: DP (Standard).
8   - dp[i] = Length of LIS ending at index i.
9   - Loop j < i. If nums[i] > nums[j], dp[i] = max(dp[i], 1 + dp[j]).
10  - Complexity: O(N^2).
11  - Accept: Good for Interviews.
12 - Approach 2: Binary Search (Patience Sort).
13   - Maintain 'tails' array.
14   - If num > all tails, append.
15   - Else, replace first tail >= num.
16   - Complexity: O(N log N).
17   - Accept: Optimal "Strong Hire".
18
19 3. INDENTED ENGLISH (DP APPROACH - EASIER TO EXPLAIN)
20 function lengthOfLIS(nums):
21     dp = [1] * N
22     for i from 1 to N:
23         for j from 0 to i:
24             if nums[i] > nums[j]:
25                 dp[i] = max(dp[i], 1 + dp[j])
26     return max(dp)
27 """
```

The Magic Question (Get Buy-In)

"I can solve this in $O(N^2)$ using DP, where 'dp[i]' tracks the longest sequence ending at 'i'. Or, I can use a Binary Search approach to build the solution in $O(N \log N)$. Which do you prefer?"

```
1 def lengthOfLIS(nums: List[int]) -> int:
2     # DP Approach O(N^2) - often sufficient and easier to derive
3     dp = [1] * len(nums)
4
5     for i in range(len(nums)):
6         for j in range(i):
7             if nums[i] > nums[j]:
8                 dp[i] = max(dp[i], 1 + dp[j])
9
10    return max(dp) if nums else 0
11
12    # Binary Search Approach O(N log N)
13    # tails = []
14    # for n in nums:
15    #     import bisect
16    #     idx = bisect.bisect_left(tails, n)
17    #     if idx < len(tails): tails[idx] = n
18    #     else: tails.append(n)
19    # return len(tails)
```

34 1-D DP: Knapsacks & Math

34.1 Partition Equal Subset Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: List of positive integers.
4 - Q: Goal? A: Split into two subsets with equal sum.
5 - Q: Equivalence? A: Find if a subset sums to Total / 2.
6
7 2. LOGIC (0/1 KNAPSACK - SET)
8 - Target = sum(nums) / 2. If odd, return False.
```

```

9   - We need to find if *any* combination sums to Target.
10  - Use a Set 'dp' storing all achievable sums.
11  - For every num, new_sums = {s + num for s in dp}.
12  - Update dp = dp U new_sums.
13  """
14
15  # --- THE MAGIC QUESTION ---
16  # "This is a 0/1 Knapsack problem. I need to find if a subset sums to exactly half the
17  # total sum.
18  # I'll maintain a Set of all reachable sums. For every number, I add it to every
19  # existing sum in the Set.
20  # If the target is in the Set, return True. O(N * Sum)."
```

```

21  def canPartition(nums: List[int]) -> bool:
22      if sum(nums) % 2 != 0:
23          return False
24
25      target = sum(nums) // 2
26      dp = {0} # Set of reachable sums
27
28      for n in nums:
29          # Create a new set to avoid modifying while iterating
30          next_dp = set()
31          for t in dp:
32              if t + n == target:
33                  return True
34              if t + n < target:
35                  next_dp.add(t + n)
36                  next_dp.add(t) # Keep existing sums
37          dp = next_dp
38
39      return False # Target not reached

```

34.2 Combination Sum IV

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Order matters? A: Yes, (1,2) != (2,1). This is Permutations.
4     - Q: Distinct inputs? A: Yes.
5
6 2. LOGIC (DP - PERMUTATIONS)
7     - Similar to Coin Change.
8     - dp[i] = number of ways to get sum 'i'.
9     - dp[i] = sum(dp[i - num]) for num in nums.
10    - Base case: dp[0] = 1.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "Since order matters, this is a Permutation problem, not a Combination one.
15 # 'dp[i]' represents the number of sequences that sum to 'i'.
16 # For every target value, I iterate through all numbers and add the ways to reach '
17     target - num'. O(Target * N)."
18
19 def combinationSum4(nums: List[int], target: int) -> int:
20     dp = {0: 1} # target -> count
21
22     # Memoization approach is often cleaner for large gaps
23     def dfs(rem):
24         if rem == 0: return 1
25         if rem < 0: return 0
26         if rem in dp: return dp[rem]
27
28         res = 0
29         for n in nums:
30             res += dfs(rem - n)
31
32         dp[rem] = res
33         return res
34
35     return dfs(target)
36
37     # Iterative Approach
38     # dp = [0] * (target + 1)
39     # dp[0] = 1
40     # for i in range(1, target + 1):
41     #     for n in nums:
42     #         if i - n >= 0: dp[i] += dp[i-n]
43     # return dp[target]
```

34.3 Perfect Squares

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Integer n.
4   - Q: Goal? A: Min number of perfect squares summing to n.
5   - Q: Coins? A: 1, 4, 9, 16... up to n.
6
7 2. LOGIC (DP - COIN CHANGE VARIANT)
8   - This is exactly "Coin Change" where coins are squares.
9   -  $dp[i] = 1 + \min(dp[i - s])$  for all squares  $s \leq i$ .
10  - Base case:  $dp[0] = 0$ .
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "This is the Coin Change problem in disguise. The 'coins' are perfect squares (1, 4,
15 # 9...)."
16 # I want the minimum number of 'coins' to reach sum 'n'.  $O(n * \sqrt{n})$ ."
17
18 def numSquares(n: int) -> int:
19     # dp[i] = min squares to sum to i
20     # Init with max value (n ones)
21     dp = [n] * (n + 1)
22     dp[0] = 0
23
24     for target in range(1, n + 1):
25         s = 1
26         while s * s <= target:
27             square = s * s
28             dp[target] = min(dp[target], 1 + dp[target - square])
29             s += 1
30
31     return dp[n]
```

34.4 Integer Break

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: n >= 2.
4 - Q: Action? A: Break into k >= 2 integers.
5 - Q: Goal? A: Maximize product.
6
7 2. APPROACHES
8 - Approach 1: DP.
9   - dp[i] = max product for number i.
10  - Split i into (j) and (i-j).
11  - Product is max(j * (i-j), j * dp[i-j]).
12  - Complexity: O(N^2).
13 - Approach 2: Math (Greedy).
14  - Optimal factor is 3. Break n into as many 3s as possible.
15  - If remainder 4, use 2*2 (better than 3*1).
16  - If remainder 2, use 2.
17  - Complexity: O(N).
18  - Accept: Optimal.
19
20 3. INDENTED ENGLISH (DP)
21 function integerBreak(n):
22     dp = [1] * (n + 1)
23     for i in 2..n:
24         for j in 1..i:
25             val = max(j * (i-j), j * dp[i-j])
26             dp[i] = max(dp[i], val)
27     return dp[n]
28 """
```

The Magic Question (Get Buy-In)

"I can use DP to check every possible split, which is $O(N^2)$. However, mathematically, decomposing the number into as many 3s as possible yields the maximum product. Do you prefer the DP solution or the $O(N)$ Math greedy approach?"

```
1 def integerBreak(n: int) -> int:
2     # DP Approach (Generalizable)
3     dp = {1: 1}
4
5     for num in range(2, n + 1):
6         dp[num] = 0 if num == n else num # Base case: num itself if not n
7         for i in range(1, num // 2 + 1):
8             # Split into i and (num - i)
9             # Either use the number (num-i) raw, or its broken down max product dp[num-i]
10        ]
11        val = max(i * (num - i), i * dp[num - i])
12        dp[num] = max(dp[num], val)
13
14    return dp[n]
15
16    # Math Approach (Optimal)
17    # if n <= 3: return n - 1
18    # res = 1
19    # while n > 4:
20    #     res *= 3
21    #     n -= 3
22    # return res * n
```

34.5 Delete and Earn

```
1 """
2 Note: Often the last problem in this module.
3
4 1. CLARIFYING QUESTIONS
5 - Q: Rules? A: Pick nums[i], delete all nums[i]-1 and nums[i]+1.
6 - Q: Duplicates? A: Yes, picking 'x' lets you pick ALL 'x's.
7
8 2. LOGIC (REDUCE TO HOUSE ROBBER)
9 - Taking 'x' deletes 'x-1' and 'x+1'. This is exactly House Robber!
10 - We cannot take adjacent numbers.
11 - Transform input: 'values' array where index 'i' holds 'sum(all i's in nums)'.
12 - Run House Robber on 'values'.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "This reduces to the House Robber problem. If I pick a number 'x', I get all copies of
17 # 'x',
18 # but I can't pick 'x-1' or 'x+1'. I'll create a frequency array where index 'i' stores
19 # the total value of number 'i'.
20 # Then I simply run the House Robber algorithm on that array. O(N + MaxVal)."
```

```
20 def deleteAndEarn(nums: List[int]) -> int:
21     if not nums: return 0
22
23     # Transform to House Robber array
24     # Index i holds total value of number i
25     max_val = max(nums)
26     values = [0] * (max_val + 1)
27
28     for n in nums:
29         values[n] += n
30
31     # Standard House Robber logic
32     rob1, rob2 = 0, 0
33     for val in values:
34         new_rob = max(val + rob1, rob2)
35         rob1 = rob2
36         rob2 = new_rob
37
38     return rob2
```

35 2-D DP: Grids & Sequences

35.1 Unique Paths

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: m x n grid.
4 - Q: Movement? A: Right or Down only.
5 - Q: Obstacles? A: No (handled in Unique Paths II).
6
7 2. LOGIC (GRID DP)
8 - dp[r][c] = unique paths to reach cell (r, c).
9 - To reach (r, c), you must come from Top (r-1, c) or Left (r, c-1).
10 - Formula: dp[r][c] = dp[r-1][c] + dp[r][c-1].
11 - Base Case: First row and First col are all 1 (only one straight line path).
12 - Optimization: Use 1D array (row) since we only need previous row.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "To reach any cell, I must come from the cell above it or the cell to the left.
17 # Therefore, the number of paths to '(r,c)' is simply the sum of paths to '(r-1,c)' and
18 # '(r,c-1)'.
19 # I can optimize this to O(N) space."
```

```
20 def uniquePaths(m: int, n: int) -> int:
21     # 1D DP Optimization: Store only the previous row
22     row = [1] * n
```

```
23
24     for i in range(m - 1): # Iterate rows
25         newRow = [1] * n
26         for j in range(1, n): # Iterate cols
27             # Current = Top (row[j]) + Left (newRow[j-1])
28             newRow[j] = row[j] + newRow[j - 1]
29         row = newRow
30
31     return row[-1]
```

35.2 Unique Paths II

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Obstacle? A: 1 means blocked.
4     - Q: Start blocked? A: Return 0 immediately.
5
6 2. LOGIC (GRID DP + ZEROING)
7     - Same as Unique Paths I.
8     - Modification: If grid[r][c] == 1, then dp[r][c] = 0 (0 ways to reach/pass through).
9     - Else: dp[r][c] = dp[r-1][c] + dp[r][c-1].
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "This is the same as Unique Paths, but if I encounter an obstacle at '(r,c)',
14 # I set the number of paths to that cell to 0. This naturally propagates 'zero paths'
15 # to any downstream cells blocked by it. O(M*N)."
```

```
16
17 def uniquePathsWithObstacles(obstacleGrid: List[List[int]]) -> int:
18     M, N = len(obstacleGrid), len(obstacleGrid[0])
19     dp = [0] * N
20     dp[0] = 1 # Base case: Start
21
22     for r in range(M):
23         for c in range(N):
24             if obstacleGrid[r][c] == 1:
25                 dp[c] = 0
26             elif c > 0:
27                 dp[c] += dp[c - 1] # dp[c] currently holds 'top', add 'left'
28
29     return dp[N - 1]
```

35.3 Minimum Path Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Movement? A: Right/Down only.
4   - Q: Values? A: Non-negative integers.
5
6 2. LOGIC (IN-PLACE DP)
7   - dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1]).
8   - Base Cases:
9     - Top-Left: grid[0][0].
10    - First Row: Can only come from Left.
11    - First Col: Can only come from Top.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use Dynamic Programming. The cost to reach '(r,c)' is the value at that cell
16 # plus
17 # the minimum cost to reach either the cell above or the cell to the left.
18 # I can overwrite the grid in-place to save space. O(M*N)."
19
20 def minPathSum(grid: List[List[int]]) -> int:
21     ROWS, COLS = len(grid), len(grid[0])
22
23     # Iterate top-left to bottom-right
24     for r in range(ROWS):
25         for c in range(COLS):
26             if r == 0 and c == 0:
27                 continue # Start point
28             elif r == 0:
29                 grid[r][c] += grid[r][c - 1] # Only from left
30             elif c == 0:
31                 grid[r][c] += grid[r - 1][c] # Only from top
32             else:
33                 grid[r][c] += min(grid[r - 1][c], grid[r][c - 1])
34
35     return grid[ROWS - 1][COLS - 1]
```

35.4 Longest Common Subsequence

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Subsequence? A: Non-contiguous ("ace" is sub of "abcde").
4 - Q: Input? A: Two strings text1, text2.
5
6 2. LOGIC (2D MATRIX)
7 - dp[i][j] = LCS of text1[:i] and text2[:j].
8 - If chars match (t1[i] == t2[j]):
9   - dp[i][j] = 1 + dp[i+1][j+1] (Diagonal + 1).
10 - If mismatch:
11   - dp[i][j] = max(dp[i+1][j], dp[i][j+1]) (Max of Right or Down).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use a 2D DP table. If characters match, I extend the LCS from the diagonal ('1 +
16 #   previous').
17 # If they don't, I carry over the best result found so far from either skipping a
18 #   character in string A
19 # or string B ('max(right, down)'). O(M*N)."
20
21 def longestCommonSubsequence(text1: str, text2: str) -> int:
22     # 2D grid initialized to 0
23     # Size (N+1)x(M+1) for easier base cases (out of bounds = 0)
24     dp = [[0 for _ in range(len(text2) + 1)]
25           for _ in range(len(text1) + 1)]
26
27     # Iterate backwards (bottom-up)
28     for i in range(len(text1) - 1, -1, -1):
29         for j in range(len(text2) - 1, -1, -1):
30             if text1[i] == text2[j]:
31                 # Diagonal + 1
32                 dp[i][j] = 1 + dp[i + 1][j + 1]
33             else:
34                 # Max(Right, Down)
35                 dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])
36
37     return dp[0][0]
```

35.5 Last Stone Weight II

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Rules? A: Smash x, y -> abs(x-y) remains.
4   - Q: Goal? A: Minimize remaining stone weight.
5   - Q: Insight? A: This is Partition Equal Subset Sum!
6     - Smashed stones effectively subtract. Remaining = Sum(SetA) - Sum(SetB).
7     - Minimize (Sum - 2*SubsetSum).
8
9 2. LOGIC (0/1 KNAPSACK)
10   - Target: Find subset sum closest to TotalSum / 2.
11   - Use "Partition Equal Subset Sum" logic.
12   - dp[s] = True if sum 's' is possible.
13   - Max subset sum <= Total/2 is our 'S2'.
14   - Answer = Total - 2 * S2.
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "This reduces to the Knapsack problem. Minimizing the final stone weight is equivalent
19 # to splitting the stones into two piles with the smallest difference in sum.
20 # I'll find the largest possible subset sum $\\le$ Total/2. O(N * Sum)."
```

```
21
22 def lastStoneWeightII(stones: List[int]) -> int:
23     stone_sum = sum(stones)
24     target = stone_sum // 2
25
26     # Logic: Find max achievable sum <= target
27     dp = {0} # Reachable sums
28
29     for s in stones:
30         new_sums = set()
31         for x in dp:
32             if x + s <= target:
33                 new_sums.add(x + s)
34                 new_sums.add(x)
35         dp = new_sums
36
37     max_s2 = max(dp)
38     return stone_sum - 2 * max_s2
```

35.6 Best Time to Buy and Sell Stock with Cooldown

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Cooldown? A: After sell, must wait 1 day before buy.
4 - Q: Goal? A: Max profit.
5
6 2. LOGIC (STATE MACHINE DP)
7 - Three states:
8   1. Held (Have stock): Can Sell or Hold.
9   2. Sold (Just sold): Forced Cooldown next.
10  3. Reset (Ready to buy): Can Buy or Wait.
11 - Transitions:
12 - held[i] = max(held[i-1], reset[i-1] - price)
13 - sold[i] = held[i-1] + price
14 - reset[i] = max(reset[i-1], sold[i-1])
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll model this as a State Machine. I maintain three states for each day: 'Held', '
19 #   Sold', and 'Reset'.
20 # The transitions enforce the cooldown: entering 'Held' comes from 'Reset', and 'Reset'
21 #   comes from 'Sold' or itself. O(N)."
```

```
21 def maxProfit(prices: List[int]) -> int:
22     # Initial states
23     sold = float("-inf")
24     held = float("-inf")
25     reset = 0
26
27     for price in prices:
28         pre_sold = sold
29
30         # Logic flow:
31         # Sold today: came from Held yesterday
32         sold = held + price
33
34         # Held today: kept Held OR bought from Reset
35         held = max(held, reset - price)
36
37         # Reset today: kept Reset OR just cooled down from Sold
38         reset = max(reset, pre_sold)
39
40     return max(sold, reset)
```

36 2-D DP: Knapsacks, Strings & Memoization

36.1 Coin Change II

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Goal? A: Number of combinations to make amount.
4 - Q: Infinite coins? A: Yes (Unbounded Knapsack).
5 - Q: Order matters? A: No. (1, 2) is same as (2, 1).
6
7 2. LOGIC (UNBOUNDED KNAPSACK)
8 - dp[i] = number of ways to make amount 'i'.
9 - Outer loop: Coins. Inner loop: Amounts.
10 - Why? Processing one coin at a time prevents duplicates (permutations).
11 - Formula: dp[a] += dp[a - coin].
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "This is the Unbounded Knapsack problem (counting combinations). To avoid duplicates
16 #   like '(1,2)' and '(2,1)',
17 # I must iterate through the coins in the outer loop. For each coin, I update the ways
18 #   to reach every possible amount. O(N * Amount)."
```

```
18 def change(amount: int, coins: List[int]) -> int:
19     dp = [0] * (amount + 1)
```

```

20     dp[0] = 1 # Base case: 1 way to make 0 (use no coins)
21
22     for c in coins:
23         for a in range(c, amount + 1):
24             dp[a] += dp[a - c]
25
26     return dp[amount]
27
28 """
29 5. DRY RUN
30 amount=5, coins=[1, 2, 5]
31 - Init dp: [1, 0, 0, 0, 0, 0]
32 - Coin 1: dp becomes [1, 1, 1, 1, 1, 1] (1 way using only 1s)
33 - Coin 2:
34     - a=2: dp[2] += dp[0] -> 1+1=2 (1+1, 2)
35     - a=3: dp[3] += dp[1] -> 1+1=2 (1+1+1, 1+2)
36     - a=4: dp[4] += dp[2] -> 1+2=3
37     - a=5: dp[5] += dp[3] -> 1+2=3
38 - Coin 5:
39     - a=5: dp[5] += dp[0] -> 3+1=4
40 - Result: 4.
41 """

```

36.2 Target Sum

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: nums array, target int.
4   - Q: Op? A: Assign '+' or '-' to each num.
5   - Q: Goal? A: Count assignments summing to target.
6
7 2. LOGIC (SUBSET SUM REDUCTION)
8   - Let P = positive subset, N = negative subset.
9   - Sum(P) - Sum(N) = Target.
10  - Sum(P) + Sum(N) + Sum(P) - Sum(N) = Target + TotalSum.
11  - 2 * Sum(P) = Target + TotalSum.
12  - Sum(P) = (Target + TotalSum) / 2.
13  - Problem becomes: Count subsets that sum to (Target + Total) / 2.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I can reduce this to the 'Subset Sum' problem. Finding '+' and '-' assignments is
18 # mathematically equivalent
19 # to finding a subset 'P' such that '2 * Sum(P) = Total + Target'. I then count how many
20 # subsets sum to this value using DP. O(N * Sum)."
```

```
19 def findTargetSumWays(nums: List[int], target: int) -> int:
20     total_sum = sum(nums)
21
22     # Mathematical check: (Total + Target) must be even and non-negative
23     if abs(target) > total_sum or (total_sum + target) % 2 != 0:
24         return 0
25
26     subset_target = (total_sum + target) // 2
27
28     dp = [0] * (subset_target + 1)
29     dp[0] = 1
30
31     for n in nums:
32         # Iterate backwards to avoid reusing the same element in the same step
33         for s in range(subset_target, n - 1, -1):
34             dp[s] += dp[s - n]
35
36     return dp[subset_target]
```

36.3 Interleaving String

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Interleaving? A: s3 is formed by s1 and s2 shuffling, preserving relative order.
4 - Q: Length? A: len(s3) must equal len(s1) + len(s2).
5
6 2. LOGIC (2D BOOLEAN GRID)
7 - dp[i][j] = True if s1[:i] and s2[:j] form s3[:i+j].
8 - If s1[i-1] matches s3[i+j-1], we can inherit dp[i-1][j].
9 - If s2[j-1] matches s3[i+j-1], we can inherit dp[i][j-1].
10 - Base case: dp[0][0] = True.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use a 2D boolean table. 'dp[i][j]' indicates if the prefix of 's1' (length 'i')
15 # and the prefix of 's2' (length 'j')
16 # can interleave to form the prefix of 's3' (length 'i+j'). I check character matches
17 # against 's3' to transition states. O(M*N)."
```

```
17 def isInterleave(s1: str, s2: str, s3: str) -> bool:
18     if len(s1) + len(s2) != len(s3):
19         return False
20
21     dp = [[False] * (len(s2) + 1) for _ in range(len(s1) + 1)]
22     dp[0][0] = True
23
24     for i in range(len(s1) + 1):
25         for j in range(len(s2) + 1):
26             # Check if coming from Top (s1 matches)
27             if i > 0 and s1[i - 1] == s3[i + j - 1] and dp[i - 1][j]:
28                 dp[i][j] = True
29
30             # Check if coming from Left (s2 matches)
31             if j > 0 and s2[j - 1] == s3[i + j - 1] and dp[i][j - 1]:
32                 dp[i][j] = True
33
34     return dp[-1][-1]
```

36.4 Longest Increasing Path in a Matrix

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Increasing? A: Strictly increasing (val < neighbor).
4   - Q: Movement? A: 4 directions.
5   - Q: Cycle? A: Impossible in strictly increasing path.
6
7 2. LOGIC (DFS + MEMOIZATION)
8   - This is hard to iterate (no clear Top-Left order).
9   - Use DFS(r, c) = 1 + max(DFS(neighbors)).
10  - Memoize: Cache results in a 2D array.
11  - Loop through every cell to start DFS (since path can start anywhere).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "Since the path must be strictly increasing, there are no cycles. I can model this as
16 # finding the longest path
17 # in a DAG using DFS with Memoization. For each cell, I compute the longest path
18 # starting from it and cache the result. O(M*N)."
19
20 def longestIncreasingPath(matrix: List[List[int]]) -> int:
21     ROWS, COLS = len(matrix), len(matrix[0])
22     dp = {} # Map (r,c) -> max length
23
24     def dfs(r, c, prevVal):
25         if (r < 0 or r >= ROWS or c < 0 or c >= COLS or
26             matrix[r][c] <= prevVal):
27             return 0
28
29         if (r, c) in dp:
30             return dp[(r, c)]
31
32         res = 1
33         res = max(res, 1 + dfs(r + 1, c, matrix[r][c]))
34         res = max(res, 1 + dfs(r - 1, c, matrix[r][c]))
35         res = max(res, 1 + dfs(r, c + 1, matrix[r][c]))
36         res = max(res, 1 + dfs(r, c - 1, matrix[r][c]))
37
38         dp[(r, c)] = res
39         return res
40
41     longest = 0
42     for r in range(ROWS):
43         for c in range(COLS):
44             longest = max(longest, dfs(r, c, -1))
45
46     return longest
```

36.5 Distinct Subsequences

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: s = "rabbbit", t = "rabbit".
4   - Q: Goal? A: Count ways to form 't' by deleting chars from 's'.
5
6 2. LOGIC (2D DP)
7   - dp[i][j] = ways to form t[:j] using s[:i].
8   - If chars mismatch (s[i] != t[j]):
9     - We MUST skip s[i].
10    - dp[i][j] = dp[i-1][j].
11   - If chars match (s[i] == t[j]):
12     - Option 1: Use s[i]. (Take from dp[i-1][j-1]).
13     - Option 2: Skip s[i]. (Take from dp[i-1][j]).
14     - dp[i][j] = dp[i-1][j-1] + dp[i-1][j].
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll use 2D DP. If the characters match, I have two choices: use the character (
19 #   inherit from diagonal)
20 # or skip it (inherit from top). If they don't match, I must skip (inherit from top).
21 # Summing these choices gives the total distinct subsequences. O(M*N)."
```

```
21 def numDistinct(s: str, t: str) -> int:
22     # Memoization (Top-Down) is often more intuitive for this one
23     cache = {}
24
25     def dfs(i, j): # i=index in s, j=index in t
26         if j == len(t): return 1 # Found a match
27         if i == len(s): return 0 # Ran out of source chars
28
29         if (i, j) in cache: return cache[(i, j)]
30
31         if s[i] == t[j]:
32             # Match: Sum of (Include char) and (Skip char)
33             cache[(i, j)] = dfs(i + 1, j + 1) + dfs(i + 1, j)
34         else:
35             # Mismatch: Must skip
36             cache[(i, j)] = dfs(i + 1, j)
37
38         return cache[(i, j)]
39
40     return dfs(0, 0)
```

36.6 Edit Distance

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Ops? A: Insert, Delete, Replace. Cost 1.
4     - Q: Goal? A: Min ops to convert word1 to word2.
5
6 2. LOGIC (CLASSIC 2D DP)
7     - dp[i][j] = min ops for word1[:i] -> word2[:j].
8     - Match (w1[i] == w2[j]): No op. dp[i][j] = dp[i-1][j-1].
9     - Mismatch:
10        - Insert: dp[i][j-1]
11        - Delete: dp[i-1][j]
12        - Replace: dp[i-1][j-1]
13        - Take 1 + min(Insert, Delete, Replace).
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "This is the classic Edit Distance problem. I'll use a 2D DP table. If characters
18 # match, no operation is needed.
19 # If they mismatch, I take the minimum of the three possible operations (Insert, Delete,
20 # Replace) and add 1 cost. O(M*N)."
```

```
19 def minDistance(word1: str, word2: str) -> int:
20     m, n = len(word1), len(word2)
21
22     # 2D Array (m+1) x (n+1)
23     dp = [[0] * (n + 1) for _ in range(m + 1)]
24
25     # Base Cases
26     for i in range(m + 1): dp[i][0] = i # Deleting all chars
27     for j in range(n + 1): dp[0][j] = j # Inserting all chars
28
29     for i in range(1, m + 1):
30         for j in range(1, n + 1):
31             if word1[i - 1] == word2[j - 1]:
32                 # Characters match, no new op needed
33                 dp[i][j] = dp[i - 1][j - 1]
34             else:
35                 # Mismatch: 1 + min(Replace, Delete, Insert)
36                 dp[i][j] = 1 + min(
37                     dp[i - 1][j - 1], # Replace
38                     dp[i - 1][j],     # Delete
39                     dp[i][j - 1]      # Insert
40                 )
41
42     return dp[m][n]
```

37 2-D DP: Game Theory & Hard Intervals

37.1 Stone Game

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Rules? A: Alice/Bob take from start or end.
4     - Q: Goal? A: Maximize stones. Return True if Alice wins.
5     - Q: Total stones? A: Odd (no ties). Piles even.
6
7 2. APPROACHES
8     - Approach 1: Math (Cheat).
9         - Alice can always force picking all even-indexed piles OR all odd-indexed piles.
10        - Since sum is odd, one set is larger. Alice always wins.
11        - Return True. O(1)).
12     - Approach 2: DP (Minimax).
13        - dp[i][j] = max relative score (MyStones - OpponentStones) for subarray i..j.
14        - Pick Left: piles[i] - dp[i+1][j].
15        - Pick Right: piles[j] - dp[i][j-1].
16        - Accept: Expected interview answer (proves you know DP).
17
18 3. INDENTED ENGLISH (DP)
```

```

19 function stoneGame(piles):
20     N = len(piles)
21     dp = table[N][N]
22     for length from 2 to N:
23         for i from 0 to N-length:
24             j = i + length - 1
25             pickL = piles[i] - dp[i+1][j]
26             pickR = piles[j] - dp[i][j-1]
27             dp[i][j] = max(pickL, pickR)
28     return dp[0][N-1] > 0
29 """

```

The Magic Question (Get Buy-In)

”Mathematically, Alice can always force a win by choosing to take all even-indexed or all odd-indexed piles. However, I’ll implement the 2D DP solution to demonstrate the Minimax logic. ‘dp[i][j]’ stores the maximum advantage over the opponent for the subarray ‘i’ to ‘j’. $O(N^2)$.”

```

1 def stoneGame(piles: List[int]) -> bool:
2     # Math Solution (One-liner):
3     # return True
4
5     # DP Solution (Minimax)
6     n = len(piles)
7     # dp[i][j] = max relative score for subarray piles[i:j+1]
8     dp = [[0] * n for _ in range(n)]
9
10    # Base case: diagonal (only 1 pile left)
11    for i in range(n):
12        dp[i][i] = piles[i]
13
14    for length in range(2, n + 1): # Length of subarray
15        for i in range(n - length + 1):
16            j = i + length - 1
17
18            # If I pick left (i), opponent gets optimal result from [i+1...j]
19            pickLeft = piles[i] - dp[i + 1][j]
20
21            # If I pick right (j), opponent gets optimal result from [i...j-1]
22            pickRight = piles[j] - dp[i][j - 1]
23
24            dp[i][j] = max(pickLeft, pickRight)
25
26    return dp[0][n - 1] > 0

```

37.2 Stone Game II

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Rules? A: Can take 1 to 2*M piles. M = max(M, taken).
4 - Q: Goal? A: Alice wants to maximize her total stones.
5 - Q: Opponent? A: Bob plays optimally.
6
7 2. LOGIC (DP + MEMOIZATION)
8 - State: (i, m, isAlice).
9 - Alice choice: Try X from 1 to 2*M.
10 - Result = sum(piles[i:i+X]) + solve(i+X, max(m, X), Bob)
11 - Alice maximizes result.
12 - Bob choice: Try X from 1 to 2*M.
13 - Result = solve(i+X, max(m, X), Alice) (Bob takes stones, doesn't give to Alice)
14 - Bob minimizes Alice's result.
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll use DP with Memoization. The state is '(index, M, isAlice)'. Alice tries all
19 # valid moves (1 to 2M)
20 # and maximizes her score. Bob tries all moves and minimizes Alice's score (Zero-Sum
21 # Game). O(N^3)."
22
23 def stoneGameII(piles: List[int]) -> int:
24     cache = {}
25     N = len(piles)
26
27     # Suffix sums for O(1) calculation of stone ranges
28     suffix_sum = [0] * N
29     suffix_sum[-1] = piles[-1]
30     for i in range(N - 2, -1, -1):
31         suffix_sum[i] = piles[i] + suffix_sum[i + 1]
32
33     def dfs(i, m, isAlice):
34         if i >= N: return 0
35         if (i, m, isAlice) in cache: return cache[(i, m, isAlice)]
36
37         res = 0 if isAlice else float("inf")
38
39         for x in range(1, 2 * m + 1):
40             if i + x > N: break
41
42             if isAlice:
43                 # Alice takes current stones + recursively gets best from next state
44                 current_stones = suffix_sum[i] - (suffix_sum[i+x] if i+x < N else 0)
45                 res = max(res, current_stones + dfs(i + x, max(m, x), False))
46             else:
47                 # Bob takes stones (we don't add to res), tries to minimize Alice's
48                 future
49                 res = min(res, dfs(i + x, max(m, x), True))
50
51         cache[(i, m, isAlice)] = res
52         return res
53
54     return dfs(0, 1, True)
```

37.3 Burst Balloons

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Score? A: nums[i-1] * nums[i] * nums[i+1].
4   - Q: Edges? A: Implicit 1s on boundary.
5   - Q: Order? A: Determine optimal order to burst.
6
7 2. LOGIC (REVERSE THINKING / INTERVAL DP)
8   - Instead of picking which to burst FIRST, pick which to burst LAST.
9   - Why? The LAST balloon connects left boundary and right boundary.
10  - dp[i][j] = max coins for range (i, j).
11  - Iterate k from i+1 to j-1 (k is the last balloon to burst).
12  - Coins = nums[i]*nums[k]*nums[j] + dp[i][k] + dp[k][j].
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "This is a classic Interval DP problem. Instead of deciding what to burst first, I'll
17 #   decide what to burst *last*.
18 # If 'k' is the last balloon in the range '(i, j)', the cost is 'nums[i]*nums[k]*nums[j]
19 #   ' plus the subproblems 'dp[i][k]' and 'dp[k][j]'. O(N^3)."
```

```
18
19 def maxCoins(nums: List[int]) -> int:
20     # Pad with 1s
21     nums = [1] + nums + [1]
22     dp = {}
23
24     def dfs(l, r):
25         # Base case: No balloons between boundaries
26         if l + 1 == r:
27             return 0
28         if (l, r) in dp:
29             return dp[(l, r)]
30
31         res = 0
32         # Iterate through every possible LAST balloon k
33         for k in range(l + 1, r):
34             coins = nums[l] * nums[k] * nums[r]
35             res = max(res, coins + dfs(l, k) + dfs(k, r))
36
37         dp[(l, r)] = res
38         return res
39
40     return dfs(0, len(nums) - 1)
41
42 """
43 5. DRY RUN
44     nums=[3,1,5,8] -> [1,3,1,5,8,1]
45     - dfs(0, 5): Range 1..8
46       - Try k=1 (3): coins=1*3*1 + dfs(0,1)+dfs(1,5)
47       - Try k=4 (8): coins=1*8*1 + dfs(0,4)+dfs(4,5)
48       - ... Recursion solves smaller sub-ranges.
49 """
```

37.4 Regular Expression Matching

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: '.'? A: Any single char.
4 - Q: '*'? A: Zero or more of PRECEDING element.
5 - Q: '.*'? A: Zero or more of ANY sequence.
6
7 2. LOGIC (2D DP / RECURSION)
8 - Base Case: If pattern empty, string must be empty.
9 - Match: first_match = (p[0] == s[0] or p[0] == '.').
10 - Star Logic (p[1] == '*'):
11 - Choice 1: Ignore star (0 matches) -> solve(s, p[2:]).
12 - Choice 2: Use star (1+ matches) -> first_match AND solve(s[1:], p).
13 - Else (No star):
14 - first_match AND solve(s[1:], p[1:]).
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll use DFS with memoization. The tricky part is the '*'. If the next character is
19 # '.*', I have two choices:
20 # ignore the 'x*' entirely (match 0 times), or if the first character matches, consume
21 # one character from 's' and keep the 'x*' pattern active. O(M*N)."
```

```
21 def isMatch(s: str, p: str) -> bool:
22     cache = {}
23
24     def dfs(i, j):
25         if (i, j) in cache: return cache[(i, j)]
26
27         if j >= len(p):
28             return i >= len(s)
29
30         match = i < len(s) and (s[i] == p[j] or p[j] == ".")
31
32         # Check if next char is '*'
33         if j + 1 < len(p) and p[j + 1] == "*":
34             # Option 1: Don't use '*' (skip p[j] and p[j+1])
35             # Option 2: Use '*' (if match, move s index, keep p index)
36             res = (dfs(i, j + 2) or
37                   (match and dfs(i + 1, j)))
38         else:
39             # Standard match
40             res = match and dfs(i + 1, j + 1)
41
42         cache[(i, j)] = res
43         return res
44
45     return dfs(0, 0)
```

38 Greedy: Change, Subarrays & Jumps

38.1 Lemonade Change

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: List of bills [5, 10, 20].
4 - Q: Price? A: 5 each.
5 - Q: Start? A: 0 change.
6 - Q: Goal? A: Return True if we can provide change for everyone.
7
8 2. LOGIC (SIMULATION)
9 - Maintain counters for $5 and $10 bills.
10 - If customer gives 5: five++.
11 - If customer gives 10: ten++, five-- (Must give 5 back).
12 - If customer gives 20:
13 - Preference: Give 10 + 5 (Get rid of 10s first!).
14 - Else: Give 5 + 5 + 5.
15 - Else: False.
16 """
```

```

17
18 # --- THE MAGIC QUESTION ---
19 # "I'll simulate the transaction queue. The greedy key is: when giving change for a $20,
20 # I always prioritize giving a $10 and a $5 over three $5s, because $5 bills are more
   versatile."
21
22 def lemonadeChange(bills: List[int]) -> bool:
23     five, ten = 0, 0
24
25     for bill in bills:
26         if bill == 5:
27             five += 1
28         elif bill == 10:
29             if five == 0: return False
30             five -= 1
31             ten += 1
32         else: # bill == 20
33             # Greedy: Try to spend a 10 first
34             if ten > 0 and five > 0:
35                 ten -= 1
36                 five -= 1
37             elif five >= 3:
38                 five -= 3
39             else:
40                 return False
41
42     return True

```

38.2 Maximum Subarray

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Negatives? A: Yes.
4   - Q: Empty? A: Assume valid input with at least 1 number.
5   - Q: Return? A: The Sum (not the subarray).
6
7 2. LOGIC (KADANE'S ALGORITHM)
8   - Concept: "Should I start a new subarray, or extend the current one?"
9   - Rule: If 'current_sum' becomes negative, it's contributing garbage. Reset it to 0.
10  - Loop: 'curSum += n'. 'maxSub = max(maxSub, curSum)'. 'if curSum < 0: curSum = 0'.
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "I'll use Kadane's Algorithm. I iterate through the array, adding to a running sum.
15 # If the running sum ever drops below zero, I reset it, effectively discarding the
16 #   negative prefix.
17 # I track the maximum sum seen so far. O(N)."
```

```
18 def maxSubArray(nums: List[int]) -> int:
19     maxSub = nums[0]
20     curSum = 0
21
22     for n in nums:
23         if curSum < 0:
24             curSum = 0
25         curSum += n
26         maxSub = max(maxSub, curSum)
27
28     return maxSub
29
30 """
31 5. DRY RUN
32     nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
33     - -2: cur=-2. max=-2. cur reset to 0.
34     - 1: cur=1. max=1.
35     - -3: cur=-2. max=1. cur reset to 0.
36     - 4: cur=4. max=4.
37     - ...
38 """
```

38.3 Maximum Sum Circular Subarray

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Circular? A: Subarray can wrap from end to start.
4   - Q: All negatives? A: Return the max element (least negative).
5
6 2. LOGIC (MIN/MAX KADANE)
7   - Case 1: Max subarray is in the middle (Normal Kadane).
8   - Case 2: Max subarray wraps around.
9     - Algebra: MaxWrap = TotalSum - MinSubarray.
10    - Why? Removing the "worst" middle part leaves the "best" circular part.
11   - Result: max(MaxSubarray, Total - MinSubarray).
12   - Edge Case: If all numbers are negative, MaxSubarray is the answer (Total - Min
13     would be 0, which is empty, invalid).
14
15 # --- THE MAGIC QUESTION ---
16 # "A circular max subarray is either a standard max subarray, or the total sum minus the
17   standard *minimum* subarray (wrapping around). I'll compute both using Kadane's
18   logic and take the maximum. O(N)."
19
20 def maxSubarraySumCircular(nums: List[int]) -> int:
21     globalMax, globalMin = nums[0], nums[0]
22     curMax, curMin = 0, 0
23     total = 0
24
25     for n in nums:
26         curMax = max(curMax + n, n)
27         curMin = min(curMin + n, n)
28         total += n
29
30         globalMax = max(globalMax, curMax)
31         globalMin = min(globalMin, curMin)
32
33     # Edge case: If all nums are negative, globalMax is the largest single negative
34     if globalMax < 0:
35         return globalMax
36
37     return max(globalMax, total - globalMin)
```

38.4 Longest Turbulent Subarray

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Turbulent? A: Comparisons flip: > < > < >.
4     - Q: Equality? A: Equality breaks turbulence (e.g., 5, 5).
5
6 2. LOGIC (SLIDING WINDOW / GREEDY)
7     - Use pointers 'l' and 'r'.
8     - Iterate 'r'. Check comparison between 'r' and 'r-1'.
9     - If sign flips compared to previous, extend window ('r - l + 1').
10    - If sign stays same (e.g., < then <), reset 'l = r - 1'.
11    - If equal, reset 'l = r'.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll use a sliding window. As long as the comparison sign (> or <) flips at each step
16 # I expand the window. If the sign matches the previous one or values are equal,
17 # I reset the window start. O(N)."
18
19 def maxTurbulenceSize(arr: List[int]) -> int:
20     N = len(arr)
21     if N == 1: return 1
22
23     res = 1
24     l, r = 0, 1
25     prev = "" # ">", "<", or "="
26
27     while r < N:
28         if arr[r-1] > arr[r]:
29             curr = ">"
30         elif arr[r-1] < arr[r]:
31             curr = "<"
32         else:
33             curr = "="
34
35         if curr == "=":
36             l = r # Reset completely
37         elif curr == prev:
38             l = r - 1 # Reset to last valid pair
39
40         # Else: curr != prev and curr != "=", valid flip, just continue
41
42         res = max(res, r - l + 1)
43         prev = curr
44         r += 1
45
46     return res
```

38.5 Jump Game

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Value? A: Max jump length from that position.
4   - Q: Goal? A: Reach last index.
5
6 2. APPROACHES
7   - Approach 1: DP.  $O(N^2)$ . Too slow.
8   - Approach 2: Greedy (Shift Goal Post).
9     - Start from end ( $N-1$ ).
10    - Can index 'i' reach the current goal? ( $i + \text{nums}[i] \geq \text{goal}$ ).
11    - If yes, 'i' becomes the new goal.
12    - If final goal is 0, True.
13    - Accept: Optimal  $O(N)$ .
14
15 3. INDENTED ENGLISH (GREEDY)
16  function canJump(nums):
17      goal = len(nums) - 1
18      for i from len-1 down to 0:
19          if i + nums[i] >= goal:
20              goal = i
21      return goal == 0
22 """
```

The Magic Question (Get Buy-In)

"I'll work backwards from the last index. I maintain a 'goal' index, initially the last element. If the current index 'i' can jump to the 'goal', then 'i' becomes the new 'goal'. If the goal reaches 0, we win. $O(N)$."

```
1 def canJump(nums: List[int]) -> bool:
2     goal = len(nums) - 1
3
4     for i in range(len(nums) - 1, -1, -1):
5         if i + nums[i] >= goal:
6             goal = i
7
8     return goal == 0
9
10 """
11 5. DRY RUN
12  nums = [2, 3, 1, 1, 4]
13  - Goal=4.
14  - i=3 (1): 3+1 >= 4? Yes. Goal=3.
15  - i=2 (1): 2+1 >= 3? Yes. Goal=2.
16  - i=1 (3): 1+3 >= 2? Yes. Goal=1.
17  - i=0 (2): 0+2 >= 1? Yes. Goal=0.
18  - Return True.
19 """
```

39 Greedy: Advanced Jumps & Simulation

39.1 Jump Game II

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Goal? A: Min number of jumps to reach end.
4   - Q: Guaranteed? A: Yes.
5
6 2. LOGIC (IMPLICIT BFS)
7   - Do not use DP ( $O(N^2)$ ). Use BFS logic in  $O(N)$ .
8   - We jump in "levels".
9   - Level 0: Index 0.
10  - Level 1: All nodes reachable from Level 0.
11  - Level 2: All nodes reachable from Level 1 (that weren't in L0).
12  - Track 'l' and 'r' of the current level.
```

```

13     - Farthest reach = max(i + nums[i]) for i in l..r.
14     """
15
16     # --- THE MAGIC QUESTION ---
17     # "I'll use BFS principles without a queue. I maintain a window '[l, r]' representing
18     # the range of the current jump number.
19     # I iterate through this window to find the farthest point reachable, which defines the
20     # window '[r+1, farthest]' for the next jump. O(N)."
21
22     def jump(nums: List[int]) -> int:
23         jumps = 0
24         l, r = 0, 0
25
26         while r < len(nums) - 1:
27             farthest = 0
28             for i in range(l, r + 1):
29                 farthest = max(farthest, i + nums[i])
30
31             l = r + 1
32             r = farthest
33             jumps += 1
34
35         return jumps

```

39.2 Jump Game VII

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: Binary string s, minJump, maxJump.
4 - Q: Rules? A: Jump from i to j if s[j]=='0' and i+min <= j <= i+max.
5 - Q: Goal? A: Reach last index.
6
7 2. APPROACHES
8 - Approach 1: BFS/DP (Naive). O(N * (max-min)). TLE.
9 - Approach 2: Sliding Window / Queue.
10 - We process index 'i'. We want to "activate" range [i+min, i+max].
11 - Use a Queue to store indices we can reach.
12 - When processing 'i' (from queue), add all valid '0's in range.
13 - Optimization: Don't re-scan range. Start scanning from 'max(current_idx,
14   last_scanned_max + 1)'.
15 - Complexity: O(N).
16 """
17 # --- THE MAGIC QUESTION ---
18 # "I'll use BFS with a Queue. To avoid O(N^2) behavior, I'll ensure I never process
19 # the same index twice.
20 # When I pop an index 'i', I scan for valid jump targets only starting from the '
21 # farthest' point I've already checked. O(N)."
```

```
20
21 def canReach(s: str, minJump: int, maxJump: int) -> bool:
22     q = deque([0])
23     farthest = 0
24
25     while q:
26         i = q.popleft()
27
28         # Start scanning from where we left off, or the start of the new jump range
29         start = max(i + minJump, farthest + 1)
30         end = min(i + maxJump + 1, len(s))
31
32         for j in range(start, end):
33             if s[j] == "0":
34                 if j == len(s) - 1: return True
35                 q.append(j)
36
37         # Update farthest to avoid re-checking
38         farthest = i + maxJump
39
40     return False
```

39.3 Gas Station

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Unique solution? A: Guaranteed unique or none.
4   - Q: Empty? A: No.
5
6 2. LOGIC (TOTAL VS CURRENT SURPLUS)
7   - Condition 1: If sum(gas) < sum(cost), impossible. Return -1.
8   - Condition 2: If solvable, there exists a start index.
9   - Greedy: Iterate. Track 'total'. If 'total' drops below 0, it means the current
10     start position failed.
11   - Reset start to 'i + 1' and reset 'total' to 0.
12   - Why? If A can't reach B, no point between A and B can reach B either.
13 """
14 # --- THE MAGIC QUESTION ---
15 # "First, if total gas is less than total cost, it's impossible. Otherwise, a solution
16 # is guaranteed.
17 # I'll iterate through stations accumulating a tank surplus. If the tank drops below
18 # zero,
19 # I know the starting point must be after the current station, so I reset the start
20 # pointer. O(N)."
```

```
18
19 def canCompleteCircuit(gas: List[int], cost: List[int]) -> int:
20     if sum(gas) < sum(cost):
21         return -1
22
23     total = 0
24     start = 0
25
26     for i in range(len(gas)):
27         total += (gas[i] - cost[i])
28
29         if total < 0:
30             total = 0
31             start = i + 1
32
33     return start
```

39.4 Hand of Straights

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Hand of cards, groupSize W.
4   - Q: Goal? A: Split all cards into groups of W consecutive numbers.
5   - Q: [1,2,3,6,2,3,4,7,8], W=3 -> [1,2,3], [2,3,4], [6,7,8].
6
7 2. LOGIC (MIN-HEAP / SORTED MAP)
8   - Greedy: We MUST use the smallest card available to start a sequence.
9   - Why? The smallest card 'x' cannot be the middle of a sequence (requires 'x-1').
10  - Algo:
11     1. Count frequency of all cards.
12     2. Min-Heap of unique cards.
13     3. Pop min 'start'. Check if 'start+1', 'start+2'... exist.
14     4. Decrement counts.
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I must always process the smallest available card first, as it can only be the start
19 # of a sequence.
20 # I'll use a Hash Map for counts and a Min-Heap to track the minimum starting card. O(N
21 # log N)."
```

```
21 def isNStraightHand(hand: List[int], groupSize: int) -> bool:
22     if len(hand) % groupSize != 0:
23         return False
24
25     count = collections.Counter(hand)
26     minH = list(count.keys())
27     heapq.heapify(minH)
28
29     while minH:
30         first = minH[0]
31
32         for i in range(first, first + groupSize):
33             if i not in count:
34                 return False
35
36             count[i] -= 1
37             if count[i] == 0:
38                 # Optimized: We expect to remove the min element
39                 if i != minH[0]:
40                     return False
41                 heapq.heappop(minH)
42
43     return True
```

39.5 Dota2 Senate

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Rules? A: Ban the next opponent senator.
4     - Q: Round based? A: Yes, if you ban, you stay for next round.
5
6 2. LOGIC (TWO QUEUES)
7     - Store indices of 'R' and 'D' in two separate queues.
8     - Compare front of R_queue vs D_queue.
9     - Smaller index acts first -> Bans the larger index.
10    - Winner is pushed to back of queue with index 'i + N' (next round).
11    - Loser is popped permanently.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll simulate the voting using two queues: one for Radiant and one for Dire, storing
16   their indices.
17 # In each turn, the senator with the smaller index bans the other. The winner gets re-
18   added to the queue
19 # with an offset of 'N' to represent the next round. O(N)."
```

```
19 def predictPartyVictory(senate: str) -> str:
20     senate = list(senate)
21     R, D = deque(), deque()
22
23     # Fill queues
24     for i, c in enumerate(senate):
25         if c == "R": R.append(i)
26         else: D.append(i)
27
28     N = len(senate)
29     while R and D:
30         rTurn = R.popleft()
31         dTurn = D.popleft()
32
33         if rTurn < dTurn:
34             R.append(rTurn + N)
35         else:
36             D.append(dTurn + N)
37
38     return "Radiant" if R else "Dire"
```

40 Greedy: Constructive Logic

40.1 Merge Triplets to Form Target Triplet

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Operation? A: max(a[i], b[i]) for all i.
4     - Q: Goal? A: Reach exactly [t1, t2, t3].
5     - Q: Logic? A: If ANY value in a triplet is > target, it's useless.
6         - Why? Taking max() with a larger value ruins the target forever.
7
8 2. LOGIC (FILTER & VERIFY)
9     - Step 1: Filter out "poison" triplets (where triplet[i] > target[i]).
10    - Step 2: From remaining valid triplets, check if we can form the target.
11    - We need at least one triplet where trip[0] == target[0].
12    - We need at least one triplet where trip[1] == target[1].
13    - We need at least one triplet where trip[2] == target[2].
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "A triplet is 'poison' if any of its values exceed the target, as 'max()' can never
18   reduce a value.
19 # I'll ignore those. For the valid triplets, I just need to verify that we can
20   eventually find the target value
21 # for each of the three positions independently. O(N)."
```

```
21 def mergeTriplets(triplets: List[List[int]], target: List[int]) -> bool:
```

```
22 good = set()
23
24 for t in triplets:
25     # If triplet is valid (no element exceeds target)
26     if t[0] > target[0] or t[1] > target[1] or t[2] > target[2]:
27         continue
28
29     # Check which indices match the target
30     for i, v in enumerate(t):
31         if v == target[i]:
32             good.add(i)
33
34 return len(good) == 3
```

40.2 Partition Labels

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: "ababcbacadefegdehijhklij".
4   - Q: Rule? A: Each letter appears in at most one part.
5   - Q: Goal? A: List of sizes of partitions.
6
7 2. LOGIC (LAST OCCURRENCE MAP)
8   - Insight: If we have 'a' in a partition, the partition MUST extend to the *last*
9     occurrence of 'a'.
10   - Algo:
11     1. Map 'last = { char : index }'.
12     2. Iterate string. Maintain 'end' of current partition.
13     3. 'end = max(end, last[char])'.
14     4. If 'i == end', we found a cut point. Store size, reset.
15
16 # --- THE MAGIC QUESTION ---
17 # "To ensure every letter appears in only one partition, a partition starting with
18   character 'C'
19 # must extend to at least the last occurrence of 'C'. I'll iterate through the string,
20   dynamically
21 # extending the partition's end index. When the current index meets the end index, I cut
22   . O(N)."
```

```
20
21 def partitionLabels(s: str) -> List[int]:
22     lastIndex = {c: i for i, c in enumerate(s)}
23     res = []
24     size, end = 0, 0
25
26     for i, c in enumerate(s):
27         size += 1
28         end = max(end, lastIndex[c])
29
30         if i == end:
31             res.append(size)
32             size = 0
33
34     return res
```

40.3 Valid Parenthesis String

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: '(', ')', '*''.
4 - Q: Star? A: Can be '(', ')', or '*'.
5 - Q: Goal? A: Return True if valid.
6
7 2. LOGIC (RANGE OF POSSIBILITIES)
8 - We can't use a simple counter because of '*'.
9 - Maintain a range '[minOpen, maxOpen]'.
10 - '(': min++, max++.
11 - ')': min--, max--.
12 - '*': min-- (treat as ')'), max++ (treat as '(').
13 - Rules:
14   - 'minOpen' can't be negative (reset to 0).
15   - 'maxOpen' can't be negative (return False immediately).
16   - At end, 'minOpen' must be 0.
17 """
18
19 # --- THE MAGIC QUESTION ---
20 # "Because '*' introduces ambiguity, I'll track the *range* of possible open parentheses
21 #   count '[min, max]'.
22 # '('*' broadens the range. If 'max' ever drops below zero, it's invalid.
23 # If 'min' ends at zero, it's valid. O(N)."
24
25 def checkValidString(s: str) -> bool:
26     leftMin, leftMax = 0, 0
27
28     for c in s:
29         if c == "(":
30             leftMin += 1
31             leftMax += 1
32         elif c == ")":
33             leftMin -= 1
34             leftMax -= 1
35         else: # c == '*'
36             leftMin -= 1 # Treat as )
37             leftMax += 1 # Treat as (
38
39         if leftMax < 0:
40             return False # Too many )
41
42         if leftMin < 0:
43             leftMin = 0 # Can't have negative open brackets
44
45     return leftMin == 0
```

40.4 Candy

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Rule 1? A: Each child has at least 1 candy.
4     - Q: Rule 2? A: Higher rating neighbors get more candy.
5     - Q: Goal? A: Min total candy.
6
7 2. LOGIC (TWO PASS GREEDY)
8     - Initialize all to 1.
9     - Left-to-Right Pass: If ratings[i] > ratings[i-1], candies[i] = candies[i-1] + 1.
10    - Right-to-Left Pass: If ratings[i] > ratings[i+1], candies[i] = max(candies[i],
11      candies[i+1] + 1).
12    - Why max? We must satisfy both neighbors.
13 """
14 # --- THE MAGIC QUESTION ---
15 # "I'll use a two-pass greedy approach. First, I scan left-to-right to ensure children
16   with higher ratings
17 # than their left neighbor get more candy. Then, I scan right-to-left for the right
18   neighbor condition.
19 # Taking the maximum of both passes satisfies all constraints. O(N)."
```

```
18
19 def candy(ratings: List[int]) -> int:
20     arr = [1] * len(ratings)
21
22     # Pass 1: Left to Right
23     for i in range(1, len(ratings)):
24         if ratings[i] > ratings[i - 1]:
25             arr[i] = arr[i - 1] + 1
26
27     # Pass 2: Right to Left
28     for i in range(len(ratings) - 2, -1, -1):
29         if ratings[i] > ratings[i + 1]:
30             arr[i] = max(arr[i], arr[i + 1] + 1)
31
32     return sum(arr)
```

41 Intervals: Merging & Schedules

41.1 Insert Interval

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Input sorted? A: Yes, by start time.
4     - Q: Overlap? A: Merge if necessary.
5
6 2. LOGIC (LINEAR SCAN)
7     - Three phases:
8         1. Add all intervals ending before newInterval starts.
9         2. Merge overlapping intervals (update newInterval).
10            - Start = min(old.start, new.start)
11            - End = max(old.end, new.end)
12         3. Add all remaining intervals.
13     - Complexity: O(N).
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "Since the list is already sorted, I can iterate in one pass. I'll add all intervals
18   that come strictly before the new one.
19 # Then, I'll merge all overlapping intervals into the 'newInterval'. Finally, I'll add
20   the rest. O(N)."
```

```
19
20 def insert(intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
21     res = []
22
23     for i in range(len(intervals)):
24         # Case 1: Current interval is completely after newInterval
25         if newInterval[1] < intervals[i][0]:
26             res.append(newInterval)
```

```
27         return res + intervals[i:]
28
29     # Case 2: Current interval is completely before newInterval
30     elif newInterval[0] > intervals[i][1]:
31         res.append(intervals[i])
32
33     # Case 3: Overlap -> Merge
34     else:
35         newInterval = [
36             min(newInterval[0], intervals[i][0]),
37             max(newInterval[1], intervals[i][1])
38         ]
39
40     res.append(newInterval)
41     return res
```

41.2 Merge Intervals

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Sorted? A: No guarantees.
4     - Q: Output? A: Non-overlapping list.
5
6 2. LOGIC (SORT & MERGE)
7     - Step 1: Sort by start time.
8     - Step 2: Iterate.
9         - If current.start <= last_merged.end: Overlap!
10            - last_merged.end = max(last_merged.end, current.end).
11        - Else: No overlap. Append current.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll sort the intervals by start time. Then I'll iterate through them.
16 # If the current interval starts before the previous one ends, I merge them by extending
17 # the end time.
18 # Otherwise, I simply add the interval to the result. O(N log N)."
```

```
18
19 def merge(intervals: List[List[int]]) -> List[List[int]]:
20     intervals.sort(key=lambda x: x[0])
21     output = [intervals[0]]
22
23     for start, end in intervals[1:]:
24         lastEnd = output[-1][1]
25
26         if start <= lastEnd:
27             # Merge
28             output[-1][1] = max(lastEnd, end)
29         else:
30             # Add new
31             output.append([start, end])
32
33     return output
```

41.3 Non-overlapping Intervals

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Goal? A: Min number of intervals to remove to make rest valid.
4   - Q: Strategy? A: Greedy.
5
6 2. LOGIC (SORT BY START / CHECK END)
7   - Sort by start time.
8   - If two intervals overlap (start < prevEnd):
9     - We must remove one. Which one?
10    - The one that ends LATER (it causes more future collisions).
11    - Keep the one that ends SOONER (min of ends).
12    - Count++.
13   - Else: Update prevEnd = current.end.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll sort by start time and iterate. If two intervals overlap, I must remove one.
18 # Greedily, I should remove the one that ends *later*, as it blocks more future
19 # intervals.
20 # So I keep the one with the smaller end time. O(N log N)."
```

```
21 def eraseOverlapIntervals(intervals: List[List[int]]) -> int:
22     intervals.sort()
23     res = 0
24     prevEnd = intervals[0][1]
25
26     for start, end in intervals[1:]:
27         if start >= prevEnd:
28             # No overlap, safe update
29             prevEnd = end
30         else:
31             # Overlap! Remove one
32             res += 1
33             # Keep the one that ends first (to minimize future overlaps)
34             prevEnd = min(prevEnd, end)
35
36     return res
```

41.4 Meeting Rooms

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: List of [start, end].
4   - Q: Goal? A: Can one person attend ALL meetings? (No overlaps).
5
6 2. LOGIC (SORT CHECK)
7   - Sort by start time.
8   - Check if current.start < prev.end.
9   - If yes, return False.
10 """
11
12 # --- THE MAGIC QUESTION ---
13 # "I'll sort the meetings by start time. If any meeting starts before the previous one
14 #   ends,
15 # it's impossible to attend both. O(N log N)."
```

```
16 def canAttendMeetings(intervals: List[List[int]]) -> bool:
17     intervals.sort(key=lambda x: x[0])
18
19     for i in range(1, len(intervals)):
20         if intervals[i][0] < intervals[i - 1][1]:
21             return False
22
23     return True
```

41.5 Meeting Rooms II

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Goal? A: Min conference rooms required.
4 - Q: Overlap? A: If overlap, need new room.
5
6 2. APPROACHES
7 - Approach 1: Min-Heap.
8   - Heap stores 'end times' of occupied rooms.
9   - If new_start >= min_end (heap top): Room free! Pop heap.
10  - Always push new_end.
11  - Result: len(heap).
12  - Accept: Intuitive.
13 - Approach 2: Chronological Ordering (Two Pointers).
14  - Separate 'starts' and 'ends'. Sort both.
15  - Iterate 'starts'. If start < end, need room (count++).
16  - Else (meeting ended), decrement count (or just move end ptr).
17  - Accept: Optimal.
18
19 3. INDENTED ENGLISH (TWO POINTERS)
20 function minMeetingRooms(intervals):
21     starts = sorted([i.start for i in intervals])
22     ends = sorted([i.end for i in intervals])
23     res, count = 0, 0
24     s, e = 0, 0
25
26     while s < len(intervals):
27         if starts[s] < ends[e]:
28             count += 1
29             s += 1
30         else:
31             count -= 1
32             e += 1
33         res = max(res, count)
34     return res
35 """
```

The Magic Question (Get Buy-In)

"I'll separate the start times and end times into two sorted arrays. I'll iterate through them chronologically. If a meeting starts before the earliest ending meeting finishes, I increment the room count. Otherwise, I free up a room. $O(N \log N)$."

```
1 def minMeetingRooms(intervals: List[List[int]]) -> int:
2     starts = sorted([i[0] for i in intervals])
3     ends = sorted([i[1] for i in intervals])
4
5     res, count = 0, 0
6     s, e = 0, 0
7
8     while s < len(intervals):
9         # A meeting starts before the nearest one ends
10        if starts[s] < ends[e]:
11            count += 1
12            s += 1
13        # A meeting ended
14        else:
15            count -= 1
16            e += 1
17        res = max(res, count)
18
19    return res
```

42 Intervals: Advanced Heaps

42.1 Meeting Rooms III

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Goal? A: Room with MOST meetings.
4 - Q: Tie-break? A: Lower room index.
5 - Q: Delay rule? A: If no room free, delay start until earliest room finishes.
6 - Q: Input? A: n rooms, meetings list.
7
8 2. LOGIC (TWO HEAPS)
9 - Sort meetings by start time.
10 - Heap 1 ('available'): Min-Heap of room indices (starts with 0..n-1).
11 - Heap 2 ('busy'): Min-Heap of (end_time, room_index).
12 - Iterate through meetings:
13     1. Release rooms: If 'busy.top.end <= start', move from 'busy' to 'available'.
14     2. If 'available' exists: Pop min index, add to 'busy' with normal end time.
15     3. If 'available' empty: Pop earliest 'busy' room.
16         - New End = old_end + duration.
17         - Add back to 'busy'.
18     4. Track count array.
19 """
20
21 # --- THE MAGIC QUESTION ---
22 # "I'll manage the rooms using two Min-Heaps: 'available' (room indices) and 'busy' (end
23 # times).
24 # I sort meetings by start time. If no room is free, I delay the meeting by picking the
25 # earliest finishing room
26 # from the 'busy' heap and extending its end time. O(M log N)."
27
28 def mostBooked(n: int, meetings: List[List[int]]) -> int:
29     meetings.sort()
30
31     # available: heap of room indices [0, 1, 2...]
32     available = [i for i in range(n)]
33     heapq.heapify(available)
34
35     # busy: heap of (end_time, room_index)
36     busy = []
37
38     count = [0] * n
39
40     for start, end in meetings:
41         # 1. Free up finished rooms
42         while busy and busy[0][0] <= start:
43             _, room = heapq.heappop(busy)
44             heapq.heappush(available, room)
45
46         # 2. Assign room
47         if available:
48             room = heapq.heappop(available)
49             heapq.heappush(busy, (end, room))
50         else:
51             # Must wait for earliest room
52             finish_time, room = heapq.heappop(busy)
53             # New end time is calculated with DELAY
54             new_end = finish_time + (end - start)
55             heapq.heappush(busy, (new_end, room))
56
57         count[room] += 1
58
59     return count.index(max(count))

```

42.2 Minimum Interval to Include Each Query

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Intervals [s, e], Queries [q1, q2...].
4   - Q: Goal? A: Smallest interval size containing q.
5   - Q: -1? A: If no interval covers q.
6
7 2. LOGIC (OFFLINE QUERY PROCESSING)
8   - Sort Intervals by start.
9   - Sort Queries (keep original index to map answer back).
10  - Min-Heap: Stores '(size, end_time)'.
11  - Iterate through sorted queries:
12    1. Add all intervals where 'start <= query'.
13    2. Remove all intervals from heap top where 'end < query' (Invalid).
14    3. Heap top is now the smallest VALID interval.
15  - Complexity: O(N log N + Q log Q).
16 """
17
18 # --- THE MAGIC QUESTION ---
19 # "I'll use the 'Offline Query' pattern. I sort both intervals and queries.
20 # As I iterate through queries, I add 'started' intervals to a Min-Heap (based on size)
21 # and remove 'ended' intervals from the top. The top of the heap is always the smallest
22 # valid interval. O(N log N + Q log Q)."
```

```
23 def minInterval(intervals: List[List[int]], queries: List[int]) -> List[int]:
24     intervals.sort()
25
26     # Store queries with original index: (query_val, index)
27     sorted_queries = sorted([(q, i) for i, q in enumerate(queries)])
28
29     res = [-1] * len(queries)
30     minHeap = [] # (size, end)
31     i = 0
32
33     for q, idx in sorted_queries:
34         # 1. Add valid intervals (that have started)
35         while i < len(intervals) and intervals[i][0] <= q:
36             l, r = intervals[i]
37             size = r - l + 1
38             heapq.heappush(minHeap, (size, r))
39             i += 1
40
41         # 2. Remove invalid intervals (that ended before q)
42         while minHeap and minHeap[0][1] < q:
43             heapq.heappop(minHeap)
44
45         # 3. Top is answer
46         if minHeap:
47             res[idx] = minHeap[0][0]
48
49     return res
50
51 """
52 5. DRY RUN
53 Intervals: [[1,4], [2,4], [3,6]] -> Sorted: Same
54 Queries: [2, 3] -> Sorted: Same
55 - Query 2:
56   - Add [1,4] (size 4), Add [2,4] (size 3). Heap: [(3,4), (4,4)].
57   - Remove ended? None.
58   - Top: 3. Res[0] = 3.
59 - Query 3:
60   - Add [3,6] (size 4). Heap: [(3,4), (4,4), (4,6)].
61   - Remove ended? None.
62   - Top: 3. Res[1] = 3.
63 """
```

43 Math & Geometry: Matrix Manipulation

43.1 Transpose Matrix

```

1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: 2D integer array (m x n).
4 - Q: Goal? A: Flip over main diagonal. (r, c) -> (c, r).
5 - Q: Dimensions? A: m x n becomes n x m.
6
7 2. LOGIC (NEW ARRAY)
8 - Cannot always do in-place easily if m != n (rectangular).
9 - Create new matrix 'res' of size n x m.
10 - res[c][r] = matrix[r][c].
11 """
12
13 # --- THE MAGIC QUESTION ---
14 # "Since the matrix can be rectangular ($M \neq N$), in-place transposition is complex.
15 # I'll simply create a new $N \times M$ matrix and map 'matrix[r][c]' to 'res[c][r]'. O(
    M*N)."
16
17 def transpose(matrix: List[List[int]]) -> List[List[int]]:
18     ROWS, COLS = len(matrix), len(matrix[0])
19     res = [[0] * ROWS for _ in range(COLS)]
20
21     for r in range(ROWS):
22         for c in range(COLS):
23             res[c][r] = matrix[r][c]
24
25     return res

```

43.2 Rotate Image

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Direction? A: 90 degrees clockwise.
4 - Q: Constraint? A: In-place (O(1) extra space).
5 - Q: Matrix? A: n x n square.
6
7 2. APPROACHES
8 - Approach 1: Transpose + Reverse.
9   - Logic: Transpose (swap(r,c) with (c,r)). Then reverse each row.
10  - Accept: Standard linear algebra trick.
11 - Approach 2: Four-way swap (Layer by Layer).
12   - Save TopLeft. TL = BL, BL = BR, BR = TR, TR = SavedTL.
13   - Complexity: O(N^2).
14
15 3. INDENTED ENGLISH (TRANSPOSE + REVERSE)
16 function rotate(matrix):
17     # 1. Transpose
18     for r in range(n):
19         for c in range(r, n):
20             swap(matrix[r][c], matrix[c][r])
21     # 2. Reverse rows
22     for row in matrix:
23         row.reverse()
24 """
```

The Magic Question (Get Buy-In)

"Rotating 90 degrees clockwise is mathematically equivalent to Transposing the matrix (swapping rows and cols) and then Reversing each row. I'll do both operations in-place. $O(N^2)$."

```
1 def rotate(matrix: List[List[int]]) -> None:
2     N = len(matrix)
3
4     # 1. Transpose (Swap diagonal)
5     for r in range(N):
6         for c in range(r, N): # Start from r to avoid double swapping
7             matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
8
9     # 2. Reverse Rows
10    for row in matrix:
11        row.reverse()
12
13    """
14 5. DRY RUN
15    [[1,2],
16     [3,4]]
17 - Transpose:
18   - (0,1) swap (1,0) -> 2 swap 3.
19   - [[1,3],
20     [2,4]]
21 - Reverse:
22   - [1,3] -> [3,1]
23   - [2,4] -> [4,2]
24 - Final: [[3,1], [4,2]]. Correct.
25 """
```

43.3 Spiral Matrix

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: m x n matrix.
4 - Q: Output? A: List of integers in spiral order.
5
6 2. LOGIC (BOUNDARIES)
7 - Maintain 4 pointers: top, bottom, left, right.
8 - Loop while left <= right and top <= bottom.
9 - 1. Traverse Top Row (left -> right). top++.
10 - 2. Traverse Right Col (top -> bottom). right--.
11 - 3. Check break condition.
12 - 4. Traverse Bottom Row (right -> left). bottom--.
13 - 5. Traverse Left Col (bottom -> top). left++.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll simulate the spiral path using four boundary pointers: 'top', 'bottom', 'left',
18 # 'right'.
19 # I iterate in a loop: traverse right, down, left, up, shrinking the boundaries after
20 # each pass.
21 # Crucially, I'll check if boundaries have crossed after the first two traversals to
22 # avoid duplicates. O(M*N)."
```

```
20
21 def spiralOrder(matrix: List[List[int]]) -> List[int]:
22     res = []
23     left, right = 0, len(matrix[0])
24     top, bottom = 0, len(matrix)
25
26     while left < right and top < bottom:
27         # 1. Top Row (left to right)
28         for i in range(left, right):
29             res.append(matrix[top][i])
30             top += 1
31
32         # 2. Right Col (top to bottom)
33         for i in range(top, bottom):
34             res.append(matrix[i][right - 1])
35             right -= 1
36
37         if not (left < right and top < bottom):
38             break
39
40         # 3. Bottom Row (right to left)
41         for i in range(right - 1, left - 1, -1):
42             res.append(matrix[bottom - 1][i])
43             bottom -= 1
44
45         # 4. Left Col (bottom to top)
46         for i in range(bottom - 1, top - 1, -1):
47             res.append(matrix[i][left])
48             left += 1
49
50     return res
```

43.4 Set Matrix Zeroes

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Goal? A: If cell is 0, set entire row/col to 0.
4 - Q: Constraint? A: O(1) space "Strong Hire".
5
6 2. LOGIC (USE FIRST ROW/COL AS FLAGS)
7 - Instead of new arrays, use row 0 and col 0 to store flags.
8 - Exception: The overlapping cell 'matrix[0][0]'.
9 - Use 'rowZero' boolean for the first row's status.
10 - Use 'matrix[0][0]' for the first column's status.
11 - Pass 1: Mark flags.
12 - Pass 2: Iterate inner grid (1..R, 1..C) and set zeros.
13 - Pass 3: Handle first row/col separately using flags.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "To achieve O(1) space, I'll use the first row and first column of the matrix itself
18 # to store the 'zero flags'.
19 # I'll keep a separate variable 'rowZero' to handle the first row's overlap.
20 # First, I mark the flags; then I use them to set the inner matrix to zero; finally, I
21 # handle the first row and column. O(M*N)."
22
23 def setZeroes(matrix: List[List[int]]) -> None:
24     ROWS, COLS = len(matrix), len(matrix[0])
25     rowZero = False # Flag for 0th row
26
27     # 1. Mark Flags
28     for r in range(ROWS):
29         for c in range(COLS):
30             if matrix[r][c] == 0:
31                 matrix[0][c] = 0 # Mark column
32
33                 if r > 0:
34                     matrix[r][0] = 0 # Mark row
35             else:
36                 rowZero = True
37
38     # 2. Set Zeros (Inner Grid)
39     for r in range(1, ROWS):
40         for c in range(1, COLS):
41             if matrix[0][c] == 0 or matrix[r][0] == 0:
42                 matrix[r][c] = 0
43
44     # 3. Handle First Col
45     if matrix[0][0] == 0:
46         for r in range(ROWS):
47             matrix[r][0] = 0
48
49     # 4. Handle First Row
50     if rowZero:
51         for c in range(COLS):
52             matrix[0][c] = 0
```

43.5 Detect Squares

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Stream of points (add), and count(point).
4   - Q: Axis-aligned? A: Yes.
5   - Q: Goal? A: Count squares with 'point' as one corner.
6
7 2. LOGIC (DIAGONAL FINDING)
8   - Store points in a HashMap (Coordinate -> Count).
9   - Query(px, py):
10    - Iterate through ALL existing points (x, y).
11    - Check if (x, y) forms a diagonal with (px, py).
12    - Condition: abs(px - x) == abs(py - y) AND (px != x).
13    - If diagonal exists, check if the other two corners (x, py) and (px, y) exist.
14    - Add count[(x, y)] * count[(x, py)] * count[(px, y)].
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll store point counts in a Hash Map. For the 'count' query, I iterate through all
19 #   known points
20 # and treat each as a potential diagonal corner. If a point forms a diagonal with the
21 #   query point,
22 # I check the map for the existence of the other two corners to complete the square. O(N
23 ) per query."
24
25 class DetectSquares:
26
27     def __init__(self):
28         self.ptsCount = collections.defaultdict(int)
29         self.pts = []
30
31     def add(self, point: List[int]) -> None:
32         # Use tuple for hashability
33         self.ptsCount[tuple(point)] += 1
34         self.pts.append(point)
35
36     def count(self, point: List[int]) -> int:
37         res = 0
38         px, py = point
39
40         for x, y in self.pts:
41             # Check Diagonal: |dx| == |dy| and not same point
42             if (abs(px - x) != abs(py - y)) or px == x:
43                 continue
44
45             # If (x, y) is diagonal, corners are (x, py) and (px, y)
46             res += self.ptsCount[(x, py)] * self.ptsCount[(px, y)]
47
48         return res
```

44 Math & Geometry: Number Theory & GCD

44.1 Greatest Common Divisor of Strings

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Two strings s1, s2.
4   - Q: Definition of Divisor? A: 't' divides 's' if 's' is just 't' repeated multiple
5     times.
6   - Q: Goal? A: Largest string 'x' that divides both s1 and s2.
7
8 2. LOGIC (MATH PROPERTY)
9   - Property: If s1 + s2 != s2 + s1, no common divisor exists. Return "".
10  - If they are compatible, the GCD string length is simply gcd(len(s1), len(s2)).
11  - Result: s1[:gcd_len].
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "First, I verify if a common divisor is even possible by checking if 's1 + s2 == s2 +
16 #   s1'.
```

```

15 # If they match, the problem reduces to finding the Greatest Common Divisor of their
    lengths.
16 # The answer is the prefix of that length. O(N)."
17
18 def gcdOfStrings(str1: str, str2: str) -> str:
19     # Check if they have a common pattern
20     if str1 + str2 != str2 + str1:
21         return ""
22
23     from math import gcd
24     # The length of the GCD string is the GCD of the lengths
25     length = gcd(len(str1), len(str2))
26
27     return str1[:length]

```

44.2 Insert Greatest Common Divisors in Linked List

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Head of Linked List.
4   - Q: Operation? A: Insert node with val = gcd(curr.val, next.val) between them.
5   - Q: End condition? A: Traversal stops at last node.
6
7 2. LOGIC (POINTER MANIPULATION)
8   - Iterate while 'curr.next' exists.
9   - 1. Calculate 'g_val = gcd(curr.val, curr.next.val)'.
10  - 2. Create 'node = ListNode(g_val)'.
11  - 3. Wiring: 'node.next = curr.next'; 'curr.next = node'.
12  - 4. Advance 'curr' to 'node.next' (skip the new node).
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll traverse the list. Between every pair of nodes 'curr' and 'curr.next',
17 # I'll calculate their GCD, create a new node, and insert it. Then I advance 'curr'
18 # past the new node to process the next original pair. O(N)."
```

```
19
20 def insertGreatestCommonDivisors(head: Optional[ListNode]) -> Optional[ListNode]:
21     import math
22
23     curr = head
24     while curr and curr.next:
25         val1, val2 = curr.val, curr.next.val
26         g = math.gcd(val1, val2)
27
28         # Insert new node
29         new_node = ListNode(g)
30         new_node.next = curr.next
31         curr.next = new_node
32
33         # Advance pointers (skip the inserted node)
34         curr = new_node.next
35
36     return head
```

44.3 Happy Number

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Process? A: Replace number with sum of squares of digits.
4 - Q: Happy? A: Reaches 1.
5 - Q: Unhappy? A: Enters a cycle (does not include 1).
6
7 2. APPROACHES
8 - Approach 1: HashSet.
9   - Store seen numbers. If we see a number again, cycle detected.
10 - Approach 2: Floyd's Cycle Finding (Tortoise and Hare).
11   - Slow runner does 1 step (sum of squares).
12   - Fast runner does 2 steps.
13   - If Fast == Slow (and not 1), cycle detected -> False.
14   - If Fast == 1, True.
15   - Accept: O(1) Space.
16
17 3. INDENTED ENGLISH (FLOYD'S)
18     function isHappy(n):
19         slow, fast = n, sumSq(n)
20         while fast != 1 and slow != fast:
21             slow = sumSq(slow)
22             fast = sumSq(sumSq(fast))
23         return fast == 1
24 """
```

The Magic Question (Get Buy-In)

"This is a cycle detection problem. The sequence either reaches 1 or enters a loop. I'll use Floyd's Tortoise and Hare algorithm (two pointers) to detect the cycle in $O(1)$ space."

```
1 def isHappy(n: int) -> bool:
2
3     def sumSq(num):
4         output = 0
5         while num:
6             digit = num % 10
7             output += digit ** 2
8             num //= 10
9         return output
10
11     slow, fast = n, sumSq(n)
12
13     # Continue while fast hasn't reached 1 and hasn't caught slow
14     while fast != 1 and slow != fast:
15         slow = sumSq(slow)
16         fast = sumSq(sumSq(fast))
17
18     return fast == 1
```

44.4 Plus One

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: List of digits [1, 2, 3] represents 123.
4   - Q: Goal? A: Add one: [1, 2, 4].
5   - Q: Tricky case? A: [9, 9] -> [1, 0, 0].
6
7 2. LOGIC (REVERSE ITERATION)
8   - Iterate backwards from end.
9   - If digit < 9: Increment and return immediately.
10  - If digit == 9: Set to 0, continue to next digit (carry).
11  - If loop finishes (all 9s): Return [1] + digits (e.g., [1, 0, 0]).
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll iterate backwards. If I find a digit less than 9, I increment it and return
16 # immediately.
17 # If it's a 9, I set it to 0 and carry over. If I finish the loop, it means the number
18 # was all 9s,
19 # so I prepend a 1. O(N)."
20
21 def plusOne(digits: List[int]) -> List[int]:
22     n = len(digits)
23
24     for i in range(n - 1, -1, -1):
25         if digits[i] < 9:
26             digits[i] += 1
27             return digits
28
29         digits[i] = 0
30
31     # If we are here, it means all digits were 9 (e.g., 999 -> 000)
32     return [1] + digits
```

45 Math & Geometry: String Logic & Arithmetic

45.1 Roman to Integer

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Valid Roman numeral string (e.g., "IV", "MCMXCIV").
4   - Q: Range? A: 1 to 3999.
5
6 2. LOGIC (LOOKAHEAD)
7   - Standard Rule: Add the value.
8   - Exception: If current value < next value, SUBTRACT it.
9     - "IV" -> 1 < 5 -> Subtract 1, Add 5. Result 4.
10    - "VI" -> 5 > 1 -> Add 5, Add 1. Result 6.
11   - Algo: Map chars to ints. Iterate. Compare s[i] with s[i+1].
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll map each Roman character to its integer value. As I iterate through the string,
16 # if the current value is less than the next one (like 'IV'), I subtract it.
17 # Otherwise, I add it. O(N)."
18
19 def romanToInt(s: str) -> int:
20     roman = {
21         "I": 1, "V": 5, "X": 10, "L": 50,
22         "C": 100, "D": 500, "M": 1000
23     }
24
25     res = 0
26
27     for i in range(len(s)):
28         # Lookahead check
29         if i + 1 < len(s) and roman[s[i]] < roman[s[i + 1]]:
30             res -= roman[s[i]]
31         else:
```

```
32         res += roman[s[i]]
33
34     return res
```

45.2 Excel Sheet Column Title

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: Integer column number (1 -> "A", 28 -> "AB").
4 - Q: Base? A: Base-26, but 1-indexed (no 0).
5
6 2. LOGIC (BASE CONVERSION)
7 - Normal Base-10 to Base-26: 'digit = n % 26', 'n //= 26'.
8 - Issue: In Excel, 'A' is 1, not 0. 26 is 'Z', 27 is 'AA'.
9 - Fix: Decrement n by 1 before modulo operation.
10 - 'n = 26' -> 'n-1 = 25' -> '25 % 26 = 25' ('Z'). Correct.
11 - 'n = 27' -> 'n-1 = 26' -> '26 % 26 = 0' ('A'). Correct.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "This is essentially Base-26 conversion. The trick is that Excel columns are 1-indexed
16 # , not 0-indexed.
17 # To handle this, I'll decrement 'n' by 1 at each step before taking the modulo 26 to
18 # map it to 'A'-'Z'. O(log N)."
19
20 def convertToTitle(columnNumber: int) -> str:
21     res = ""
22
23     while columnNumber > 0:
24         columnNumber -= 1 # Adjust for 1-based indexing
25
26         remainder = columnNumber % 26
27         # 'A' is 65
28         res += chr(ord('A') + remainder)
29
30         columnNumber //= 26
31
32     return res[::-1] # Reverse since we built it backwards
33
34 """
35 5. DRY RUN
36 n = 28
37 - n-=1 (27). Rem = 27 % 26 = 1 ('B'). n //= 26 -> 1.
38 - n-=1 (0). Rem = 0 % 26 = 0 ('A'). n //= 26 -> 0.
39 - Res = "BA" -> Reverse -> "AB".
40 """
```

45.3 Pow(x, n)

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: n negative? A: Yes, return 1 / x|n|.
4 - Q: Constraints? A: O(log N) required.
5
6 2. APPROACHES
7 - Approach 1: Loop n times. O(N). TLE.
8 - Approach 2: Binary Exponentiation (Recursive).
9   - xn = x(n/2) * x(n/2) if n is even.
10  - xn = x * x(n-1) if n is odd.
11  - Complexity: O(log N).
12  - Accept: Optimal.
13
14 3. INDENTED ENGLISH
15 function myPow(x, n):
16     if n == 0: return 1
17     if n < 0: return myPow(1/x, -n)
18     if n % 2 == 0:
19         half = myPow(x, n // 2)
20         return half * half
21     else:
22         return x * myPow(x, n - 1)
23 """
```

The Magic Question (Get Buy-In)

"I'll use Binary Exponentiation. Instead of multiplying x repeatedly, I calculate $x^{n/2}$ once and square it. If n is odd, I multiply by one extra x . This reduces the complexity to $O(\log N)$."

```
1 def myPow(x: float, n: int) -> float:
2     # Handle negative exponent
3     if n < 0:
4         x = 1 / x
5         n = -n
6
7     def helper(n):
8         if n == 0: return 1
9
10        half = helper(n // 2)
11
12        if n % 2 == 0:
13            return half * half
14        else:
15            return x * half * half
16
17     return helper(n)
```

45.4 Multiply Strings

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Two non-negative integers as strings.
4   - Q: Built-in BigInt? A: No, simulate manually.
5   - Q: Output? A: Product as string.
6
7 2. LOGIC (SIMULATION)
8   - Result array size: len1 + len2 (Max possible digits).
9   - Iterate i from num1, j from num2 (reversed).
10  - Product = n1 * n2.
11  - Add to 'res[i + j + 1]' (ones place).
12  - Carry over to 'res[i + j]' (tens place).
13  - Skip leading zeros at the end.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll simulate manual multiplication using an array of size 'len1 + len2'.
18 # For every pair of digits at indices 'i' and 'j', their product contributes to indices
19 # 'i+j' and 'i+j+1' in the result array. O(M*N)."
```

```
19
20 def multiply(num1: str, num2: str) -> str:
21     if "0" in [num1, num2]:
22         return "0"
23
24     res = [0] * (len(num1) + len(num2))
25     num1, num2 = num1[::-1], num2[::-1] # Reverse to align indices
26
27     for i in range(len(num1)):
28         for j in range(len(num2)):
29             digit1 = int(num1[i])
30             digit2 = int(num2[j])
31
32             # Position indices in result array
33             # Note: Because we reversed input, indices are straight forward
34             # i+j is low position, i+j+1 is carry position
35             val = res[i + j] + (digit1 * digit2)
36
37             res[i + j] = val % 10
38             res[i + j + 1] += val // 10
39
40     # Reverse back and remove leading zeros
41     res = res[::-1]
42
43     # Locate first non-zero
44     beg = 0
45     while beg < len(res) and res[beg] == 0:
46         beg += 1
47
48     # Convert to string
49     return "".join(map(str, res[beg:]))
```

46 Bit Manipulation: XOR & Essentials

46.1 Single Number

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Non-empty array of integers.
4   - Q: Rule? A: Every element appears twice except for one.
5   - Q: Constraint? A: Linear time O(N) and Constant space O(1).
6
7 2. LOGIC (XOR CANCELLATION)
8   - Property: A ^ A = 0.
9   - Property: A ^ 0 = A.
10  - Property: XOR is commutative (order doesn't matter).
11  - If we XOR all numbers, pairs cancel out (become 0).
12  - The result is the single unique number.
13 """
```

```

14
15 # --- THE MAGIC QUESTION ---
16 # "I'll use the XOR property where 'A ^ A = 0'. By XORing all numbers in the array
    together,
17 # the duplicate pairs will cancel each other out, leaving only the single unique number.
    O(N)."
18
19 def singleNumber(nums: List[int]) -> int:
20     res = 0
21     for n in nums:
22         res ^= n
23     return res
24
25 """
26 5. DRY RUN
27     nums = [4, 1, 2, 1, 2]
28     - res=0
29     - res^4 = 4
30     - res^1 = 5 (binary 101)
31     - res^2 = 7 (binary 111)
32     - res^1 = 6 (binary 110) -> The '1's cancelled
33     - res^2 = 4 (binary 100) -> The '2's cancelled
34     - Final: 4.
35 """

```

46.2 Number of 1 Bits

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: Unsigned integer.
4 - Q: Goal? A: Count set bits ('1's').
5
6 2. APPROACHES
7 - Approach 1: Loop 32 times. Check 'n & 1', then 'n >>= 1'.
8 - Approach 2: Brian Kernighan's Algorithm.
9 - Logic: 'n = n & (n - 1)' removes the rightmost '1' bit.
10 - Complexity: O(Number of set bits), faster than O(32) for sparse numbers.
11 - Accept: Optimal.
12
13 3. INDENTED ENGLISH (KERNIGHAN)
14 function hammingWeight(n):
15     count = 0
16     while n > 0:
17         n = n & (n - 1)
18         count += 1
19     return count
20 """
```

The Magic Question (Get Buy-In)

"I'll use the trick 'n = (n - 1)', which always flips the least significant set bit to 0. I simply repeat this operation until 'n' becomes 0, counting the steps. This is strictly faster than iterating all 32 bits."

```
1 def hammingWeight(n: int) -> int:
2     count = 0
3     while n:
4         n &= (n - 1) # Removes the rightmost '1'
5         count += 1
6     return count
```

46.3 Counting Bits

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Goal? A: Return array where ans[i] is number of 1s in i, for 0 <= i <= n.
4   - Q: Constraints? A: O(N) time. (Cannot run Hamming Weight N times).
5
6 2. LOGIC (DP / OFFSET)
7   - Pattern:
8     0: 0
9     1: 1 (1+dp[0])
10    2: 1 (1+dp[0])
11    3: 2 (1+dp[1])
12   - Formula: dp[i] = dp[i >> 1] + (i & 1).
13   - Logic: The bits of 'i' are the bits of 'i // 2' shifted left, plus the last bit (
      odd/even).
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "This is a Dynamic Programming problem. The number of set bits in 'i' is equal to the
      number of set bits
18 # in 'i >> 1' (integer division by 2) plus 1 if 'i' is odd. O(N)."
19
20 def countBits(n: int) -> List[int]:
21     dp = [0] * (n + 1)
22
23     for i in range(1, n + 1):
24         # i >> 1 reuses value for i//2
25         # i & 1 adds 1 if odd
26         dp[i] = dp[i >> 1] + (i & 1)
27
28     return dp
```

46.4 Missing Number

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Array [0, n] missing one number.
4   - Q: Unsorted? A: Yes.
5   - Q: Constraints? A: O(N) time, O(1) space.
6
7 2. APPROACHES
8   - Approach 1: Sum formula. 'Exp = n*(n+1)/2'. 'Missing = Exp - Actual'.
9   - Approach 2: XOR.
10    - Logic: XOR all indices [0..n] and XOR all values.
11    - Result: '(0^0) ^ (1^1) ... ^ (missing)'. Everything cancels except missing.
12    - Accept: Safe from overflow (though Python handles overflow).
13
14 3. INDENTED ENGLISH (XOR)
15   function missingNumber(nums):
16       res = len(nums)
17       for i, n in enumerate(nums):
18           res ^= i ^ n
19       return res
20 """
```

The Magic Question (Get Buy-In)

"I'll XOR all the indices from 0 to N together with all the values in the array. Since every number appears once in the index sequence and once in the value sequence (except the missing one), everything else cancels out. $O(N)$."

```
1 def missingNumber(nums: List[int]) -> int:
2     # Initialize with n because loop only goes to n-1
3     res = len(nums)
4
5     for i in range(len(nums)):
6         # XOR the index and the value
7         res ^= i ^ nums[i]
8
9     return res
```

46.5 Reverse Bits

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Input? A: 32-bit unsigned integer.
4     - Q: Output? A: Value with bits reversed.
5
6 2. LOGIC (BIT SHIFTING)
7     - Result 'res' starts at 0.
8     - Iterate 32 times:
9         1. Shift 'res' Left ('res << 1') to make room.
10        2. Get last bit of 'n' ('n & 1').
11        3. Add to 'res' (OR or ADD).
12        4. Shift 'n' Right ('n >> 1') to process next bit.
13 """
14
15 # --- THE MAGIC QUESTION ---
16 # "I'll iterate 32 times. In each step, I shift my result to the left to open a spot,
17 # then extract the least significant bit from the input 'n' and add it to that spot. 0
18 # (1)."
```

```
18
19 def reverseBits(n: int) -> int:
20     res = 0
21     for _ in range(32):
22         # 1. Shift result left
23         res = res << 1
24
25         # 2. Get last bit of n
26         bit = n & 1
27
28         # 3. Add bit to result
29         res = res | bit
30
31         # 4. Shift n right
32         n = n >> 1
33
34     return res
```

47 Bit Manipulation: Arithmetic & Ranges

47.1 Add Binary

```
1 """
2 1. CLARIFYING QUESTIONS
3     - Q: Input? A: Two binary strings "11", "1".
4     - Q: Output? A: Sum as binary string "100".
5     - Q: Built-in allowed? A: Ideally assume no 'int(a, 2)'.
6
7 2. LOGIC (MANUAL ADDITION)
8     - Pointers at end of strings 'a' and 'b'.
9     - Maintain 'carry'.
10    - Loop while pointers valid or carry > 0.
11    - 'total = digitA + digitB + carry'.
12    - 'res.append(total % 2)'.
13    - 'carry = total // 2'.
14    - Reverse result.
15 """
16
17 # --- THE MAGIC QUESTION ---
18 # "I'll simulate manual binary addition iterating from right to left.
19 # I keep track of a 'carry'. At each position, the sum is 'digitA + digitB + carry'.
20 # The new digit is 'sum % 2' and the new carry is 'sum // 2'. O(N)."
```

```
21
22 def addBinary(a: str, b: str) -> str:
23     res = []
24     carry = 0
25     i, j = len(a) - 1, len(b) - 1
26
27     while i >= 0 or j >= 0 or carry:
28         digitA = int(a[i]) if i >= 0 else 0
```

```
29     digitB = int(b[j]) if j >= 0 else 0
30
31     total = digitA + digitB + carry
32
33     # Binary math: 1+1=0 (carry 1), 1+0=1, 1+1+1=1 (carry 1)
34     res.append(str(total % 2))
35     carry = total // 2
36
37     i -= 1
38     j -= 1
39
40     return "".join(res[::-1])
```

47.2 Sum of Two Integers

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Constraints? A: No '+' or '-' operators.
4 - Q: Negatives? A: Yes.
5
6 2. LOGIC (HARDWARE ADDER)
7 - XOR ('a ^ b') adds bits where at least one is 1 (but not both). It's "Sum without
8   Carry".
9 - AND ('a & b') finds bits where BOTH are 1. Shifting this left ('<< 1') is the "
10  Carry".
11 - Loop until Carry is 0.
12 - Python Caveat: Integers are infinite. Must mask with '0xffffffff' to simulate 32-
13   bit overflow.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "I'll simulate a hardware adder. 'a ^ b' gives the sum without carry, and '(a & b) <<
18   1' gives the carry.
19 # I repeat 'a = sum', 'b = carry' until the carry is zero. In Python, I mask with 0
20   xFFFFFFFF to handle negative number representation correctly. 0(1)."
```

```
16
17 def getSum(a: int, b: int) -> int:
18     mask = 0xffffffff # 32-bit mask
19
20     while (b & mask) > 0:
21         carry = (a & b) << 1
22         a = a ^ b
23         b = carry
24
25     # Handle negative result for Python (if MSB is 1)
26     return (a & mask) if b > 0 else a
```

47.3 Reverse Integer

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Range? A: 32-bit signed.
4 - Q: Overflow? A: Return 0 if result overflows.
5
6 2. LOGIC (MODULO 10)
7 - Extract digit: 'n % 10'.
8 - Build result: 'res = res * 10 + digit'.
9 - Overflow Check: Check *before* multiplying.
10 - 'if res > MAX // 10', fail.
11 - Python handles large ints automatically, but interviews want logic.
12 """
13
14 # --- THE MAGIC QUESTION ---
15 # "I'll extract digits using modulo 10 and build the reversed number.
16 # To handle 32-bit overflow strictly, I check if 'res' exceeds 'MAX_INT // 10' *before*
17 # adding the next digit. O(log N)."
```

```
18 def reverse(x: int) -> int:
19     # Handle negative sign
20     sign = -1 if x < 0 else 1
21     x = abs(x)
22
23     res = 0
24     while x:
25         digit = x % 10
26         x //= 10
27
28         # Check overflow (2^31 - 1 approx 2.14 billion)
29         if res > 214748364: # Pre-check (MAX_INT // 10)
30             return 0
31
32         res = res * 10 + digit
33
34     return res * sign
```

47.4 Bitwise AND of Numbers Range

```
1 """
2 1. CLARIFYING QUESTIONS
3   - Q: Input? A: Range [left, right].
4   - Q: Goal? A: AND of all numbers in range.
5   - Q: Naive? A: Loop left to right? No, TLE if range is large.
6
7 2. LOGIC (COMMON PREFIX)
8   - If numbers differ, the lower bits will eventually flip (0->1->0) across the range,
9     resulting in 0 AND.
10  - We only care about the "Common Prefix" of 'left' and 'right'.
11  - Shift both right until they equal. Count shifts.
12  - Shift result left by count.
13 """
14 # --- THE MAGIC QUESTION ---
15 # "The Bitwise AND of a range is determined by the common high-order bits (prefix) of
16 # the start and end numbers.
17 # I'll shift both numbers right until they match, tracking the shift count, then shift
18 # the result back. O(log N)."
19
20 def rangeBitwiseAnd(left: int, right: int) -> int:
21     shift = 0
22
23     # Find common prefix
24     while left < right:
25         left >>= 1
26         right >>= 1
27         shift += 1
28
29     # Shift back to fill low-bits with 0
30     return left << shift
31
32 """
33 5. DRY RUN
34 left=5 (101), right=7 (111)
35 - 5 < 7? Yes. L=2 (10), R=3 (11). s=1.
36 - 2 < 3? Yes. L=1 (01), R=1 (01). s=2.
37 - 1 < 1? No.
38 - Return 1 << 2 = 4 (100).
39 - Logic: 5&6&7 = 101 & 110 & 111 = 100. Correct.
40 """
```

47.5 Minimum Array End

```
1 """
2 1. CLARIFYING QUESTIONS
3 - Q: Input? A: n (size), x (AND result).
4 - Q: Goal? A: Create increasing array of size n, where AND(all) == x.
5   Return min possible value of last element.
6 - Q: Logic? A: To keep AND == x, every number must have bits of x set.
7   - The variable bits are the 0-bits of x.
8   - We construct the (n-1)th number using the "empty slots" (0-bits) of x.
9
10 2. LOGIC (BIT INTERLEAVING)
11 - We need to form the (n-1)th number (0-indexed). Let's call it 'k = n - 1'.
12 - We map the bits of 'k' into the zero-bit positions of 'x'.
13 - Why? We preserve the 1s of x (to satisfy AND condition) and fill gaps to make the
   sequence increasing.
14 """
15
16 # --- THE MAGIC QUESTION ---
17 # "To ensure the AND of the array is 'x', every element must have the bits of 'x' set to
   1.
18 # To find the minimum ending value, I take 'n-1' and insert its bits into the *zero-bit
   positions* of 'x'. O(64) ~ O(1)."
19
20 def minEnd(n: int, x: int) -> int:
21     # We need to construct the (n-1)-th variation
22     k = n - 1
23     res = x
24
25     # Position in k (the source bits)
26     k_pos = 0
27     # Position in x (the target bits)
28     x_pos = 0
29
30     while (k >> k_pos) > 0:
31         # If x has a 0 bit at this position, we can use it
32         if not (res & (1 << x_pos)):
33             # Get the bit from k
34             bit = (k >> k_pos) & 1
35             # Set it in res
36             res |= (bit << x_pos)
37             k_pos += 1
38
39         x_pos += 1
40
41     return res
```